

Programski jezici

<http://www.programskijezici.matf.bg.ac.rs/>

**Univerzitet u Beogradu
Matematički fakultet**

Programske paradigme

Materijali za vežbe

**Nastavnik: Milena Vujošević Janičić
Asistenti: Milan Čugurović, Ivan Ristović**

**Beograd
2020.**

Priprema materijala:

dr Milena Vujošević Jančić, docent na Matematičkom fakultetu u Beogradu

Marjana Šolajić, asistent na Matematičkom fakultetu u Beogradu

Branislava Živković

Nemanja Mićović, asistent na Matematičkom fakultetu u Beogradu

Milica Selaković, asistent na Matematičkom fakultetu u Beogradu

Milan Čugurović, asistent na Matematičkom fakultetu u Beogradu

Ivan Ristović, asistent na Matematičkom fakultetu u Beogradu

Sadržaj

1	Skript programiranje	3
1.1	Žašto Python?	3
1.2	Uvod, funkcije, kolekcije, moduli	3
1.2.1	Uvodni primeri	3
1.2.2	Zadaci za samostalni rad sa rešenjima	14
1.2.3	Zadaci za vežbu	15
1.3	OOP koncepti	18
1.4	Datoteke, serijalizacija, sortiranje	22
1.4.1	Datoteke	22
1.4.2	Serijalizacija, JSON	23
1.4.3	os modul	24
1.4.4	Sortiranje	25
1.4.5	Zadaci za samostalni rad sa rešenjima	26
1.4.6	Zadaci za vežbu	29
1.5	Rešenja	31
2	Programiranje ograničenja - Python	37
2.1	Programiranje ograničenja	37
2.1.1	Uvodni primeri	37
2.1.2	Zadaci za samostalni rad sa rešenjima	38
2.1.3	Zadaci za vežbu	42
2.2	Rešenja	45
3	Funkcionalni koncepti u programskom jeziku Python	55
3.1	Funkcionalno programiranje	55
3.1.1	Uvodni primeri	55
3.1.2	Zadaci za samostalni rad sa rešenjima	56
3.1.3	Zadaci za vežbu	58
3.2	Rešenja	60
4	Komponentno programiranje u programskom jeziku Python	63
4.1	PyQt5 - Instalacija potrebnih alata	63
4.1.1	Instalacija PyQt5	63
4.1.2	Instalacija dodatka za Visual Studio Code	63
4.2	Uvodni primeri	64
4.3	Zadaci za samostalni rad sa rešenjima	70
4.4	Rešenja	72
5	Funkcionalno programiranje	75
5.1	Uvod	75
5.1.1	Uvodni primeri	75
5.1.2	Zadaci za samostalni rad sa rešenjima	79
5.1.3	Zadaci za vežbu	80
5.2	Liste, funkcije višeg reda	81
5.2.1	Zadaci za samostalni rad sa rešenjima	86
5.2.2	Zadaci za vežbu	88
5.3	Korisnički tipovi, Napredni koncepti	89

5.3.1	Uvodni primeri	89
5.3.2	Zadaci za samostalni rad sa rešenjima	93
5.3.3	Zadaci za vezbu	95
5.4	Rešenja	96
6	Konkurentno programiranje	103
6.1	Jezik Scala	103
6.2	Instalacija potrebnih alata	104
6.2.1	Pravljenje projekta	104
6.2.2	Inicijalizacija projekta	104
6.2.3	Konfiguracija projekta	104
6.3	Uvod u jezik Scala	105
6.4	Zadaci	109
6.5	Zadaci za vezbu sa rešenjima	110
6.5.1	Zadaci za vezbu	111
6.6	Rešenja	112
7	Distribuirano programiranje	125
7.1	O Apache Spark-u	125
7.2	Uputstvo za Apache Spark	127
7.2.1	Potreban softver i alati	127
7.2.2	Pravljenje projekta	127
7.2.3	Dodavanje biblioteke u <code>build.sbt</code>	127
7.2.4	Pokretanje programa	128
7.3	Zadaci sa rešenjima	128
7.4	Zadaci za vezbu	130
7.5	Rešenja	130
8	Logičko programiranje	139
8.1	Jezik Prolog	139
8.2	Instalacija BProlog-a	139
8.3	Uvod	139
8.3.1	Uvodni primeri	139
8.3.2	Zadaci za samostalni rad sa rešenjima	144
8.3.3	Zadaci za vezbu	144
8.4	Liste	145
8.4.1	Uvodni primeri	145
8.4.2	Zadaci za samostalni rad sa rešenjima	146
8.4.3	Zadaci za vezbu	147
8.5	Razni zadaci	147
8.5.1	Zadaci sa rešenjima	147
8.5.2	Zadaci za vezbu	149
9	Programiranje ograničenja - Prolog	151
9.1	Programiranje ograničenja	151
9.1.1	Uvodni primeri	151
9.1.2	Zadaci za samostalni rad sa rešenjima	152
9.1.3	Zadaci za vezbu	154
9.2	Rešenja	156

1

Skript programiranje

Potrebno je imati instaliran Python 3.7 na računaru.

Literatura:

- (a) <https://www.python.org/>
- (b) <http://www.tutorialspoint.com/python>
- (c) <https://wiki.python.org/moin/>

1.1 Žašto Python?

Programski jezik Python predstavlja trenutno najpopularniji programski jezik. Koristi se svuda, od prosvete preko privrede i industrije do medicine i mnogih drugih oblasti. Najbolji pokazatelj toga su trenutne rang liste popularnosti programskih jezika, na kojima Python dominira. Organizacija IEEE rangirala je Python kao #1 jezik za 2018. godinu, pre čega je bio rangiran kao #1 u 2017. godini, a #3 u 2016. godini. GitHub, vizualizacija GitHub zastupljenosti jezika, stavlja Python na izuzetno visoku #3 poziciju.

Python kultura propagira open-source ideje, zajednicu koja je povezana kako na lokalnom tako i na globalnom nivou, koja održava svoj jezik, i deli svoje znanje sa drugima. Filozofija jezika je toliko jaka da je čak ugrađena i u sam jezik. Ovo se može videti tako što se interpreteru zada komanda "import this". Tom prilikom na ekranu se prikaže kompletan manifest jezika kao i osnovne ideje i vrednosti istog.

Osnovne prednosti jezika jesu jasnost, jednostavnost, intuitivnost, konciznost i ekspresivnost, kao i izuzetno jaka i aktivna zajednica. Dodatno sjajne biblioteke koje implementiraju mnogobrojne funkcionalnosti. Cena svega ovoga jeste efikasnost, često su Python programi dosta sporiji od osnovnih konkurenata. Međutim, na ovome se aktivno radi, pišu se biblioteke u Pythonu koje su jako efikasne (primer *numpy*) i koje umnogome popravljaju efikasnost rada u Pythonu.

Detaljnije poređenje može se videti na linku: <https://benchmarksgame-team.pages.debian.net/benchmarksgame/>

1.2 Uvod, funkcije, kolekcije, moduli

1.2.1 Uvodni primeri

Zadatak 1.1 Napisati program koji na standardni izlaz ispisuje poruku *Hello world!*.

```
1 # Ovako se pisu komentari
2 #
3 # Pokretanje programa iz terminala:
4 # $python hello.py
5
6 """
7 Tradicionalni program koji na standardni izlaz ispisuje Zdravo svete.
8 Ilustruje prednosti programskog jezika Python.
9 """
10 print("Hello world.")
```


Zadatak 1.2 Napisati program koji za uneta dva cela broja i nisku ispisuje najpre unete vrednosti, a zatim i zbir brojeva, njihovu razliku, proizvod i količnik.

```

1 # Skup (Set)
2 imena = set()
3 for ime in ['Milan', 'Ivan', 'Milan', 'Ana', 'Ana', 'Milan']:
4     imena.add(ime)
5 print(imena)
6 print(set(['Milan', 'Ivan', 'Milan', 'Ana', 'Ana', 'Milan']))
7
8 # Stek (LIFO struktura)
9
10 # Funkcije:
11 # * empty() - da li je stek prazan? - Time Complexity : O(1)
12 # * size() - čvelina steka? - Time Complexity : O(1)
13 # * top() - čVraa referencu na element sa vrha steka - Time Complexity : O(1)
14 # * push(g) - Dodaje element g na vrh steka - Time Complexity : O(1)
15 # * pop() - Skida element sa vrha steka - Time Complexity : O(1)
16 #
17 # Implementacije: collections.deque, queue.LifoQueue etc. ili jednostavno koristiti
18 listu!
19 stack = []
20 stack.append('a')
21 stack.append('b')
22 stack.append('c')
23 print('Inicijalni stek:', end=':')
24 print(stack)
25 print('pop():')
26 print(stack.pop())
27 print(stack.pop())
28 print(stack.pop())
29 print('Stanje steka:', end=':')
30 print(stack)
31
32 # Mapa (Dictionary)
33 # Notacija: {kljuc:vrednost, kljuc:vrednost, ...} - JSON format
34 prosek = {'Milan':9.45, 'Ana':9.87, 'Nikola':6.76}
35 print(prosek)
36 print(type(prosek))
37 print(prosek.keys())
38 print(prosek.values())
39 print('Milan' in prosek)
40
41 # Torke (Tuple)
42 # Torka predstavlja strukturu podataka koja je uredjena i imutabilna
43 torka = ("jabuka", "banana", "visnja", "pomorandza", "kivi", "lubenica", "mango")
44 print(torka[2:5])
45 try:
46     torka[0] = 11
47 except TypeError:
48     print("Torke su imutabilne.")

```

Zadatak 1.3 Napisati program koji sa standardnog ulaza učitava jedan string i jedan ceo broj, a zatim ispisuje (dinamicki određene) tipove učitanih promenljivih. Nakon toga, na standardni izlaz ispisati poruku Ispisivanje vise stringova koriscenjem separatora =, <-> i novog reda, redom.

```

1 # Hip (red sa prioriteto)
2 # https://docs.python.org/3.7/library/heapq.html
3 # Funkcije:
4 # * heapify - Konvertuje listu u hip u mestu.
5 # * heappush - Dodaje element u hip.
6 # * heappop - Vraca koren hipa.
7 # * heapreplace - Zamenjuje koren hipa novom vrednoscu prosledjenoj funkciji.
8 import heapq
9
10 H = [21,1,45,78,3,5]
11 heapq.heapify(H)
12 print(H)
13 heapq.heappush(H,8)
14 print(H)
15

```

```

# Heapsort
17 def heapsort(iterable):
    h = []
    19     for value in iterable:
        heapq.heappush(h, value)
    21     return [heapq.heappop(h) for i in range(len(h))]
print(heapsort([1, 3, 5, 7, 9, 2, 4, 6, 8, 0]))

```

Zadatak 1.4 Napisati program koji sa standardnog ulaza ucitava ceo broj n , a zatim na standardni izlaz ispisuje poruku o ostatku koji taj broj daje kada se podeli sa pet. Nakon toga, koristeći for i while petlju ispisati prvih n prirodnih brojeva.

```

# Grananje
2 print("if/elif: ")
x = int(input("Unesi jedan broj: "))
4 if x%5==0:
    print("Uneti je broj deljiv sa 5.")
6 elif x%5==1 or x%5==2:
    print("Uneti broj daje ostatak 1 ili 2 pri deljenju sa 5.")
8 elif x%5==3:
    print("Uneti broj daje ostatak 3 pri deljenju sa 5.")
10 else:
    print("Preostaje da uneti broj daje ostatak 4 pri deljenju sa 5.")
12
# While petlja
14 print("while: ")
i = 0
16 while i < 10:
    print(i, end=' ')
18     i += 1 # i++ nije podrzano

# For petlja
20 print("for: ")
22 for i in range(10):
    print(i, end=' ')

```

Zadatak 1.5 Napisati program koji sa standardnog ulaza ucitava ceo broj, koji je otporan na pogresan ulaz. Nakon toga, ilustrovati imutabilnost stringova.

```

1 # Try catch
try:
3     n = int(input("Unesite ceo broj: "))
except ValueError:
5     print("Greska. Parsiranje celog broja.")
    exit(1)
7
# Imutabilnost stringova
9 x = "Hello world"
print("x: ", x, id(x))
11 y = "Hello world"
print("y: ", y, id(y))
13 print(x==y)
#x[0] = '7' # Type Error
15 #print("x: ", x, id(x))

```

Zadatak 1.6 Ako je prvi dan u mesecu ponedeljak napisati funkciju `radni_dan(dan)` koja kao argument dobija dan u mesecu i vraća tačno ako je dan radni dan. Napisati program koji testira ovu funkciju, korisnik sa standardnog ulaza u petlji unosi deset dana i dobija o poruku o tome da li su uneti dani radni ili ne.

```

1 # Funkcije
#
3 # def ime_funkcije(argumenti):
#     telo funkcije
5
def radni_dan(broj):
7     # Osnovne logicke operacije:
    # not, and, or
9

```

```

11         if broj % 7 == 1 or broj % 7 == 2 or broj % 7 == 3:
12             return True
13         elif broj % 7 == 4 or broj % 7 == 5:
14             return True
15         # Naredbi <<elif>> moze biti vise
16         else:
17             return False
18
19 # Blokovi se ne ogranicavaju viticastim zagradama kao sto je u C-u
20 # vec moraju biti indentovani. Naredbe na istom nivou indentacije se
21 # smatraju istim naredbama istog bloka
22
23 i = 0
24 while i <= 10:
25     dan = input("Unesite dan: ")
26     try:
27         dan = int(dan)
28     except ValueError:
29         print("Greska. Parsiranje rednog broja dana.")
30         exit(1)
31     if radni_dan(dan):
32         print("Uneti dan je radni dan.")
33     else:
34         print("Uneti dan je neradan.")
35     i = i + 1 # i++ ne postoji, moze ili ovako ili i += 1
36
37 # Naredba <<break>> iskace iz bloka, isto kao i u C-u

```

Zadatak 1.7 Napisati program koji ilustruje osnovne sistemske funkcije i module programskog jezika Python tako škoristi funkcije faktoriyel, logaritam (math), generise pseudoslucajne brojeve (modul random), ispisuje argumente komandne linije na izlaz (modul sys), poziva sistemski poziv listanja tekuceg direktorijuma (modul os), ispisuje broj sekundi od 1.1.1970. godine , finalno unosi string sa ulaza za koji proverava (korišćenjem regularnih izraza) da li predstavlja zapis celog broja.

```

2     """
3     Python moduli.
4     """
5
6     # U <<math>> modulu se nalaze brojne naprednije matematicke funkcije kao sto su:
7     # math.sqrt(broj)
8     # math.log(broj, osnova)
9     # math.sin(ugao_u_radijanima), math.cos(), ...
10    # math.exp(stepen)
11    # math.factorial(broj)
12    import math
13    print(math.factorial(23))
14    print(math.log(225, 10))
15
16    # Pseudo slucajni brojevi nalaze se u modulu <<random>>
17    import random
18    print(random.random()) # [0.0, 1.0)
19    print(random.randint(0,1)) # {0,1,...,9,10}
20
21    # Preko <<sys>> modula se moze pristupiti objektima koje interpreter koristi,
22    # kao sto su npr argumenti komandne linije
23    import sys
24    print(sys.argv)
25
26    # U <<os>> modulu se nalaze, izmedju ostalog, sistemski pozivi
27    import os
28    os.system('ls -lah')
29
30    # Alias za modul
31    import time as t
32    print(t.time())
33
34    # Selektovani import
35    from re import search as re_src
36    print(re_src(r'[0-9]+', input('Unesite string za match: ')))

```

Zadatak 1.8 Napisati program koji na standardni izlaz ispisuje vrednost $6!$, $\log_5 125$ i pseudo

slučajan broj iz opsega $[0, 1)$

```

1  # Matematicke funkcije
3  # Uključujemo modul <<math>>
   import math
5
   # U ovom moduli se nalaze brojne funkcije kao sto su:
7  #
   # math.sqrt(broj)
9  # math.log(broj, osnova)
   # math.sin(ugao_u_radijanima), math.cos(), ...
11 # math.exp(stepen)
   # math.factorial(broj)
13 # i druge...
   print("\n-----Matematicke funkcije-----\n")
15 print(math.factorial(6))
   print(math.log(125, 5))
17
   # Pseudo slucajni brojevi
19
   # Uključujemo modul <<random>>
21 import random
23
   # Funkcija random() vraca pseudo slucajan broj tipa float iz opsega [0.0, 1.0)
   print("\n-----Pseudo slucajni brojevi-----\n")
25 print("Pseudo slucajan broj iz opsega [0.0,1.0)\n")
   print(random.random())
27
   # Korisne funkcije:
29 #
   # randint(a,b) - vraca pseudo slucajan ceo broj n iz opsega [a,b]
31 # choice(lista) - vraca pseudo slucajan element iz liste

```

Zadatak 1.9 Napisati program koji definiše praznu (bez tela) funkciju tri argumenta, kvadratnu funkciju, kao i funkciju translacije, koja prosleđeni argument umanjuje za jedan. Ilustrovati kompoziciju kvadratne funkcije i translacije. Funkciju kvadriranja definisati kao anonimnu funkciju.

```

1  """
   Python funkcije.
3  """
5
   # Sintaksa:
   # def name(args):
7   #   body
9
   # Definicija funkcije (prazna funkcija)
   def empty(x, y, z):
11     pass
   print(empty(1,2,3))
13
   # Definicija anonimne funkcije (funkcija sabiranja)
15 sum = lambda x, y: x + y
   print(sum(1,2))
17 print(sum)
19
   def square(x):
       """ Vraca kvadrat svog argumenta """
21     return x**2
23
   print(square.__repr__())
   print(help(square))
25 x = 9
   print(square(x))
27 x = 9.3
   print(square(x))
29
   square = lambda x : x**2
31 print(square)
   print(square(7))

```

Zadatak 1.10 Ilustrovati prenos argumenata funkciji: definisati dve funkcije koje rade sa listom, jednu koja menja, a drugu koja pravi kopiju prosleđenog joj argumenta.

```
# Prenos argumenata funkciji
2 # Postoje dva tipa objekata u Python-u: mutabilni i imutabilni
# - Za imutabilne objekte, menjanje unutar funkcije kreira novu instancu
4 # i originali se ne menjaju. Imutabilni objekti su niske, brojevi i torke.
# - Za mutabilne objekte, svaka izmena unutar funkcije menja original.
6 # Medjutim, ponovna dodela instance nece promeniti instancu izvan funkcije.
# Mutabilni objekti su (izmedju ostalih): liste, mape i klasne instance.

8
def translate(x):
10     x = x-1
    return x
12
a = 5
14 print('a:', a)
    translate(a)
16 print('a:', a)

18 l = [1, 2, 3]
def translate(l):
20     l = l[:-1]
    print('l: ', l)
22     translate(l)
    print(l)
24
l = [1, 2, 3]
26 def translate(l):
    l[0] = "M"
28 print('l: ', l)
    translate(l)
30 print(l)

32 x = 17
print("x=",x, " id=",id(x))
34 def ref_demo(x):
    print("x=",x, " id=",id(x))
36     x=42
    print("x=",x, " id=",id(x))
38 ref_demo(x)

40 x = 17
print("x=",x, " id=",id(x))
42 def ref_demo1(x):
    print("x=",x, " id=",id(x))
44     x -= 1
    print("x=",x, " id=",id(x))
46 ref_demo1(x)
```

Zadatak 1.11 Definisati funkciju printf, koja na standardni izlaz stampa svoje argumente (ne praviti nikakve pretpostavke o broju argumenata). Modifikovati definisanu funkciju tako da kao argumente prima proizvoljan imenovanih argumenata.

Definisati funkciju koja racuna aritmeticku sredinu svojih argumenata (pretpostaviti da je nad argumentima definisane operacije deljenja i sabiranja, kao i da će broj samih argumenata biti veći od jedan.

```
# Funkcije sa nepoznatim brojem argumenata
2 def printf(*x):
    print(x)
4 printf()
    printf(14)
6 printf([1,2,3])
    printf(1,2,3,translate, printf)
8
def aritm_sredina(head, *tail):
10     sum = head
    for i in tail:
12         sum += i
    return sum / (1.0 + len(tail))
14 print(aritm_sredina(1,2,3,4,5))
```

```

16 # Proizvoljan broj imenovanih argumenata
def printf(**x):
18     print(x)
printf()
20 printf(de="German",en="English",fr="French")

22 # Lokalne funkcije
def average(a, b, c):
24     def total(a, b, c):
        return a+b+c
26     return total(a,b,c)/3
print(average(1, 2, 3))
28

# REFERENCE:
30 # https://www.python-course.eu/passing_arguments.php
# https://www.zentut.com/python-tutorial/advanced-python-function/

```

Zadatak 1.12 Napisati program koji sa standardnog ulaza učitava listu celih brojeva, a zatim na standardni izlaz ispisuje njenu dužinu, sumu njenih elemenata, njen maksimalni element kao i broj pojavljivanja broja jedan u unetoj listi.

```

# Lista (List)
2 l = [1, 2, 3, 4, 5]
print(1, type(1))
4 l += [-3, -3, -3]
print(l)
6 print('len:', len(l))
print('sum:', sum(l))
8 l.append(-6)
print(l)
10 l.pop()
print(l)
12 l = [1, 2, "Matf", 12.3, [1, 2, 3], [[1,2], [3,4]], 1]
print(l)
14 print(l[2])
print(l[-2])
16 print(l[2:])
print(l[:-2])
18 print(l.count(1))
l1 = [2, 3, 34, 341, 2, -23, 213]
20

l = list(input())
22 print('len:', len(l))
print('sum:', sum(l))
24 print('max: ', max(l1))
print(l.count(1))
26

# Poredjenje razlicitih tipova
28 lista = [[1,2,3], [1,2,5], ['abc','abc','abc'], ['abc', 'ab', 'abcd'], ['a','b','c']
    > ['a', 'b']]
try:
30     print(max(lista))
except TypeError as err:
32     print("Greska. Python3 ne poredi razlicite tipove: ", err)

```

Zadatak 1.13 Napisati program kojim se ilustruje upotreba struktura podataka: steka, skupa, mape i torke.

```

# Skup (Set)
2 imena = set()
for ime in ['Milan', 'Ivan', 'Milan', 'Ana', 'Ana', 'Milan']:
4     imena.add(ime)
print(imena)
6 print(set(['Milan', 'Ivan', 'Milan', 'Ana', 'Ana', 'Milan']))

8

# Stek (LIFO struktura)
10 # Funkcije:
# * empty() - da li je stek prazan? - Time Complexity : O(1)
12 # * size() - čveliina steka? - Time Complexity : O(1)

```

```

# * top() - ĆVraa referencu na element sa vrha steka - Time Complexity : O(1)
# * push(g) - Dodaje element g na vrh steka - Time Complexity : O(1)
# * pop() - Skida element sa vrha steka - Time Complexity : O(1)
#
# Implementacije: collections.deque, queue.LifoQueue etc. ili jednostavno koristiti
# listu!
stack = []
stack.append('a')
stack.append('b')
stack.append('c')
print('Inicijalni stek:', end=':')
print(stack)
print('pop():')
print(stack.pop())
print(stack.pop())
print(stack.pop())
print('Stanje steka:', end=':')
print(stack)

# Mapa (Dictionary)
# Notacija: {kljuc:vrednost, kljuc:vrednost, ...} - JSON format
prosek = {'Milan':9.45, 'Ana':9.87, 'Nikola':6.76}
print(prosek)
print(type(prosek))
print(prosek.keys())
print(prosek.values())
print('Milan' in prosek)

# Torke (Tuple)
# Torka predstavlja strukturu podataka koja je uredjena i imutabilna
torka = ("jabuka", "banana", "visnja", "pomorandza", "kivi", "lubenica", "mango")
print(torka[2:5])
try:
    torka[0] = 11
except TypeError:
    print("Torke su imutabilne.")

```

Zadatak 1.14 Korišćenjem hipa implementirati algoritam sortiranja hip-sort.

```

# Hip (red sa prioriteto)
# https://docs.python.org/3.7/library/heapq.html
# Funkcije:
# * heapify - Konvertuje listu u hip u mestu.
# * heappush - Dodaje element u hip.
# * heappop - Vraca koren hipa.
# * heapreplace - Zamenjuje koren hipa novom vrednoscu prosledjenoj funkciji.
import heapq

H = [21,1,45,78,3,5]
heapq.heapify(H)
print(H)
heapq.heappush(H,8)
print(H)

# Heapsort
def heapsort(iterable):
    h = []
    for value in iterable:
        heapq.heappush(h, value)
    return [heapq.heappop(h) for i in range(len(h))]
print(heapsort([1, 3, 5, 7, 9, 2, 4, 6, 8, 0]))

```

Zadatak 1.15 Napisati program koji imitira rad bafera. Maksimalni broj elemenata u baferu je 5. Korisnik sa standardnog ulaza unosi podatke do unosa reči *quit*. Program ih smešta u bafer, posto se bafer napuni unosi se ispisuju na standarni izlaz i bafer se prazni.

```

# LISTA
#
# Notacija: [element1, element2, ...]
#
# Liste mogu sadrzati razlicite tipove podataka

```

```

6 # lista = [1,2,3.4,"Another brick in the wall",True,[5,False,4.4,'Layla']]

8 # Prazna lista
buffer = []
10 i = 0
while True:
12     unos = input()
    if unos == 'quit':
14         break
    # Ubacivanje elementa na kraj
16     buffer.append(unos)
    i += 1
18     if i == 5:
        # Prolazak kroz listu
20         for unos in buffer:
            print(unos)
22         # Praznimo bafer
        buffer = []
24         i = 0

```

Zadatak 1.16 Korisnik sa standardnog ulaza unosi ceo broj n , a potom ciklično pomeren rastuće sortiran niz (pr. 56781234) koji ima n elemenata. Napisati program koji na standardni izlaz ispisuje sortiran niz bez ponavljanja elementa.

```

# Korisne funkcije za liste:
2 #
# list.remove(x) - izbacuje prvo pojavljivanje elementa x iz liste
4 # list.count(x) - vraca broj koliko puta se element x nalazi u listi
# list.index(x) - vraca indeks prvog pojavljivanja elementa x u listi
6 # len(lista) - vraca broj elemenata liste
# del lista[a:b] - brise elemente liste od pozicije a do b

8
try:
10     n = int(input("Unesite broj elemenata: "))
except ValueError:
12     print("Greska. Parsiranje broja elemenata.")
    exit(1)

14
elementi = []
16 # Funkcija <<range>>
#
18 # range(kraj)
# range(pocetak, kraj[, korak])

20
for i in range(n):
22     element = int(input())
    # Provera da li se element nalazi u listi
24     if not element in elementi:
        # Ubacivanje elementa u listi
26         elementi.append(element)

28 k = 0
for i in range(n-1):
30     # Pristupanje elementima liste
    if elementi[i] > elementi[i+1]:
32         k = i + 1
        break
34
prvi_deo = elementi[0:k]
36 drugi_deo = elementi[k:]

38 # Nadovezivanje dve liste
sortirani = drugi_deo + prvi_deo
40 print("Sortirani elementi: ")
for element in sortirani:
42     print(element)

```

Zadatak 1.17 Napisati funkciju `max_list(lista)` koja vraća najveći element u listi `lista`. Napisati program koji testira ovu funkciju.

```
def max_list(lista):
```



```

2      # Mozemo indeksirati liste unazad, pozicija -1 odgovara poslednjem elementu
      maximum = lista[-1]
4      for element in lista:
          if maximum < element:
              maximum = element
6      return maximum
8
lista = [1, 4, -6, 7, 9, 0, 1]
10 print(max_list(lista))

```

Zadatak 1.18 Napisati program za rad sa stekom.

- Definirati stek koji sadrži elemente 9, 8, 7
- Dodati na stek elemente 6 i 5
- Skinuti sa steka element i ispisati ga na standardni izlaz

```

# Koriscenje liste kao stek strukture podataka
2 stek = [9,8,7]
# Operacija push je implementirana funkcijom append
4 stek.append(6)
stek.append(5)
6 print("\n-----Ispisujemo stek-----\n")
print(stek)
8 # Operacija pop je implementirana funkcijom pop
print("\n-----Ispisujemo element dobijem funkcijom pop-----\n")
10 print(stek.pop())
print("\n-----Ispisujemo znanje nakon pozivanja funkcije pop-----\n")
12 print(stek)

```

Zadatak 1.19 Napisati program koji za uneti prirodan broj n ispisuje vrednosti funkcije x^2 u celobrojnim tačkama u intervalu $[0, n]$. Zadatak rešiti korišćenjem mape.

```

# KATALOG
#
2 # Katalog je kolekcija uredjenih parova oblika (kljuc, vrednost)
#
4 # Notacija: {kljuc:vrednost, kljuc:vrednost, ...}
#
6 mapa = {} # prazna mapa
#
8 try:
    n = int(input("Unesite n: "))
10 except ValueError as err:
    print("Greska. Parsiranje broja elemenata: ", err)
    exit(1)
12 for x in range(n):
    mapa[x] = x**2
14
16 # Prolazak kroz mapu
18 print("\n-----Prolazak kroz katalog-----\n")
for kljuc in mapa:
20     print("f({0:s}) = {1:s}\n".format(str(kljuc), str(mapa[kljuc])))
22
# Korisne funkcije
#
24 # map.keys() - vraca listu kljuceva iz kataloga
# map.values() - vraca listu vrednosti iz kataloga
# map.has_key(kljuc) - vraca True/False u zavisnosti od toga da li se element
26 # sa kljucem kljuc nalazi u katalogu

```

Zadatak 1.20 Sa standardnog ulaza se unose reči do reči *quit*. Napisati program koji ispisuje unete reči eliminišući duplikate.

```

1 lista = []
#
3 while True:
    unos = input()

```

```

5     if unos == 'quit':
6         break
7     lista.append(unos)

9 # Pravljenje skupa od liste
10 skup = set(lista)
11
12 for i in skup:
13     print(i)

```

Zadatak 1.21 Napisati funkciju `min_torka(lista)` koja vraća najmanji element u torci torki. Napisati program koji ovu funkciju testira.

```

1 # Uredjene N-TORKE
2 print("\n-----Torke-----\n")
3 # torka = ("Daffy", "Duck", 11)

5 def min_torka(torke):
6     # Pristupanje elementima u torki
7     minimum = torke[0]
8     for torka in torke:
9         # Ukoliko torke ne sadrže elemente istog tipa na istim pozicijama, i dalje ih
10         # možemo porediti,
11         # ali poredjenje se vrši na osnovu imena tipa elementa leksikografski
12         # npr. element tipa List < element tipa String < element tipa Tuple i slično
13         if minimum > torka:
14             minimum = torka
15     return minimum

16 torke1 = ((1, 2, 'a'), (1, 2, 'b'), (1, 3, 'z'), (2, 2, '5'))
17 print(min_torka(torke1))

19 torke2 = ((1,2,'a'),(1,2,'b'),([1,2,3], 'Bugs', 4), ([1,1,1], 'Bunny', 6),(1,2,['a','b']),
20          (1,2,'ab'))
21 try:
22     print(min_torka(torke2))
23 except TypeError as err:
24     print("Greska. Python3 ne poredi razlicite tipove: ", err)
25     exit(1)

```

1.2.2 Zadaci za samostalni rad sa rešenjima

Zadatak 1.22 Pogodi broj Napisati program koji implementira igricu "Pogodi broj". Na početku igre računar zamišlja jedan slučajan broj u intervalu [0,100]. Nakon toga igrač unosi svoje ime i započinje igru. Igrač unosi jedan po jedan broj sve dok ne pogodi koji broj je računar zamislio. Svaki put kada igrač unese broj, u zavisnosti od toga da li je broj koji je unet veći ili manji od zamišljenog broja ispisuje se odgovarajuća poruka. Igra se završava u trenutku kada igrač pogodio zamišljen broj.

Primer 1

```
INTERAKCIJA SA PROGRAMOM:
POKRETANJE: pogodi_broj
----- IGRA: Pogodi broj -----
Unesite Vase ime:
Milica
Zdravo Milica. :)
Zamislilo sam neki broj od 1 do 100. Da li mozes da pogodis koji je to broj?
Unesi broj
50
Broj koji sam zamislilo je MANJI od 50
Unesi broj
25
Broj koji sam zamislilo je VECI od 25
Unesi broj
30
Broj koji sam zamislilo je MANJI od 30
Unesi broj
27
"BRAVO!!! Pogodio si! Zamislilo sam 27. Bilo je lepo igrati se sa tobom. :)
```

[Rešenje 1.22]

Zadatak 1.23 Aproksimacija broja PI metodom Monte Karlo Napisati program koji aproksimira broj PI koriscenjem metode Monte Karlo. Sa standardnog ulaza unosi se broj N. Nakon toga N puta se bira tačka na slučajan način tako da su obe koordinate tačke iz intervala [0,1]. Broj PI se računa po sledecoj formuli:

$$PI = 4 * A/B$$

- A je broj slučajno izabranih tačaka koje pripadaju krugu poluprečnika 0.5, sa centrom u tački (0.5, 0.5)
- B je broj slučajno izabranih tačaka koje pripadaju kvadratu čija temena su tačke (0, 0), (0, 1), (1, 1), (1, 0).

Primer 1

```
INTERAKCIJA SA PROGRAMOM:
POKRETANJE: aproksimacija_PI
Izracunavanje broja PI metodom Monte Karlo
Unesite broj iteracija:
5
Tacka:
(0.14186247318019474, 0.15644650897353152)
Tacka:
(0.8910898038304426, 0.2200563958299553)
Tacka:
(0.641604107090444, 0.03712366524007682)
Tacka:
(0.4893727376942526, 0.17230005349431587)
Tacka:
(0.6199558112390107, 0.32122922953511124)
Tacka:
(0.5821041171248978, 0.025052299437462566)
Broj PI aproksimiran metodom Monte Karlo: 4.0
```

[Rešenje 1.23]

Zadatak 1.24 X-O Napisati program koji implementira igricu X-O sa dva igrača.

Primer 1

```

POKRETANJE: XO
INTERAKCIJA SA PROGRAMOM:
  IGRA: X-O pocinje
  Unesite ime prvog igraca:
    Ana
  Zdravo Ana
  Unesite ime drugog igraca:
    Petar
  Zdravo Petar!
  Igrac ('Ana', 'X') igra prvi.
  X : ('Ana', 'X')
  O : ('Petar', 'O')
  Zapocnimo igru
  TABLA
  1 2 3
  ---
  1 | - | - | - |
  ---
  2 | - | - | - |
  ---
  3 | - | - | - |
  ---
  Ana unesite koordinate polja koje
  zelite da popunite u posebnim linijama:
  Unesite vrstu:
    1
  Unesite kolonu:
    1
  TABLA
  1 2 3
  ---
  1 | X | - | - |
  ---
  2 | - | - | - |
  ---
  3 | - | - | - |
  ---
  Petar unesite koordinate polja koje
  zelite da popunite u posebnim linijama:
  Unesite vrstu:
    1
  Unesite kolonu:
    2
  TABLA
  1 2 3
  ---
  1 | X | O | - |
  ---
  2 | - | - | - |
  ---
  3 | - | - | - |
  ---

```

```

Ana unesite koordinate polja koje
zelite da popunite u posebnim linijama:
Unesite vrstu:
2
Unesite kolonu:
2
TABLA
1 2 3
---
1 | X | O | - |
---
2 | - | X | - |
---
3 | - | - | - |
---
Petar unesite koordinate polja koje
zelite da popunite u posebnim linijama:
Unesite vrstu:
2
Unesite kolonu:
3
TABLA
1 2 3
---
1 | X | O | - |
---
2 | - | X | O |
---
3 | - | - | - |
---
Ana unesite koordinate polja koje
zelite da popunite u posebnim linijama:
Unesite vrstu:
3
Unesite kolonu:
3
TABLA
1 2 3
---
1 | X | O | - |
---
2 | - | X | O |
---
3 | - | - | X |
---
BRAVO!!!!!! Igrac Ana je pobedio!

```

[Rešenje 1.24]

1.2.3 Zadaci za vežbu

Zadatak 1.25 Ajnc Napisati program koji implementira igricu Ajnc sa jednim igračem. Igra se sa špilom od 52 karte. Na početku igrač unosi svoje ime nakon čega računar deli dve karte igraču i dve karte sebi. U svakoj sledećoj iteraciji računar deli po jednu kartu igraču i sebi. Cilj igre je sakupiti karte koje u zbiru imaju 21 poen. Karte sa brojevima nose onoliko bodova koliki je broj, dok žandar, dama, kralj nose 10 bodova. Karta As može da nosi 1 ili 10 bodova, u zavisnosti od toga kako igraču odgovara. Igrać koji sakupi 21 je pobedio. Ukoliko igrač premaši 21 bod, pobednik je njegov protivnik. Detaljan opis igre može se naći na narednoj adresi: <https://en.wikipedia.org/wiki/Blackjack>

Primer 1

```

POKRETANJE: ajnc
INTERAKCIJA SA PROGRAMOM:
----- IGRA: Ajnc -----
Unesite Vase ime:
Pavle
Zdravo Pavle. :)
Igra pocinje
Vase karte su:
1 Herc 5 karo
Hit ili stand?[H/S]
H
Vase karte su:
1 Herc 5 karo 5 tref
Cestitam!!! Pobedio si!
Bilo je lepo igrati se sa tobom. :)

```

Primer 2

```

POKRETANJE: ajnc
INTERAKCIJA SA PROGRAMOM:
----- IGRA: Ajnc -----
Unesite Vase ime:
Pavle
Zdravo Pavle. :)
Igra pocinje
Vase karte su:
Q Tref 7 karo
Hit ili stand?[H/S]
H
Vase karte su:
Q Tref 7 karo K Herc
Zao mi je, izgubio si!:(
Bilo je lepo igrati se sa tobom. :)

```

Zadatak 1.26 4 u liniji Napisati program koji implementira igricu 4 u nizu sa dva igrača. Tabla za igru je dimenzije 8x8. Igrači na početku unose svoja imena, nakon čega računar nasumično dodeljuje crvenu i žutu boju igračima. Igrač sa crvenom bojom igra prvi i bira kolonu u koju ce da spusti svoju lopticu. Cilj igre je da se sakupe 4 loptice iste boje u liniji. Prvi igrač koji sakupi 4 loptice u liniji je pobedio. Detaljan opis igre može se naći na narednoj adresi: https://en.wikipedia.org/wiki/Connect_Four

Primer 1

```

POKRETANJE: cetri_u_nizu
INTERAKCIJA SA PROGRAMOM:
IGRA: Cetiri u nizu pocinje
Unesite ime prvog igraca:
Ana
Zdravo Ana
Unesite ime drugog igraca:
Petar
Zdravo Petar!
Igrac ('Ana', 'C') igra prvi.
C : ('Ana', 'C')
Z : ('Petar', 'Z')
Zapocnimo igru
TABLA
 1 2 3 4 5 6 7 8
-----
1 | - | - | - | - | - | - | - |
-----
2 | - | - | - | - | - | - | - |
-----
3 | - | - | - | - | - | - | - |
-----
4 | - | - | - | - | - | - | - |
-----
5 | - | - | - | - | - | - | - |
-----
6 | - | - | - | - | - | - | - |
-----
7 | - | - | - | - | - | - | - |
-----
8 | - | - | - | - | - | - | - |
-----
Ana unesite koordinate polja
koje zelite da popunite
u posebnim linijama:
Unesite vrstu:
1
Unesite kolonu:
1

```

```

TABLA
 1 2 3 4 5 6 7 8
-----
1 | c | - | - | - | - | - | - |
-----
2 | - | - | - | - | - | - | - |
-----
3 | - | - | - | - | - | - | - |
-----
4 | - | - | - | - | - | - | - |
-----
5 | - | - | - | - | - | - | - |
-----
6 | - | - | - | - | - | - | - |
-----
7 | - | - | - | - | - | - | - |
-----
8 | - | - | - | - | - | - | - |
-----
Petar unesite koordinate polja
koje zelite da popunite
u posebnim linijama:
Unesite vrstu:
2
Unesite kolonu:
1
TABLA
 1 2 3 4 5 6 7 8
-----
1 | c | - | - | - | - | - | - |
-----
2 | z | - | - | - | - | - | - |
-----
3 | - | - | - | - | - | - | - |
-----
4 | - | - | - | - | - | - | - |
-----
5 | - | - | - | - | - | - | - |
-----
6 | - | - | - | - | - | - | - |
-----
7 | - | - | - | - | - | - | - |
-----
8 | - | - | - | - | - | - | - |
-----

```

Ana unesite koordinate polja
koje zelite da popunite
u posebnim linijama:
Unesite vrstu:

1

Unesite kolonu:

2

TABLA

1 2 3 4 5 6 7 8

```

-----
1 | c | c | - | - | - | - | - |
-----
2 | z | - | - | - | - | - | - |
-----
3 | - | - | - | - | - | - | - |
-----
4 | - | - | - | - | - | - | - |
-----
5 | - | - | - | - | - | - | - |
-----
6 | - | - | - | - | - | - | - |
-----
7 | - | - | - | - | - | - | - |
-----
8 | - | - | - | - | - | - | - |

```

Petar unesite koordinate polja
koje zelite da popunite
u posebnim linijama:

Unesite vrstu:

2

Unesite kolonu:

2

TABLA

1 2 3 4 5 6 7 8

```

-----
1 | c | c | - | - | - | - | - |
-----
2 | z | z | - | - | - | - | - |
-----
3 | - | - | - | - | - | - | - |
-----
4 | - | - | - | - | - | - | - |
-----
5 | - | - | - | - | - | - | - |
-----
6 | - | - | - | - | - | - | - |
-----
7 | - | - | - | - | - | - | - |
-----
8 | - | - | - | - | - | - | - |

```

Ana unesite koordinate polja
koje zelite da popunite
u posebnim linijama:

Unesite vrstu:

1

Unesite kolonu:

3

TABLA

1 2 3 4 5 6 7 8

```

-----
1 | c | c | c | - | - | - | - |
-----
2 | z | z | - | - | - | - | - |
-----
3 | - | - | - | - | - | - | - |
-----
4 | - | - | - | - | - | - | - |
-----
5 | - | - | - | - | - | - | - |
-----
6 | - | - | - | - | - | - | - |
-----
7 | - | - | - | - | - | - | - |
-----
8 | - | - | - | - | - | - | - |

```

Petar unesite koordinate polja
koje zelite da popunite
u posebnim linijama:

Unesite vrstu:

2

Unesite kolonu:

3

TABLA

1 2 3 4 5 6 7 8

```

-----
1 | c | c | c | - | - | - | - |
-----
2 | z | z | z | - | - | - | - |
-----
3 | - | - | - | - | - | - | - |
-----
4 | - | - | - | - | - | - | - |
-----
5 | - | - | - | - | - | - | - |
-----
6 | - | - | - | - | - | - | - |
-----
7 | - | - | - | - | - | - | - |
-----
8 | - | - | - | - | - | - | - |

```

Ana unesite koordinate polja
koje zelite da popunite
u posebnim linijama:

Unesite vrstu:

1

Unesite kolonu:

4

TABLA

1 2 3 4 5 6 7 8

```

-----
1 | c | c | c | c | - | - | - |
-----
2 | z | z | z | - | - | - | - |
-----
3 | - | - | - | - | - | - | - |
-----
4 | - | - | - | - | - | - | - |
-----
5 | - | - | - | - | - | - | - |
-----
6 | - | - | - | - | - | - | - |
-----
7 | - | - | - | - | - | - | - |
-----
8 | - | - | - | - | - | - | - |

```

BRAVO!!!!!! Igrac Ana je pobedio!

1.3 OOP koncepti

Zadatak 1.27 Korišćenjem liste implementirati klasu stek.

```
"""
2   Python klase. Uvod. Konstruktor. Instanciranje. self promenljiva.
"""
4   class Stek:
6       # konstruktor
7       def __init__(self):
8           self.stek = []
10      def push(self, elem):
11          self.stek.append(elem)
12
13      def pop(self):
14          try:
15              return self.stek.pop()
16          except IndexError:
17              print("Stek prazan, nemoguće izvesti pop.")
18              return None
19
20      def peek(self):
21          try:
22              return self.stek[-1]
23          except IndexError:
24              print("Stek prazan, nemoguće izvesti peek.")
25              return None
26
27  s = Stek()
28  s.push(1)
29  s.push(2)
30  s.push(3)
31  print(s.peek())
32  print(s.pop())
33  print(s.pop())
34  print(s.pop())
35  print(s.pop())
36  print(s.peek())
```

Zadatak 1.28 Implementirati klasu Robot, koja će biti svesna broja trenutno živih instanci ove klase. Klasa kao atribut ima naziv robota. Od metoda definisati metod oglašavanja robota, kao i statički metod koji na standardni izlaz ispisuje broj trenutno aktivnih instanci ove klase. Ilustrovati upotrebu klase.

```
"""
2   Python klase. Destruktor, promenljive.
"""
4   class Robot:
5       """Predstavlja robota sa imenom."""
6
7       # Mozemo pristupiti dokumentaciji preko __doc__() metoda
8
9       populacija = 0 # klasna promenljiva
10
11      def __init__(self, name):
12          """Konstruktor."""
13          self.name = name
14          Robot.populacija += 1
15          print(f"{self.name} je kreiran!")
16
17      def __del__(self):
18          """Destruktor."""
19          print(f"{self.name} se unistava!")
20          Robot.populacija -= 1
21
22      def oglasi_se(self):
23          """Oglasavanje za trenutnog robota."""
24          print(f"Zdravo, moje ime je {self.name}.")
```

```

26     @classmethod
27     def koliko(cls):
28         """Ispisuje broj trenutno aktivnih robota."""
29         print(f"Imamo {cls.populacija} aktivnih robota.")
30
31         # postoji i @staticmethod, takve funkcije nemaju parametre
32         # koriste se kada nije potrebno da metod pristupa atributima klase
33
34
35 droid1 = Robot("R2-D2")
36 droid1.oglasise()
37 Robot.koliko()
38
39 droid2 = Robot("C-3PO")
40 droid2.oglasise()
41 Robot.koliko()
42
43 print("\nRoboti rade posao ovde...\n")
44
45 print("Roboti su završili posao. Obrisimo ih.")
46 del droid1
47 del droid2
48
49 Robot.koliko()
50
51 # REFERENCE:
52 # https://python.swaroopch.com/oop.html

```

Zadatak 1.29 Implementirati klasu Oblik, koja kao atribut ima boju kao i informaciju o popunjenosti unutrašnjosti oblika bojom. Definirati metode za dohvaćanje i postavljanje ovih atributa. Definirati klase Krug i Pravougaonik koje nasleđuju klasu oblik, od dodatnih atributa imaju informaciju o poluprečniku odnosno stranicama redom, i implementiraju funkcije računanja obima i površine. Ilustrovati upotrebu ovih klasa.

```

"""
2     Python klase. Nasledjivanje.
3     """
4
5     import math
6
7     class Oblik:
8         def __init__(self, boja='crna', popuna=False):
9             # protected promenljive pocinju sa _
10            # private promenljive pocinju sa __
11            # Ovo je samo konvencija o imenovanju, ne postoje zaista privatne promenljive
12            # u Pythonu.
13            # Radi se o kvazi privatnim, Python promenljivim koje čpocinju sa __ menja
14            # naziv interno,
15            # i na taj čnain švri skrivanje imena.
16            self.__boja = boja
17            self.__popuna = popuna
18
19        def get_boja(self):
20            return self.__boja
21
22        def get_popuna(self):
23            return self.__popuna
24
25        def set_popuna(self, popuna):
26            self.__popuna = popuna
27
28
29        class Pravougaonik(Oblik):
30            def __init__(self, a, b):
31                super().__init__(boja='crvena')
32                self.__a = a
33                self.__b = b
34
35            def površina(self):
36                return self.__a * self.__b
37
38            def obim(self):
39                return 2 * (self.__a + self.__b)
40
41
42        class Circle(Oblik):
43            def __init__(self, r):

```



```

38         super().__init__(boja='plava')
39         self.__r = r
40     def površina(self):
41         return math.pi * self.__r ** 2
42     def obim(self):
43         return 2 * math.pi * self.__r
44
45 r1 = Pravougaonik(10.5, 2.5)
46 print("Površina pravougaonika r1:", r1.površina())
47 print("Obim pravougaonika r1:", r1.obim())
48 print("Boja pravougaonika r1:", r1.get_boja())
49 print("Da li je r1 popunjen? ", r1.get_popuna())
50 r1.set_popuna(True)
51 print("Da li je r1 popunjen? ", r1.get_popuna())
52
53 c1 = Circle(12)
54 print("\nPovršina kruga c1:", format(c1.površina(), "0.2f"))
55 print("Obim kruga c1:", format(c1.obim(), "0.2f"))
56 print("Boja kruga c1:", c1.get_boja())
57 print("Da li je c1 popunjen? ", c1.get_popuna())
58 c1.set_popuna(True)
59 print("Da li je c1 popunjen? ", c1.get_popuna())
60
61 """
62 Višestruko nasleđjivanje:
63 class A:
64     def __init__(self):
65         super(A, self).__init__()
66         print('A')
67 class B:
68     def __init__(self):
69         super(B, self).__init__()
70         print('B')
71 class C:
72     def __init__(self):
73         super(C, self).__init__()
74         print('C')
75 class X(A, B, C):
76     def __init__(self):
77         super(X, self).__init__()
78 x = X()
79 """
80
81 """
82 Virtualne funkcije:
83 class A:
84     def metod(self):
85         print("metod() iz klase A")
86 class B(A):
87     def metod(self):
88         print("metod() iz klase B")
89 b_obj = B()
90 a_obj = A()
91 b_obj.metod()
92 a_obj.metod()
93 """
94
95 """
96 Predefinisanje operatora:
97 class Tacka:
98     def __init__(self, x = 0, y = 0):
99         self.x = x
100         self.y = y
101     def __str__(self):
102         return "({0},{1})".format(self.x,self.y)
103     def __add__(self,other):
104         x = self.x + other.x
105         y = self.y + other.y
106         return Tacka(x,y)
107 p1 = Tacka(2,3)
108 p2 = Tacka(-1,2)
109 print(p1 + p2)

```

```

110 """
112 # REFERENCE:
    # https://overiq.com/python-101/inheritance-and-polymorphism-in-python/

```

Zadatak 1.30 Implementirati klasu MyIterator koja predstavlja iterator od 0 do n, pri čemu je n atribut klase.

```

1  """
2      Iteratori.
3  """
4  for i in [1,2,3,4]:
5      print(i)
6  for i in ('a', 'b', 'c', 'd'):
7      print(i)
8
9  # Da bi Python objekat bio iterabilan, potrebno je da implementira čsledee metode:
10 # * __iter__ metod koji se poziva prilikom inicijalizacije iteratora
11 #   čvraa objekat koji implementira __next__ metod.
12 # * __next__ metoda čvraa čsledeu vrednost iteratora.
13 #   Kada se iterator objekat nalazi u okviru 'for' in petlje, petlja
    implicitno
14 #       poziva __next__() metod na iterator objektu.
15 #       Metod žpodie izuzetak StopIteration kao signal kraja iteriranja.
16
17 class MojIterator:
18     def __init__(self, limit):
19         self.limit = limit
20
21     def __iter__(self):
22         self.x = 0
23         return self
24
25     def __next__(self):
26         # Treba nam trenutna vrednost x
27         x = self.x
28         if x > self.limit:
29             raise StopIteration
30         else:
31             self.x += 1
32             return x
33
34 for i in MojIterator(10):
35     print(i)
36
37 # Ovo se prevodi u (približno, naravno):
38 # foo = MojIterator(10)
39 # foo = foo.__iter__()
40 # try:
41 #     while (True):
42 #         i = foo.__next__()
43 #         print(i)
44 # except StopIteration:
45 #     pass

```

Zadatak 1.31 Implementirati generator Fibonačijevih brojeva.

```

1 def Fibonacci():
2     a, b = 0, 1
3     while True:
4         yield a
5         a, b = b, a + b
6
7 f = Fibonacci()
8 for i in range(10):
9     print(f, f.__next__())

```

1.4 Datoteke, serijalizacija, sortiranje

Serijalizacija predstavlja proces prevođenja struktura podataka ili objekata u format koji može da se čuva ili prenosi, takav da je moguće rekonstruisati polaznu strukturu/objekat. Da bismo demonstrirali serijalizaciju, prvo ćemo pokazati kako se radi sa datotekama u Python-u, a zatim ćemo serijalizovati objekte u datoteke u JSON formatu.

1.4.1 Datoteke

Zadatak 1.32 Korisnik na standardni ulaz unosi dve niske. Napisati program koji prvo pojavljivanje druge niske u prvoj zamenjuje karakterom \$. U slučaju da nema pojavljivanja druge niske u prvoj i da je druga niska kraća ispisuje nadovezane niske sa separatorom -. Ako je druga niska duža od prve program treba da ispiše drugu nisku i njenu dužinu.

```
1 # Niske
2 #
3 # Mozemo ih pisati izmedju jednostrukih i dvostrukih navodnika
4
5 niska1 = input("Unesite nisku: ")
6 niska2 = input("Unesite nisku: ")
7
8 # Duzinu niske racunamo koristeci funkciju len(niska)
9 if len(niska1) >= len(niska2):
10     # Funkcija count
11     # niska.count(podniska [, pocetak [, kraj]]) - vraca broj koliko se puta
12     # podniska nalazi u niski (u intervalu od pocetak do kraj)
13     n = niska1.count(niska2)
14     if n > 0:
15         # Funkcija find
16         # niska.find(podniska [, pocetak [, kraj]]) - vraca poziciju prvog
17         # pojavljivanja
18         # podniska u niski (u intervalu od pocetak do kraj), -1 ukoliko se podniska
19         # ne nalazi u niski
20         i = niska1.find(niska2)
21         # Karakterima u niski mozemo pristupati koristeci notaciju [] kao kod listi
22         niska1 = niska1[0 : i] + '$' + niska1[i+len(niska2) : ]
23         print(niska1)
24     else:
25         # Funkcija join
26         # niska_separator.join([niska1,niska2,niska3,...]) - spaja listu niski
27         # separatorom
28         print('-'.join([niska1,niska2]))
29 else:
30     print("Duzina niske {0:s} je {1:d}".format(niska2, len(niska2)))
31
32 # Korisne funkcije za rad sa niskama:
33 #
34 # niska.isalnum()
35 #     isalpha()
36 #     isdigit()
37 #     islower()
38 #     isspace()
39 #     isupper()
40
41 # niska.split(separator) - razlaze nisku u listu koristeci separator
42 # niska.replace(stara, nova [, n]) - zamenjuje svako pojavljivanje niske stara
43 # niskom nova (ukoliko je zadat broj n, onda zamenjuje najvise n pojavljivanja)
```

Zadatak 1.33 Napisati program koji ispisuje sadržaj datoteke *datoteka.txt* na standardni izlaz karakter po karakter.

```
1 # Datoteku otvaramo koristeci funkciju open koja vraca
2 # referencu na otvoreni tok podataka.
3 #
4 # rezimi:
5 # - "r" -> read
6 # - "w" -> write
7 # - "a" -> append
8 # - "r+" -> read + append
9 #
```

```

# Datoteku smo dužni da zatvorimo sa 'datoteka.close()',
11 #
# datoteka = open("datoteka.txt","r")
13 # kod koji obradjuje datoteku
# ...
15 # datoteka.close()

17 # Python nudi 'with' koji omogućava da
# se datoteka automatski zatvori, čak i u situaciji
19 # kada se ispali izuzetak. Ovo je preporuceni nacin
# za citanje tokova podataka u Python-u.
21 with open("datoteka.txt", "r") as datoteka:
    # Citamo datoteku liniju po liniju, a potom
    # u liniji citamo karakter po karakter.
23     for linija in datoteka:
25         for karakter in linija:
                print(karakter)
27 # datoteka.close() nam nije neophodno,
# Python ce sam zatvoriti datoteku kada
29 # se završi 'with' blok

```

Zadatak 1.34 Napisati program koji ispisuje sadržaj datoteke *datoteka.txt* na standardni izlaz liniju po liniju.

```

# Ispitivanje da li je otvaranje datoteke uspelo:
2 try:
    with open("datoteka.txt","r") as g:
4         # Liniju po liniju mozemo učitavati koristeći petlju
        # tako što 'iteriramo' kroz Datoteku
6         print("-----Iteriranje kroz datoteku <<for>> petljom-----\n")
        # Metod f.readline() čita jednu liniju iz Datoteke
8         for linija in g:
                print(linija)
10 except IOError:
    # Ukoliko ne uspe otvaranje datoteke, Python ispaljuje
12    # izuzetak 'IOError'.
    print("Nije uspelo otvaranje datoteke.")

```

Zadatak 1.35 Napisati program koji dodaje u datoteku *datoteka.txt* nisku *water* a potom ispisuje njen sadržaj na standardni izlaz.

```

1 # f.readlines() i list(f)
# vraćaju listu linija datoteke
3 #
# f.write(niska) upisuje nisku u datoteku
5 print("-----Upisivanje u datoteku-----\n")
# razresiti konfuziju između a+ (upisuje na kraj) i r+ (upisuje na pocetak, tj.
# prepisuje postojeći sadržaj)
7 with open("datoteka.txt","a+") as h:
    h.write("water\n")
9    # h.flush() - da budemo sigurni da je prebaceno iz bafera
    # nakon write file pointer se pomerio pa da bismo procitali ceo sadržaj vraćamo
    se na pocetak datoteke
11    h.seek(0,0)
    print(h.readlines())

```

1.4.2 Serijalizacija, JSON

JSON (<https://www.json.org/>) reprezentacija objekata predstavlja jednostavni i pregledan način za serijalizaciju objekata. Koristi se svuda, pre svega u web programiranju kada je potrebno proslediti objekte preko mreže ali i lokalno kada je na primer potrebno sačuvati neki objekat u bazi podataka. JSON je nezamenjiv kod takozvanih *RESTful servisa* (servisa zasnovanih na REST-u, videti https://en.wikipedia.org/wiki/Representational_state_transfer). Korisnici obično pošalju zahtev preko HTTP protokola, a servis odgovara JSON reprezentacijom objekta koji ima informacije koje je korisnik tražio.

Zadatak 1.36 Korisnik na standardni ulaz unosi podatke o imenu, prezimenu i godinama. Program potom kreira JSON objekat *junak*, koji ima podatke *Ime*, *Prezime* i *Godine*, i ispisuje ga na standardni izlaz, a potom i u datoteku *datoteka.txt*.

```
1 # JSON format
2 #
3 # Funkcije za rad sa JSON formatom se nalaze u modulu json
4 import json
5
6 ime = input("Unesite ime: ")
7 prezime = input("Unesite prezime: ")
8 godine = int(input("Unesite godine: "))
9
10 # json.dumps(objekat) vraca string koji sadrzi JSON reprezentaciju objekta x
11
12 print("\n-----JSON reprezentacija objekta-----\n")
13 junak = {"Ime": ime, "Prezime":prezime, "Godine":godine}
14 print(json.dumps(junak))
15
16 # json.dump(x,f) upisuje string sa JSON reprezentacijom objekta x u datoteku f
17
18 f = open("datoteka.json","w")
19 json.dump(junak,f)
20 f.close()
```

Zadatak 1.37 Napisati program koji iz datoteke *datoteka.txt* učitava JSON objekat, a potom na standardni izlaz ispisuje podatke o *imenu*, *prezimenu* i *godinima*.

```
1 import json
2 # json.load(f) učitava iz datoteke string koji sadrži
3 # JSON format objekta i vraca referencu na mapu koja
4 # je konstruisana na osnovu .json datoteke.
5 print("\n-----Ucitavanje objekta iz datoteke-----\n")
6 f = open("dat4.json","r")
7 x = json.load(f)
8 print(x['Ime'])
9 print(x['Prezime'])
10 print(x['Godine'])
11 f.close()
```

Zadatak 1.38 Napisati program koji od RESTful servisa <https://quotes.rest> uzima citat dana pomoću GET zahteva na <https://quotes.rest/qod> i ispisuje citat na standardni izlaz. Napomena: Kako bi kod radio za Python2.7 potrebno je instalirati modul *requests* (`pip install requests`, *pip* je Python modul menadžer, ukoliko nije instaliran može se instalirati pomoću `apt-get install python-pip` ili bilo kod drugog package-managera).

```
1 import requests
2
3 url = 'https://quotes.rest/qod'
4 response = requests.get(url)
5
6 if response.ok:
7     json = response.json()
8     print(json)
9     citat = json['contents']['quotes'][0]['quote']
10    print("-----")
11    print(citat)
12 else:
13    print("Neuspesno dovlacenje citata.")
```

1.4.3 os modul

Zadatak 1.39 Napisati program koji na standardni izlaz ispisuje argumente komandne linije.

```
1 # modul sys ima definisan objekat argv koji predstavlja listu argumenata komandne
2   linije (svi argumenti se cuvaju kao niske karaktera)
```

```

3 import sys

5 if len(sys.argv) == 1:
    print("Niste naveli argumente komandne linije")
    # funkcija exit() iz modula sys prekida program
    # (ukoliko se ne prosledi argument, podrazumevano
    # se salje None objekat)
    exit()

11 # ispisujemo argumente komandne linije
13 # prvi argument, tj. sys.argv[0] je uvek ime skript fajla koji se pokrece
    for item in sys.argv:
15         print(item)

```

Zadatak 1.40 Napisati program koji na standardni izlaz ispisuje oznaku za tekući i roditeljski direktorijum, kao i separator koji se koristi za pravljenje putanje.

```

import os

2
print(os.getcwd())      # oznaka tekućeg direktorijuma
4 print(os.pardir)       # oznaka za roditeljski direktorijum tekućeg direktorijuma
print(os.sep)           # ispisuje separator koji koristi za pravljenje putanja

```

Zadatak 1.41 Napisati program koji imitira rad komande *ls*. Program na standardni izlaz ispisuje sadržaj tekućeg direktorijuma.

```

1 import os

3 # funkcija za prosledjenu putanju direktorijuma vraća listu imena
# svih fajlova u tom direktorijumu, . je zamena za putanju tekućeg direktorijuma
5 print(os.listdir(os.getcwd()))

7 # os.walk() - vraća listu torki (trenutni_direktorijum, poddirektorijumi, datoteke)
# os.path.join(putanja, ime) - pravi putanju tako sto nadovezuje na
9 #   prosledjenu putanju zadato ime odvojeno /

11 print("\n-----Prolazak kroz zadati direktorijum-----\n")
    for (trenutni_dir, poddirektorijumi, datoteke) in os.walk(os.getcwd()):
13         print(trenutni_dir)
            for datoteka in datoteke:
15                 print(os.path.join(trenutni_dir, datoteka))

17 # os.path.abspath(path) - vraća apsolutnu putanju za zadatu relativnu putanju nekog
#   fajla
# os.path.isdir(path) - vraća True ako je path putanja direktorijuma, inace vraća
#   False
19 # os.path.isfile(path) - vraća True ako je path putanja regularnog fajla, inace vraća
#   False

```

Zadatak 1.42 Napisati program koji na standardni izlaz ispisuje sve apsolutne putanje regularnih fajlova koji se nalaze u tekućem direktorijumu.

```

1 import os

3 print("\n-----Regularni fajlovi zadatog direktorijuma-----\n")
    for ime in os.listdir(os.getcwd()):
5         # Funkcija 'join' vrsi konkatenciju putanja koristeći sistemski separator
            if os.path.isfile(os.path.join(os.getcwd(), ime)):
7                 print(os.path.abspath(os.path.join(os.getcwd(), ime)))

```

1.4.4 Sortiranje

Zadatak 1.43 U datoteci *tacke.json* se nalaze podaci o tačkama u sledećem formatu.

Listing 1.1: *tacke.json*

```

1 [ {"teme": "A" , "koordinate": [10.0, 1.1]},

```

```
2 {"teme": "B" , "koordinate": [1.0, 15.0]},
3 {"teme": "C" , "koordinate": [-1.0, 5.0]} ]
```

Napisati program koji učitava podatke o tačkama iz datoteke *tacke.json* i sortira i po udaljenosti od koordinatnog početka. Na standardni izlaz ispisati podatke pre i posle sortiranja.

```
1 # Sortiranje
2 #
3 # sorted(kolekcija [, kljuc [, obrni]]) - vraca sortiranu kolekciju
4 #
5 # kolekcija - kolekcija koju zelimo da sortiramo
6 # kljuc - funkcija koja vraca kljuc po kome se poredi
7 # obrni - True/False (opadajuće/rastuće)
8 #
9 # Ukoliko zelimo da sortiramo po odredjenoj funkciji, mozemo iskoristiti
10 # funkciju iz `functools` modula `cmp_to_key()` koja ce prosledjenu
11 # funkciju "transformisati" u argument kljuc po kojem se poredi.
12 # Funkcija koja se prosledjuje funkciji cmp_to_key() ima potpis:
13 # foo(x, y) -> integer
14 # i vraca negativnu vrednost ako je x < y, 0 za x == y i pozitivnu vrednost za x > y
15 #
16 # primer:
17 #
18 # def foo(x, y):
19 #     return x + y
20 # sorted(kolekcija, key=cmp_to_key(foo))
21 #
22 # napomena: foo se prosledjuje bez zagrada jer se prosledjuje objekat koji
23 # predstavlja funkcija a ne povratna vrednost funkcije (zagrade znace da se
24 # funkcija evaluira za date argumente)
25
26 import json
27 import math
28
29 # l = ["A", "C", "D", "5", "1", "3"]
30 # print(l)
31 # print("sortirana lista: ", sorted(l)) # definisan operator < nad niskama
32
33 # U sledecem primeru je neophodno da definisemo svoje funkcije za vracanje kljuc po
34 # kom se poredi jer je kolekcija lista recnika i nad njima nije definisan operator
35 # '<'
36 with open("tacke.json", "r") as f:
37     tacke = json.load(f)
38
39 # Za definisanje kljuc poredjenja tacaka iskoristicemo anonimnu (lambda) funkciju (
40 # na mala vrata uvodimo ono sto cemo raditi detaljnije u programskom jeziku Haskell
41 # )
42 sortirane_tacke = sorted(tacke, key=lambda x: math.sqrt(x['koordinate'][0]**2 + x['
43     koordinate'][1]**2)) # Optimizacija: bez .sqrt; reverse=True za obratan redosled
44 print("Tacke pre sortiranja:")
45 for item in tacke:
46     print(item["teme"],)
47 print("\nTacke nakon sortiranja: ")
48 for item in sortirane_tacke:
49     print(item["teme"],)
50 print()
51
52 # import functools
53 # TODO Probati isto koristeći cmp_to_key()
```

1.4.5 Zadaci za samostalni rad sa rešenjima

Zadatak 1.44 Napisati program koji sa standardnog ulaza učitava ime datoteke i broj n i računa broj pojavljivanja svakog n -grama u datoteci koji su sačinjeni od proizvoljnih karaktera i rezultat upisuje u datoteku *rezultat.json*.

Primer 1

```

POKRETANJE: python n-anagram.py
INTERAKCIJA SA PROGRAMOM:
  Unesite ime datoteke:
    datoteka.txt
  Unesite n
    2

```

Sadržaj ulaznih datoteka koje se koriste u prethodnom primeru upotrebe programa:

Listing 1.2: *datoteka.txt*

```
1 Ovo je datoteka dat
```

Listing 1.3: *rezultat.json*

```

1 {
2   'a': 1, 'ka': 1, 'ot': 1, 'ek': 1,
3   'd': 2, 'j': 1, 'da': 2, 'e': 1,
4   'o': 1, 'to': 1, 'at': 2, 'je': 1,
5   'ov': 1, 'te': 1, 'vo': 1
6 }

```

[Rešenje 1.44]

Zadatak 1.45

U datoteci `korpa.json` se nalazi spisak kupljenog voća u json formatu:

```
1 [ { 'ime' : ime_voca, 'kolicina' : broj_kilograma } , ... ]
```

U datotekama `maxi_cene.json`, `idea_cene.json`, `shopngo_cene.json` se nalaze cene voća u json formatu:

```
1 [ { 'ime' : ime_voca, 'cena' : cena_po_kilogramu } , ... ]
```

Napisati program koji izračunava ukupan račun korpe u svakoj prodavnici i ispisuje cene na standardni izlaz.

Primer 1

```

POKRETANJE: python korpa.py
INTERAKCIJA SA PROGRAMOM:
  Maxi: 631.67 dinara
  Idea: 575.67 dinara
  Shopngo: 674.67 dinara

```

[Rešenje 1.45]

Sadržaj ulaznih datoteka koje se koriste u prethodnom primeru upotrebe programa:

Listing 1.4: *korpa.json*

```

1 [ {"ime" : "jabuke" , "kolicina": 3.3},
2   {"ime": "kruske" , "kolicina": 2.1},
3   {"ime": "đgroze" , "kolicina": 2.6},

```

Listing 1.5: *maksi_cene.json*

```

1 [ {"ime" : "jabuke" , "cena" : 59.9},
2   {"ime" : "kruske" , "cena" : 120},
3   {"ime" : "đgroze" , "cena" : 70},
4   {"ime" : "narandze" , "cena" : 49.9},
5   {"ime" : "breskve" , "cena" : 89.9} ]

```


Listing 1.6: *idea_cene.json*

```
1 [ {"ime" : "jabuke", "cena" : 39.9},
2 {"ime" : "kruske", "cena" : 100},
3 {"ime" : "đgroze", "cena" : 90},
4 {"ime" : "breskve", "cena" : 59.9} ]
```

Listing 1.7: *shopngo_cene.json*

```
1 [ {"ime" : "jabuke", "cena" : 69.9},
2 {"ime" : "kruske", "cena" : 100},
3 {"ime" : "đgroze", "cena" : 90},
4 {"ime" : "maline", "cena" : 290},
```

[Rešenje 1.45]

Zadatak 1.46 Napisati program koji računa odnos kardinalnosti skupova duže i šire za zadati direktorijum. Datoteka pripada skupu duže ukoliko ima više redova od maksimalnog broja karaktera po redu, u suprotnom pripada skupu šire. Sa standardnog ulaza se unosi putanja do direktorijuma. Potrebno je obići sve datoteke u zadatom direktorijumu i njegovim poddirektorijumima (koristiti funkciju `os.walk()`) i ispisati odnos kardinalnosti skupova duže i šire.

Primer 1

```
POKRETANJE: python duze_sire.py
INTERAKCIJA SA PROGRAMOM:
  Unesite putanju do direktorijuma:
  ..
  Kardinalnost skupa duze : Kardinalnost skupa sire
  10 : 15
```

[Rešenje 1.46]

Zadatak 1.47 Napisati program koji obilazi direktorijume rekurzivno i računa broj datoteka za sve postojeće ekstenzije u tim direktorijumima. Sa standardnog ulaza se unosi putanja do početnog direktorijuma, a rezultat se ispisuje u datoteku `rezultat.json`.

Primer 1

```
POKRETANJE: python ekstenzije.py
INTERAKCIJA SA PROGRAMOM:
  Unesite putanju do direktorijuma:
  .
```

Sadržaj datoteke koja se generise u prethodnom primeru upotrebe programa:

Listing 1.8: *rezultat.txt*

```
1 {
2   'txt' : 14,
3   'py' : 12,
4   'c' : 10
5 }
```

[Rešenje 1.47]

Zadatak 1.48 U datoteci `radnici.json` nalaze se podaci o radnom vremenu zaposlenih u preduzeću u sledecem formatu:

```
1 [ { 'ime' : ime\_radnika, 'odmor' : [pocetak, kraj], 'radno_vreme'
   : [pocetak, kraj] }, ... ]
```

Napisati program koji u zavisnosti od unete opcije poslodavcu ispisuje trenutno dostupne radnike odnosno radnike koji su na odmoru. Moguće opcije su 'd' - trenutno dostupni radnici i 'o' - radnici koji su na odmoru. Radnik je dostupan ukoliko nije na odmoru i trenutno vreme je u okviru njegovog radnog vremena.

Primer 1

```

POKRETANJE: python odmor.py
INTERAKCIJA SA PROGRAMOM:
Unesite opciju koju zelite
d - dostupni radnici
o - radnici na odmoru :
m
Uneta opcija nije podrzana

```

Primer 2

```

POKRETANJE: python odmor.py
INTERAKCIJA SA PROGRAMOM:
Unesite opciju koju zelite
d - dostupni radnici
o - radnici na odmoru :
d
Pera Peric

```

[Rešenje 1.48]

Sadržaj ulaznih datoteka koje se koriste u prethodnom primeru upotrebe programa:

Listing 1.9: *radnici.json*

```

1 [ { 'ime' : 'Pera Peric',
2   'odmor' : ['21.08.2016.', '31.08.2016.'],
3   'radno_vreme' : ['08:30', '15:30'] } ]

```

Zadatak 1.49 Napisati program koji učitava ime datoteke sa standardnog ulaza i na standardni izlaz ispisuje putanje do svih direktorijuma u kojima se nalazi ta datoteka.

Primer 1

```

POKRETANJE: python pojavljivanja.py
INTERAKCIJA SA PROGRAMOM:
Unesite ime datoteke:
1.py
/home/student/vezbe/cas1/1.py
/home/student/vezbe/cas7/1.py
/home/student/vezbe/cas9/1.py

```

[Rešenje 1.49]

1.4.6 Zadaci za vežbu

Zadatak 1.50 Napisati program koji iz datoteke *ispiti.json* učitava podatke o ispitima i njihovim datumima. Ispisati na standardni izlaz za svaki ispit njegovo ime i status "Prosao" ukoliko je ispit prosao, odnosno "Ostalo je jos n dana.", gde je n broj dana od trenutnog datuma do datuma ispita.

Primer 1

```

POKRETANJE: python ispiti.py
INTERAKCIJA SA PROGRAMOM:
Relacione baze podataka Prosao
Vestacka inteligencija Prosao
Linearna algebra i analiticka geometrija Prosao

```

Sadržaj ulaznih datoteka koje se koriste u prethodnom primeru upotrebe programa:

Listing 1.10: *ispiti.json*

```

1 [ { 'ime': 'Relacione baze podataka',
2   'datum': '21.09.2016.' },
3   { 'ime': 'Vestacka inteligencija',
4   'datum': '17.06.2017.' },
5   { 'ime': 'Linearna algebra i analiticka geometrija',
6   'datum': '08.02.2017.' } ]

```

Zadatak 1.51 Napisati program koji izdvaja sve jednolinijske i višelinjske komentare iz .c datoteke čije ime se unosi sa standardnog ulaza, listu jednih i drugih komentara upisuje u datoteku *komentari.json*. Jednolinijski komentari se navode nakon `//` a višelinjski između `/*` i `*/`.

Primer 1

```
POKRETANJE: python komentari.py
INTERAKCIJA SA PROGRAMOM:
Unesite ime datoteke:
program.c
```

Sadrzaj ulaznih datoteka koje se koriste u prethodnom primeru kao i datoteke koja se generiše:

Listing 1.11: *program.c*

```
1  #include <stdio.h>
2
3  // Primer jednolinijskog komentara
4
5  int main(){
6  /*
7  Na ovaj nacin ispisujemo tekst
8  na standardni izlaz koristeći jezik C.
9  */
10 printf("Hello world!");
11
12 // Na ovaj nacin se ispisuje novi red
13 printf("\n");
14 /*
15 Ukoliko se funkcija uspesno završila
16 vracamo 0 kao njen rezultat.
17 */
18 return 0;
19 }
```

Listing 1.12: *komentari.json*

```
1 {
2   'jednolinijski' : ['Primer jednolinijskog komentara',
3                     'Na ovaj nacin se ispisuje novi red'],
4   'viselinejski'  : ['Na ovaj nacin ispisujemo tekst na standardni
5                     izlaz koristeći jezik C.',
6                     'Ukoliko se funkcija uspesno završila
7                     vracamo 0 kao njen rezultat.'],
8 }
```

Zadatak 1.52 Napisati program upoređuje dve datoteke čija imena se unose sa standardnog ulaza. Rezultat upoređivanja je datoteka *razlike.json* koja sadrži broj linija iz prve datoteke koje se ne nalaze u drugoj datoteci i obratno. *Napomena* Obratiti pažnju na efikasnost.

Primer 1

```
POKRETANJE: python razlika.py
INTERAKCIJA SA PROGRAMOM:
Unesite ime datoteke:
dat1.txt
Unesite ime datoteke:
dat2.txt
```

Sadrzaj ulaznih datoteka koje se koriste u prethodnom primeru kao i datoteke koja se generiše::

Listing 1.13: *dat1.txt*

```
1 //netacno
2
3 same=1;
4
5 for(i=0;s1[i]!='\0' && s2[i]!='\0';i++) {
6   if(s1[i]!=s2[i]) {
7     same=0;
8     break;
9   }
10 }
```

```
return same;
```

Listing 1.14: *dat2.txt*

```
1 //tacno
3 for(i=0;s1[i]!='\0' && s2[i]!='\0';i++){
5 if(s1[i]!=s2[i])
   return 0;
7 }
   return s1[i]==s2[i];
```

Listing 1.15: *razlike.json*

```
1 {
2   'dat1.txt' : 7,
3   'dat2.txt' : 4
4 }
```

Zadatak 1.53 Napisati program koji ispisuje na standardni izlaz putanje do lokacija svih Apache virtuelnih hostova na računaru. Smatrati da je neki direktorijum lokacija Apache virtuelnog hosta ukoliko u sebi sadrži `index.html` ili `index.php` datoteku.

Primer 1

```
POKRETANJE: python apache.py
INTERAKCIJA SA PROGRAMOM:
/home/student/PVEB/prviPrimer
/home/student/licna_strana
/home/student/PVEB/ispit/jun
```

Zadatak 1.54 Napisati program koji realizuje autocomplete funkcionalnost. Sa standardnog ulaza korisnik unosi delove reči sve dok ne unese karakter !. Nakon svakog unetog dela reči ispisuju se reči koje počinju tim karakterima. Spisak reči koje program može da predloži se nalazi u datoteci `reci.txt`.

Primer 1

```
POKRETANJE: python autocomplete.py
INTERAKCIJA SA PROGRAMOM:
ma
mac macka mama maceta madionicar
mac
mac macka maceta
!
```

Sadržaj ulaznih datoteka koje se koriste u prethodnom primeru upotrebe programa:

Listing 1.16: *reci.txt*

```
1 mac pesma skola macka mama maceta igra dmadionicar
```

1.5 Rešenja

Rešenje 1.22 Pogodi broj

```
1 # Pogodi broj
3 import random
5 print("----- IGRA: Pogodi broj -----\\n")
7 zamisljen_broj = random.randint(0,100)
9 ime = input("Unesite Vase ime: ")
```

```

11 print("Zdravo {0:s}. :) \nZamislio sam neki broj od 1 do 100. Da li mozes da pogodis
    koji je to broj?".format(ime))

13 pogodio = False
while not pogodio:
15     print("Unesi broj:")
    broj = int(input())
17     if broj == zamisljen_broj:
        pogodio = True
19     elif broj > zamisljen_broj:
        print("Broj koji sam zamislio je MANJI od {0:d}.".format(broj))
21     else:
        print("Broj koji sam zamislio je VECI od {0:d}.".format(broj))
23
print("BRAVO!!! Pogodio si! Zamislio sam {0:d}. Bilo je lepo igrati se sa tobom. :)".
    format(zamisljen_broj))

```

Rešenje 1.23 Aproksimacija broja PI metodom Monte Karlo

```

# Aproksimacija broja PI metodom Monte Karlo

2
import random, math

4
def dist(A, B):
6     """Funkcija izracunava euklidsko rastojanje izmedju tacaka A i B"""
    return math.sqrt((A[0]-B[0])**2 + (A[1]-B[1])**2)

8
print("Izracunavanje broja PI metodom Monte Karlo \n")
try:
10     N = int(input("Unesite broj iteracija: "))
except ValueError as err:
12     print("Greska. Parsiranje broja iteracija: ", err)
14     exit(1)

A = 0 # Broj tacaka u krugu
16 B = 0 # Broj tacaka u kvadratu

18 i = N
while i >= 0:
20     tacka = (random.random(), random.random())
    # Ukoliko se tacka nalazi u krugu, povecavamo broj tacaka u krugu
22     if dist(tacka, (0.5, 0.5)) <= 0.5:
        A = A + 1
24     B = B + 1
    i = i - 1
26
print("Broj PI aproksimiran metodom Monte Karlo: ")
28 print(4.0*A/B)

```

Rešenje 1.24 X-O

```

# X-O
2
#
# - | 0 | X
4
# --- ---
# X | - | -
6
# --- ---
# - | X | 0

8
import random

10
def ispisi_tablu(tabla):
12     print("\n      TABLA \n")
    print("      1   2   3   ")
14     print("      --- --- --- ")
    indeks = 1
16     for i in tabla:
        print(indeks, "|", i[0], "|", i[1], "|", i[2], "|")
18     print("      --- --- --- ")
    indeks = indeks + 1

```

```

20     print("\n")

22 def pobedio(tabla):
    if (tabla[0][0] != "-" and tabla[0][2] != "-") and ((tabla[0][0] == tabla[1][1]
    == tabla[2][2]) or (tabla[0][2] == tabla[1][1] == tabla[2][0])):
24         return True
    for i in range(3):
26         if (tabla[0][i] != "-" and tabla[i][0] != "-") and ((tabla[0][i] == tabla[1][
    i] == tabla[2][i]) or (tabla[i][0] == tabla[i][1] == tabla[i][2])):
            return True
28     return False

30 def ucitaj_koordinate(ime):
    while True:
32         print("{0:s} unesite koordinate polja koje zelite da popunite u posebnim
    linijama:\n".format(ime))
        x = int(input("Unesite vrstu: "))
34         y = int(input("Unesite kolonu: "))
        if 1<=x<=3 and 1<=y<=3:
36             return x-1,y-1
        else:
38             "Morate uneti brojeve 1,2 ili 3\n"

40 def korak(igrac):
    while True:
42         x,y = ucitaj_koordinate(igrac[0])
        if tabla[x][y] == "-":
44             tabla[x][y] = igrac[1]
            ispisi_tablu(tabla)
46             break
        else:
48             print(tabla[x][y])
            print("Uneto polje je popunjeno!\n")

50 print("IGRA: X-O pocinje\n")

52 ime1 = input("Unesite ime prvog igraca: ")
54 print("Zdravo {0:s}!\n".format(ime1))
    ime2 = input("Unesite ime drugog igraca: ")
56 print("Zdravo {0:s}!\n".format(ime2))

58 indikator = random.randint(1,2)
    if indikator == 1:
60         prvi_igrac = (ime1, "X")
        drugi_igrac = (ime2, "O")
62     else:
        prvi_igrac = (ime2, "X")
64         drugi_igrac = (ime1, "O")

66 print("Igrac {0:s} igra prvi. \n".format(prvi_igrac[0]))
    print("X : {0:s}\n".format(prvi_igrac[0]))
68 print("O : {0:s}\n".format(drugi_igrac[0]))

70 tabla = [['-', '-', '-'], ['- ', '- ', '- '], ['- ', '- ', '- ']]

72 print("Zapocnimo igru \n")

74 ispisi_tablu(tabla)

76 na_redu = 0
    iteracija = 0
78 igraci = [prvi_igrac, drugi_igrac]
    while iteracija < 9:
80         korak(igraci[na_redu])
        if pobedio(tabla) == True:
82             print("BRAVO!!!!!! Igrac {0:s} je pobedio!\n".format(igraci[na_redu][0]))
            break
84         na_redu = (na_redu+1)%2
        iteracija = iteracija + 1

86
88 if iteracija == 9:
    print("NERESEN! Pokusajte ponovo.\n")

```

Rešenje 1.44

```
# dat.txt:
2 # Ovo je datoteka dat
#
4 # rezultat.json:
#
6 # {"a ": 1, "ka": 1, "ot": 1, "ek": 1, " d": 2, " j": 1, "da": 2, "e ": 1, "o ": 1, "
   to": 1, "at": 2, "je": 1, "Ov": 1, "te": 1, "vo": 1}

8 import json

10 ime_datoteke = input("Unesite ime datoteke: ")
n = int(input("Unesite broj n: "))

12 # Otvaramo datoteku i citamo njen sadrzaj
14 f = open(ime_datoteke, "r")
sadrzaj = f.read()
16 f.close()

18 recnik = {}
i = 0
20 # Prolazimo kroz sadrzaj i uzimamo jedan po jedan n-gram
while i < len(sadrzaj) - n:
22     ngram = sadrzaj[i : i+n]
    # Ukoliko se n-gram vec nalazi u recniku,
24     # povecavamo mu broj pojavljivanja
    if ngram in recnik:
26         recnik[ngram] = recnik[ngram]+1
    # Dodajemo n-gram u recnik i postavljamo mu broj na 1
28     else:
        recnik[ngram] = 1
30     i = i + 1

32 f = open("rezultat.json", "w")
json.dump(recnik,f)
34 f.close()
```

Rešenje 1.45

```
import json

2
def cena_voca(prodavnica, ime_voca):
4     for voce in prodavnica:
        if voce['ime'] == ime_voca:
8             return voce['cena']

# Ucitavamo podatke iz datoteka
10 f = open('korpa.json', "r")
korpa = json.load(f)
f.close()

12 f = open('maxi_cene.json', "r")
maxi_cene = json.load(f)
f.close()

14 f = open('idea_cene.json', "r")
idea_cene = json.load(f)
f.close()

16 f = open('shopngo_cene.json', "r")
shopngo_cene = json.load(f)
f.close()

20
22 maxi_racun = 0
24 idea_racun = 0
26 shopngo_racun = 0
28 i = 0
# Za svako voce u korpi dodajemo njegovu cenu u svaki racun posebno
30 while i < len(korpa):
    ime_voca = korpa[i]['ime']
32     maxi_racun = maxi_racun + korpa[i]['kolicina']*cena_voca(maxi_cene, ime_voca)
```

```

34     idea_racun = idea_racun + korpa[i]['kolicina']*cena_voca(idea_cene, ime_voca)
    shopngo_racun = shopngo_racun + korpa[i]['kolicina']*cena_voca(shopngo_cene,
        ime_voca)
    i += 1
36
37 print("Maxi: " + str(maxi_racun) + " dinara")
38 print("Idea: " + str(idea_racun) + " dinara")
    print("Shopngo: " + str(shopngo_racun) + " dinara")

```

Rešenje 1.46

```

import os
2
dat_u_duze = 0
4 dat_u_sire = 0

6 # Funkcija koja obilazi datoteku i vraca 1 ukoliko datoteka pripada skupu duze
# odnosno 0 ukoliko datoteka pripada skupu sire
8 def obilazak(ime_datoteke):
    br_linija = 0
10    najduza_linija = 0
    with open(ime_datoteke, "r") as f:
12        for linija in f:
            br_linija = br_linija + 1
14            if len(linija) > najduza_linija:
                najduza_linija = len(linija)
16    if br_linija > najduza_linija:
        return 1
18    else:
        return 0
20
ime_direktorijuma = input("Unesite putanju do direktorijuma: ")
22
for (tren_dir, pod_dir, datoteke) in os.walk(ime_direktorijuma):
24    for dat in datoteke:
        if obilazak(os.path.join(tren_dir, dat)) == 0:
26            dat_u_sire += 1
        else:
28            dat_u_duze += 1
30
print("Kardinalnost skupa duze: kardinalnost skupa sire")
print(str(dat_u_duze)+" "+str(dat_u_sire))

```

Rešenje 1.47

```

1 import os
import json
3
ime_direktorijuma = input("Unesite putanju do direktorijuma: ")
5
ekstenzije = {}
7
for (tren_dir, pod_dir, datoteke) in os.walk(ime_direktorijuma):
9    for dat in datoteke:
        pozicija = dat.find(".")
11        # Ukoliko datoteka ima ekstenziju, pretpostavljamo da su datoteke imenovane
        tako da posle . ide ekstenzija u ispravnom obliku
        if pozicija >= 0:
13            # Ukoliko ekstenzija postoji u mapi, povecavamo njen broj
            if dat[pozicija:] in ekstenzije:
15                ekstenzije[dat[pozicija:]] += 1
            else:
17                # Dodajemo novu ekstentiju u mapu i postavljamo njen broj na 1
                ekstenzije[dat[pozicija:]] = 1
19
with open("rezultat.json", "w") as f:
21    json.dump(ekstenzije, f)

```

Rešenje 1.48


```
1 import json, os, sys
2 from datetime import datetime
3
4 try:
5     with open("radnici.json", "r") as f:
6         radnici = json.load(f)
7 except IOError:
8     print("Otvaranje datoteke nije uspjelo!")
9     sys.exit()
10
11 opcija = input("Unesite opciju koju zelite (d - dostupni radnici, o - radnici na
12             odmoru): \n")
13
14 if opcija != "d" and opcija != "o":
15     print("Uneta opcija nije podrzana.")
16     exit()
17
18 tren_dat = datetime.now()
19
20 # funkcija datetime.strptime(string, format) pravi objekat tipa datetime na osnovu
21 # zadatih podataka u stringu i odgovarajuceg formata, na primer ako je datum
22 # zapisan kao "21.08.2016" odgovarajuci format je "%d.%m.%Y." pa se funkcija poziva
23 # sa datetime.strptime("21.08.2016", "%d.%m.%Y.")
24
25 for radnik in radnici:
26     kraj_odmora = datetime.strptime(radnik['odmor'][1], "%d.%m.%Y.").date()
27     pocetak_odmora = datetime.strptime(radnik['odmor'][0], "%d.%m.%Y.").date()
28     kraj_rad_vrem = datetime.strptime(radnik['radno_vreme'][1], "%H:%M").time()
29     pocetak_rad_vrem = datetime.strptime(radnik['radno_vreme'][0], "%H:%M").time()
30     if opcija == "o":
31         # Ukoliko je radnik trenutno na odmoru ispisujemo ga
32         if pocetak_odmora < tren_dat.date() < kraj_odmora:
33             print(radnik["ime"])
34     else:
35         # Ukoliko je radnik trenutno dostupan i nije na odmoru, ispisujemo ga
36         if not (pocetak_odmora < tren_dat.date() < kraj_odmora) and pocetak_rad_vrem
37         < tren_dat.time() < kraj_rad_vrem:
38             print(radnik["ime"])
```

Rešenje 1.49

```
1 import os
2
3 ime_datoteke = input("Unesite ime datoteke: ")
4
5 # pretrazujemo ceo fajl sistem, odnosno pretragu krecemo od root direktorijuma /
6 # imajte u vidu da ce vreme izvršavanja ovog programa biti veliko posto se pretrazuje
7 # ceo fajl sistem, mozete ga prekinuti u svakom trenutku sa CTRL+C
8 for (tren_dir, pod_dir, datoteke) in os.walk("/"):
9     # objekat datoteke predstavlja listu imena datoteka iz direktorijuma
10    # ta imena poredimo sa zadatim
11    for dat in datoteke:
12        # ako smo naisli na trazenu datoteku, pravimo odgovarajucu putanju
13        if dat == ime_datoteke:
14            print(os.path.join(os.path.abspath(tren_dir), ime_datoteke))
```

2

Programiranje ograničenja - Python

Potrebno je imati instaliran Python 3.7 i biblioteku python-constraint. Na Ubuntu operativnom sistemu, biblioteka python-constraint se može instalirati pomoću Pip alata:

```
sudo apt-get -y install python3-pip
sudo pip3 install python-constraint
```

Korisni linkovi i literatura:

<http://labix.org/doc/constraint/>
<https://pypi.python.org/pypi/python-constraint>
http://www.hakank.org/constraint_programming_blog/

2.1 Programiranje ograničenja

2.1.1 Uvodni primeri

Zadatak 2.1 Napisati program koji na standardni izlaz ispisuje sve kombinacije oblika xyz , gde je $x \in \{a, b, c\}$, $y \in \{1, 2, 3\}$ i $z \in \{0.1, 0.2, 0.3\}$ tako da važi da je $10 \cdot z = y$.

```
1 # Programiranje ograničenja
3 # Uključujemo modul za rad sa ograničenjima
import constraint
5
# Definiramo problem
7 problem = constraint.Problem()
# Dodajemo promenljive
9 #
# problem.addVariable(ime_promenljive, domen_lista)
11 # problem.addVariables(lista_imena_promenljivih, domen_lista)
problem.addVariable('x', ['a', 'b', 'c'])
13 problem.addVariable('y', [1, 2, 3])
# Ispisujemo rešenja
15 # print(problem.getSolutions())

17 problem.addVariable('z', [0.1, 0.2, 0.3])
# Dodajemo ograničenja
19 #
# problem.addConstraint(ogranicenje [, redosled_promenljivih])
21 #
# ograničenje može biti:
23 # constraint.AllDifferentConstraint() - različite vrednosti svih promenljivih
# constraint.AllEqualConstraint() - iste vrednosti svih promenljivih
25 # constraint.MaxSumConstraint(s [,tezine]) - suma vrednosti promenljivih (pomnožena
sa tezina) ne prelazi s
# constraint.MinSumConstraint(s [,tezine]) - suma vrednosti promenljivih (pomnožena
sa tezina) nije manja od s
27 # constraint.ExactSumConstraint(s [,tezine]) - suma vrednosti promenljivih (
pomnožena sa tezina) je s
# constraint.InSetConstraint(skup) - vrednosti promenljivih se nalaze u skupu skup
```

```

29 # constraint.NotInSetConstraint(skup) - vrednosti promenljivih se ne nalaze u skupu
    skup
    # constraint.SomeInSetConstraint(skup) - vrednosti nekih promenljivih se nalaze u
    skupu skup
31 # constraint.SomeNotInSetConstraint(skup) - vrednosti nekih promenljivih se ne
    nalaze u skupu skup
    #
33 # redosled_promenljivih predstavlja listu promenljivih
    # i zadaje se zbog definisanja tacnog redosleda
35 # ogracenja koja se primenjuju na promenljive
    #
37 # Mozemo napraviti i svoju funkciju ogracenja
def ogracenje(y,z):
39     if y / 10.0 == z:
        return True
41
    # Prosledjujemo funkciju ogracenja i redosled promenljivih koji treba da odgovara
    redosledu argumenata funkcije ogracenja
43 problem.addConstraint(ogracenje,['y','z'])
    resenja = problem.getSolutions()
45 print("\n-----Resenja-----\n")
    for resenje in resenja:
47     print(str(resenje['x']) + " " + str(resenje['y']) + " " + str(resenje['z']))

```

2.1.2 Zadaci za samostalni rad sa rešenjima

Zadatak 2.2 Napisati program koji pronalazi trocifren broj ABC tako da je količnik $ABC / (A + B + C)$ minimalan i A, B i C su različiti brojevi. Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```

1 print """
min_resenje['A']*100 + min_resenje['B']*10 + min_resenje['C']
3 """

```

[Rešenje 2.2]

Zadatak 2.3 Dati su novčići od 1, 2, 5, 10, 20 dinara. Napisati program koji pronalazi sve moguće kombinacije tako da zbir svih novčića bude 50.

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```

1 print """
2 1 din: {0:d}
2 2 din: {1:d}
4 5 din: {2:d}
10 din: {3:d}
6 20 din: {4:d}
"""
6 """
    .format(r["1 din"], r["2 din"], r["5 din"], r["10 din"], r["20 din"])

```

[Rešenje 2.3]

Zadatak 2.4 Napisati program koji reda brojeve u magičan kvadrat. Magičan kvadrat je kvadrat dimenzija 3x3 takav da je suma svih brojeva u svakom redu, svakoj koloni i svakoj dijagonali jednak 15 i svi brojevi različiti. Na primer:

```

4 9 2
3 5 7
8 1 6

```

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```

1 print " _____ "
2 print " | {0:d} {1:d} {2:d} | ".format(r['a'], r['b'], r['c'])
3 print " | {0:d} {1:d} {2:d} | ".format(r['d'], r['e'], r['f'])
4 print " | {0:d} {1:d} {2:d} | ".format(r['g'], r['h'], r['i'])
5 print " _____ "

```

[Rešenje 2.4]

Zadatak 2.5 Napisati program koji pronalazi sve vrednosti promenljivih X, Y i Z za koje važi da je $X \geq Z$ i $X * 2 + Y * X + Z \leq 34$ pri čemu promenljive pripadaju narednim domenima $X \in \{1, 2, \dots, 90\}$, $Y \in \{2, 4, 6, \dots, 60\}$ i $Z \in \{1, 4, 9, 16, \dots, 100\}$

[Rešenje 2.5]

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```

1 print " _____ "
2 print "X = {0:d} , Y = {1:d} , Z = {2:d} ".format(r['X'], r['Y'], r['Z'])

```

[Rešenje 2.5]

Zadatak 2.6 Napisati program koji dodeljuje različite vrednosti različitim karakterima tako da suma bude zadovoljena:

```

TWO
+TWO
-----
FOUR

```

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```

1 print " _____ "
2 print " " + str(r['T']) + str(r['W']) + str(r['O'])
3 print " +" + str(r['T']) + str(r['W']) + str(r['O'])
4 print "=" + str(r['F']) + str(r['O']) + str(r['U']) + str(r['R'])

```

[Rešenje 2.6]

Zadatak 2.7 Napisati program koji pronalazi sve vrednosti promenljivih X, Y, Z i W za koje važi da je $X \geq 2 * W$, $3 + Y \leq Z$ i $X - 11 * W + Y + 11 * Z \leq 100$ pri čemu promenljive pripadaju narednim domenima $X \in \{1, 2, \dots, 10\}$, $Y \in \{1, 3, 5, \dots, 51\}$, $Z \in \{10, 20, 30, \dots, 100\}$ i $W \in \{1, 8, 27, \dots, 1000\}$.

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```

1 print " _____ "
2 print "X = {0:d} , Y = {1:d} , Z = {2:d}, W = {3:d} ".format(r['X'], r['Y'], r['Z'], r['W'])

```

[Rešenje 2.7]

Zadatak 2.8 Napisati program koji raspoređuje brojeve 1-9 u dve linije koje se seku u jednom broju. Svaka linija sadrži 5 brojeva takvih da je njihova suma u obe linije 25 i brojevi su u rastućem redosledu.

```

1 3
2 4
5
6 8
7 9

```

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "_____"
2 print "{0:d} {1:d}".format(r['a'], r['A'])
3 print "{0:d} {1:d} ".format(r['b'], r['B'])
4 print " {0:d} ".format(r['c'])
5 print "{0:d} {1:d} ".format(r['D'], r['d'])
6 print "{0:d} {1:d}".format(r['E'], r['e'])
7 print "_____"
```

[Rešenje 2.8]

Zadatak 2.9 Pekara *Kiflica* proizvodi hleb i kifle. Za mešenje hleba potrebno je 10 minuta, dok je za kiflu potrebno 12 minuta. Vreme potrebno za pečenje ćemo zanemariti. Testo za hleb sadrži 300g brašna, a testo za kiflu sadrži 120g brašna. Zarada koja se ostvari prilikom prodaje jednog hleba je 7 dinara, a prilikom prodaje jedne kifle je 9 dinara. Ukoliko pekara ima 20 radnih sati za mešenje peciva i 20kg brašna, koliko komada hleba i kifli treba da se umesi kako bi se ostvarila maksimalna zarada (pod pretpostavkom da će pekara sve prodati)?

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "_____"
2 print "Maksimalna zarada je {0:d}, komada hleba je {1:d}, a komada kifli {2:d}".format
   (7*max_H + 9*max_K, max_H, max_K)
3 print "_____"
```

[Rešenje 2.9]

Zadatak 2.10 Napisati program pronalazi vrednosti A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S (svako slovo predstavlja različit broj) koje su poredane u heksagon na sledeći način:

A,B,C
D,E,F,G
H,I,J,K,L
M,N,O,P
Q,R,S

tako da zbir vrednosti duž svake horizontalne i dijagonalne linije bude 38 ($A+B+C = D+E+F+G = \dots = Q+R+S = 38$, $A+D+H = B+E+I+M = \dots = L+P+S = 38$, $C+G+L = B+F+K+P = \dots = H+M+Q = 38$).

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "_____"
2 print "{0:d},{1:d},{2:d}".format(r['A'], r['B'], r['C'])
3 print "{0:d},{1:d},{2:d},{3:d}".format(r['D'], r['E'], r['F'], r['G'])
4 print "{0:d},{1:d},{2:d},{3:d},{4:d}".format(r['H'], r['I'], r['J'], r['K'], r['L'])
5 print "{0:d},{1:d},{2:d},{3:d}".format(r['M'], r['N'], r['O'], r['P'])
6 print "{0:d},{1:d},{2:d}".format(r['Q'], r['R'], r['S'])
7 print "_____"
```

[Rešenje 2.10]

Zadatak 2.11 Kompanija Start ima 250 zaposlenih radnika. Rukovodstvo kompanije je odlučilo da svojim radnicima obezbedi dodatnu edukaciju. Da bi se radnik obučio programskom jeziku Elixir potrebno je platiti 100 evra po osobi za kurs, ali bi njegovo produktivno znanje ovog programskog jezika donelo 150 projekat/sati mesečno, što bi za kompaniju značilo dobit od 5 evra po projekat/satu. Da bi se radnik obučio programskom jeziku Dart potrebno je platiti 105 evra po osobi za kurs, ali bi njegovo produktivno znanje ovog programskog jezika donelo 170 projekat/sati mesečno, koji bi za kompaniju značili dobit od 6 evra po satu. Ukoliko Start ima na raspolaganju 26000 evra za obuku i maksimalan broj 51200 mogućih projekat/sati mesečno, odrediti na koji način kompanija treba da obuči svoje zaposlene kako bi ostvarila maksimalnu dobit.

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print """Maksimalna zarada je {0:d},
   broj radnika koje treba poslati na kurs Elixir je {1:d},
3 a broj radnika koje treba poslati na kurs Dart je {2:d}.
   """.format(170*6*max_E + 150*5*max_D - (100*max_E + 150*max_D) , max_E, max_D)
```

[Rešenje 2.11]

Zadatak 2.12 Napisati program koji raspoređuje 8 topova na šahovsku tablu tako da se nikoja dva topa ne napadaju.

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "_____"
   for i in "12345678":
3     for j in range(1,9):
       if r[i] == j:
5         print "T",
       else:
7         print "_",
   print ""
9 print "_____"
```

[Rešenje 2.12]

Zadatak 2.13 Napisati program koji raspoređuje 8 dama na šahovsku tablu tako da se nikoje dve dame ne napadaju.

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "_____"
   for i in "12345678":
3     for j in range(1,9):
       if r[i] == j:
5         print "D",
       else:
7         print "_",
   print ""
9 print "_____"
```

[Rešenje 2.13]

Zadatak 2.14 Napisati program koji učitava tablu za Sudoku iz datoteke čije ime se zadaje sa standardnog ulaza i korišćenjem ograničenja rešava Sudoku zagonetku.

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "=====
   for i in range(1,10):
3     print "|",
     for j in range(1,10):
5         if j%3 == 0:
           print str(r[i*10+j])+" | ",
7         else:
           print str(r[i*10+j]),
9     print ""
   if i%3 == 0 and i!=9:
11    print "=====
   print "=====
```

Primer 1

```
POKRETANJE: python sudoku.py
INTERAKCIJA SA PROGRAMOM:
Unesite ime datoteke sa tablom za sudoku:
sudoku.json
=====
| 6 3 8 | 7 5 2 | 1 9 4 |
| 9 2 7 | 4 1 8 | 5 6 3 |
| 5 4 1 | 6 3 9 | 2 7 8 |
=====
| 7 1 4 | 5 9 6 | 3 8 2 |
| 2 8 9 | 1 7 3 | 4 5 6 |
| 3 5 6 | 8 2 4 | 7 1 9 |
=====
| 8 9 5 | 3 4 1 | 6 2 7 |
| 4 7 2 | 9 6 5 | 8 3 1 |
| 1 6 3 | 2 8 7 | 9 4 5 |
=====
```

[Rešenje 2.14]

Sadržaj datoteke koja se koriste u primeru 2.14:

Listing 2.1: *sudoku.json*

```
1 [[6, 3, 8, 7, 0, 0, 0, 0, 0],
2 [0, 2, 7, 0, 0, 0, 5, 0, 0],
3 [5, 4, 0, 6, 0, 9, 2, 0, 0],
4 [0, 1, 0, 5, 9, 0, 3, 0, 0],
5 [0, 0, 9, 0, 7, 0, 4, 0, 0],
6 [0, 0, 6, 0, 2, 4, 0, 1, 0],
7 [0, 0, 5, 3, 0, 1, 0, 2, 7],
8 [0, 0, 2, 0, 0, 0, 8, 3, 0],
9 [0, 0, 0, 0, 0, 7, 9, 4, 5]]
```

2.1.3 Zadaci za vežbu

Zadatak 2.15 Za svaku narednu zagonetku, napisati program koji dodeljuje različite vrednosti različitim karakterima tako da suma bude zadovoljena:

```
GREEN + ORANGE = COLORS
MANET + MATISSE + MIRO + MONET + RENOIR = ARTISTS
COMPLEX + LAPLACE = CALCULUS
THIS + IS + VERY = EASY
CROSS + ROADS = DANGER
FATHER + MOTHER = PARENT
WE + WANT + NO + NEW + ATOMIC = WEAPON
EARTH + AIR + FIRE + WATER = NATURE
SATURN + URANUS + NEPTUNE + PLUTO = PLANETS
SEE + YOU = SOON
NO + GUN + NO = HUNT
WHEN + IN + ROME + BE + A = ROMAN
DONT + STOP + THE = DANCE
HERE + THEY + GO = AGAIN
OSAKA + HAIKU + SUSHI = JAPAN
MACHU + PICCHU = INDIAN
SHE + KNOWS + HOW + IT = WORKS
COPY + PASTE + SAVE = TOOLS
```

Sve rezultate ispisati na standardni izlaz koristeći sledeći format komande ispisa. Primer za prvu zagonetku.

KOMANDA ISPISA REŠENJA:

```
print " " + str(r['G']) + str(r['R']) + str(r['E']) + str(r['E']) + str(r['N'])
```

```

2 print " + " + str(r['O']) + str(r['R']) + str(r['A']) + str(r['N']) + str(r['G']) + str(r
  ['E'])
print "= " + str(r['C']) + str(r['O']) + str(r['L']) + str(r['O']) + str(r['R']) + str(r[
  'S'])

```

Zadatak 2.16 Za svaku narednu zagonetku, napisati program koji dodeljuje različite vrednosti različitim karakterima tako da suma bude zadovoljena:

```

THREE + THREE + ONE = SEVEN
NINE + LESS + TWO = SEVEN
ONE + THREE + FOUR = EIGHT
THREE + THREE + TWO + TWO + ONE = ELEVEN
SIX + SIX + SIX = NINE + NINE
SEVEN + SEVEN + SIX = TWENTY
ONE + ONE + ONE + THREE + THREE + ELEVEN = TWENTY
EIGHT + EIGHT + TWO + ONE + ONE = TWENTY
ELEVEN + NINE + FIVE + FIVE = THIRTY
NINE + SEVEN + SEVEN + SEVEN = THIRTY
TEN + SEVEN + SEVEN + SEVEN + FOUR + FOUR + ONE = FORTY
TEN + TEN + NINE + EIGHT + THREE = FORTY
FOURTEEN + TEN + TEN + SEVEN = FORTYONE
NINETEEN + THIRTEEN + THREE + TWO + TWO + ONE + ONE + ONE = FORTYTWO
FORTY + TEN + TEN = SIXTY
SIXTEEN + TWENTY + TWENTY + TEN + TWO + TWO = SEVENTY
SIXTEEN + TWELVE + TWELVE + TWELVE + NINE + NINE = SEVENTY
TWENTY + TWENTY + THIRTY = SEVENTY
FIFTY + EIGHT + EIGHT + TEN + TWO + TWO = EIGHTY
FIVE + FIVE + TEN + TEN + TEN + TEN + THIRTY = EIGHTY
SIXTY + EIGHT + THREE + NINE + TEN = NINETY
ONE + NINE + TWENTY + THIRTY + THIRTY = NINETY

```

Sve rezultate ispisati na standardni izlaz koristeći sledeći format komande ispisa. Primer za prvu zagonetku.

KOMANDA ISPISA REŠENJA:

```

1 print " " + str(r['T']) + str(r['R']) + str(r['E']) + str(r['E'])
2 print " " + str(r['T']) + str(r['R']) + str(r['E']) + str(r['E'])
3 print " + " + str(r['O']) + str(r['N']) + str(r['E'])
print "= " + str(r['S']) + str(r['E']) + str(r['V']) + str(r['E']) + str(r['N'])

```

Zadatak 2.17 Za svaku narednu zagonetku, napisati program koji dodeljuje različite vrednosti različitim karakterima tako da jednakost bude zadovoljena:

```

MEN * AND = WOMEN
COGITO = ERGO * SUM
((JE + PENSE) - DONC) + JE = SUIS
FERMAT * S = LAST + THEOREM.
WINNIE / THE = POOH
TWO * TWO + EIGHT = TWELVE

```

Sve rezultate ispisati na standardni izlaz koristeći sledeći format komande ispisa. Primer za prvu zagonetku.

KOMANDA ISPISA REŠENJA:

```

2 print " " + str(r['M']) + str(r['A']) + str(r['N'])
print " * " + str(r['A']) + str(r['N']) + str(r['D'])
print "= " + str(r['W']) + str(r['O']) + str(r['M']) + str(r['E']) + str(r['N'])

```

Zadatak 2.18 Uraditi sve zadatke koji su pobrojani ovde:
<http://www.primepuzzle.com/leeslatest/alphameticpuzzles.html>

Zadatak 2.19 Napisati program koji učitava ceo broj n i ispisuje magičnu sekvencu S brojeva od 0 do $n - 1$. $S = (x_0, x_1, \dots, x_{n-1})$ je magična sekvenca ukoliko postoji x_i pojavljivanja broja i za $i = 0, 1, \dots, n - 1$.

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "( ",
  for i in range(n-1):
3     print str(r["x"+str(i)]) + ", ",
  print str(r["xn-1"]) + ")"
```

Primer 1

```
POKRETANJE: python sudoku.py
INTERAKCIJA SA PROGRAMOM:
  Unesite dužinu magične sekvence
  4
  (1,2,1,0)
  (2,0,2,0)
```

Zadatak 2.20 Čistačica Mica sređuje i čisti kuće i stanove. Da bi sredila i počistila jedan stan potrebno joj je 1 sat, dok joj je za kuću potrebno 1.5 sati. Prilikom čišćenja, Mica potroši neku količinu deterdženta, 120ml po stanu, odnosno 100ml po kući. Mica zaradi 1000 dinara po svakom stanu, odnosno 1500 dinara po kući. Ukoliko Mica radi 40 sati nedeljno i ima 5l deterdženta na raspolaganju, koliko stanova i kuća je potrebno da očisti kako bi imala najveću zaradu?

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
print "Potrebno je da ocisti {0:d} kuca i {1:d} stanova da bi zarada bila najveća".
      format(r["kuca"], r["stan"])
```

Zadatak 2.21 Marija se bavi грнčarstvom i pravi šolje i tanjire. Da bi se napravila šolja, potrebno je 6 minuta, dok je za tanjir potrebno 3 minuta. Pri pravljenju šolje potroši se 75 gr, dok se za tanjir potroši 100 gr gline. Ukoliko ima 20 sati na raspolaganju za izradu svih proizvoda i 250 kg gline, a zarada koju ostvari iznosi 2 evra po svakoj šolji i 1.5 evra po tanjiru, koliko šolja i tanjira treba da napravi kako bi ostvarila maksimalnu zaradu?

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "Potrebno je da napravi {0:d} solja i {1:d} tanjira da bi zarada bila najveća".
      format(r["solja"], r["tanjir"])
```

Zadatak 2.22 Jovanin komšija preprodaje računare i računarsku opremu. Očekuje isporuku računara i štampača. Pri tom, računari su spakovani tako da njihova kutija zauzima 360 kubnih decimetara prostora, dok se štampači pakuju u kutijama koje zauzimaju 240 kubnih decimetara prostora. Komšija se trudi da mesečno proda najmanje 30 računara i da taj broj bude bar za 50% veći od broja prodatih štampača. Računari koštaju 200 evra po nabavnoj ceni, a prodaju se po ceni od 400 evra, dok štampači koštaju u nabavci 60 evra i prodaju se za 140 evra. Magacin kojim komšija raspolaže ima svega 30000 kubnih decimetara prostora i mesečno može da nabavi robu u iznosu od najviše 14000 evra. Koliko računara, a koliko štampača komšija treba da proda kako bi se maksimalno obogatio?

Sve rezultate ispisati na standardni izlaz koristeći datu komandu ispisa.

KOMANDA ISPISA REŠENJA:

```
1 print "Potrebno je da proda {0:d} stampaca i {1:d} racunara da bi se maksimalno obogatio
      ".format(r["stampac"], r["racunar"])
```

2.2 Rešenja

Rešenje 2.2

```

1 import constraint
3 problem = constraint.Problem()
# Definisemo promenljive i njihove vrednosti
5 problem.addVariable('A',range(1,10))
problem.addVariable('B',range(10))
7 problem.addVariable('C',range(10))
# Dodajemo ogranicenje da su vrednosti svih promenljivih razlicite
9 problem.addConstraint(constraint.AllDifferentConstraint())
resenja = problem.getSolutions()
11 # Znamo da minimalni kolicnik mora biti manji od 999
min_kolicnik = 999
min_resenje = {}
13 for resenje in resenja:
15     a = resenje['A']
    b = resenje['B']
17     c = resenje['C']
    kolicnik = (a*100 + b*10 + c) / (a+b+c)
19     if kolicnik < min_kolicnik:
        min_kolicnik = kolicnik
21     min_resenje = resenje
print(min_resenje['A']*100 + min_resenje['B']*10 + min_resenje['C'])

```

Rešenje 2.3

```

1 import constraint
3 problem = constraint.Problem()
# Definisemo promenljive za svaki novcic
5 # ako bi se zahtevalo da u kombinaciji bude od svake vrednosti
# bar po jedan novcic samo treba promeniti da domen za svaku
7 # promenljivu kreće od 1
problem.addVariable("1 din",range(0,51))
9 problem.addVariable("2 din",range(0,26))
problem.addVariable("5 din",range(0,11))
11 problem.addVariable("10 din",range(0,6))
problem.addVariable("20 din",range(0,3))
13
# Problem koji je uocen pri ispisu resenja je sledeci,
15 # redosled u kom ce biti dodate promenljive problemu ne
# mora uvek da odgovara redosledu kojim smo mi definisali promenljive,
17 # u konkretnom primeru (videti oblik u kom ispisuje resenje),
# promenljive ce se dodati u sledecem redosledu:
19 # '1 din', '2 din', '10 din', '20 din', '5 din'
# (nacin na koji se kljucevi organizuju u rečniku nije striktno definisan,
21 # primetimo da niske nisu sortirane)
# posledica je da postavljanje ogranicenja
23 # problem.addConstraint(constraint.ExactSumConstraint(50,[1,2,5,10,20]))
# neće ispravno dodeliti tezine, na primer,
25 # tezinu 5 dodeli promenljivoj '10 din' umesto '5 din' kako bismo ocekivali
#
27 # I nacin da se resi ovaj problem je da redosled promenljivih
# koji odgovara redosledu tezina za ExactSumConstraint prosledimo
29 # kao dodatni argument za funkciju addConstraint
#
31 problem.addConstraint(
    constraint.ExactSumConstraint(50,[1,2,5,10,20]),
33     ["1 din", "2 din", "5 din","10 din", "20 din"])
#
35 # II nacin je da definisemo svoju funkciju koja predstavlja ogranicenje, samo ce sada
# solver nesto sporije da radi posto ugradjene funkcije imaju optimizovanu
# pretragu i brze dolaze do resenja
#
37 #def o(a, b, c, d, e):
# if a + 2*b + 5*c + 10*d + 20*e == 50:
39 #     return True

```

```
#
41 #problem.addConstraint(o, ["1 din", "2 din", "5 din","10 din", "20 din"])
#
43 resenja = problem.getSolutions()

45 for r in resenja:
    print("----")
    print("""1 din: {0:d}
2 din: {1:d}
49 5 din: {2:d}
10 din: {3:d}
51 20 din: {4:d}""".format(r["1 din"],r["2 din"],r["5 din"], r["10 din"], r["20 din"]))
    # Provera da je suma bas 50
53 print("Ukupno:", r["1 din"] + r["2 din"]*2 + r["5 din"]*5 + r["10 din"]*10 + r["20
    din"]*20)
    print("----")
```

Rešenje 2.4

```
1 # 4 9 2
# 3 5 7
3 # 8 1 6
#

5
import constraint

7
def o(x,y,z):
9     if x+y+z == 15:
        return True

11
problem = constraint.Problem()
13 # Promenljive:
# a b c
15 # d e f
# g h i
17 problem.addVariables("abcdefghi", range(1,10))
problem.addConstraint(constraint.AllDifferentConstraint())
19 # Dodajemo ogranicenja za svaku vrstu
problem.addConstraint(o,"abc")
21 problem.addConstraint(o,"def")
problem.addConstraint(o,"ghi")
23 # Dodajemo ogranicenja za svaku kolonu
problem.addConstraint(o,"adg")
25 problem.addConstraint(o,"beh")
problem.addConstraint(o,"cfi")
27 #Dodajemo ogranicenja za dijagonale
problem.addConstraint(o,"aei")
29 problem.addConstraint(o,"ceg")

31 resenja = problem.getSolutions()
for r in resenja:
33     print(" ----- ")
    print("| {0:d} {1:d} {2:d} |".format(r['a'],r['b'],r['c']))
35     print("| {0:d} {1:d} {2:d} |".format(r['d'],r['e'],r['f']))
    print("| {0:d} {1:d} {2:d} |".format(r['g'],r['h'],r['i']))
37     print(" ----- ")
```

Rešenje 2.5

```
# X,Y,Z
2 #
# X >= Z
4 # X*2 + X*Y + Z <= 34
#
6 # X <- {1,2,3,...90}
# Y <- {2,4,6,...60}
8 # Z <- {1,4,9,16,...100}
#

10
import constraint
```

```

12 problem = constraint.Problem()
14 # Dodajemo promenljivu X i definisemo njen domen
16 problem.addVariable('X', range(1,91))
18 # Dodajemo promenljivu Y i definisemo njen domen
19 problem.addVariable('Y', range(2,61,2))
20
21 domenZ = [];
22 for i in range(1,11):
23     domenZ.append(i*i)
24
25 # Dodajemo promenljivu Z i definisemo njen domen
26 problem.addVariable('Z', domenZ)
27
28 def o1(x,z):
29     if x >= z:
30         return True
31
32 def o2(x,y,z):
33     if x*2 + x*y + z <= 34:
34         return True;
35
36 # Dodajemo ogranicenja
37 problem.addConstraint(o1, 'XZ')
38 problem.addConstraint(o2, 'XYZ')
39
40 resenja = problem.getSolutions()
41
42 for r in resenja:
43     print("-----")
44     print("X = {0:d} , Y = {1:d} , Z = {2:d}".format(r['X'],r['Y'],r['Z']))
45     print("-----")

```

Rešenje 2.6

```

# TWO
# +TWO
# -----
# FOUR
#
import constraint

problem = constraint.Problem()
# Definisemo promenljive i njihove vrednosti
problem.addVariables("TF",range(1,10))
problem.addVariables("WOUR",range(10))

# Definisemo ogranicenje za cifre
def o(t, w, o, f, u, r):
    if 2*(t*100 + w*10 + o) == f*1000 + o*100 + u*10 + r:
        return True

# Dodajemo ogranicenja za cifre
problem.addConstraint(o,"TWOFOUR")
# Dodajemo ogranicenje da su sve cifre razlicite
problem.addConstraint(constraint.AllDifferentConstraint())

resenja = problem.getSolutions()

for r in resenja:
    print("-----")
    print("  "+str(r['T'])+str(r['W'])+str(r['O']))
    print(" "+str(r['T'])+str(r['W'])+str(r['O']))
    print("="+str(r['F'])+str(r['O'])+str(r['U'])+str(r['R']))

```

Rešenje 2.7

```

# Dati sistem nejednacina nema resenje, tj. metog getSolutions() vraca praznu listu
# Ukoliko se za promenljivu W domen promeni na {1,...,100} sistem ce imati resenje
import constraint

problem = constraint.Problem()

# Dodajemo promenljivu X i definisemo njen domen
problem.addVariable('X', range(1,11))

# Dodajemo promenljivu Y i definisemo njen domen
problem.addVariable('Y', range(1,52,2))

domenZ = []
domenW = []

for i in range(1,11):
    domenZ.append(i*10)
    domenW.append(i**3)

# Dodajemo promenljivu Z i definisemo njen domen
problem.addVariable('Z', domenZ)

# Dodajemo promenljivu W i definisemo njen domen
problem.addVariable('W', domenW)

# Za ovako definisan domen za promenljivu W sistem ce imati resenje
# problem.addVariable('W', range(1,101))

def o1(x,w):
    if x >= 2*w:
        return True

def o2(y,z):
    if 3 + y <= z:
        return True

def o3(x,y,z,w):
    if x - 11*w + y + 11*z <= 100:
        return True;

# Dodajemo ogranicenja
problem.addConstraint(o1, 'XW')
problem.addConstraint(o2, 'YZ')
problem.addConstraint(o3, 'XYZW')

resenja = problem.getSolutions()
# Proveravamo da li postoji resenje za sistem nejednacina
if resenja==[]:
    print("Sistem nema resenje.")
else:
    for r in resenja:
        print("-----")
        print("X = {0:d} , Y = {1:d} , Z = {2:d}, W = {3:d}".format(r['X'],r['Y'],r['Z'], r['W']))
        print("-----")

```

Rešenje 2.8

```

#      1   3
#      2   4
#      5
#      6   8
#      7   9
#
import constraint

# Definisemo ogranicenje za jednu dijagonalu
def o(a,b,c,d,e):
    if a<b<c<d<e and a+b+c+d+e==25:
        return True

```

```

15 problem = constraint.Problem()
# Definiramo promenljive za svaku poziciju
17 problem.addVariables('abcdeABDE',range(1,10))
# Dodajemo ogranicenja za obe dijagonale
19 problem.addConstraint(o,'abcde')
problem.addConstraint(o,'ABcDE')
21 # Dodajemo ogranicenje da su vrednosti svih promenljivih razlicite
problem.addConstraint(constraint.AllDifferentConstraint())
23
resenja = problem.getSolutions()
25 for r in resenja:
    print("-----")
    print("{0:d}   {1:d}".format(r['a'],r['A']))
27    print(" {0:d} {1:d} ".format(r['b'],r['B']))
29    print("  {0:d}  ".format(r['c']))
    print(" {0:d} {1:d} ".format(r['D'],r['d']))
31    print("{0:d}   {1:d} ".format(r['E'],r['e']))
    print("-----")

```

Rešenje 2.9

```

1 #
# Potrebno je napraviti H komada hleba i K komada kifli
#
3 # Zarada iznosi:
# - 7din/hleb, tj. zarada za H komada hleba bice 7*H
# - 9din/kifla tj. zarada za K komada kifli bice 9*K
#
5 # Ukupna zarada iznosi:
# 7*H + 9*K - funkcija koju treba maksimizovati
#
7 # Ogranicenja vremena:
# - vreme potrebno za mesenje jednog hleba je 10min,
13 #   tj. za mesenje H komada hleba potrebno je 10*H minuta
# - vreme potrebno za mesenje jedne kifle je 12min,
15 #   tj. za mesenje K komada kifli potrebno je 12*K minuta
#
17 # Ukupno vreme koje je na raspolaganju iznosi 20h, tako da je:
# 10*H + 12*K <= 1200
19 #
# Ogranicenje materijala:
21 # - za jedan hleb potrebno je 300g brasna, a za H komada hleba potrebno je H*300
   grama
# - za jednu kifli potrebno je 120g brasna, a za K komada kifli potrebno je K*120
   grama
23 #
# Ukupno, na raspolaganju je 20kg brasna, tako da je:
25 # 300*H + 120*K <= 20000
#
27 # Broj kifli i hleba je najmanje 0, tako da:
# H>=0
29 # K>=0
#
31 # S obzirom na to da imamo 20kg brasna na raspolaganju, mozemo napraviti:
# - najvise 20000/120 kifli
33 # - najvise 20000/300 hleba
#
35 # H <= 20000/120 ~ 167
# K <= 20000/300 ~ 67
37 #
# S obzirom na to da imamo 20h na raspolaganju, mozemo napraviti:
39 # - najvise 1200/12 kifli
# - najvise 1200/10 hleba
41 #
# H <= 1200/10 = 120
43 # K <= 1200/12 = 100
#
45 # najoptimalnije je za gornju granicu domena postaviti
# minimum od dobijenih vrednosti,
47 # tj. sve ukupno H <= 120, K <= 67

```

```

49 import constraint

51 problem = constraint.Problem()

53 # Dodajemo promenljivu H i definisemo njen domen
problem.addVariable('H', range(0,121))

55 # Dodajemo promenljivu K i definisemo njen domen
57 problem.addVariable('K', range(0,68))

59 def ogranicenje_vremena(h,k):
    if 10*h + 12*k <= 1200:
61         return True

63 def ogranicenje_materijala(h,k):
    if 300*h + 120*k <= 20000:
65         return True;

67 # Dodajemo ogranicenja vremena i materijala
problem.addConstraint(ogranicenje_vremena, 'HK')
69 problem.addConstraint(ogranicenje_materijala, 'HK')

71 resenja = problem.getSolutions()

73 # Pronalazimo maksimalnu vrednost funkcije cilja
max_H = 0
75 max_K = 0

77 for r in resenja:
    if 7*r['H'] + 9*r['K'] > 7*max_H + 9*max_K:
79         max_H = r['H']
            max_K = r['K']

81
83 print("-----")
print("Maksimalna zarada je {0:d}, komada hleba je {1:d}, a komada kifli {2:d}".
    format(7*max_H + 9*max_K, max_H, max_K))
print("-----")

```

Rešenje 2.10

```

1
import constraint

3
def o1(x,y,z):
    if x+y+z == 38:
5         return True

7 def o2(x,y,z,w):
    if x+y+z+w == 38:
9         return True

11 def o3(x,y,z,w,h):
    if x+y+z+w+h == 38:
        return True

13
problem = constraint.Problem()
15 problem.addVariables("ABCDEFGHIJKLMNOPQRST", range(1,38))
problem.addConstraint(constraint.AllDifferentConstraint())

17 # Dodajemo ogranicenja za svaku horizontalnu liniju
# A,B,C
19 # D,E,F,G
#H,I,J,K,L
21 # M,N,O,P
# Q,R,S

23 problem.addConstraint(o1,"ABC")
problem.addConstraint(o2,"DEFG")
25 problem.addConstraint(o3,"HIJKL")
problem.addConstraint(o2,"MNOP")
27 problem.addConstraint(o1,"QRS")

29 # Dodajemo ogranicenja za svaku od glavnih dijagonala
# A,B,C
31 # D,E,F,G
#H,I,J,K,L

```

```

33 # M,N,O,P
34 # Q,R,S
35
36 problem.addConstraint(o1,"HMQ")
37 problem.addConstraint(o2,"DINR")
38 problem.addConstraint(o3,"AEJOS")
39 problem.addConstraint(o2,"BFKP")
40 problem.addConstraint(o1,"CGL")
41
42 # Dodajemo ogranicjenja za svaku od sporednih dijagonala
43 # A,B,C
44 # D,E,F,G
45 #H,I,J,K,L
46 # M,N,O,P
47 # Q,R,S
48
49 problem.addConstraint(o1,"ADH")
50 problem.addConstraint(o2,"BEIM")
51 problem.addConstraint(o3,"CFJNQ")
52 problem.addConstraint(o2,"GKOR")
53 problem.addConstraint(o1,"LPS")
54
55 resenja = problem.getSolutions()
56 for r in resenja:
57     print(" -----")
58     print(" {0:d},{1:d},{2:d}".format(r['A'],r['B'],r['C']))
59     print(" {0:d},{1:d},{2:d},{3:d}".format(r['D'],r['E'],r['F'],r['G']))
60     print(" {0:d},{1:d},{2:d},{3:d},{4:d}".format(r['H'],r['I'],r['J'],r['K'],r['L']))
61     print(" {0:d},{1:d},{2:d},{3:d}".format(r['M'],r['N'],r['O'],r['P']))
62     print(" {0:d},{1:d},{2:d}".format(r['Q'],r['R'],r['S']))
63     print(" -----")

```

Rešenje 2.11

```

import constraint
2
problem = constraint.Problem()
3 # Kompanija ima 250 zaposlenih radnika
4 # za sve njih organizuje dodatnu obuku
5 # ako je E promenljiva za Elixir, a D za Dart
6 # mora da vazi E<=250, D<=250 i E + D = 250
7 #
8 # Dodajemo promenljivu E i definisemo njen domen
9 problem.addVariable('E', range(0,251))
10
11 # Dodajemo promenljivu D i definisemo njen domen
12 problem.addVariable('D', range(0,251))
13
14 def ukupno_radnika(e,d):
15     if e+d == 250:
16         return True
17
18 def ogranicenje_projekat_sati(e,d):
19     if 150*e + 170*d <= 51200:
20         return True
21
22 def ogranicenje_sredstava(e,d):
23     if 100*e + 105*d <= 26000:
24         return True;
25
26 # Dodajemo ogranicjenja za broj projekat/sati i ukupna sredstva
27 # na raspolaganju kao i za broj radnika u firmi
28 problem.addConstraint(ogranicenje_projekat_sati, 'ED')
29 problem.addConstraint(ogranicenje_sredstava, 'ED')
30 problem.addConstraint(ukupno_radnika, 'ED')
31
32 resenja = problem.getSolutions()
33
34 # Pronalazimo maksimalnu vrednost funkcije cilja
35 max_E = 0
36 max_D = 0
37
38 # Od ostvarene dobiti preko broja projekat/sati oduzimamo gubitak za placanje kurseva

```



```

        radnicima
for r in resenja:
    if 150*5*r['E'] + 170*6*r['D'] - (100*r['E'] + 105*r['D']) > 150*5*max_E + 170*6*
max_D - (100*max_E + 105*max_D) :
        max_E = r['E']
        max_D = r['D']

print("Maksimalna zarada je {0:d}, broj radnika koje treba poslati na kurs Elixir je
{1:d}, a broj radnika koje treba poslati na kurs Dart je {2:d}.".format(170*6*
max_E + 150*5*max_D - (100*max_E + 105*max_D) , max_E, max_D))

```

Rešenje 2.12

```

1 # Jedan od mogućih rasporeda
2 # sva ostala resenja su permutacije
3 # takve da je u svakoj vrsti i koloni samo jedan top
4 #
5 # 8 T - - - - -
6 # 7 - T - - - -
7 # 6 - - T - - -
8 # 5 - - - T - -
9 # 4 - - - - T -
10 # 3 - - - - - T
11 # 2 - - - - - T
12 # 1 - - - - - T
13 # 1 2 3 4 5 6 7 8
14 #
15
16 import constraint
17
18 problem = constraint.Problem()
19 # Dodajemo promenljive za svaku kolonu i njihove vrednosti 1-8
20 problem.addVariables("12345678", range(1,9))
21 # Dodajemo ograničenje da se topovi ne napadaju medjusobno po svakoj vrsti
22 problem.addConstraint(constraint.AllDifferentConstraint())
23 resenja = problem.getSolutions()
24
25 # Broj svih mogućih permutacija je 8! = 40320
26 # Za prikaz svih najbolje pozvati program sa preusmerenjem izlaznih podataka
27 # python 2_12.py > izlaz.txt
28 print("Broj resenja je: {0:d}.".format(len(resenja)))
29
30 for r in resenja:
31     print("-----")
32     for i in "12345678":
33         for j in range(1,9):
34             if r[i] == j:
35                 print("T", end='')
36             else:
37                 print("-", end='')
38         print("")
39     print("-----")

```

Rešenje 2.13

```

1 # 8 D - - - - -
2 # 7 - - - D - -
3 # 6 - D - - - -
4 # 5 - - - - D -
5 # 4 - - D - - -
6 # 3 - - - - D -
7 # 2 - - - D - -
8 # 1 - - - - - D
9 # 1 2 3 4 5 6 7 8
10 #
11
12 import constraint
13 import math

```

```

15 problem = constraint.Problem()
# Dodajemo promenljive za svaku kolonu i njihove vrednosti 1-8
17 problem.addVariables("12345678", range(1,9))
# Dodajemo ogranicenje da se dame ne napadaju medjusobno po svakoj vrsti
19 problem.addConstraint(constraint.AllDifferentConstraint())

21 for k1 in range(1,9):
    for k2 in range(1,9):
23         if k1 < k2:
            # Definisemo funkciju ogranicenja za dijagonale
25             def o(vrsta1, vrsta2, kolona1=k1, kolona2=k2):
                if math.fabs(vrsta1 - vrsta2) != math.fabs(kolona1 - kolona2):
27                     return True
            problem.addConstraint(o, [str(k1),str(k2)])
29
resenja = problem.getSolutions()
31 # Za prikaz svih najbolje pozvati program sa preusmerenjem izlaznih podataka
# python 2_13.py > izlaz.txt
33 print("Broj resenja je: {0:d}.".format(len(resenja)))
for r in resenja:
35     print("-----")
    for i in "12345678":
37         for j in range(1,9):
            if r[i] == j:
39                 print("D", end='')
            else:
41                 print("-", end='')
        print("")
43     print("-----")

```

Rešenje 2.14

```

1 # 9 - - - - -
# 8 - - - - -
3 # 7 - - - - -
# 6 - - - - -
5 # 5 - - - - -
# 4 - - - - -
7 # 3 - - - - -
# 2 - - - - -
9 # 1 - - - - -
# 1 2 3 4 5 6 7 8 9
11 #

13 import constraint
import json

15 problem = constraint.Problem()

17 # Dodajemo promenljive za svaki red i njihove vrednosti 1-9
19 for i in range(1, 10):
    problem.addVariables(range(i * 10 + 1, i * 10 + 10), range(1, 10))

21 # Dodajemo ogranicenja da se u svakoj vrsti nalaze razlicite vrednosti
23 for i in range(1, 10):
    problem.addConstraint(constraint.AllDifferentConstraint(), range(i * 10 + 1, i *
    10 + 10))

25 # Dodajemo ogranicenja da se u svakoj koloni nalaze razlicite vrednosti
27 for i in range(1, 10):
    problem.addConstraint(constraint.AllDifferentConstraint(), range(10 + i, 100 + i,
    10))

29 # Dodajemo ogranicenja da svaki podkvadrat od 3x3 promenljive ima razlicite vrednosti
31 for i in [1,4,7]:
    for j in [1,4,7]:
33         pozicije = [10*i+j,10*i+j+1,10*i+j+2,10*(i+1)+j,10*(i+1)+j+1,10*(i+1)+j
        +2,10*(i+2)+j,10*(i+2)+j+1,10*(i+2)+j+2]
        problem.addConstraint(constraint.AllDifferentConstraint(),pozicije)
35

37 ime_datoteke = input("Unesite ime datoteke sa tablom za sudoku: ")

```

```
f = open(ime_datoteke, "r")
39 tabla = json.load(f)
f.close()

41
42 # Dodajemo ogranicenja za svaki broj koji je zadat na tabli
43 for i in range(9):
44     for j in range(9):
45         if tabla[i][j] != 0:
46             def o(vrednost_promenljive, vrednost_na_tabli = tabla[i][j]):
47                 if vrednost_promenljive == vrednost_na_tabli:
48                     return True
49
50             problem.addConstraint(o, [((i+1)*10 + (j+1))])
51
52 resenja = problem.getSolutions()
53
54 for r in resenja:
55     print("=====")
56     for i in range(1,10):
57         print("|", end='')
58         for j in range(1,10):
59             if j%3 == 0:
60                 print(str(r[i*10+j])+" | ", end='')
61             else:
62                 print(str(r[i*10+j]), end='')
63         print("")
64         if i%3 == 0 and i!=9:
65             print("-----")
66     print("=====")
```

3

Funkcionalni koncepti u programskom jeziku Python

Literatura:

(a) <https://docs.python.org/3.7/howto/functional.html>

3.1 Funkcionalno programiranje

3.1.1 Uvodni primeri

Zadatak 3.1 Napisati program koji sa standardnog ulaza učitava dva broja i na standardni izlaz ispisuje njihov zbir.

```
1 def suma(a, b):  
    return a + b  
3  
suma2 = lambda a, b : a + b  
5  
a = int(input("Unesi a: "))  
7 b = int(input("Unesi b: "))  
print("suma: {}".format(suma(a, b)))  
9 print("suma1: {}".format(suma2(a, b)))
```

Zadatak 3.2 Napisati funkciju `parovi(a,b,c, d)` koja generiše listu parova celih brojeva (x, y) , za koje x pripada segmentu $[a, b]$, a y pripada segmentu $[c, d]$. Testirati rad funkcije pozivom u programu.

```
1 a = int(input("Unesi a: "))  
b = int(input("Unesi b: "))  
3 c = int(input("Unesi c: "))  
d = int(input("Unesi d: "))  
5  
lista = []  
7 for i in range(a, b):  
    for j in range(c, d):  
9         lista.append((i, j))  
print(lista)  
11  
lista1 = [(i, j) for i in range(a, b) for j in range(c, d)]  
13 print(lista1)
```

Zadatak 3.3 Marko, Petar i Pavle su polagali ispit iz predmeta programske paradigme. Napisati program koji sa standardnog ulaza učitava ocene koji su dobili, a potom ispisuje listu parova (student, ocena) na standardni izlaz.

```
1 imena = ["Marko", "Petar", "Pavle"]  
ocene = [int(input("unesi ocenu koju je dobio "+ime+": ")) for ime in imena]  
3
```

```
print(zip(imena, ocene))
5
for par in zip(imena, ocene):
7     print(par)
```

Zadatak 3.4 Napisati program koji sa standardnog ulaza učitava nisku, a na standardni izlaz ispisuje nisku u kojoj su sva mala slova pretvorena u velika.

```
1 # pseudo kod za map.
2 # def map(func, seq):
3 #     # vraca `Map` objekat gde
4 #     # funkcija func primenjena na svaki element
5 #     return Map(
6 #         func(x)
7 #         for x in seq
8 #     )
9 niska = input("Unesi zeljenu nisku: ")
10 print(map(lambda c: c.upper(), niska)) # map object
11 print(list(map(lambda c: c.upper(), niska))) # lista karaktera
12 print(''.join(list(map(lambda c: c.upper(), niska)))) # niska
```

Zadatak 3.5 Napisati program koji sa standardnog ulaza učitava nisku, a na standardni izlaz ispisuje sve karaktere koji nisu slova.

```
1 # Funkcija filter
2 # - kao prvi argument uzima funkciju uslova (funkcija koja vraca logicku vrednost)
3 # i izdvaja iz liste koju uzima kao drugi argument
4 # sve elemente koji zadovoljavaju zadat uslov
5 #
6 # def filter(evaluate , seq):
7 #     return Map(
8 #         x for x in seq
9 #         if evaluate(x) is True
10 #     )
11 niska = input("Unesite zeljenu nisku: ")
12 print(list(filter(lambda c: not c.isalpha(), niska))) # lista karaktera koji nisu slova
```

Zadatak 3.6 Napisati program koji na standardni izlaz ispisuje sumu, koristeći funkciju `reduce` prvih n prirodnih brojeva gde se n unosi sa standardnog ulaza.

```
1 # funkcija koja odgovara funkciji fold u Haskelu je funkcija reduce
2 # ona se nalazi u modulu functools
3 from functools import reduce
4
5 try:
6     n = int(input("Unesi n: "))
7 except ValueError:
8     print("Error: cast error")
9
10 brojevi = range(0, n+1)
11 print("Suma(prvih {}) = {}".format(n, reduce(lambda x, y: x+y, brojevi)))
```

3.1.2 Zadaci za samostalni rad sa rešenjima

Zadatak 3.7 U datoteci `korpa.json` se nalazi spisak kupljenog voća u json formatu:

```
1 [ { 'ime' : ime_voca, 'kolicina' : broj_kilograma } , ... ]
```

U datotekama `maxi_cene.json`, `idea_cene.json`, `shopngo_cene.json` se nalaze cene voća u json formatu:

```
1 [ { 'ime' : ime_voca, 'cena' : cena_po_kilogramu } , ... ]
```

Napisati program koji izračunava ukupan račun korpe u svakoj prodavnici i ispisuje cene na standardni izlaz.

Primer 1

```

|| POKRETANJE: python korpa.py
|| INTERAKCIJA SA PROGRAMOM:
||   Maxi: 631.67 dinara
||   Idea: 575.67 dinara
||   Shopngo: 674.67 dinara

```

Sadržaj datoteka koje se koriste u primeru 3.7:

Listing 3.1: *korpa.json*

```

1 [ {"ime" : "jabuke" , "kolicina": 3.3},
2 {"ime" : "kruske" , "kolicina": 2.1},
3 {"ime" : "grozdje" , "kolicina": 2.6},

```

Listing 3.2: *maksi_cene.json*

```

1 [ {"ime" : "jabuke" , "cena" : 59.9},
2 {"ime" : "kruske" , "cena" : 120},
3 {"ime" : "grozdje" , "cena" : 70},
4 {"ime" : "narandze" , "cena" : 49.9},
5 {"ime" : "breskve" , "cena" : 89.9} ]

```

Listing 3.3: *idea_cene.json*

```

1 [ {"ime" : "jabuke" , "cena" : 39.9},
2 {"ime" : "kruske" , "cena" : 100},
3 {"ime" : "grozdje" , "cena" : 90},
4 {"ime" : "breskve" , "cena" : 59.9} ]

```

Listing 3.4: *shopngo_cene.json*

```

1 [ {"ime" : "jabuke" , "cena" : 69.9},
2 {"ime" : "kruske" , "cena" : 100},
3 {"ime" : "grozdje" , "cena" : 90},
4 {"ime" : "maline" , "cena" : 290},

```

[Rešenje 3.7]

Zadatak 3.8 U datoteci *tacke.json* se nalaze podaci o tačkama u sledećem formatu.

Listing 3.5: *tacke.json*

```

1 [ {"teme": teme , "koordinate": [x, y]}, ... ]

```

Napisati program koji učitava podatke o tačkama iz datoteke *tacke.json* i sortira i po udaljenosti od koordinatnog početka opadajuće. Program na standardni izlaz treba da ispiše tačku iz zadatog kvadranta, koja je najudaljenija od koordinatnog početka. Kvadrant se zadaje kao argument komandne linije.

Primer 1

```

|| POKRETANJE: python tacke.py 1
|| INTERAKCIJA SA PROGRAMOM:
||   B

```

Sadržaj datoteke koja se koristi u primeru 3.8:

Listing 3.6: *tacke.json*

```

1 [ {"teme": "A" , "koordinate": [10.0, 1.1]},
2 {"teme": "B" , "koordinate": [1.0, 15.0]},
3 {"teme": "C" , "koordinate": [-1.0, 5.0]} ]

```

[Rešenje 3.8]

3.1.3 Zadaci za vežbu

Zadatak 3.9 U datoteci *knjige.json* se nalaze podaci o knjigama u knjižari u sledećem formatu.

Listing 3.7: *knjige.json*

```
1 [ {"oznaka": broj , "naziv_autor": naziv knjige i autor, "kolicina":  
    kolicina, "cena": cena po komadu}, ...]
```

Napisati program koji ispisuje listu parova (oznaka, ukupna vrednost). Ukupna vrednost se sračunava na osnovu cene i broja primeraka, a povećava se za 10 dinara ukoliko ukupna cena knjiga sa istom oznakom ne prelazi minimalnu cenu od 100 dinara.

Primer 1

```
POKRETANJE: python knjige.py  
INTERAKCIJA SA PROGRAMOM:  
[('34587', 163.8), ('98762', 284.0), ('77226', 108.85000000000001), ('88112', 84.97)]
```

Sadržaj datoteke koja se koristi u primeru 3.9:

Listing 3.8: *knjige.json*

```
1 [ {"oznaka": 34587, "naziv_autor": "Learning Python, Mark Lutz", "  
    kolicina": 4, "cena": 40.95},  
2 {"oznaka": 98762, "naziv_autor": "Programming Python, Mark Lutz", "  
    kolicina": 5, "cena": 56.80},  
3 {"oznaka": 77226, "naziv_autor": "Head First Python, Paul Barry", "  
    kolicina": 3, "cena": 32.95},  
4 {"oznaka": 88112, "naziv_autor": "Einfuhrung in Python3, Bernd Klein",  
    "kolicina": 3, "cena": 24.99} ]
```

Zadatak 3.10 Datoteka *fudbaleri.json* sadrži podatke o fudbalerima (ime, nacionalnost i broj golova) u sledećem formatu:

```
1 [ { "Ime" : "Alexis Sanchez", "Nacionalnost" : "Cile", "Golovi" : 17 } ,  
    ...]
```

- (a) Definirati funkciju *uporedi* koristeći *lambda* izraz, koja poredi dva fudbalera po broju postignutih golova. Funkcija vraća -1, 0 ili 1 ukoliko je prvi fudbaler postigao manji, jednak i veći broj golova u odnosu na drugog fudbalera
- (b) Napisati program koji iz izabrane datoteke izdvaja fudbalere određene nacionalnosti i sortira ih rastuće po broju golova. Željena nacionalnost (npr. 'Engleska') i ime datoteke u kojoj se nalaze podaci o fudbalerima se zadaju kao argumenti komandne linije, a rezultat rada programa se upisuje u datoteku *izabrana_nacionalnost.json* (npr. *Engleska_nacionalnost.json*). U slučaju greške prilikom pokretanja programa, ispisati tekst **Greska** na standardni izlaz.

Primer 1

```
POKRETANJE: python 1.py Engleska arsenal.json  
ENGLESKA_NACIONALNOST.JSON  
[{"Nacionalnost": "Engleska",  
  "Ime": "Alex Oxlade-Chamberlain",  
  "Golovi": 2},  
 {"Nacionalnost": "Engleska",  
  "Ime": "Theo Walcott", "Golovi": 8}]
```

Sadržaj datoteke koja se koristi u primeru 3.10:

Listing 3.9: *fudbaleri.json*

```
1  
2 [{"Nacionalnost": "Cile", "Ime": "Alexis Sanchez", "Golovi" : 17 },
```

```

3 {"Nacionalnost": "Francuska", "Ime": "Olivier Giroud", "Golovi" : 8},
4 {"Nacionalnost": "Engleska", "Ime": "Theo Walcott", "Golovi" : 8},}
5 {"Nacionalnost": "Engleska", "Ime": "Alex Oxlade-Chamberlain", "Golovi"
  : 2},
6 {"Nacionalnost": "Francuska", "Ime": "Laurent Koscielny", "Golovi" : 2
  } ]

```

Zadatak 3.11 Datoteka `utakmice.json` sadrži podatke o utakmicama (timovi koji se takmiče i vreme početka utakmice) za svaki sport posebno u sledećem formatu:

```

1 [ { "Timovi": "Liverpool-Arsenal", "Vreme": "18:00"} , ... ]

```

- Napisati funkciju `u_vremenskom_intervalu(utakmice, pocetak, kraj)` koja vraća listu utakmica čiji se početak nalazi u vremenskom opsegu početak kraj. Funkciju implementirati bez korišćenja petlji.
- Napisati program koji sa standardnog ulaza učitava podatke o početku i kraju vremenskog intervala u formatu `%H:%M` i iz datoteka učitava podatke o utakmicama za sve sportove, i na standardni izlaz ispisuje listu utakmica čije se vreme početka nalazi u opsegu unešenog vremenskog intervala.

Primer 1

```

POKRETANJE: python 1.py
ULAZ:
  Unesite početak intervala: 17:00
  Unesite kraj intervala: 18:00
IZLAZ:
  {"Timovi": "Liverpool-Arsenal", "Vreme": "18:00"}
  {"Timovi": "Milan-Cheivo", "Vreme": "17:00"}
  {"Timovi": "Koln-Bayern", "Vreme": "17:30"}

```

Sadržaj datoteke koja se koristi u primeru 3.11:

Listing 3.10: `utakmice.json`

```

1 [{"Timovi": "Liverpool-Arsenal", "Vreme": "18:00"},
2 {"Timovi": "Milan-Cheivo", "Vreme": "17:00"},
3 {"Timovi": "Eibar-Real Madrid", "Vreme": "18:30"},
4 {"Timovi": "Roma-Napoli", "Vreme": "16:15"},
5 {"Timovi": "Koln-Bayern", "Vreme": "17:30"} ]

```

Zadatak 3.12 Napisati program koji na standardni izlaz ispisuje apsolutne putanje do svih datoteka koje sadrže reč koja se unosi sa standardnog ulaza ili u slučaju da ni jedna datoteka ne sadrži zadatu reč ispisati poruku `Ni jedna datoteka ne sadrzi zadatu rec`. Program napisati korišćenjem lambda izraza i bez korišćenja petlji

Primer 1

```

POKRETANJE: python 1.py
ULAZ:
  Unesite zadatu rec: include
IZLAZ:
  /home/student/programiranje2/ispit/1.c
  /home/student/programiranje2/ispit/2.c
  /home/student/programiranje2/ispit/3.c

```

Primer 1

```

POKRETANJE: python 1.py
ULAZ:
  Unesite zadatu rec: Lenovo
IZLAZ:
  Ni jedna datoteka ne sadrzi zadatu rec

```


3.2 Rešenja

Rešenje 3.7

```
import json
2 from functools import reduce

4 def cena_voca(prodavnica, ime_voca): # [{"ime" : "jabuke", "cena" : 69.9},...], "
    jabuke"
    for voce in prodavnica:
6         if voce['ime']==ime_voca:
            return voce['cena']

8 # Ucitavam podatke
10 try:
    with open('korpa.json', 'r') as f: # [{"ime" : "jabuke", "kolicina": 3.3},...]
12         korpa = json.load(f)
    with open('maxi_cene.json', 'r') as f:
14         maxi_cene = json.load(f)
    with open('idea_cene.json', 'r') as f:
16         idea_cene = json.load(f)
    with open('shopngo_cene.json', 'r') as f:
18         shopngo_cene = json.load(f)
except IOError:
20     print("Error: open json file")
    exit(1)

22 maxi_racun = reduce(lambda a, b: a+b, [voce['kolicina']*cena_voca(maxi_cene, voce['
    ime']) for voce in korpa])
24 idea_racun = reduce(lambda a, b: a+b, [voce['kolicina']*cena_voca(idea_cene, voce['
    ime']) for voce in korpa])
shopngo_racun = reduce(lambda a, b: a+b, [voce['kolicina']*cena_voca(shopngo_cene,
    voce['ime']) for voce in korpa])

26 print("Racuni u prodavnicama: ")
28 print("Maxi: " + str(maxi_racun) + " dinara")
print("Idea: " + str(idea_racun) + " dinara")
30 print("Shopngo: " + str(shopngo_racun) + " dinara")
```

Rešenje 3.8

```
1 import json
import math
3 import sys
from functools import partial

5 # funkcija koja proverava da li tacka pripada kvadrantu
7 def pripada(kvadrant, tacka): # 1, {'teme': 'A', 'koordinate': [10.0, 1.1]}
    koordinate = tacka["koordinate"]
9     if kvadrant == 1:
        return koordinate[0] >=0 and koordinate[1] >=0
11     elif kvadrant == 2:
        return koordinate[0] <=0 and koordinate[1] >=0
13     elif kvadrant == 3:
        return koordinate[0] <=0 and koordinate[1] <=0
15     elif kvadrant == 4:
        return koordinate[0] >=0 and koordinate[1] <=0
17     else:
        print("Error: Kvadrant nije ispravno unet")
        exit()

19
21 if len(sys.argv) != 2:
    print("Niste naveli dovoljno argumenta komandne linije: kvadrant")
23     exit(1)

25 try:
    with open("tacke.json","r") as f: # [{'teme': 'A', 'koordinate': [10.0, 1.1]},
        ...]
27         tacke = json.load(f)
except IOError:
```

```
29 print('Error: open json file')
   exit(1)
31
32 try:
33     kvadrant = int(sys.argv[1])
34 except ValueError:
35     print('Error: invalid cast')
36     exit(1)
37
38 # sa partial fiksiramo jedan argument: kvadrant
39 # partial vraca objekat koji se moze pozvati kao funkcija (u konkretnom slucaju
   jednog argumenta)
40 uslov = partial(pripada, kvadrant)
41 kvadrant_tacke = list(filter(uslov, tacke)) # listu dobijamo od filter objecta
42
43 # Ispis
44 if len(kvadrant_tacke) > 0:
45     print(min(kvadrant_tacke, key = lambda x: x['koordinate'][0]**2 + x['koordinate'
   ][1]**2))
46 else:
47     print("Trazena tacka ne postoji")
48
49 # Eventualno resavanje zadatka sortiranjem: (nije efikasno)
50 # sortirane_tacke = sorted(tacke, key = lambda x: x['koordinate'][0]**2 + x['
   koordinate'][1]**2)
51 # print(sortirane_tacke)
```


4

Komponentno programiranje u programskom jeziku Python

Potrebno je imati instalirano:

- (a) Python (verzija 3.7) <https://www.python.org/>
- (b) pip3 (python3-pip, može se instalirati preko apt-a za *ubuntu distribucije)

4.1 PyQt5 - Instalacija potrebnih alata

4.1.1 Instalacija PyQt5

Za distribucije zasnovane na Ubuntu distribuciji, dovoljno je pokrenuti sledeće komande:

```
1 $ pip3 install --user pyqt5
2 $ sudo apt-get install python3-pyqt5
3 $ sudo apt-get install pyqt5-dev-tools
4 $ sudo apt-get install qttools5-dev-tools
```

Slično važi i za ostale distribucije.

Kada instaliramo PyQt5 alate, možemo kreirati .ui fajlove koristeći Qt Designer. Kreirane .ui fajlove možemo zatim prevesti u Python3 kod tako što koristimo alat `pyuic5` na sledeći način:

```
$ pyuic5 ui_fajl.ui
```

Opcije (videti `man pyuic5`):

- `-o ime_fajla.py` - specificira ime generisanog fajla
- `-p` - samo prikazuje UI, bez prevoda u Python kod
- `-x` - dodatno generiše kod koji dozvoljava pokretanje generisanog Python fajla

Često ćemo pokretati `pyuic5` na sledeći način:

```
1 $ pyuic5 ui_fajl.ui -x -o ui_fajl.py
```

4.1.2 Instalacija dodatka za Visual Studio Code

<https://marketplace.visualstudio.com/items?itemName=zhoufeng.pyqt-integration>

Potrebno je podesiti lokaciju Qt Designer izvršnog fajla u podešavanjima dodatka. Ukoliko je izabrana podrazumevana lokacija prilikom instalacije Qt5, Qt alati se nalaze na lokaciji `/usr/lib/qt5/bin`. Dakle, putanja do Qt Designer-a bi u tom slučaju bila: `/usr/lib/qt5/bin/designer`.

4.2 Uvodni primeri

Zadatak 4.1 Napisati program koji ilustruje osnovni rad sa komponentama PyQt5 okruženja, integrišući par osnovnih komponenti po izboru.



```

1  # -*- coding: utf-8 -*-
2
3  # Form implementation generated from reading ui file '1.ui'
4  #
5  # Created by: PyQt5 UI code generator 5.12.3
6  #
7  # WARNING! All changes made in this file will be lost!
8
9
10 from PyQt5 import QtCore, QtGui, QtWidgets
11
12
13 class Ui_MainWindow(object):
14     def setupUi(self, MainWindow):
15         MainWindow.setObjectName("MainWindow")
16         MainWindow.resize(393, 529)
17         self.centralwidget = QtWidgets.QWidget(MainWindow)
18         self.centralwidget.setObjectName("centralwidget")
19         self.verticalLayoutWidget = QtWidgets.QWidget(self.centralwidget)
20         self.verticalLayoutWidget.setGeometry(QtCore.QRect(90, 20, 211, 241))
21         self.verticalLayoutWidget.setObjectName("verticalLayoutWidget")
22         self.verticalLayout = QtWidgets.QVBoxLayout(self.verticalLayoutWidget)
23         self.verticalLayout.setContentsMargins(0, 0, 0, 0)
24         self.verticalLayout.setObjectName("verticalLayout")
25         self.pushButton = QtWidgets.QPushButton(self.verticalLayoutWidget)
26         self.pushButton.setObjectName("pushButton")
27         self.verticalLayout.addWidget(self.pushButton)
28         self.checkBox = QtWidgets.QCheckBox(self.verticalLayoutWidget)
29         self.checkBox.setObjectName("checkBox")
30         self.verticalLayout.addWidget(self.checkBox)
31         self.radioButton = QtWidgets.QRadioButton(self.verticalLayoutWidget)
32         self.radioButton.setObjectName("radioButton")
33         self.verticalLayout.addWidget(self.radioButton)
34         self.label = QtWidgets.QLabel(self.centralwidget)
35         self.label.setGeometry(QtCore.QRect(96, 296, 201, 121))
36         font = QtGui.QFont()
37         font.setPointSize(20)
38         font.setItalic(True)
39         self.label.setFont(font)

```

```

41         self.label.setObjectName("label")
42         MainWindow.setCentralWidget(self.centralwidget)
43         self.menubar = QtWidgets.QMenuBar(MainWindow)
44         self.menubar.setGeometry(QtCore.QRect(0, 0, 393, 29))
45         self.menubar.setObjectName("menubar")
46         self.menuInfo = QtWidgets.QMenu(self.menubar)
47         self.menuInfo.setObjectName("menuInfo")
48         MainWindow.setMenuBar(self.menubar)
49         self.statusbar = QtWidgets.QStatusBar(MainWindow)
50         self.statusbar.setObjectName("statusbar")
51         MainWindow.setStatusBar(self.statusbar)
52         self.actionAbout = QtWidgets.QAction(MainWindow)
53         self.actionAbout.setObjectName("actionAbout")
54         self.menuInfo.addAction(self.actionAbout)
55         self.menubar.addAction(self.menuInfo.menuAction())
56
57         self.retranslateUi(MainWindow)
58         QtCore.QMetaObject.connectSlotsByName(MainWindow)
59
60     def retranslateUi(self, MainWindow):
61         _translate = QtCore.QCoreApplication.translate
62         MainWindow.setWindowTitle(_translate("MainWindow", "MainWindow"))
63         self.pushButton.setText(_translate("MainWindow", "Hello Push Button"))
64         self.checkBox.setText(_translate("MainWindow", "Hello Check Box"))
65         self.radioButton.setText(_translate("MainWindow", "Hello Radio Button"))
66         self.label.setText(_translate("MainWindow", "Hello Text Label"))
67         self.menuInfo.setTitle(_translate("MainWindow", "Hello Menu"))
68         self.actionAbout.setText(_translate("MainWindow", "Hello Menu Item"))
69
70 if __name__ == "__main__":
71     import sys
72     app = QtWidgets.QApplication(sys.argv)
73     MainWindow = QtWidgets.QMainWindow()
74     ui = Ui_MainWindow()
75     ui.setupUi(MainWindow)
76     MainWindow.show()
77     sys.exit(app.exec_())

```

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <ui version="4.0">
3      <class>MainWindow</class>
4      <widget class="QMainWindow" name="MainWindow">
5          <property name="geometry">
6              <rect>
7                  <x>0</x>
8                  <y>0</y>
9                  <width>393</width>
10                 <height>529</height>
11             </rect>
12         </property>
13         <property name="windowTitle">
14             <string>MainWindow</string>
15         </property>
16         <widget class="QWidget" name="centralwidget">
17             <widget class="QWidget" name="verticalLayoutWidget">
18                 <property name="geometry">
19                     <rect>
20                         <x>90</x>
21                         <y>20</y>
22                         <width>211</width>
23                         <height>241</height>
24                     </rect>
25                 </property>
26                 <layout class="QVBoxLayout" name="verticalLayout">
27                     <item>
28                         <widget class="QPushButton" name="pushButton">
29                             <property name="text">
30                                 <string>Hello Push Button</string>
31                             </property>
32                         </widget>
33                     </item>
34                 </layout>
35             </widget>
36         </widget>
37     </widget>
38 </ui>

```

```

35     <widget class="QCheckBox" name="checkBox">
36         <property name="text">
37             <string>Hello Check Box</string>
38         </property>
39     </widget>
40 </item>
41 <item>
42     <widget class="QRadioButton" name="radioButton">
43         <property name="text">
44             <string>Hello Radio Button</string>
45         </property>
46     </widget>
47 </item>
48 </layout>
49 </widget>
50 <widget class="QLabel" name="label">
51     <property name="geometry">
52         <rect>
53             <x>96</x>
54             <y>296</y>
55             <width>201</width>
56             <height>121</height>
57         </rect>
58     </property>
59     <property name="font">
60         <font>
61             <pointsize>20</pointsize>
62             <italic>true</italic>
63         </font>
64     </property>
65     <property name="text">
66         <string>Hello Text Label</string>
67     </property>
68 </widget>
69 </widget>
70 <widget class="QMenuBar" name="menubar">
71     <property name="geometry">
72         <rect>
73             <x>0</x>
74             <y>0</y>
75             <width>393</width>
76             <height>29</height>
77         </rect>
78     </property>
79     <widget class="QMenu" name="menuInfo">
80         <property name="title">
81             <string>Hello Menu</string>
82         </property>
83         <addaction name="actionAbout"/>
84     </widget>
85     <addaction name="menuInfo"/>
86 </widget>
87 <widget class="QStatusBar" name="statusbar"/>
88 <action name="actionAbout">
89     <property name="text">
90         <string>Hello Menu Item</string>
91     </property>
92 </action>
93 </widget>
94 <resources/>
95 <connections/>
</ui>

```

U buduću nećemo prikazivati generisane .ui fajlove.

Zadatak 4.2 Napisati program koji učitava tekst korišćenjem jednolinijskog polja za unos i isti ispisuje u labelu ispod jednolinijskog polja za unos.



```

2  # -*- coding: utf-8 -*-
3
4  # Form implementation generated from reading ui file '2.ui'
5  #
6  # Created by: PyQt5 UI code generator 5.12.3
7  #
8  # WARNING! All changes made in this file will be lost!
9
10 from PyQt5 import QtCore, QtGui, QtWidgets
11
12 class Ui_MainWindow(object):
13     def setupUi(self, MainWindow):
14         MainWindow.setObjectName("MainWindow")
15         MainWindow.resize(687, 386)
16         self.centralwidget = QtWidgets.QWidget(MainWindow)
17         self.centralwidget.setObjectName("centralwidget")
18         self.label = QtWidgets.QLabel(self.centralwidget)
19         self.label.setGeometry(QtCore.QRect(66, 26, 341, 61))
20         font = QtGui.QFont()
21         font.setFamily("Tlwg Typo")
22         font.setPointSize(14)
23         font.setItalic(True)
24         self.label.setFont(font)
25         self.label.setObjectName("label")
26         self.label_2 = QtWidgets.QLabel(self.centralwidget)
27         self.label_2.setGeometry(QtCore.QRect(130, 370, 191, 71))
28         font = QtGui.QFont()
29         font.setPointSize(20)
30         font.setItalic(True)
31         self.label_2.setFont(font)
32         self.label_2.setText("")
33         self.label_2.setObjectName("label_2")
34         self.horizontalLayoutWidget = QtWidgets.QWidget(self.centralwidget)
35         self.horizontalLayoutWidget.setGeometry(QtCore.QRect(70, 100, 571, 71))
36         self.horizontalLayoutWidget.setObjectName("horizontalLayoutWidget")
37         self.horizontalLayout = QtWidgets.QHBoxLayout(self.horizontalLayoutWidget)
38         self.horizontalLayout.setContentsMargins(0, 0, 0, 0)
39         self.horizontalLayout.setObjectName("horizontalLayout")
40         self.label_3 = QtWidgets.QLabel(self.horizontalLayoutWidget)
41         font = QtGui.QFont()
42         font.setPointSize(16)
43         font.setItalic(True)
44         self.label_3.setFont(font)
45         self.label_3.setObjectName("label_3")
46         self.horizontalLayout.addWidget(self.label_3)
47         self.lineEdit = QtWidgets.QLineEdit(self.horizontalLayoutWidget)
48         self.lineEdit.setObjectName("lineEdit")
49         self.horizontalLayout.addWidget(self.lineEdit)
50

```



```

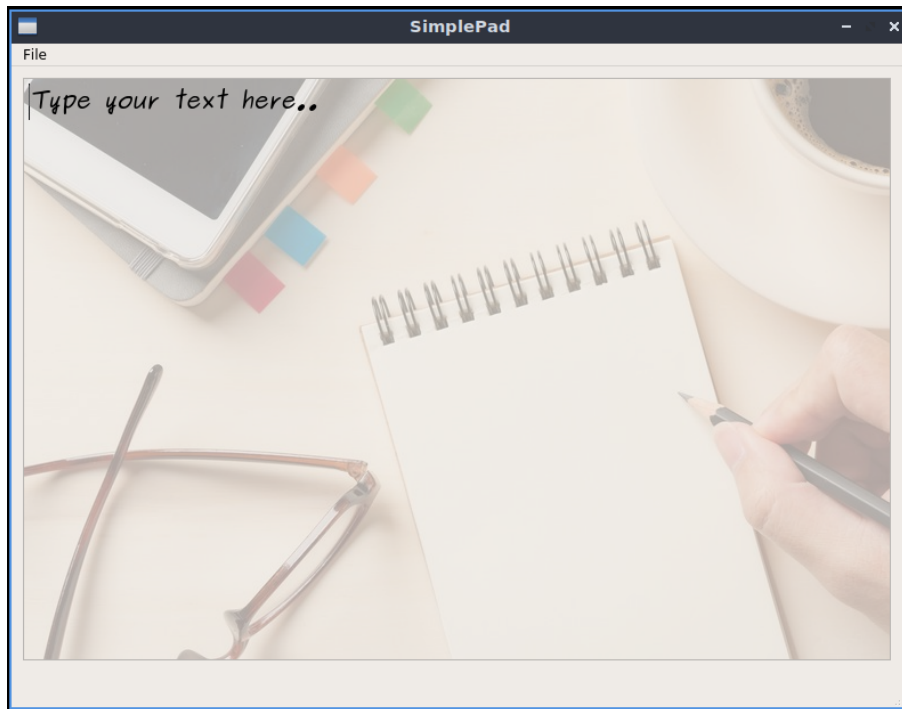
52     self.label_4 = QtWidgets.QLabel(self.centralwidget)
53     self.label_4.setGeometry(QtCore.QRect(190, 190, 291, 71))
54     font = QtGui.QFont()
55     font.setFamily("Garuda")
56     font.setPointSize(26)
57     font.setBold(True)
58     font.setItalic(True)
59     font.setUnderline(True)
60     font.setWeight(75)
61     self.label_4.setFont(font)
62     self.label_4.setText("")
63     self.label_4.setObjectName("label_4")
64     self.pushButton = QtWidgets.QPushButton(self.centralwidget)
65     self.pushButton.setGeometry(QtCore.QRect(270, 290, 111, 51))
66     self.pushButton.setObjectName("pushButton")
67     MainWindow.setCentralWidget(self.centralwidget)
68     self.menubar = QtWidgets.QMenuBar(MainWindow)
69     self.menubar.setGeometry(QtCore.QRect(0, 0, 687, 29))
70     self.menubar.setObjectName("menubar")
71     MainWindow.setMenuBar(self.menubar)
72     self.statusbar = QtWidgets.QStatusBar(MainWindow)
73     self.statusbar.setObjectName("statusbar")
74     MainWindow.setStatusBar(self.statusbar)

75
76     self.retranslateUi(MainWindow)
77     self.lineEdit.textChanged['QString'].connect(self.label_4.setText)
78     self.pushButton.clicked.connect(self.label_4.clear)
79     self.pushButton.clicked.connect(self.lineEdit.clear)
80     QtCore.QMetaObject.connectSlotsByName(MainWindow)
81
82     def retranslateUi(self, MainWindow):
83         _translate = QtCore.QCoreApplication.translate
84         MainWindow.setWindowTitle(_translate("MainWindow", "MainWindow"))
85         self.label.setText(_translate("MainWindow", "Hello World to Signal Handler"))
86         self.label_3.setText(_translate("MainWindow", "Insert your name: "))
87         self.pushButton.setText(_translate("MainWindow", "clear name"))
88
89 if __name__ == "__main__":
90     import sys
91     app = QtWidgets.QApplication(sys.argv)
92     MainWindow = QtWidgets.QMainWindow()
93     ui = Ui_MainWindow()
94     ui.setupUi(MainWindow)
95     MainWindow.show()
96     sys.exit(app.exec_())

```

Ubuduće nećemo prikazivati generisane .py fajlove.

Zadatak 4.3 Napisati program koji implementira *SimplePad* aplikaciju: jednostavan tekstualni editor koji podržava osnovne komande (kreiranja, otvaranja i cuvanja dokumenata), dodatno opcije kopiranja, lepljenja i isecanja delova teksta - bazne operacije za rad sa tekstom.



```

import ui_4_03
2 from PyQt5 import QtCore, QtGui, QtWidgets
from PyQt5.QtWidgets import QFileDialog
4 import sys
import os

6
__author__ = 'Milan Cugurovic & Ivan Ristovic, 2020.'

8
class SimplePadClass(ui_4_03.Ui_MainWindow, QtWidgets.QMainWindow):
10     def __init__(self):
        super(SimplePadClass, self).__init__()
12         self.setupUi(self)

14         # Putanja do trenurno otvorenog fajla
        self.path = None

16
        # Akcije
18         self.actionNew.triggered.connect(lambda: self.OpenMe())
        self.actionSave.triggered.connect(lambda: self.SaveMe())
20         self.actionSaveAs.triggered.connect(lambda: self.SaveAsMe())

22
        # Azuriram naslov prozora
        def __title(self):
24             self.setWindowTitle("{} - SimplePad".format(os.path.basename(self.path) if
self.path is not None else 'Untitled'))

26
        # Poruka za korisnika
        def __dialog(self, msg):
28             box = QMessageBox(self)
            box.setText(msg)
30             box.show()

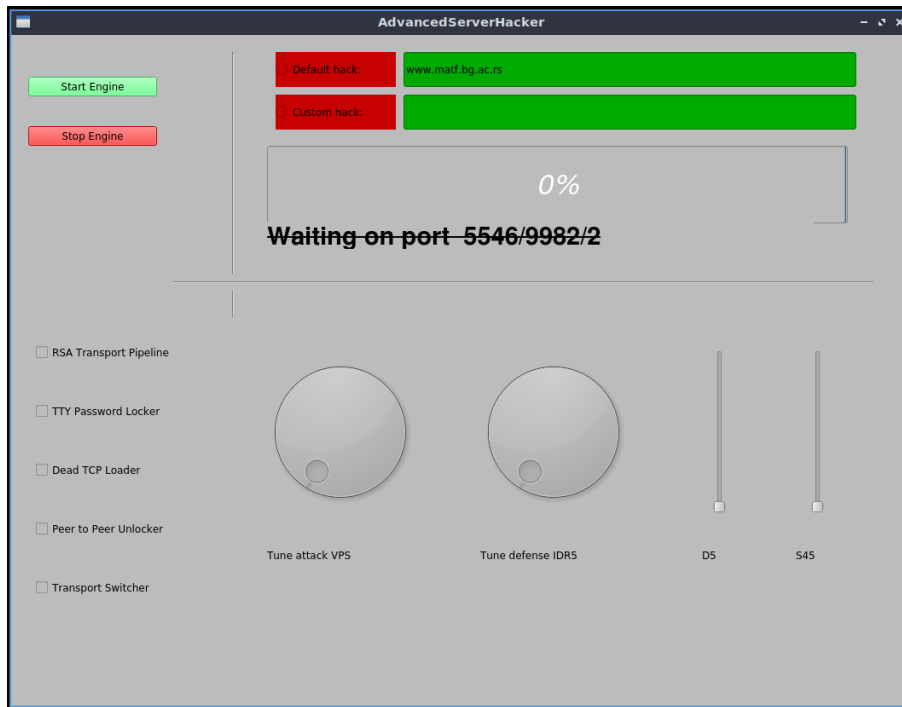
32
        # Cuvam na fajl sistemu
        def __save_util(self, path):
34             data = self.editor.toPlainText()
            try:
36                 with open(path, "w") as f:
                     f.write(data)
            except Exception as e:
38                 self.__dialog(str(e))
            else:
40                 self.path = path
42                 self.__title()

```

```
44 # ctr+O
45 def OpenMe(self):
46     path, _ = QFileDialog.getOpenFileName(self, "Open file", "", "All files (*.*)
47     ;")
48     if path:
49         try:
50             with open(path, "r") as f:
51                 data = f.read()
52             except Exception as e:
53                 self.__dialog(str(e))
54             else:
55                 self.path = path
56                 self.editor.setPlainText(data)
57                 self.__title()
58
59 # Ctr+S
60 def SaveMe(self):
61     if self.path is None:
62         return self.SaveAsMe()
63     self.__save_util(self.path)
64
65 # Ctrl+Alt+S
66 def SaveAsMe(self):
67     path, _ = QFileDialog.getSaveFileName(self, "Save file", "", "All files (*.*)
68     ;")
69     if not path:
70         return # cancelled
71     self.__save_util(path)
72
73 if __name__ == "__main__":
74     app = QtWidgets.QApplication(sys.argv)
75     simplePad = SimplePadClass()
76     simplePad.show()
77     sys.exit(app.exec_())
```

4.3 Zadaci za samostalni rad sa rešenjima

Zadatak 4.4 Napisati program koji implementira aplikaciju koja simulira hakerske napade. Aplikacija klikom na dugme za start, ažurira progres bar (od 0 do 100%), svake sekunde napredujući za po jedan procenat. Dodatno, aplikacija podržava čekiranje dodatnih opcija hakovanja, kao i prodešavanje kontinualnih parametara.



[Rešenje 4.4]

Zadatak 4.5 Implementirati digitron koji podržava osnovne aritmetičke operacije sabiranja, oduzimanja, množenja i deljenja. Implementirati brisanje unetih brojeva (back dugme) kao i resetovanje rada digitrona klikom na odgovarajuće dugme.



[Rešenje 4.5]

4.4 Rešenja

Rešenje 4.4

```
1 from PyQt5 import QtCore, QtGui, QtWidgets
2 import ui_4_04
3 import sys, os, random, string, time
4
5 __author__ = 'Milan Cugurovic & Ivan Ristovic, 2020.'
6
7 class HackerApp(ui_4_04.Ui_MainWindow, QtWidgets.QMainWindow):
8     def __init__(self):
9         super(HackerApp, self).__init__()
10        self.setupUi(self)
11
12        # Start button
13        self.start.clicked.connect(self.pressed)
14
15        def pressed(self):
16            print("engine started...")
17            for i in range(0, 100):
18                print("...{0}...".format(''.join(random.choices(string.ascii_uppercase +
19                    string.digits, k=random.randint(20, 40)))))
20                self.progressBar.setProperty("value", i)
21                if i%2==0:
22                    self.label_msg.setText(''.join(random.choices(string.ascii_uppercase
23                        + string.digits, k=random.randint(20, 40)))))
24                    time.sleep(1)
25
26 if __name__=="__main__":
27     app = QtWidgets.QApplication(sys.argv)
28     hacker = HackerApp()
29     hacker.show()
30     sys.exit(app.exec_())
```

Rešenje 4.5

```
1 import ui_4_05
2 from PyQt5 import QtCore, QtGui, QtWidgets
3 import sys
4 import re
5
6 __author__ = 'Milan Cugurovic & Ivan Ristovic, 2020.'
7
8 class CalculatorClass(ui_4_05.Ui_Calculator, QtWidgets.QMainWindow):
9     def __init__(self):
10        super(CalculatorClass, self).__init__()
11        self.setupUi(self)
12
13        # Dodajem akcije za press the number
14        self.pushButton_d0.clicked.connect(lambda: self.display_screen('0'))
15        self.pushButton_d1.clicked.connect(lambda: self.display_screen('1'))
16        self.pushButton_d2.clicked.connect(lambda: self.display_screen('2'))
17        self.pushButton_d3.clicked.connect(lambda: self.display_screen('3'))
18        self.pushButton_d4.clicked.connect(lambda: self.display_screen('4'))
19        self.pushButton_d5.clicked.connect(lambda: self.display_screen('5'))
20        self.pushButton_d6.clicked.connect(lambda: self.display_screen('6'))
21        self.pushButton_d7.clicked.connect(lambda: self.display_screen('7'))
22        self.pushButton_d8.clicked.connect(lambda: self.display_screen('8'))
23        self.pushButton_d9.clicked.connect(lambda: self.display_screen('9'))
24
25        # Dodajem akciju za clear
26        self.pushButton_clr.clicked.connect(self.lineEdit.clear)
27
28        # Dodajem akciju za backspace
29        self.pushButton_del.clicked.connect(self.lineEdit.backspace)
30
31        # Dodajem akcije za operacije
32        self.pushButton_add.clicked.connect(lambda: self.display_operator('+'))
33        self.pushButton_sub.clicked.connect(lambda: self.display_operator('-'))
```

```

34         self.pushButton_mul.clicked.connect(lambda: self.display_operator('*'))
35         self.pushButton_div.clicked.connect(lambda: self.display_operator('/'))
36         self.pushButton_mod.clicked.connect(lambda: self.display_operator('%'))

38         # Dodajem finalno akciju za =
39         self.pushButton_eq.clicked.connect(self.calculate)

40
41     # Ispisujem broj value na ekran
42     def display_screen(self, digit):
43         self.lineEdit.insert(digit)
44
45     def display_operator(self, operator):
46         if len(self.lineEdit.text())>0 and self.lineEdit.text()[-1] not in ['+', '-',
47             '*', '/']:
48             self.lineEdit.insert(operator)
49
50     def calculate(self):
51         data = str(self.lineEdit.text())
52         operands = re.split('\+|\-|', data)
53         operators = list(filter(lambda c: c in ['+', '-', '*', '/'], data))
54         assert len(operators) + 1 == len(operands)
55         operands = list(map(lambda expression: self.evaluate(expression), operands))
56
57         result = int(operands[0])
58         for i in range(0, len(operators)):
59             nextOperand = operands[i+1]
60             nextOperator = operators[i]
61
62             if nextOperator=='+':
63                 result += int(nextOperand)
64             else:
65                 result -= int(nextOperand)
66         self.lineEdit.setText(str(result))
67
68     def evaluate(self, data):
69         operands = re.split('\*|/', data)
70         if len(operands)==1:
71             return data
72         operators = list(filter(lambda c: c in ['*', '/'], data))
73         assert len(operators) + 1 == len(operands)
74
75         result = int(operands[0])
76         for i in range(0, len(operators)):
77             nextOperand = operands[i+1]
78             nextOperator = operators[i]
79
80             if nextOperator=='*':
81                 result *= int(nextOperand)
82             else:
83                 result = int(result/float(nextOperand))
84         return result
85
86 if __name__ == "__main__":
87     app = QtWidgets.QApplication(sys.argv)
88     calculator = CalculatorClass()
89     calculator.show()
90     sys.exit(app.exec_())

```


5

Funkcionalno programiranje

Potrebno je imati instaliran GHC kompajler na računaru. Za projekte je potrebno instalirati Haskell Stack sistem.

Literatura:

- (a) <https://www.haskell.org/>
- (b) <https://wiki.haskell.org/Haskell>
- (c) <https://docs.haskellstack.org/en/stable/README/>

5.1 Uvod

5.1.1 Uvodni primeri

Zadatak 5.1 Napisati funkciju main koja ispisuje poruku Zdravo! :).

```
2  -- Ovako pisemo jednolinijske komentare
3
4  {-
5      Ovako pisemo
6      viselinijске
7      komentare
8  -}
9
10 {-
11     Pretpostavljamo da smo instalirali Glasgow Haskell Compiler - ghc
12     (uz njega dolazi i interpreter - ghci)
13
14     Pokrenimo interpreter (ghci):
15
16     GHCi, version 8.0.1: http://www.haskell.org/ghc/  :? for help
17     Prelude>
18
19     Sa interpreterom mozemo direktno komunicirati, npr:
20     Prelude> print "Zdravo! :)"
21     "Zdravo! :)"
22
23     Specijalne komande za interpreter pocinju sa `:`
24
25     Interpretatoru dodajemo mogucnost izvorsavanja funkcija iz izvorne
26     datoteke ime_programa.hs komandom `:load` ili krace `:l`:
27
28     :load ime_programa.hs
29
30     Nakon toga mozemo pokrenuti sve funkcije koje su u toj datoteci
31     definisane, npr.
32
33     Prelude> main
34     "Zdravo! :)"
35
36     Ukoliko promenimo sadrzaj izvornih fajlova, mozemo naterati
```



```

36  interpreter da ih ponovo ucita komandom `:r`
38  Iz interpretera se izlazi komandom `:quit` ili krace `:q`
39  Prelude> :quit
40
41  Haskell programe takodje mozemo kompajlirati uz pomoc ghc-a:
42
43  ghc ime_programa.hs
44
45  I onda ih normalno pokrenuti:
46
47  ./ime_programa
48 -}
49
50 main = print "Zdravo! :)"

```

Zadatak 5.2 Napisati funkciju `duplo n` koja računa dvostruku vrednost celog broja `n`.

```

{-
2   Osnovni tipovi:
3   Bool
4   Char
5   String
6   Int
7   Integer
8   Float
9   Double
10
11  Tipiski razredi (definisu funkcije koje neki tip mora da implementira):
12  Eq - tipovi sa jednakoscu (==,/=)
13  Ord - tipovi sa uredjenjem (<,<=,>,>=,...)
14  Num - numericki tipovi (+,-,*,...)
15  Integral - celobrojni tipovi (div, mod,...)
16  Fractional - razlomacki tipovi (/ , recip,...)
17
18  Potpis funkcije (primeri):
19  f :: Int -> Int           - 1 argument
20  f :: Int -> Int -> Int -> Int - 3 argumenta
21
22  Informacije o bilo kom objektu mozete dobiti naredbom:
23  :info ime_objekta
24
25  Tip funkcije (ili izraza) se moze dobiti naredbom
26  :type izraz
27 -}
28
29 duplo :: Int -> Int
30 duplo a = a * 2
31
32 -- posto je (*) funkcija dva argumenta, mozemo da
33 -- iskoristimo cinjenicu da se Karijevim postupkom od (*)
34 -- moze dobiti funkcija jednog argumenta parcijalnom
35 -- primenom (*) na 2 - isto vazi za svaku funkciju
36 duplo' :: Int -> Int
37 duplo' = (*2)

```

Zadatak 5.3 Napisati funkciju `ostatak3 n` koja računa ostatak pri deljenju broja `n` brojem tri.

```

1  {-
2   Aritmetičke operacije nad tipovima Int, Integer, Float:
3   +, -, *, /,
4   ^ - stepen (tip Int)
5   ** - stepen (tip Float)
6
7   Funkcije:
8   mod, div, log, sqrt, sin, cos, tan
9
10 -}
11
12 ostatak3 n = mod n 3

```

```

13 -- posto je mod funkcija, pise se prefiksno
15 -- mozemo funkcije pisati infiksno izmedju simbola `:
   ostatak3' n = n `mod` 3
17
19 -- slicno, infiksne operatore mozemo pisati prefiksno u zagradama:
   -- zbir a b = (+) a b

```

Zadatak 5.4 Napisati funkciju `korenCeli n` koja računa realni koren celog broja `n` korišćenjem ugrađene funkcije `sqrt`.

```

1 -- ukoliko koristimo funkcije koje su definisane za realne
2 -- tipove nad argumentima koji su celobrojnog tipa potrebno
3 -- je prevesti vrednost u nadklasu Num koriscenjem funkcije
4 -- fromIntegral - Haskell je striktno tipiziran
5
6 korenCeli :: Int -> Double
7 korenCeli n = sqrt (fromIntegral n)

```

Zadatak 5.5 Napisati funkciju `sumaPrvih n` koja računa sumu prvih `n` prirodnih brojeva (rekurzivno, bez korišćenja formule).

```

1 -- Ne postoje petlje, jedini nacin ponavljanja je rekurzija
2
3 -- Blokovi se u Haskelu odvajaju indentacijom
4 -- Haskell je striktan sto se tice poravnanja - blokove moramo
5 -- lepo ravnati inace cemo dobiti gresku
6
7 -- Uslovni izraz (mora imati else granu)
   sumaPrvih n =
8     if n == 0 then 0
9     else n + sumaPrvih(n-1)
10
11 -- Ogradjene jednacine (guards) - mozemo ih koristiti umesto
12 -- uslovnih izraza, bitan je redosled navodjenja, otherwise
13 -- slucaj treba uvek staviti na kraju
   sumaPrvih' n      -- primetimo da nema znaka "=" !
14   | n == 0 = 0
15   | otherwise = n + sumaPrvih'(n-1)
16
17 -- jasno, ovaj zadatak se trivijalno moze resiti bez rekurzije
18 -- ako znamo liste (vraticemo se na ovaj primer u nastavku):
19 sumaPrvih'' n = sum [1..n]
20
21

```

Zadatak 5.6 Napisati funkciju `parMax p` koja određuje veći element iz para realnih brojeva `p`.

```

1 {-
2   Torke (Tuples) su uredjeni niz vrednosti nekog tipa
3   (ne nuzno istog):
4
5   ('a',1,[1.2, 2.3],"torka")
6
7   Funkcije definisane za parove (n=2):
8   fst - vraca prvi element para
9   snd - vraca drugi element para
10  -}
11
12 parMax :: (Double, Double) -> Double
13 parMax p
14   | fst p <= snd p = snd p
15   | otherwise = fst p
16
17 -- mozemo koristiti ugradjenu funkciju max:
18 parMax' :: (Double, Double) -> Double
19 parMax' p = max (fst p) (snd p)
20
21 {-
22   Uparivanje sablona (pattern matching)
23 -}

```

```

not :: Bool -> Bool
25 not True = False
not False = True

27
Ako pozovemo not True uparice se prvi sablon,
29 a ako pozovemo not False uparice se drugi sablon

31 Dzoker (wildcard)

33 prvi nacin (ako bismo zamenili redosled sablona, javlja pattern match overlapped)
(&&) :: Bool -> Bool -> Bool
35 True && True = True
_ && _ = False --- moglo bi i: n && m = False, bitno je samo da su razlicito
imenovani

37
drugi nacin, efikasniji:
39 (&&) :: Bool -> Bool -> Bool
True && x = x
41 False && _ = False

43 _ predstavlja dzoker, tj. sablon koji odgovara bilo kojoj vrednosti (obratiti
paznju na redosled definisanja sablona!)

45 Sabloni torki:
prviParan (x,_) = even x
47 -}

49 parMax'' :: (Double, Double) -> Double
parMax'' (l,r) = max l r

```

Zadatak 5.7 Napisati funkciju lista a b koja pravi listu celih brojeva iz segmenta $[a, b]$. U slučaju da granice segmenta nisu ispravne, rezultat je prazna lista.

```

1 {-
Liste je kolekcija vrednosti istog tipa:
3 [element1,element2,...]

5 U Haskell-u je lista osnovna struktura podataka za cuvanje
kolekcija - lista je jednostruko ulancana i imutabilna

7
Trebalo bi imati u vidu da, zbog implementacije, neke operacije su
9 znatno sporije od ostalih jer pristup elementima nije konstantan
kao kod niza npr.

11
Operacije:
13 - konstrukcija liste: operator (:)
(:) :: a -> [a] -> [a]
15 primer: 5 : [1,2,3] (rezultat je [5,1,2,3])
- konkatencija listi: operator (++)
17 (++) :: [a] -> [a] -> [a]
primer: [1,2] ++ [3,4] (rezultat je [1,2,3,4])
19 - indeksiranje liste: operator (!!)
(!!) :: [a] -> Int -> a
21 primer: [1,2,3] !! 1 (rezultat je 2)
- raspon (range):
23 [a..b]
primer: [1..5] (rezultat je [1,2,3,4,5])
25 primer: [5,4..1] (rezultat je [5,4,3,2,1])
primer: [1,3..10] (rezultat je [1,3,5,7,10])

27
Neke korisne funkcije:
29 - head :: [a] -> a
vraca glavu liste
31 - tail :: [a] -> [a]
vraca rep liste
33 - length :: [a] -> Int
vraca duzinu liste
35 - take :: Int -> [a] -> [a]
uzima zadati broj elemenata sa pocetka liste
37 - drop :: Int -> [a] -> [a]
izbacuje zadati broj elemenata sa pocetka liste
39 - null :: [a] -> Bool

```

```

41     testira da li je lista prazna
- elem :: Eq a => a -> [a] -> Bool
    (potpis je pojednostavljen, elem ne radi samo za liste)
43     testira da li se element nalazi u listi
    elementi se porede po jednakosti, stoga elementi liste
45     moraju biti u tipskom razredu Eq - operator (==)

47
    Niske su liste karaktera, na primer:
49     niska1 = ['A','l','a','d','d','i','n']
    niska2 = "Aladdin"

51
    Mozemo niske tretirati kao [Char] ili kao String, nema razlike
53 -}

55 lista :: Int -> Int -> [Int]
    lista a b = [a..b]

57
-- Napomena:
59 -- posto (..) nije operator vec deo sintakse, _ne mozemo_
-- ukloniti argumente tako sto napisemo:
61 -- lista = (..)

```

5.1.2 Zadaci za samostalni rad sa rešenjima

Zadatak 5.8 Napisati funkciju `proizvodPrvih n` koja računa proizvod prvih n prirodnih brojeva (rekurzivno, bez korišćenja formule). Pretpostaviti da je argument ispravan.

<i>Primer 1</i> <pre> POKRETANJE: proizvodPrvih 5 IZLAZ: 120 </pre>	<i>Primer 2</i> <pre> POKRETANJE: proizvodPrvih 30 IZLAZ: 26525285981219105863630848000000 </pre>
--	--

[Rešenje 5.8]

Zadatak 5.9 Napisati funkciju `prost n` koja vraća `True` ako je n prost, `False` inače. Pretpostaviti da je argument ispravan.

<i>Primer 1</i> <pre> POKRETANJE: prost 5 IZLAZ: True </pre>	<i>Primer 2</i> <pre> POKRETANJE: prost 12 IZLAZ: False </pre>
---	---

[Rešenje 5.9]

Zadatak 5.10 Napisati funkciju `nzd a b` koja računa najveći zajednički delilac brojeva a i b (koristiti Euklidov algoritam). Pretpostaviti da su argumenti ispravni.

<i>Primer 1</i> <pre> POKRETANJE: nzd 12 15 IZLAZ: 3 </pre>	<i>Primer 2</i> <pre> POKRETANJE: nzd 12 13 IZLAZ: 1 </pre>
--	--

[Rešenje 5.10]

Zadatak 5.11 Napisati funkciju `tipJednacine a b c` koja vraća tip kvadratne jednačine $a * x^2 + b * x + c = 0$ (Degenerisana, Jedno resenje, Dva resenja, Bez resenja).

<i>Primer 1</i> <pre> POKRETANJE: tipJednacine 1 2 1 IZLAZ: "Jedno resenje" </pre>	<i>Primer 2</i> <pre> POKRETANJE: tipJednacine (-1) 8 5 IZLAZ: "Dva resenja" </pre>
---	--

[Rešenje 5.11]

Zadatak 5.12 Napisati funkciju `izDekadne x osn` koja prebacuje broj `x` iz dekadne u osnovu `osn` i funkciju `uDekadnu x osn` koja prebacuje broj `x` iz osnove `osn` u dekadnu osnovu. Pretpostaviti da je $osn > 1$ i $osn < 10$.

Primer 1

```

|| POKRETANJE: izDekadne 101 2
|| IZLAZ:
|| 1100101

```

Primer 2

```

|| POKRETANJE: uDekadnu 765 8
|| IZLAZ:
|| 501

```

[Rešenje 5.12]

Zadatak 5.13 Napisati funkciju `ceoDeo x` koja računa ceo deo korena pozitivnog broja `x` (bez korišćenja ugrađenih funkcija za koren i/ili stepen).

Primer 1

```

|| POKRETANJE: ceoDeo 15
|| IZLAZ:
|| 3

```

Primer 2

```

|| POKRETANJE: ceoDeo 100
|| IZLAZ:
|| 10

```

[Rešenje 5.13]

Zadatak 5.14 Napisati funkciju `harm n` koja pravi listu prvih `n` elemenata harmonijskog reda. Pretpostaviti da je argument ispravan.

Primer 1

```

|| POKRETANJE: harm 2
|| IZLAZ:
|| [1.0,0.5]

```

Primer 2

```

|| POKRETANJE: harm 5
|| IZLAZ:
|| [1.0,0.5,0.3333333333333333,0.25,0.2]

```

[Rešenje 5.14]

Zadatak 5.15 Napisati funkciju `delioci n` koja pravi listu svih pravih delioca pozitivnog broja `n`. Pretpostaviti da je argument ispravan.

Primer 1

```

|| POKRETANJE: delioci 12
|| IZLAZ:
|| [2,3,4,6]

```

Primer 2

```

|| POKRETANJE: delioci 13
|| IZLAZ:
|| []

```

[Rešenje 5.15]

Zadatak 5.16 Napisati funkciju `nadovezi l1 l2 n` koja nadovezuje na listu `l1` `n` puta listu `l2`. Pretpostaviti da je argument `n` pozitivan broj.

Primer 1

```

|| POKRETANJE: nadovezi [1] [2] 3
|| IZLAZ:
|| [1,2,2,2]

```

Primer 2

```

|| POKRETANJE: nadovezi [] [1,2,3] 2
|| IZLAZ:
|| [1,2,3,1,2,3]

```

[Rešenje 5.16]

5.1.3 Zadaci za vežbu

Zadatak 5.17 Napisati funkciju `sumaKvadrata n` koja računa sumu kvadrata prvih `n` prirodnih brojeva (rekurzivno, bez korišćenja formule).

Primer 1

```

|| POKRETANJE: sumaKvadrata 5
|| IZLAZ:
|| 55

```

Primer 2

```

|| POKRETANJE: sumaKvadrata 10
|| IZLAZ:
|| 385

```

Zadatak 5.18 Napisati funkciju `brojDelilaca n` koja vraća broj pravih delilaca prirodnog broja `n`.

Primer 1

```

|| POKRETANJE: brojDelilaca 5
|| IZLAZ:
|| 0

```

Primer 2

```

|| POKRETANJE: brojDelilaca 12
|| IZLAZ:
|| 4

```

Zadatak 5.19 Napisati funkciju `fib n` koja računa n -ti element Fibonačijevog niza. Pretpostaviti da je argument ispravan.

Primer 1

```

|| POKRETANJE: fib 5
|| IZLAZ:
|| 5

```

Primer 2

```

|| POKRETANJE: fib 12
|| IZLAZ:
|| 144

```

Zadatak 5.20 Napisati funkciju `osnova x osn1 osn2` koja prebacuje broj x iz osnove `osn1` u osnovu `osn2`. Pretpostaviti da su `osn1` i `osn2` brojevi veći od 1 i manji od 10.

Primer 1

```

|| POKRETANJE: osnova 100 10 3
|| IZLAZ:
|| 10201

```

Primer 2

```

|| POKRETANJE: osnova 525 8 2
|| IZLAZ:
|| 101010101

```

Zadatak 5.21 Napisati funkciju `parni n` koja pravi listu prvih n parnih prirodnih brojeva. Pretpostaviti da je argument ispravan.

Primer 1

```

|| POKRETANJE: parni 3
|| IZLAZ:
|| [2,4,6]

```

Primer 2

```

|| POKRETANJE: parni 10
|| IZLAZ:
|| [2,4,6,8,10,12,14,16,18,20]

```

Zadatak 5.22 Napisati funkciju `fibLista n` koja pravi listu prvih n elemenata Fibonačijevog niza. Pretpostaviti da je argument ispravan.

Primer 1

```

|| POKRETANJE: fibLista 5
|| IZLAZ:
|| [1,1,2,3,5]

```

Primer 2

```

|| POKRETANJE: fibLista 10
|| IZLAZ:
|| [1,1,2,3,5,8,13,21,34,55]

```

Zadatak 5.23 Napisati funkciju `jednocifreniDelioci n` koja pravi listu svih jednocifrenih delilaca prirodnog broja n . Pretpostaviti da je argument ispravan.

Primer 1

```

|| POKRETANJE: jednocifreniDelioci 15
|| IZLAZ:
|| [1,3,5]

```

Primer 2

```

|| POKRETANJE: jednocifreniDelioci 40
|| IZLAZ:
|| [1,2,4,5,8]

```

5.2 Liste, funkcije višeg reda

Zadatak 5.24 Definirati funkciju `uvecaj lista` koja svaki element celobrojne liste uvećava za jedan.

```

1 {-
Neke jednostavne funkcije viseg reda:
3   map :: (a -> b) -> [a] -> [b]
   primenjuje funkciju (a -> b) na listu elemenata tipa a
5   moze se reci da transformise listu koristeći datu funkciju ili
   da selektuje finalni rezultat (slicno SELECT-u u SQL upitima) -
7   stoga se u nekim jezicima zove "transform" (C++) ili "Select" (C#)

9   primer: map (\t -> t + 1) [1, 2, 3]    -- vraca [2,3,4]
   primer: map (+1) [1, 2, 3]             -- vraca [2,3,4]
11  primer: map fst [(1,2), (3,4)]         -- vraca [1,3]

13  filter :: (a -> Bool) -> [a] -> [a]

```

```

15     filtrira listu koristeći predikat (a -> Bool) - svi elementi
    za koje taj predikat vrati True se nalaze u rezultatu

17     primer: filter (\t -> t > 2) [1, 2, 3] -- vraca [3]
    primer: filter (>2) [1, 2, 3]          -- vraca [3]

19
21     all :: (a -> Bool) -> [a] -> Bool    *
    proverava da li svi elementi date liste zadovoljavaju predikat

23     * tip nije bas ovakav ako se proveru u ghci, razlog je zato sto je all
    jos vise genericki - radi ne samo za liste nego za bilo koji tip
25     koja pripada odredjenoj klasi (Foldable - definisani foldl i foldr
    - videti opise za foldl i foldr u nastavku teksta)

27
    primer: all (\t -> t < 2) [1, 2, 3] -- vraca False
29     primer: all (<5) [1, 2, 3]      -- vraca True

31     any :: (a -> Bool) -> [a] -> Bool
    slicno kao all samo sto vraca True ako neki od elemenata zadovoljava
33     predikat

35     primer: any (\t -> t < 2) [1, 2, 3] -- vraca True
    primer: any (>5) [1, 2, 3]          -- vraca False
37 -}

39     uvecaj lista = map (+1) lista

41 -- ili krace:
    uvecaj' = map (+1)

```

Zadatak 5.25 Definirati funkciju `spoji lista1 lista2` koja pravi listu uređenih parova tako što spaja redom elemente prve liste sa elementima druge liste u parove rezultujuće liste.

```

{-
2     Funkcija zip pravi listu torki od dve liste
    tako sto spaja elemente prve liste sa elementima druge liste
4 -}

6     spoji lst1 lst2 = zip lst1 lst2

8 -- ili krace
    spoji' = zip

```

Zadatak 5.26 Definirati funkciju `pozitivni lista` koja izdvaja sve pozitivne elemente iz liste.

```

{-
2     Lambda funkcije - sintaksa:
    \arg1 arg2 ... argn -> izraz
4
6     Funkcija filter ocekuje predikat, tj funkciju tipa:
    a -> Bool
-}

8     pozitivni lista = filter (\x -> x > 0) lista

10 -- ili krace
12     pozitivni = filter (>0)

14 -- naravno, preferiramo (>0) jer je krace i citljivije

```

Zadatak 5.27 Definirati funkciju `prviNegativni lista` koja izdvaja najduži prefiks negativnih elemenata liste.

```

1 {-
    Jos neke funkcije viseg reda za rad sa listama:
3
    any uslov lista - vraca True ako postoji element u listi koji zadovoljava uslov,
    False inace

```

```

5  all uslov lista - vraća True ako svi elementi u listi zadovoljavaju uslov, False
    inace
    takeWhile uslov lista - izdvaja jedan po jedan element liste sve dok ne stigne do
    elementa koji ne zadovoljava uslov
7  dropWhile uslov lista - izbacuje jedan po jedan element liste sve dok ne stigne do
    elementa koji ne zadovoljava uslov
    sum lista - sabira elemente liste
9  product lista - množi elemente liste
    reverse lista - obrće listu
11 unzip - razdvaja listu parova u liste čiji su elementi prvi, odnosno drugi elementi
    parova
    concat lista - spaja liste iz liste listi u jednu listu
13 elem e lista - vraća True ako e postoji u listi, False inace
    replicate n x - kopira broj x n puta
15 -}

17 prviNegativni lista = takeWhile (<0) lista

```

Zadatak 5.28 Definirati funkciju `sum lista` koja računa sumu elemenata celobrojne liste korišćenjem ugrađene funkcije `foldr`.

```

{-
2  foldl :: (b -> a -> b) -> b -> [a] -> b      *
  foldr :: (a -> b -> b) -> b -> [a] -> b
4  (izraz fold cemo koristiti kad je nebitno o kojem tacno fold-u je rec)
    pocevsi od binarne funkcije, pocetne vrednosti (akumulator) i kolekcije,
6  fold primenjuje funkciju redom na elemente kolekcije i akumulator i
    rezultat primene funkcije postaje novi akumulator
8  (u nekim jezicima se zove "reduce" (Python), "accumulate" (C++) mada nisu
    svuda isti parametri)

10
    razlike izmedju foldl i foldr:
12     - foldl radi sleva na desno (normalni poredak)
    - foldr radi zdesna na levo (obrnuti poredak)
14     - NAPOMENA: primetimo da nije isti redosled akumulatora i elementa!

16
    primer za foldl:
    funkcija: (+) ; acc: 0 ; kolekcija: [1, 2, 3, 4, 5]
18     koraci koje foldl radi:
    0 + 1 = 1 -- 0 je acc, 1 je trenutni element, 1 postaje acc
20     1 + 2 = 3 -- 1 je acc, 2 je trenutni element, 3 postaje acc
    3 + 3 = 6 -- 3 je acc, 3 je trenutni element, 6 postaje acc
22     6 + 4 = 10 -- 6 je acc, 4 je trenutni element, 10 postaje acc
    10 + 5 = 15 -- 10 je acc, 5 je trenutni element, 15 postaje acc
24     -- nema vise elemenata, rezultat foldl-a je 15 - zbir liste

26
    postoje i varijante foldl1/foldr1 u slucajevima kada za akumulator
    zelimo da uzmemo prvi/poslednji element kolekcije
28
    * slicno kao all i any, i fold je genericki, radi ne samo za liste
    potpis iznad je pojednostavljen da bi bilo lakse razumevanje
30 -}

32 sum1 lista = foldl (+) 0 lista
34 sum2 lista = foldr (+) 0 lista

36 -- probati za (-), koji nije komutativan

```

Zadatak 5.29 Definirati funkciju `absSume lista_listi` koja na osnovu liste listi celih brojeva pravi listu apsolutnih suma elemenata liste listi korišćenjem kompozicije funkcija za rad sa listama.

```

1  {-
    Kompozicija funkcija
3
    (f . g) x <-> f (g x)
5  -}

7  -- sum - ugradjena funkcija koja odredjuje sumu elemenata liste
  -- abs - ugradjena funkcija za odredjivanje apsolutne vrednosti broja
9
  absSume lista_listi = map (abs . sum) lista_listi

```


Zadatak 5.30 Definirati funkciju `sledbenici l` koja vraća listu sledbenika elemenata liste `l` koji su prirodni brojevi.

```
1 {-
2   Cesto nam je potrebno da listu zadamo na sledeci nacin:
3
4   lista = [f(x) | x<-D, p(x)]
5
6   to mozemo jednostavno uraditi koristeći funkcije map i filter
7
8   map f (filter p xs)
9 -}
10
11 sledbenici = map (+1) (filter (>0) lista)
```

Zadatak 5.31 Definirati rekurzivnu funkciju `list_elem x l` koja proverava da li lista `l` sadrži dati element `x`. Dodatno, definisati prethodnu funkciju korišćenjem funkcija `or` i `map`.

```
1 list_elem :: Eq a => a -> [a] -> Bool
2 list_elem x = or . map (== x)
3
4 -- ili krace
5 list_elem' x = any (==x)
```

Zadatak 5.32 Napisati biblioteku za rad sa tačkama u 2D ravni. Definirati tipove `Tacka` (koordinate su realni brojevi) i `Putanja` (niz tačaka). Implementirati funkcije:

- `tacka x y` - konstruiše tačku sa datim koordinatama
- `putanja [t1, t2...]` - konstruiše putanju od date liste tačaka
- `duzinaPutanje p` - vraća dužinu putanje `p`
- `translirajTacku t dx dy` - translira tacku `t` za vektor `(dx, dy)`
- `rastojanje t1 t2` - vraća rastojanje između tačaka `t1` i `t2`
- `uKrugu r [t1, t2...]` - iz liste tačaka vraća one koje se nalaze u krugu poluprečnika `r` od tačke `(0,0)`
- `translirajPutanju p dx dz` - translira sve tačke date putanje za vektor `(dx, dy)`
- `nadovezi t p` - nadovezuje tačku `t` na kraj putanje `p`
- `spojiPutanja p1 p2` - spaja putanje `p1` i `p2` i vraća novu putanju
- `centroid [t1, t2...]` - vraća centroid liste tačaka
- `kvadrantTacke t` - vraća kvadrant tačke, 0 ako je to koordinatni početak
- `kvadrantPutanje p` - vraća kvadrant u kom se nalazi putanja ako je ona u celosti sadržana u nekom od njih, 0 inače
- `tackeUKvadrantu kv [t1, t2...]` - iz liste tačaka vraća samo one koje su u kvadrantu `kv`
- `tackeVanKvadranta kv [t1, t2...]` - iz liste tačaka vraća one koje nisu u kvadrantu `kv`
- `maksimumi [t1, t2...]` - vraća uređeni par koji predstavlja maksimume `x` i `y` koordinata tačaka iz date liste

```
1 module Lib where
2
3 -- Kreiramo biblioteku za rad sa grafikom u 2D
4
5 -- Pravimo alias za tip uredjenog para realnih brojeva,
6 -- nesto slicno kao `using` u C++-u
7 type Tacka = (Float, Float)
```

```

9  -- Funkcija koja pravi tacku od datih koordinata
-- Primetimo da korisnik ne zna internu reprezentaciju
11 -- (proveriti sa :t tacka)
-- Naravno, sa :info Tacka vidi se definicija tipa Tacka
13 tacka :: Float -> Float -> Tacka
tacka x y = (x,y)
15
-- Funkcija koja predstavlja koordinatni pocetak
17 -- Ne mozemo koristiti 0 zato sto funkcije moraju poceti
-- malim slovom
19 o :: Tacka
o = (0.0, 0.0)
21
-- Putanja predstavlja spisak tacaka
23 type Putanja = [Tacka]
25
-- Posto je putanja lista tacaka, vracamo argument bez izmene
-- Mozemo koristiti funkciju identiteta da definisemo nasu funkciju
27 putanja :: [Tacka] -> Putanja
putanja = id
29 -- skraceno za: putanja tacke = tacke
31
-- Sabloni listi:
--   length [] = 1 -- [] predstavlja listu sa jednim elementom
33 --   length lst = 1 + length (tail lst) -- lst predstavlja listu
duzinaPutanje :: Putanja -> Int
35 duzinaPutanje = length
37
-- Translira tacku za dati pomeraj
translirajTacku :: Tacka -> Float -> Float -> Tacka
39 translirajTacku (x,y) xt yt = tacka (x + xt) (y + yt)
-- bolje nego da vracamo par (x+xt, y+yt) jer ako promenimo
41 -- implementaciju tipa Tacka u nekom trenutku, ovo ce i dalje raditi!
43
rastojanje :: Tacka -> Tacka -> Float
rastojanje (x1, y1) (x2, y2) = sqrt $ (x1-x2)^2 + (y1-y2)^2
45
-- Vraca tacke koje se nalaze u krugu datog poluprecnika sa centrom u 0
47 uKругu :: Float -> [Tacka] -> [Tacka]
uKругu r lst = [t | t <- lst, rastojanje o t < r]
49
-- Translira sve tacke date putanje za dati vektor
51 translirajPutanju :: Putanja -> Float -> Float -> Putanja
translirajPutanju putanja x y = map (\t -> translirajTacku t x y) putanja
53
-- Nadovezuje tacku na putanju
55 nadovezi :: Tacka -> Putanja -> Putanja
nadovezi t putanja = reverse $ t : (reverse putanja)
57 -- pokazujemo funkciju reverse, moze bolje naravno
59
spojiPutanje :: Putanja -> Putanja -> Putanja
spojiPutanje = (++)
61
centroid :: [Tacka] -> Tacka
centroid ts = tacka prosekX prosekY
63   where prosekX = prosek $ map fst ts
         prosekY = prosek $ map snd ts
         prosek lst = (sum lst) / (fromIntegral $ length lst)
67
kvadrantTacke :: Tacka -> Int
kvadrantTacke (x,y)
69   | x > 0 && y > 0 = 1
71   | x < 0 && y > 0 = 2
   | x < 0 && y < 0 = 3
73   | x > 0 && y < 0 = 4
   | otherwise = 0 -- koord. pocetak
75
-- Vraca kvadrant u kom se nalazi putanja ako je ona u celosti sadrzana u nekom od
77 -- njih, 0 inace
kvadrantPutanje :: Putanja -> Int
79 kvadrantPutanje lst = if istiKvadranti then head kvadranti else 0
   where kvadranti = map kvadrantTacke lst
81       istiKvadranti = all (== head kvadranti) (tail kvadranti)

```

```

83 -- Vraca samo one tacke iz datog kvadranta
tackeUKvadrantu :: Int -> [Tacka] -> [Tacka]
85 tackeUKvadrantu k = filter (\t -> kvadrantTacke t == k)
-- skraceno za (mozemo delimicno primeniti funkciju):
87 -- tackeUKvadrantu k tacke = filter (\t -> kvadrantTacke t == k) tacke

89 tackeVanKvadranta :: Int -> [Tacka] -> [Tacka]
tackeVanKvadranta k = filter (\t -> (not . (==k)) (kvadrantTacke t))
91 -- moze t != k, ovde samo pokazujemo kompoziciju funkcija - (.)

93 -- Vraca par maksimuma X i Y koordinata tacaka, redom
maksimumi :: [Tacka] -> (Float, Float)
95 maksimumi lst = (maksimum $ map fst lst, maksimum $ map snd lst)
    where maksimum (x:xs) = foldl1 (\acc e -> if e > acc then e else acc) x xs
97 -- ekvivalentno sa:
-- maksimum = foldl1 (\acc e -> e > if acc then e else acc)
99 -- ili, jos krace (i preferirano):
-- maksimum = foldl1 max

```

5.2.1 Zadaci za samostalni rad sa rešenjima

Zadatak 5.33 Napisati funkcije koje određuju glavu i rep proizvoljne liste bez korišćenja ugrađenih funkcija za rad sa listama.

Primer 1

```

|| POKRETANJE: glava [1,2,3]
|| IZLAZ:
|| 1

```

Primer 2

```

|| POKRETANJE: rep [2]
|| IZLAZ:
|| []

```

[Rešenje 5.33]

Zadatak 5.34 Napisati funkciju *parni a b* koja generiše listu parnih celih brojeva iz segmenta $[a, b]$ i funkciju *neparni a b* koja generiše listu neparnih celih brojeva iz segmenta $[a, b]$.

Primer 1

```

|| POKRETANJE: parni 1 10
|| IZLAZ:
|| [2,4,6,8,10]

```

Primer 2

```

|| POKRETANJE: neparni [2, 7]
|| IZLAZ:
|| [3,5,7]

```

[Rešenje 5.34]

Zadatak 5.35 Napisati funkciju *parovi a b c d* koja generiše listu parova celih brojeva (x, y) , za koje x pripada segmentu $[a, b]$, a y pripada segmentu $[c, d]$.

Primer 1

```

|| POKRETANJE: parovi 1 3 2 4
|| IZLAZ:
|| [(1,2),(1,4),(3,2),(3,4)]

```

Primer 2

```

|| POKRETANJE: parovi 1 2 3 4
|| IZLAZ:
|| [(1,3),(1,4),(2,3),(3,4)]

```

[Rešenje 5.35]

Zadatak 5.36 Napisati funkciju *zavisnoY a b* koja generiše listu parova celih brojeva (x, y) , za koje x pripada segmentu $[a, b]$, a y pripada segmentu $[x, b]$.

Primer 1

```

|| POKRETANJE: zavisnoY 1 3
|| IZLAZ:
|| [(1,1),(1,2),(1,3),(2,2),(2,3),(3,3)]

```

Primer 2

```

|| POKRETANJE: zavisnoY 0 2
|| IZLAZ:
|| [(0,0),(0,1),(0,2),(1,1),(1,2),(2,2)]

```

[Rešenje 5.36]

Zadatak 5.37 Napisati funkciju *bezbedanRep 1* koja ukoliko je lista 1 prazna vraća praznu listu, inače vraća rep liste 1, koristeći: a) uslovne izraze b) ograđene jednačine c) uparivanje šablona

Primer 1

```

POKRETANJE: bezbedanRep [1,2,3]
IZLAZ:
[2,3]

```

Primer 2

```

POKRETANJE: bezbedanRep []
IZLAZ:
[]

```

[Rešenje 5.37]

Zadatak 5.38 Napisati funkciju `savršeni n` koja pravi listu savršenih brojeva manjih od n . Broj je savršen ukoliko je jednak sumi svojih faktora (tj. delilaca), ne uključujući taj broj.

Primer 1

```

POKRETANJE: savršeni 50
IZLAZ:
[6,28]

```

Primer 2

```

POKRETANJE: savršeni 1000
IZLAZ:
[6,28,496]

```

[Rešenje 5.38]

Zadatak 5.39 Napisati funkciju `zbirPar n` koja pravi listu parova (a, b) takvih da su a i b prirodni brojevi čiji je zbir jednak n .

Primer 1

```

POKRETANJE: zbirPar 1
IZLAZ:
[]

```

Primer 2

```

POKRETANJE: zbirPar 10
IZLAZ:
[(1,9),(2,8),(3,7),(4,6),(5,5),(6,4),(7,3),(8,2),(9,1)]

```

[Rešenje 5.39]

Zadatak 5.40 Napisati funkciju `poslednji l` koja određuje poslednji element proizvoljne liste l .

Primer 1

```

POKRETANJE: poslednji
["ponedeljak", "nedelja"]
IZLAZ:
"nedelja"

```

Primer 2

```

POKRETANJE: poslednji [5,4,3,2,1]
IZLAZ:
1

```

[Rešenje 5.40]

Zadatak 5.41 Napisati funkciju `spoji l` koja spaja listu listi istog tipa l u jednu listu.

Primer 1

```

POKRETANJE: spoji
[["jedan"],["tri"],["pet"]]
IZLAZ:
["jedan","tri","pet"]

```

Primer 2

```

POKRETANJE: spoji [[],[1],[1,2],[1,2,3]]
IZLAZ:
[1,1,2,1,2,3]

```

[Rešenje 5.41]

Zadatak 5.42 Napisati funkciju `sufiksi l` koja pravi listu svih sufixa proizvoljne liste l .

Primer 1

```

POKRETANJE: sufixi [1,2,3]
IZLAZ:
[[1,2,3],[2,3],[3],[]]

```

Primer 2

```

POKRETANJE: sufixi []
IZLAZ:
[[]]

```

[Rešenje 5.42]

Zadatak 5.43 Napisati funkciju `izbaci k l` koja izbacuje k -ti element iz liste l . U slučaju da je zadata neispravna pozicija u listi, funkcija vraća nepromenjenu listu.

Primer 1

```

POKRETANJE: izbaci 2 [1,2,3,4,5]
IZLAZ:
[1,2,4,5]

```

Primer 2

```

POKRETANJE: izbaci (-2) [1,2,3]
IZLAZ:
[1,2,3]

```

[Rešenje 5.43]

Zadatak 5.44 Napisati funkciju `ubaci k n l` koja ubacuje u listu `l` na poziciju `k` element `n`. U slučaju da je zadata neispravna pozicija u listi, dodati element `n` na kraj liste.

Primer 1

```
|| POKRETANJE: ubaci 2 5 [1,2,3]
|| IZLAZ:
|| [1,2,5,3]
```

Primer 2

```
|| POKRETANJE: ubaci 10 5 [1,2,3,4,5]
|| IZLAZ:
|| [1,2,3,4,5,5]
```

[Rešenje 5.44]

5.2.2 Zadaci za vežbu

Zadatak 5.45 Korišćenjem funkcija `and` i `map`, definisati funkciju `list_all p l` koja proverava da li svi elementi liste `l` zadovoljavaju dato svojstvo `p`.

Primer 1

```
|| POKRETANJE: list_all even [2,4,6]
|| IZLAZ:
|| True
```

Primer 2

```
|| POKRETANJE: list_all (>3) [1,2,3,4,5]
|| IZLAZ:
|| False
```

Zadatak 5.46 Definisati funkciju `obrni l` za obrtanje liste `l` pomoću funkcije `foldl`.

Primer 1

```
|| POKRETANJE: obrni [1,2,3]
|| IZLAZ:
|| [3,2,1]
```

Primer 2

```
|| POKRETANJE: obrni "abcde"
|| IZLAZ:
|| "edcba"
```

Zadatak 5.47 Definisati funkciju `delioci n` koja pravi listu delilaca datog prirodnog broja `n` korišćenjem funkcije `filter`. Korišćenjem prethodne funkcije definisati funkciju `prost n` koja proverava da li je dati prirodan broj `n` prost. Dodatno, koristeći funkcije `prost` i generisanje listi definisati funkciju `prosti n` koja određuje sve proste brojeve od 1 do `n`.

Primer 1

```
|| POKRETANJE: prosti 5
|| IZLAZ:
|| [2,3,5]
```

Primer 2

```
|| POKRETANJE: prosti 10
|| IZLAZ:
|| [2,3,5,7]
```

Zadatak 5.48 Definisati funkciju `sufiks l` pomoću funkcije `scanr` koja određuje sve sufikse date liste.

Primer 1

```
|| POKRETANJE: sufiks [1,2,3]
|| IZLAZ:
|| [[1,2,3], [2,3], [3], []]
```

Primer 2

```
|| POKRETANJE: sufiks "abc"
|| IZLAZ:
|| ["abc", "bc", "c", ""]
```

Zadatak 5.49 Definisati funkciju `prefiks l` koja određuje sve prefikse date liste `l` (dozvoljena je rekurzija).

Primer 1

```
|| POKRETANJE: prefiks [1,2,3]
|| IZLAZ:
|| [[], [1], [1,2], [1,2,3]]
```

Primer 2

```
|| POKRETANJE: prefiks "abc"
|| IZLAZ:
|| [, "a", "ab", "abc"]
```

Zadatak 5.50 Definisati rekurzivnu funkciju `ukloniDuplikate l` koja pomoću funkcija iz familije `fold` uklanja sve duplikate iz liste `l`.

Primer 1

```
POKRETANJE: ukloniDuplikate [1,2,1,3]
IZLAZ:
[1,2,3]
```

Primer 2

```
POKRETANJE: ukloniDuplikate "abaccddb"
IZLAZ:
"abcd"
```

Zadatak 5.51 Definirati funkciju `qsort` 1 sortira listu 1 u rastućem poretku. Za pivot uzeti prvi element liste.

Primer 1

```
POKRETANJE: qsort [1,2,1,3]
IZLAZ:
[1,1,2,3]
```

Primer 2

```
POKRETANJE: qsort "abaccddb"
IZLAZ:
"aabbccdd"
```

5.3 Korisnički tipovi, Napredni koncepti

5.3.1 Uvodni primeri

Zadatak 5.52 Definirati bulovski tip podataka.

```
1 {-
2     Algebarski tipovi
3
4     Mozemo definisati tip podataka koristeći ključnu rec `data`
5 -}
6
7 data BulovskiTip = Tacno | Netacno
8
9 -- Tacno i Netacno u ovom slučaju su "konstruktori" za tip BulovskiTip
```

Zadatak 5.53 Definirati tip podataka `Oblik` koji može biti `Kvadrat` (karakterisan stranicom), `Pravougaonik` (karakterisan dvema stranicama), `Krug` (karakterisan poluprečnikom) ili `Trougao` (karakterisan sa tri stranice).

```
1 -- Konstruktori mogu imati argumente
2
3 data Oblik = Kvadrat Float
4             | Pravougaonik Float Float
5             | Krug Float
6             | Trougao Float Float Float
7             deriving Show
8
9 -- deriving Show znaci da nas tip Oblik pripada
10 -- klasi Show - koristimo činjenicu da postoji
11 -- podrazumevana implementacija funkcije show
```

Zadatak 5.54 Definirati tip podataka `Zivotinja` koji može biti `Pas`, `Macka` ili `Papagaj`. Zatim definisati tip `Ljubimac` (karakterisan imenom, godinama i tipom zivotinje).

```
1 -- Konstruktori mogu imati argumente
2
3 data Zivotinja = Pas | Macka | Papagaj
4
5 data Ljubimac = MkLjubimac { ime :: String
6                             , godine :: Int
7                             , tip :: Zivotinja
8                             }
9
10 -- MkLjubimac je konstruktor 3 argumenta,
11 -- svakom dajemo ime koje mozemo koristiti kao
12 -- getter
13
14 -- let dzeki = MkLjubimac 3 "Dzeki" Pas
15 -- let starijiDzeki = dzeki { godine = 4 }
16 -- ovo pravi kopiju objekta sa izmenjenim poljima
```

Zadatak 5.55 Definirati tip podataka `Pravougaonik` (karakterisan dvema stranicama) i instancirati klase `Show` i `Eq` nad kreiranim tipom.

```

2 data Pravougaonik = MkPravougaonik { a :: Int
                                     , b :: Int
                                     }
4
6 instance Show Pravougaonik where
    show (MkPravougaonik a b) =
8         "(" ++ show a ++ "," ++ show b ++ ")"
10
11 instance Eq Pravougaonik where
    (==) (MkPravougaonik a1 b1) (MkPravougaonik a2 b2) =
        a1 == a2 && b1 == b2 || a1 == b2 && b1 == a2

```

Zadatak 5.56 Napisati funkcije `glava` i `rep` koje bezbedno vraćaju glavu i rep liste koristeći tip `Maybe`.

```

1 {-
2     Maybe a
3
4     Predstavlja opcionu vrednost (isto kao std::optional<T>,
5     Optional<T> ili Nullable<T> u OOP jezicima)
6
7     Definisan je kao (videti :info Maybe):
8
9     data Maybe a = Nothing | Just a
10 -}
11
12 glava :: [a] -> Maybe a
13 glava []    = Nothing
14 glava (x:_) = Just x
15
16 rep :: [a] -> Maybe [a]
17 rep []      = Nothing
18 rep (_,xs) = Just xs

```

Zadatak 5.57 Napisati funkcije `glava` i `rep` koje bezbedno vraćaju glavu i rep liste koristeći tip `Either`. U slučaju da je lista prazna, vratiti `String` sa porukom o grešci.

```

1 {-
2     Either a b
3
4     Predstavlja vrednosti koje mogu biti jednog ILI drugog tipa
5     (nesto slicno std::variant<T1, T2...>, s tim sto je variant
6     mocniji jer je sablon sa proizvoljno mnogo tipova, Either je
7     fiksiran na dva)
8
9     Definisan je kao (videti :info Maybe):
10
11     data Either a b = Left a | Right b
12
13     Koristi se cesto za baratanje greskama
14 -}
15
16 glava :: [a] -> Either String a
17 glava []    = Left "Prazna lista"
18 glava (x:_) = Right x
19
20 rep :: [a] -> Either String [a]
21 rep []      = Left "Prazna lista"
22 rep (_,xs) = Right xs

```

Zadatak 5.58 Napisati biblioteku za pomoć asistentima u arhiviranju i održavanju rezultata ispita. Definirati tipove `StepenStudija` (osnovne, master, doktorske), `Student` (karakterisan brojem indeksa, imenom, prezimenom i stepenom studija) i `Rezultat` (svaki student ima opcioni rezultat predstavljen brojem poena). Obezbediti da se student može ispisati na standardni izlaz i porediti po jednakosti sa ostalim studentima po broju indeksa. Implementirati funkcije:

- `rezultati studenti` - konstruiše listu rezultata za date studente, gde je svaki rezultat trenutno prazan
- `poeni student rezultati` - vraća broj poena datog studenta iz liste rezultata ili poruku ako se student ne nalazi u istoj
- `ponisti student rezultati` - poništava poene za datog studenta iz liste rezultata
- `ponistiSve rezultati` - poništava poene za sve studente iz liste rezultata
- `studije stepenStudija rezultati` - vraća samo one rezultate za studente sa datog stepena studija
- `polozeni rezultati` - vraća samo one rezultate gde je student položio ispit
- `upisi student poeni rezultati` - upisuje novi rezultat za datog studenta u listu rezultata
- `najboljih n rezultati` - vraća n najboljih rezultata (samo broj poena) iz liste rezultata, sortiranih opadajuće

```

module Lib where
2
import Data.Maybe
4 import Data.List as List (elemIndex, sortBy)

6 -- Kreiramo tip koji predstavlja stepen studija
data StepenStudija = OsnovneStudije
8                     | MasterStudije
                     | DoktorskeStudije
10                    deriving (Show, Eq)
-- `OsnovneStudije`, `MasterStudije` i `DoktorskeStudije` su
12 -- konstruktori za tip `StepenStudija`.
-- Ukoliko zelimo da kreiramo objekat tipa `StepenStudija` koji
14 -- predstavlja `MasterStudije`, to radimo tako sto pozovemo
-- konstruktor `MasterStudije` kao bilo koju drugu funkciju.
16 -- Konstruktor je funkcija, u ovom slucaju bez argumenata.
-- Kazemo da StepenStudija implicitno pripada klasi Show, sto
18 -- znaci da moze da se pozove funkcija `show` nad objektom tog
-- tipa, pritom koristimo podrazumevanu implementaciju koja
20 -- ispisuje imena konstruktora

22 -- Napravimo tip za skracenicu, ovde je `Kratko` konstruktor
-- sa jednim argumentom
24 data KratkiStepenStudija = Kratko StepenStudija

26 -- Instancirajmo `Show` nad novim tipom
-- `Show` se sastoji samo od funkcije `show`
28 instance Show KratkiStepenStudija where
    show (Kratko OsnovneStudije)    = "BSc"
30    show (Kratko MasterStudije)    = "MSc"
    show (Kratko DoktorskeStudije) = "PhD"
32
34 -- Jasno je da smo mogli alternativno da ne napisemo `deriving`
-- Show` vec da rucno implementiramo `Show` tako da za stepen
-- studija vratimo odgovarajucu skracenicu. Ovde smo pravili novi
36 -- tip da bismo pokazali `deriving` i konstruktore sa argumentima

38 -- Definismo studenta sa imenovanim poljima (slicno klasi u OOP)
-- Paziti na indentaciju!
40 data Student = MkStudent { indeks  :: String
                             , ime    :: String
42                             , prezime :: String
                             , stepen  :: StepenStudija
44                             }
-- U ovom slucaju, `MkStudent` je konstruktor sa 4 argumenta
46 -- indeks, ime, prezime i stepen su funkcije koje sluze kao
-- "geteri"
48 -- Studenta mozemo napraviti pozivajuci konstruktor:
-- MkStudent "123/2018" "Pera" "Peric" OsnovneStudije
50 -- ili, ako je s vec postojeći objekat studenta, mozemo
-- napraviti "kopiju" sa izmenjenim poljima razdvojenim zarezima

```



```

52 -- s { indeks = "124/2018" }
53 -- s { indeks = "124/2018", prezime = "Peric" }
54
55 -- Posto Student nije u klasi Show, definisacemo metod koji
56 -- vraca String od datog Student-a
formatirajStudenta :: Student -> String
57 formatirajStudenta s =
58     let id = indeks s
59         ip = (ime s) ++ " " ++ (prezime s)
60         ss = show $ Kratko $ stepen s
61     in id ++ " : " ++ ip ++ " : " ++ ss
62
63
64 -- Instancirajmo `Show` nad studentom
instance Show Student where
65     show = formatirajStudenta
66
67
68 -- Instancirajmo `Eq` nad studentom
-- `Eq` se sastoji od funkcije `(==)`
69 instance Eq Student where
70     s1 == s2 = (indeks s1) == (indeks s2)
71     -- naravno, moze i prefiksno:
72     -- `(==) s1 s2 = (indeks s1) == (indeks s2)`
73
74
75 -- Napravimo alias za rezultate ispita
-- Maybe predstavlja opcioni tip, definisan kao:
76 -- data Maybe a = Nothing | Just a
77 -- Nesto sto je u C++-u `std::optional<T>` ili
80 -- u Javi `Optional<T>` ili u C#-u `Nullable<T>`
-- Maybe ima jedan tipski parametar, posto mi zelimo
82 -- da cuvamo `Int` ili `Nothing` za poene na ispitu
-- onda koristimo `Maybe Int`
83 type Rezultat = (Student, Maybe Int)
84
85 -- Formiramo spisak rezultata za studente
rezultati :: [Student] -> [Rezultat]
86 rezultati = map (\s -> (s, Nothing))
87
88
89 -- Vracamo poene za studenta ili poruku ako nemamo rezultat
-- Either je tip definisan kao:
90 -- data Either a b = Left a | Right b
91 -- Dakle, imamo objekat koji moze biti tipa a ILI tipa b
94 -- Either se cesto koristi za baratanje greskama, "leva" vrednost
-- se koristi kao specijalna
95 poeni :: Student -> [Rezultat] -> Either String (Maybe Int)
96 poeni s rezultati =
97     case mi of Nothing -> Left $ (indeks s) ++ " ne studira na fakultetu!"
98             (Just i) -> Right $ snd $ rezultati !! i
99     where mi = List.elemIndex s $ map fst rezultati
100
101
102 -- Ponisti poene za studenta
ponisti :: Student -> [Rezultat] -> [Rezultat]
103 ponisti s rezultati = foldr azuriraj [] rezultati
104     where azuriraj rez acc = if fst rez == s then (s, Nothing) : acc
105                               else rez : acc
106
107
108 -- Ponisti sve rezultate
ponistiSve :: [Rezultat] -> [Rezultat]
109 ponistiSve = map (\rez -> (fst rez, Nothing))
110
111
112 -- Vraca samo rezultate sa datog stepena studija
studije :: StepenStudija -> [Rezultat] -> [Rezultat]
113 studije ss = filter (\(s, _) -> stepen s == ss)
114
115
116 -- Vraca samo rezultate studenata koji su položili ispit
polozeni :: [Rezultat] -> [Rezultat]
117 polozeni = filter (\(s, mozdaRez) -> (izvuciRez mozdaRez) > 50)
118     where izvuciRez Nothing = 0
119           izvuciRez (Just rez) = rez
120
121
122 -- Funkcija koja upisuje rezultat za studenta
-- Ako se student nalazi u rezultatima, izasao je opet - azurirati
123 -- Ako se ne nalazi, onda dodati

```

```

upisi :: Student -> Int -> [Rezultat] -> [Rezultat]
126 upisi s p rezultati = if elem s studenti
                        then foldr azuriraj [] rezultati
                        else (s, Just p) : rezultati
128     where studenti = map fst rezultati
          azuriraj rez acc = if fst rez == s then (s, Just p) : acc
                              else rez : acc
132
-- Spisak prvih n najboljih rezultata (samo poeni)
134 najboljih :: Int -> [Rezultat] -> [Int]
najboljih n rezultati = take n
136     $ List.sortBy (flip compare)
    -- $ List.sortBy (\_ _ -> GT)
138     $ map (\(Just x) -> x)
    -- $ map Data.Maybe.fromJust
140     $ filter (Nothing /=)
    $ map snd
142     $ rezultati

```

5.3.2 Zadaci za samostalni rad sa rešenjima

Zadatak 5.59 Definirati rekurzivnu funkciju *varijacije 1 k* koja generiše listu koja sadrži sve varijacije sa ponavljanjem elemenata date liste 1 dužine k.

[Rešenje 5.59]

Primer 1

```

POKRETANJE: varijacije [1,2,3] 2
IZLAZ:
[[1,1],[1,2],[1,3],[2,1],[2,2],[2,3],[3,1],[3,2],[3,3]]

```

Zadatak 5.60 Data je lista 1 koja sadrži liste ocena raznih učenika osnovne škole. Definirati funkciju *prosekOdlicni 1* koja računa prosek svih odličnih učenika.

[Rešenje 5.60]

Primer 1

```

POKRETANJE: prosekOdlicni [[1,2,3],[5,5,5],[4,5,5]]
IZLAZ:
4.8333335

```

Zadatak 5.61 Ana uči brojanje i razlikovanje boja koristeći kutiju punu jednobojnih kuglica. Ona prvo žmureći iz kutije izvuče određeni broj kuglica i poreda ih u niz u redosledu izvlačenja. Zatim izabere proizvoljnu boju i odredi na kojoj se sve poziciji u nizu izvučenih kuglica nalazi kuglica željene boje. Napisati funkciju *pozicije x 1* koja vraća listu pozicija elementa x u listi 1.

[Rešenje 5.61]

Primer 1

```

POKRETANJE: pozicije "bela" ["plava","bela","bela","ljubicasta","bela"]
IZLAZ:
[1,2,4]

```

Zadatak 5.62 Učesnici nagradne igre *Sedmica* mogu proveriti dobitak putem sajta, gde se dobitna kombinacija objavljuje odmah nakon izvlačenja. Brojevi iz dobitne kombinacije se, radi jednostavnije provere pogodaka, uvek prikazuju u rastućem redosledu. Napisati funkciju *qsort 1* koja rastuće sortira listu 1 algoritmom *qsort*.

[Rešenje 5.62]

Primer 1

```

POKRETANJE: qsort [4,5,2,11,29,38,9]
IZLAZ:
[2,4,5,9,11,29,38]

```

Primer 2

```

POKRETANJE: qsort [7,1,10,15,30,32,20]
IZLAZ:
[1,7,10,15,20,30,32]

```

Zadatak 5.63 Milan obožava da sakuplja sličice fudbalera. Da bi jednostavnije pratio koje mu sličice iz kolekcije još uvek nedostaju, čuva ih rastuće sortirane po rednom broju. Planirao je da dopuni svoju kolekciju na narednoj razmeni sličica, pa za tu priliku želi da iz svoje kolekcije izbaci sve nepotrebne duplikate koje će poneti na razmenu. Napisati funkciju `brisiPonavljanja` 1 koja briše sva uzastopna ponavljanja elemenata u listi 1.

[Rešenje 5.63]

<i>Primer 1</i> <pre>POKRETANJE: brisiPonavljanja [4,5,5,2,11,11,11] IZLAZ: [4,5,2,11]</pre>	<i>Primer 2</i> <pre>POKRETANJE: brisiPonavljanja [10,10,10,11] IZLAZ: [10,11]</pre>
---	---

Zadatak 5.64 Kasirka Mica mora da ručno kuca artikle na kasi jer se pokvario skener barkodova. Pomozite Mici da taj posao obavi što brže grupisanjem artikala iste vrste na pokretnoj traci. Napisati funkciju `podlistePonavljanja` 1 koja grupiše sva uzastopna ponavljanja nekog elementa liste 1 u podlistu tako da rezultat bude lista listi.

[Rešenje 5.64]

<i>Primer 1</i> <pre>POKRETANJE: podlistePonavljanja ["jabuke","jogurt","jogurt","hleb"] IZLAZ: [["jabuke"],["jogurt","jogurt"],["hleb"]]</pre>
--

Zadatak 5.65 Napisati funkciju `broj lista` koja vraća broj određen ciframa koje se nalaze u listi čitajući ih sa početka ka kraju liste i funkciju `brojObrnut lista` koja vraća broj određen ciframa koje se nalaze u listi čitajući ih sa kraja ka početku liste.

[Rešenje 5.65]

<i>Primer 1</i> <pre>POKRETANJE: broj [1,0,1] IZLAZ: 101</pre>	<i>Primer 2</i> <pre>POKRETANJE: brojObrnut [0,5,2] IZLAZ: 250</pre>
---	---

Zadatak 5.66 U toku su prijave za plesno takmičenje parova. Pored nagrade za najbolji par, dodeljuju se i pojedinačne nagrade za najboljeg ženskog i muškog takmičara. Da bi žiri jednostavnije beležio poene i odredio nagrade, potrebno je da im se dostave, pored liste parova koji se takmiče, i liste samo muških, tj. samo ženskih takmičara, odvojeno. Napisati funkciju `listaUPar lista` koja pretvara listu parova u par dve liste, tako da prva lista sadrži prve elemente svih parova, a druga druge elemente svih parova pod pretpostavkom da je prvi u paru uvek ženska osoba, a drugi muška (implementacija funkcije `unzip`).

[Rešenje 5.66]

<i>Primer 1</i> <pre>POKRETANJE: listaUPar [("Ivana","Milan"), ("Ana","Jovan"), ("Anica","Petar")] IZLAZ: [["Ivana","Ana","Anica"], ["Milan","Jovan","Petar"]]</pre>

Zadatak 5.67 Nakon još jedne košarkaške sezone, potrebno je sumirati rezultate svih igrača. Svaki igrač ima jedinstveni redni broj, pod kojim se u bazi, u listi prezimena, čuva njegovo prezime, a u listi pogodaka, ostvaren broj poena u sezoni. Uparivanjem odgovarajućih podataka, napraviti za svakog igrača jedinstveni par oblika (`prezime`, `poeni`). Napisati funkciju `parOdListi lista1 lista2` koja pravi listu parova od dve liste, liste prezimena i liste pogodaka, tako da prvi element svakog para bude iz prve liste, a drugi element svakog para bude iz druge liste (implementacija funkcije `zip`).

[Rešenje 5.67]

<i>Primer 1</i> <pre>POKRETANJE: parOdListi ["Mikic","Peric","Jovic"] [100,76,96] IZLAZ: [("Mikic",100),("Peric",76),("Jovic",96)]</pre>

Zadatak 5.68 Formacija igre *kolo* je polukrug sa istaknutom ulogom prvog i poslednjeg igrača. Napisati funkciju `ucesljaj mIgraci zIgraci` koja pravi jednu formaciju za kolo naizmeničnim učešljanjem igrača iz date grupe muških i ženskih igrača, `mIgraci` i `zIgraci`, redom.

[Rešenje 5.68]

Primer 1

```
POKRETANJE: ucesljaj ["Petar","Aleksa","Filip"] ["Milica","Jovana","Anica"]
IZLAZ:
["Petar","Milica","Aleksa","Jovana","Filip","Anica"]
```

5.3.3 Zadaci za vežbu

Zadatak 5.69 Definirati alias za uređeni par `ParRazličitih a b` gde prvi i drugi element para nisu nužno istog tipa.

Zadatak 5.70 Definirati tip podataka `Krug` koji se karakteriše radijusom tipa `Float`. Definirati tip podataka `Pravougaonik` koji se definiše parom stranica (obe tipa `Float`). Instancirati `Eq` i `Show` klase nad kreiranim tipovima.

Zadatak 5.71 Aleksa i Luka žele da komuniciraju preko šifrovanih poruka koje predstavljaju listom niski. Aleksa šifrira željene podatke na sledeći način: ukoliko se niska koju šalje sastoji samo od cifara, na njen početak i kraj dodaje karakter `C`, ako se sastoji samo od malih slova, na njen početak i kraj dodaje karakter `S`, a inače na njen početak i kraj dodaje karakter `O`. Definirati sledeće funkcije koje pomažu Aleksi da pošalje Luki šifrovanu poruku:

- a) `broj s` - za datu nisku `s` proverava da li su svi njeni karakteri cifre

Primer 1

```
POKRETANJE: broj "123"
IZLAZ:
True
```

Primer 2

```
POKRETANJE: broj ['c','a','o']
IZLAZ:
False
```

- a) `mala s` - za datu nisku `s` proverava da li su svi njeni karakteri mala slova

Primer 1

```
POKRETANJE: mala "pozdrav"
IZLAZ:
True
```

Primer 2

```
POKRETANJE: mala ['C','a','o']
IZLAZ:
False
```

- c) `sifruj ls` - datu listu niski `ls` transformiše na sledeći način: ukoliko se niska koju šalje sastoji samo od cifara, na njen početak i kraj dodaje karakter `C`, ako se sastoji samo od malih slova, na njen početak i kraj dodaje karakter `M`, a inače na njen početak i kraj dodaje karakter `O`.

Primer 1

```
POKRETANJE: sifruj ["11","maj","petak"]
IZLAZ:
["C11C","MmajM","MpetakM"]
```

Primer 2

```
POKRETANJE: sifruj ["poz","Poz","poZ"]
IZLAZ:
["MpozM","OPozO","OpOZO"]
```

Zadatak 5.72 Za razliku od Alekse i Luke, Ana i Milica imaju problem dešifrovanja podataka. Da bi Ana dešifrovala podatke koje joj Milica pošalje potrebno je da svaku nisku iz dobijene liste transformiše na sledeći način: ukoliko dobijena niska počinje cifrom, sa njenog početka izbaciti onoliko karaktera koliko ta niska ima cifara, a ukoliko dobijena niska počinje malim slovom, sa njenog početka izbaciti onoliko karaktera koliko ta niska ima malih slova. Pretpostaviti da će ovakvim dešifrovanjem Ana uvek dobiti ispravne izvorne podatke. Definirati sledeće funkcije koje pomažu Ani da dešifruje Miličine poruke:

- a) `cifre s` - za datu nisku `s` vraća broj karaktera niske koji su cifre

Primer 1

```
POKRETANJE: cifre "11Maj"
IZLAZ:
2
```

Primer 2

```
POKRETANJE: cifre ['m','a','j']
IZLAZ:
0
```

b) mala s - za datu nisku s vraća broj karaktera niske koji su mala slova

Primer 1

```
POKRETANJE: mala "petak"
IZLAZ:
5
```

Primer 2

```
POKRETANJE: mala ['C','a','o']
IZLAZ:
2
```

c) desifruj ls - datu listu niski ls transformiše na sledeći način: ukoliko niska počinje cifrom, sa njenog početka izbacuje onoliko karaktera koliko ta niska ima cifara, a ukoliko dobijena niska počinje malim slovom, sa njenog početka izbacuje onoliko karaktera koliko ta niska ima malih slova.

Primer 1

```
POKRETANJE: desifruj ["aaa11","1minamaj2017","101petak"]
IZLAZ:
["11","maj2017","petak"]
```

Zadatak 5.73 Napisati funkciju `magicniParovi` 1 koja pravi listu parova čiji su prvi elementi, elementi liste prirodnih brojeva 1, a drugi elementi odgovarajući magični brojevi elemenata liste 1. Magičan broj prirodnog broja n se dobija kao proizvod njegovih cifara.

Primer 1

```
POKRETANJE: magicniParovi [12,101,154]
IZLAZ:
[(12,2),(101,0),(154,20)]
```

Primer 2

```
POKRETANJE: magicniParovi [1000,99,111,222]
IZLAZ:
[(1000,0),(99,81),(111,1),(222,8)]
```

Zadatak 5.74 Podaci o cenama artikala u prodavnici su zadati u obliku liste realnih brojeva. Definirati funkciju `prosek lista_cena` koja računa prosečnu cenu artikla u prodavnici.

Primer 1

```
POKRETANJE: prosek [199.99, 125, 10.99, 45.99, 123.50]
IZLAZ:
101.09400000000001
```

5.4 Rešenja

Rešenje 5.8

```
1 proizvodPrvih :: Num a => Int -> a
  proizvodPrvih n
3   | n < 1 = 0
   | n == 1 = 1
5   | otherwise = n * proizvodPrvih (n-1)
```

Rešenje 5.9

```
1 -- Ispitujemo da li je broj prost tako sto pozovemo pomocnu funkciju
  -- koja ispituje da li postoji neki broj pocev od 2 do n koji deli broj n
3
  -- Proveravamo da li postoji broj koji deli n (pocev od broja 2 - poziv iz prethodne
    funkcije)
5 -- ukoliko je k == n to znaci da smo proverili za svaki broj od 2 do n-1
```

```

-- i ustanovili da nijedan od njih ne deli n tako da vracamo True kao indikator da
    broj n jeste prost
7 -- ukoliko je n deljivo sa k, vracamo False (n nije prost)
-- ukoliko n nije deljivo sa k, pozovemo funkciju rekursivno za sledeci broj (k+1)
9
prost :: Int -> Bool
11 prost n = prostTest n 2
    where prostTest n k
13         | n == k = True
        | n `mod` k == 0 = False
15         | otherwise = prostTest n (k+1)

17 -- Na narednim casovima, kad naucimo funkcije viseg reda,
-- napravicemo Eratostenovo sito :)
19
-- Alternativno, znajuci generatore listi (u narednim primerima),
21 -- mozemo da kazemo sledece: "Broj je prost ako je lista njegovih
-- pravih delilaca prazna"
23 prost' n = null (listaDelilaca n)
    where listaDelilaca n = [x | x <- [2..n-1], mod n x == 0]
25
-- Mozemo, dakle, kao i u Python-u, da pravimo nesto nalik na
27 -- matematicke definicije skupova
-- {x | x iz 1..n i n deli x}
29 -- "skup svih x takvih da x pripada 1..n i ...."

```

Rešenje 5.10

```

1 nzd :: Int -> Int -> Int
nzd a b
3   | b == 0 = a
   | otherwise = nzd b (a `mod` b)
5
-- alternativno
7 nzd' = gcd

```

Rešenje 5.11

```

1 tipJednacine :: Double -> Double -> Double -> String
tipJednacine a b c
3   | a == 0 = "Degenerisana"
   | (b*b - 4*a*c) == 0 = "Jedno resenje"
5   | (b*b - 4*a*c) > 0 = "Dva resenja"
   | otherwise = "Nema resenja"

```

Rešenje 5.12

```

1 uDekadnu :: Int -> Int -> Int
uDekadnu x osn =
3   if x == 0 then 0
   else uDekadnu (x `div` 10) osn * osn + (mod x 10)
5
izDekadne :: Int -> Int -> Int
7 izDekadne x osn =
   if x == 0 then 0
9   else izDekadne (x `div` osn) osn * 10 + (mod x osn)

```

Rešenje 5.13

```

1 -- Trazimo ceo deo korena broja x
-- trazimo prvi broj (pocev od 1) ciji je kvadrat veci
3 -- od kvadrata naseg broja x i kao rezultat vracamo broj
-- koji je za jedan manji
5 ceoDeo :: (Ord a, Num a) => a -> a
ceoDeo x = ceoDeo' x 1
7   where ceoDeo' x i
        | (i*i) > x = (i-1)
9        | otherwise = ceoDeo' x (i+1)

```

Rešenje 5.14

```

1 -- Pravimo listu prvih n elemenata harmonijskog reda (n>0)
2 -- ukoliko je n = 1, vracamo listu koja sadrzi prvi element
3 -- harmonijskog reda inace pozovemo funkciju rekursivno
4 -- i na rezultat nadovezemo reciprocnu vrednost broja n
5 harm :: Int -> [Double]
6 harm n
7   | n == 1 = [1.0]
8   | otherwise = harm (n-1) ++ [recip n']
9   where n' = fromIntegral n
11
12 -- Pritom, ovde imamo mali problem ako navedemo potpis,
13 -- naime da bi se radio recip, argument mora biti realan
14 -- broj - stoga konvertujemo n u realan broj - nesto sto
15 -- se prilikom automatske dedukcije samo zaključuje

```

Rešenje 5.15

```

1 -- Pravimo listu delioca broja n
2 -- ukoliko je n = 1, vracamo listu koja sadrzi samo 1
3 -- inace pozivamo pomocnu funkciju sa jos jednim parametrom
4 -- koji nam govori do kog potencijalnog delioca smo stigli
5
6 -- Pravimo listu delioca broja n pocev od broja k
7 -- ukoliko smo stigli do broja n (k==n) vracamo praznu listu
8 -- inace proveravamo da li je k delioc broja n
9 -- ako jeste, stavljamo ga u listu i pozivamo funkciju rekursivno
10 -- za k+1 (sledeci potencijalni delilac)
11 -- inace samo pozivamo funkciju rekursivno za k+1
12
13 delioci n
14   | n == 1 = [1]
15   | otherwise = delioci' n 2
16   where delioci' n k
17         | k == n = []
18         | n `mod` k == 0 = [k] ++ (delioci' n (k+1))
19         | otherwise = delioci' n (k+1)

```

Rešenje 5.16

```

1 -- Nadovezujemo listu2 na listu1 n puta
2 -- ukoliko je n == 1 na listu1 nadovezemo listu2 operatorom ++
3 -- inace pozovemo funkciju rekursivno za n-1 i na rezultat nadovezemo listu2
4 nadovezi :: [a] -> [a] -> Int -> [a]
5 nadovezi l1 l2 n = l1 ++ rep
6   where rep = concat $ replicate n l2
7
8 -- replicate ponavlja element odredjeni broj puta - vraca
9 -- listu. S obzirom da je element vec lista, rezultat je
10 -- lista listi. Stoga, koristimo funkciju concat da ih
11 -- sve spojimo u jednu (flatten u nekim jezicima)

```

Rešenje 5.33

```

1 -- preslikava iz liste elemenata u jedan element
2 glava :: [a] -> a
3 glava (x:_) = x
4
5 -- preslikava iz liste u listu
6 rep :: [a] -> [a]
7 rep (_,xs) = xs

```

Rešenje 5.34

```

1 parni :: Int -> Int -> [Int]
  parni a b = [x | x <- [a..b], even x]
3
5 neparni :: Int -> Int -> [Int]
  neparni a b = [x | x <- [a..b], odd x]

```

Rešenje 5.35

```

1 -- Mozemo imati vise generatora
3 parovi :: Int -> Int -> Int -> Int -> [(Int,Int)]
  parovi a b c d = [(x,y) | x <- [a..b], y <- [c..d]]
5
7 {-
  obratiti paznju kako utice zamena redosleda
  parovi a b c d = [(x,y) | y <- [c..d], x <- [a..b]]
9 -}

```

Rešenje 5.36

```

1 -- Generatori mogu zavisiti jedni od drugih
  -- kasniji generatori mogu zavisiti samo od promenljivih
3 -- koje su uvedene ranijim generatorima citajuci sa leva na desno
  zavisnoY :: Int -> Int -> [(Int,Int)]
5 zavisnoY a b = [(x,y) | x <- [a..b], y <- [x..b]]

```

Rešenje 5.37

```

1 bezbedanRep1 :: [a] -> [a]
  bezbedanRep1 xs = if null xs then [] else tail xs
3
5 bezbedanRep2 :: [a] -> [a]
  bezbedanRep2 xs
    | xs == [] = []
    | otherwise = tail xs
7
9 bezbedanRep3 :: [a] -> [a]
  bezbedanRep3 [] = []
  bezbedanRep3 (_:xs) = xs
11 -- bezbedanRep3 xs = tail xs

```

Rešenje 5.38

```

1 savrseni :: Int -> [Int]
  savrseni n = [x | x <- [1..n-1], sum (faktori x) == x]
3   where faktori x = [i | i <- [1..x-1], x `mod` i == 0]

```

Rešenje 5.39

```

1 zbiriPar :: Int -> [(Int,Int)]
  zbiriPar n = [(a,b) | a <- [1..n], b <- [1..n], a + b == n]
3
5 -- drugi, efikasniji nacin
  zbiriPar' :: Int -> [(Int,Int)]
  zbiriPar' n = [(a,b) | a <- [1..n], b <- [n-a], b /= 0]

```

Rešenje 5.40

```

1 poslednji :: [a] -> a
  poslednji lista = lista !! poz
3   where poz = length lista - 1 -- moze i: length(tail lista)
5
  -- postoji funkcija last koja vraca poslednji element

```


Rešenje 5.41

```

1 spoji :: [[a]] -> [a]
  spoji [] = [] -- nije neophodno, prolazi drugi sablon i za praznu listu
3 spoji lista = [x | podlista <- lista, x <- podlista]

5 {-
  spoji [] = []
7 spoji (x:xs) = x ++ spoji xs
  -}

```

Rešenje 5.42

```

1 sufiksi :: [a] -> [[a]]
  sufiksi [] = [[]]
3 -- lista je sama svoj sufiks, a ostali sufiksi su sufiksi njenog repa
  sufiksi (x:xs) = (x:xs) : sufiksi xs

```

Rešenje 5.43

```

1 izbaci :: Int -> [a] -> [a]
  izbaci _ [] = []
3 izbaci k lst = foldr (\(i,x) acc -> if i == k then acc else x : acc) []
                  $ zip [0..] lst

5 {-
7   izbaci _ [] = []
   izbaci 0 (_:xs) = xs
9   izbaci k (x:xs) = x : (izbaci (k-1) xs)
  -}

```

Rešenje 5.44

```

1 ubaci :: Int -> a -> [a] -> [a]
  ubaci _ e [] = [e]
3 ubaci k e xs = foldr (\(i,x) acc -> if i == k then e : x : acc else x : acc) []
                  $ zip [0..] xs

5 {-
7   ubaci 0 n lista = n : lista
   ubaci k n [] = [n]
9   ubaci k n (x:xs) = x : (ubaci (k-1) n xs)
  -}

```

Rešenje 5.59

```

1 varijacije xs 0 = [[]]
  varijacije xs n = concat (map (\ x -> map (x:) ys) xs)
3   where ys = varijacije xs (n-1)

```

Rešenje 5.60

```

1 prosekOdlicni :: [[Integer]] -> Float
  prosekOdlicni = prosek . filter (>= 4.5) . map prosek
3   where prosek xs = realToFrac (sum xs) / fromIntegral (length xs)

```

Rešenje 5.61

```

1 -- Racunamo sve pozicije broja x u listi
  -- tako sto spajamo listu sa listom brojeva od 0 do n
3 -- i od liste parova (broj, pozicija)
  -- izdvajamo one pozicije kod kojih je broj = x
5 -- where omogucava da u okviru funkcije uvedemo novu promenljivu za neki izraz koji
  -- se koristi u definiciji

```

```

pozicije :: Eq a => a -> [a] -> [Int]
7 pozicije x [] = []
pozicije x lista = [i | (x1,i) <- zip lista [0..n], x == x1]
9     where n = length lista - 1

```

Rešenje 5.62

```

1 qsort :: Ord a => [a] -> [a]
qsort [] = []
3 qsort (x:xs) = qsort manji ++ [x] ++ qsort veci
    where manji = [a | a <- xs, a <= x]
          veci  = [b | b <- xs, b > x]
5

```

Rešenje 5.63

```

1 -- Pravimo listu bez uzastopnih ponavljanja
-- tako na glavu nadovezemo rezultat rekurzivnog poziva funkcije nad korigovanim
    repom
3 -- sa pocetka repa izbacujemo sve elemente koji su jednaki glavi
-- [1,1,1,1,2]
5 -- 1 : brojPonavljanja [2]
-- [1,2]
7 brisiPonavljanja :: Eq a => [a] -> [a]
brisiPonavljanja [] = []
9 brisiPonavljanja (x:xs) = x : brisiPonavljanja(dropWhile (==x) xs)

```

Rešenje 5.64

```

1 -- Pravimo listu koja sadrzi sva uzastopna pojavljivanja elemenata u posebnim listama
-- [1,1,1,2,2,3] -> [[1,1,1], [2,2], [3]]
3 -- tako sto pravimo listu uzastopnih pojavljivanja glave
-- i pozivamo funkciju rekurzivno pocev od elementa koji nije jednak glavi
5 -- [[1,1,1] , podlistePonavljanja [2,2,3]]
-- [[1,1,1], [2,2], podlistePonavljanja [3]]
7 -- [[1,1,1], [2,2], [3], podlistePonavljanja []]

9 podlistePonavljanja :: Eq a => [a] -> [[a]]
podlistePonavljanja [] = []
11 podlistePonavljanja (x:xs) = (x : (takeWhile (==x) xs)) : podlistePonavljanja(
    dropWhile (==x) xs)

```

Rešenje 5.65

```

1 -- [1,2,3] -> 321

3 brojObrnut :: Num a => [a] -> a
brojObrnut [] = 0
5 brojObrnut (x:xs) = (brojObrnut xs)*10 + x

7 -- [1,2,3] -> 123
-- Koriscenje funkcije brojObrnut
9 -- broj lista = brojObrnut (reverse lista)

11 -- Drugi nacin
broj [] = 0
13 broj (x:xs) = x*10^(length xs) + broj xs

```

Rešenje 5.66

```

1 -- Pravimo par listi od liste parova kao sto radi funkcija unzip
-- [(1,2), (1,2), (1,2)] -> ([1,1,1], [2,2,2])
3 -- tako sto svaki par iz liste dodajemo u akumulator cija je pocetna vrednost par
    praznih listi
-- [(1,2), (1,2), (1,2)] - ([],[ ])
5 -- [(1,2), (1,2)] - ([1],[2])

```

```

1  -- [(1,2)] -> ([1,1],[2,2])
7  -- [] -> ([1,1,1],[2,2,2])
   listaUPar :: [(a,b)] -> ([a],[b])
9  listaUPar [] = ([],[ ])
   listaUPar lista = foldr (\ (a,b) (c,d) -> (a:c, b:d)) ([],[ ]) lista
11
13 {- Drugi nacin bez foldr:
   listaUPar [] = ([],[ ])
   listaUPar ((a,b):xs) = (a:l1, b:l2)
15     where
17         l1 = fst rep
18         l2 = snd rep
19         rep = listaUPar xs
-}
```

Rešenje 5.67

```

1  -- Implementacija funkcije zip
2  -- [1,2,3] [4,5,6] -> [(1,4), (2,5), (3,6)]
3  -- Da bi radila po duzini krace liste neophodno je dodati sablone za takve situacije
   parOdListi [] _ = []
5  parOdListi _ [] = []
   -- Od glava listi pravimo par koji dodajemo na pocetak rezultujuce liste za rep
7  parOdListi (x:xs) (y:ys) = ( (x, y) : (parOdListi xs ys) )
```

Rešenje 5.68

```

1  -- [1,1,1] [2,2,2] -> [1,2,1,2,1,2]
2  -- [1,1] [2,2,2,2] -> [1,2,1,2,2,2]
3  -- Treba obuhvatiti slucajeve listi razlicitih duzina
4  -- Sledeca linija koda ne radi jer nije dozvoljeno koriscenje dzoker promenljive u
   izrazu kojim definisete vrednost funkcije
5  -- ucesljaj [] _ = _
   ucesljaj [] lista = lista
7  ucesljaj lista [] = lista
   ucesljaj (x:xs) (y:ys) = [x]++[y]++(ucesljaj xs ys)
```

6

Konkurentno programiranje

6.1 Jezik Scala

Scala (skr. *Scalable Language*) je moderni programski jezik kreiran od strane Martina Odersky-a 2003. napravljen sa ciljem da integriše osobine objektno-orijentisanih i funkcionalnih programskih jezika.

Literatura:

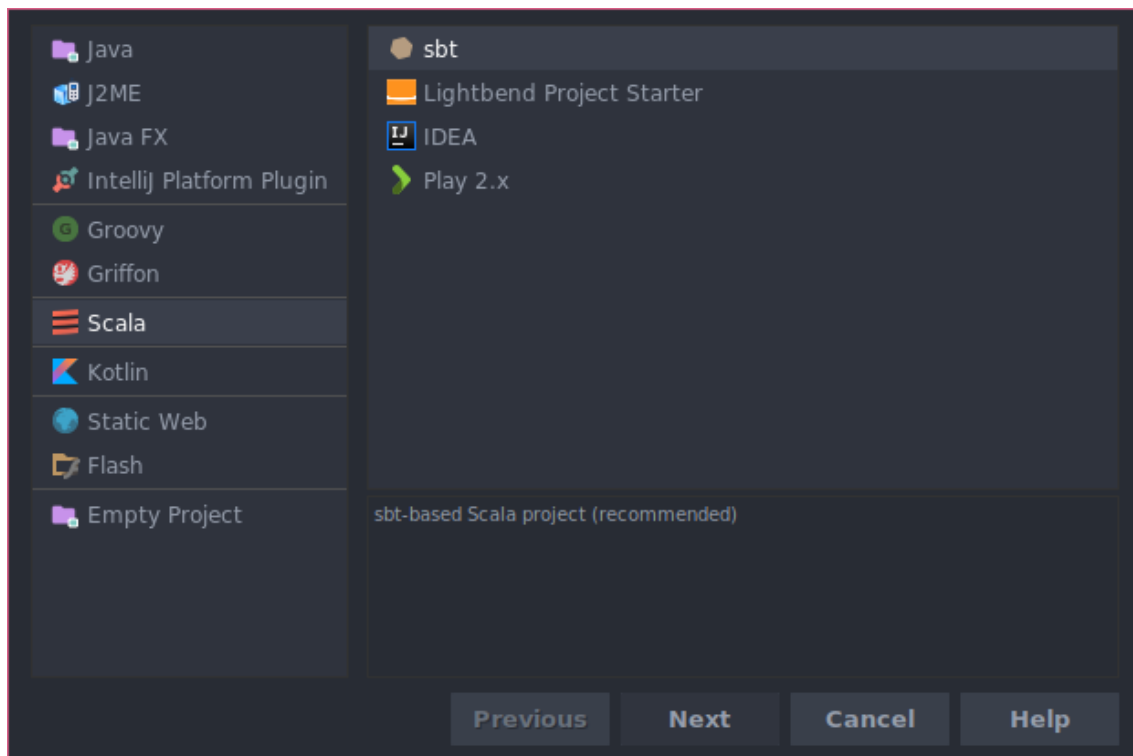
- (a) <https://docs.scala-lang.org/tour/tour-of-scala.html>
- (b) <https://www.tutorialspoint.com/scala/>
- (c) Master rad kolegice Ane Mitrović na temu primena Scala jezika u paralelizaciji rasplnutog testiranja: http://www.racunarstvo.matf.bg.ac.rs/MasterRadovi/2016_05_04_Ana_Mitrovic/rad.pdf

Neke osnovne osobine:

- (a) Scala je objektno-orijentisani jezik
Svaka vrednost je objekat, klase se mogu nasledjivati i postoje mehanizmi koji služe kao zamena za višestruko nasledjivanje
- (b) Scala je funkcionalni jezik
Svaka funkcija se tretira kao vrednost i svaka vrednost je objekat - dakle, svaka funkcija je objekat.
- (c) Scala daje konstrukte za konkurentno i sinhronizovano procesiranje
- (d) Scala je statički tipiziran jezik
- (e) Scala operiše na JVM
- (f) Scala može pokretati Java kod

Neke osnovne razlike u odnosu na Javu:

- (a) Svi tipovi su objekti (nema primitivnih tipova)
- (b) *Type-inference* - tipovi se mogu dedukovati od strane prevodioca
- (c) Funkcije se posmatraju kao objekti
- (d) Osobine, engl. *Traits* - enkapsuliraju definicije polja i metoda
- (e) Zatvorenja, engl. *Closures* - funkcije čije povratne vrednosti zavise od promenljivih deklariranih van te funkcije
- (f) Ne postoje statičke klase, umesto njih se koriste objekti (singletoni)

Slika 6.1: Početak pravljenja *sbt* projekta

6.2 Instalacija potrebnih alata

Koristimo jezik Scala (verzija 2.12.8) <http://www.scala-lang.org/>. Potrebno je imati instalirano:

- (a) IntelliJ Idea jetbrains.com/idea/
- (b) Scala plugin za IntelliJ Idea

6.2.1 Pravljenje projekta

Koristi se *sbt* (eng. Simple build tool) (koji će da kasnije da preuzima i konfiguriše dodatne biblioteke za nas). Na slici 6.1 je prikazano kako u okruženju odabrati *sbt* projekat.

Za pravljenje *sbt* projekta **neophodno** je imati aktivnu internet konekciju.

6.2.2 Inicijalizacija projekta

Okruženje se sada inicijalizuje, detektuju se Scala biblioteke i slično. Ovaj proces može da potraje od nekoliko sekundi do nekoliko minuta u zavisnosti od brzine mrežne konekcije i hardvera računara te treba biti strpljiv.

U donjem delu okruženja na sredini možete čitati poruke o tome šta se trenutno dešava, na primer:

```
sbt: dump project structure from sbt
```

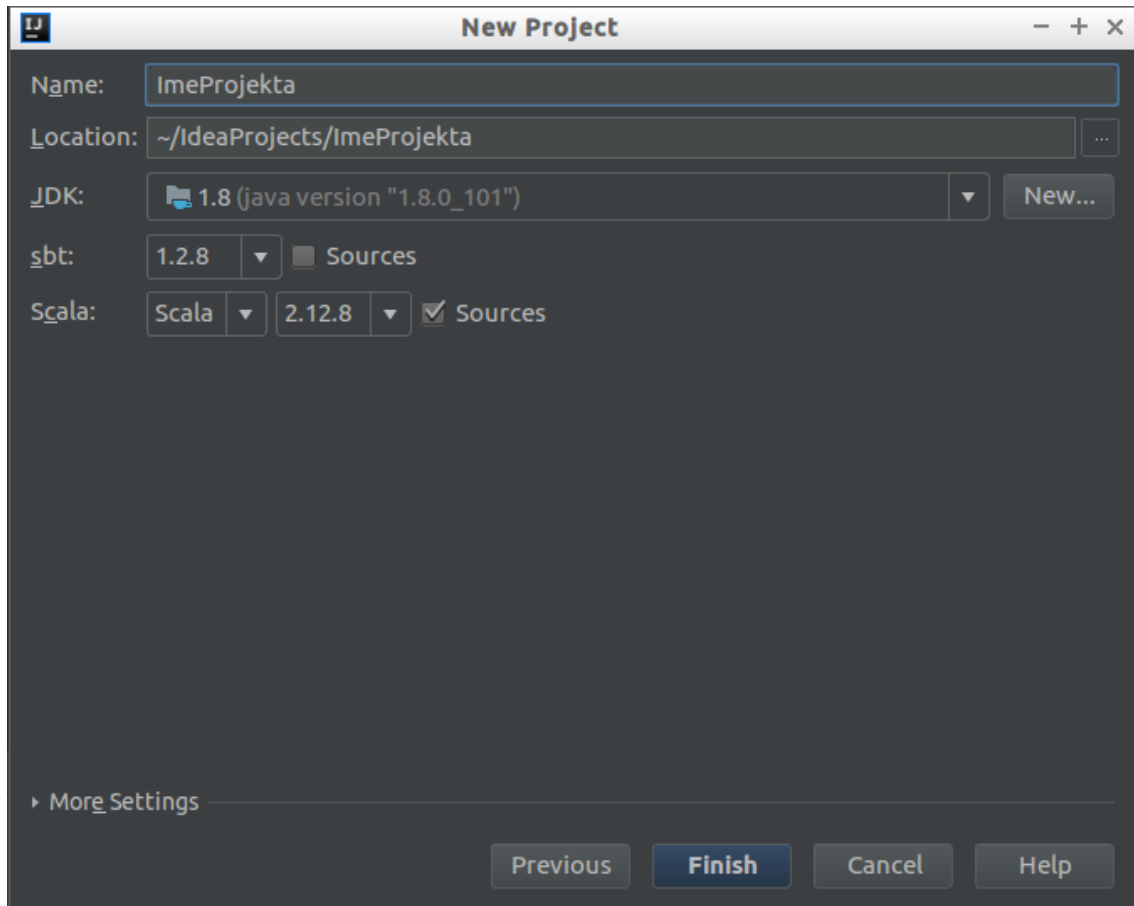
je jedan od koraka koje IntelliJ mora da izvrši kako bi pripremio sve za rad.

6.2.3 Konfiguracija projekta

Odaberite gde želite da se napravi projekat. Preporučeno je da imate direktorijum u kojem čuvate sve IntelliJ Idea projekte (uglavnom je to `home/korisnik/IdeaProjects`).

Za *sbt* i jezik Scala odaberite:

- *sbt*: 1.2.max (1.2.8)

Slika 6.2: Kraj pravljenja *sbt* projekta

- Scala: 2.12.max (2.12.8)

Na slici 6.2 prikazano je gde treba odabrati verzije za *sbt* i jezik *Scala*.
Kliknite *Finish*.

6.3 Uvod u jezik Scala

Zadatak 6.1 HelloWorld

```

1  /**
2   * Viselinijske komentare u Scali pisemo ovako (isto kao i u C-u).
3   *
4   * */
5
6  // Jednolinijske komentare pisemo ovako
7
8  object HelloWorld {
9    def main(args: Array[String]){
10      // Tacka zapeta na kraju svake naredbe je opcionalna
11      println("Hello world! :)")
12    }
13  }

```

Zadatak 6.2 Promenljive, niske, petlje, funkcije

```

1  object Uvod {
2    def main(args: Array[String]) {
3
4      println("----- Promenljive -----")
5    }
6  }

```

```

5 // Promenljive mozemo deklarirati kao konstantne
7 // ili promenljive cija se vrednost moze menjati.
8 // Ključna rec val deklarise konstantu a var promenljivu.
9 //
10 // ključnaRec imePromenljive : tipPromenljive = vrednost
11
12 val c : Int = 42
13 var x : Int = 6
14
15 x += 6
16
17 // Sledeca naredba prijavljuje gresku.
18 // `c += 4`
19
20 println("Konstanta: " + c)
21 println("Promenljiva: " + x)
22
23 if(x > 6)
24     println("Promenljiva x je veca od 6.")
25 else if(x == 6)
26     println("Promenljiva x je jednaka 6.")
27 else
28     println("Promenljiva x je manja od 6.")
29
30 println("----- Niske -----")
31
32 val crtani : String = "Maza i Lunja"
33 println("Duzina niske: '" + crtani + "' je " + crtani.length())
34
35 // Viselinjske niske pisemo izmedju trostrukih navodnika:
36 println(""" Likovi:
37         -Alisa
38         -Vendi
39         -Bambi
40         -Had
41         """)
42
43 // Interpolacija niski:
44 // Scala nam dozvoljava da ugnjezdavamo vrednosti promenljivih u niske
45 // dodavanjem karaktera s na pocetak niske (npr: s"Moja niska") omogucavamo
46 // ugnjezdavanje vrednosti promenljivih u nisku dodavajuci karakter $ pre imena
47 // promenljive
48 // (npr s"Kolicina: $kol grama")
49
50 var trajanje : Int = 76
51 println(s"Crtani film: '$crtani' - $trajanje minuta.")
52
53 println("----- Petlje -----")
54
55 var sat = 0
56 while(trajanje >= 60){
57     sat += 1
58     trajanje -= 60
59 }
60
61 println(s"'$crtani' traju $sat sat i $trajanje minuta.")
62
63 // For petljom mozemo iterirati kroz kolekcije koristeći sintaksu
64 //
65 // for(element <- kolekcija)
66 //
67 // 1 to 5 - pravi kolekciju brojeva [1,5]
68
69 println("FOR - 1 to 5 ")
70 for(i <- 1 to 5)
71     println(i)
72
73 // 1 until 5 - pravi kolekciju brojeva [1,5)
74
75 println("FOR - 1 until 5 ")
76 for(i <- 1 until 5)
77     println(i)

```

```

77 // Range(pocetak, kraj, korak) - pravi kolekciju [pocetak, kraj] sa zadatik
korakom.
79 // Korak moze biti i negativan npr. Range(10,0,-2)

81 println("FOR - Range(0,6,2) ")
83 for(i <- Range(0,6,2))
    println(i)

85
87 println("----- Nizovi -----")
87 // Pravimo ih na sledeci nacin:
87 // var niz : Array[tip] = new Array[tip](brojElemenata)
89 var crtaci : Array[String] = new Array[String](5)
91 crtaci(0) = "Petar Pan"
91 crtaci(1) = "Mulan"
93 crtaci(2) = "Aladdin"
93 crtaci(3) = "Herkules"
95 crtaci(4) = "Pocahontas"

97 println("Crtaci: ")
97 for(crtac <- crtaci)
    println(crtac)

99
101 println("----- Funkcije -----")
101 ispisiSortirano(crtaci)
103 }

103 // def imeFunkcije([listaArgumenata]) : povratnaVrednost = {
105 //     teloFunkcije
107 // }

107 // Povratna vrednost Unit je ekvivalentna void vrednosti
109 def ispisiSortirano(crtaci : Array[String]) : Unit = {
109     println("Sortirani crtaci:")
111     for(crt <- crtaci.sortWith(poredi))
113         println(crt)

113 // Anonimne funkcije
115 //
117 // ( listaArgumenata ) => { teloFunkcije }

117 println("Sortirani crtaci koristeći anonimnu funkciju:")
119 for(crt <- crtaci.sortWith((c1 , c2) => { if(c1.compareTo(c2) < 0) false else
true} ))
    println(crt)

121 }

123
125 def poredi(c1 : String, c2 : String) : Boolean = {
125     if(c1.compareTo(c2) < 0)
127         return true
127     else
129         return false
129 }
}

```

Zadatak 6.3 Klase, objekti, nasleđivanje

```

1 // Klase se u Scali konstruise na sledeci nacin:
2 //
3 // class ImeKlase {
4 //     teloKlase
5 // }
6 // ili
7 // class ImeKlase (argumentiKonstruktora) {
8 //     teloKlase
9 // }

11 class Film {
13     var naslov : String = ""
13     var trajanje : Int = 0
13     var godina : Int = 0

```



```

15 // Konstruktor
17 def this(nas : String, traj : Int, god : Int) = {
18     this()
19     this.naslov = nas
20     this.trajanje = traj
21     this.godina = god
22 }
23
24 // Metodi klase
25
26 def getNaslov() : String = {
27     // Poslednja naredba u metodu je njena povratna vrednost,
28     // tako da ne mora da se koristi `return` ključna rec
29     return this.naslov
30     // Slicno, blok ne mora da postoji ako funkcija
31     // ima samo jednu naredbu
32 }
33
34 // Konvencija je da ukoliko je metod cist (nema sporednih efekata)
35 // i ako nema argumenata, da se onda ne pisu zagrade. IntelliJ daje
36 // upozorenje kada se ova konvencija ne prati
37 def getTrajanje() : Int = this.trajanje
38
39 def getGodina : Int = this.godina
40
41 override def toString : String = {
42     "Film " + this.naslov + ", traje " + this.trajanje + " minuta, napravljen je " +
43     this.godina + " godine"
44 }
45
46 // Nasledjivanje
47 class CrtaniFilm extends Film {
48     var animator : String = ""
49
50     def this(nas : String, traj : Int, god : Int, anim : String) = {
51         this()
52         this.naslov = nas
53         this.trajanje = traj
54         this.godina = god
55         this.animator = anim
56     }
57
58     def getAnimator : String = this.animator
59
60     override def toString : String = "Crtani " + super.toString() + ", animator je " +
61     this.animator
62 }
63
64 // U Scali mozemo definisati takozvane 'singleton' objekte ključnom reci object.
65 // Garantuje se da ce postojati tacno jedan 'singleton' objekat na nivou naseg
66 // programa
67 // i on se najcesce koristi za implementaciju main metoda
68 object Program {
69     def main(args: Array[String]) {
70
71         val assassinsCreed = new Film("Assassin's Creed", 115, 2016)
72         val tarzan = new CrtaniFilm("Tarzan", 88, 1999, "Walt Disney")
73
74         println(assassinsCreed)
75         println(tarzan)
76
77         // Scala nudi brojne korisne kontejnerske i nizovske klase
78         // kao sto su Array, ArrayBuffer, Map, HashMap, Queue, Tuple1, Tuple2, i druge
79         // sa kojima cemo se susretati u narednim primerima
80         // gde ce biti vise objasnjene
81     }
82 }

```

6.4 Zadaci

Zadatak 6.4 Restoran Pet konobara radi u restoranu, i nakon što dođu na posao, poslovođa im dodeli broj stolova koje moraju da usluže. Napisati program koji učitava sa standardnog ulaza broj neusluženih stolova u restoranu i raspoređuje ih konobarima. Nakon što usluže jedan sto, konobari šalju poruku tako da poslovođa u svakom trenutku ima uvid u brojčano stanje. Konobare implementirati kao posebne niti koje uslužuju sto (spavaju nasumično izabran broj sekundi), ispisuju redni broj stola koji su uslužili i po završetku ispisuju poruku da su završili.

[Rešenje 6.4]

Zadatak 6.5 Množenje matrica Napisati program koji konkurentno množi dve matrice čiji se elementi nalaze u datotekama `matrica1.txt` i `matrica2.txt` i rezultat upisuje u datoteku `matrica3.txt`. Svaka nit treba da računa vrednosti za jednu vrstu rezultujuće matrice. Format podataka u datotekama je sledeći:

```

1  n m
2  a11 a12 a13 ... a1m
3  a21 a22 a21 ... a2m
4  ...
5  an1 an2 an3 ... anm

```

[Rešenje 6.5]

Zadatak 6.6 Broj karaktera DNK Napisati program koji konkurentno prebrojava koliko puta se koja baza (A, C, G, T) pojavljuje u DNK sekvenci koja se nalazi u višelinijskoj datoteci `bio_podaci.txt`. Sa standardnog ulaza učitati broj niti. Svakoj niti dodeliti određen broj linija koji će da obrađuje a rezultate čuvati u deljenoj mapi (pogodno je koristiti klasu `ConcurrentHashMap`).

[Rešenje 6.6]

Zadatak 6.7 Krediti Bogoljub otvara banku i poseduje određeni kapital. Kao svaki dobar ekonomista on odlučuje da zaradi na davanju kredita. Napisati program koji demonstrira rad službenica i davanje kredita u Bogoljubovoj banci. Sa standardnog ulaza učitati njegov početni kapital, kamatnu stopu i broj zaposlenih službenica. U datoteci `red_klijenata.txt` se nalazi spisak klijenata koji čekaju na red za razgovor sa službenicom u sledećem formatu:

```

1  imeKlijenta potrebnaPozajmica

```

Posao svake službenice se izvršava u posebnoj niti. Službenica poziva sledećeg klijenta iz reda na razgovor. Nakon što ustanovi koliko je novca klijentu potrebno, službenica proverava da li banka trenutno poseduje tu količinu novca i daje klijentu kredit tako što umanjuje kapital banke i klijentu računa vrednost duga u skladu sa kamatom. Ukoliko banka nije u mogućnosti da izda kredit, ispisati odgovarajuću poruku. Nakon što službenice završe sa svim klijentima iz reda ispisati poruku o ukupnoj zaradi banke.

[Rešenje 6.7]

Zadatak 6.8 Kladionica Napisati program koji simulira proces kladenja. Kladioničari se klade na ishod pet fudbalskih utakmica uplaćujući određenu količinu novca na tiket. U datoteci `kladioniciari.txt` se nalazi spisak kladioničara i njihovih tiketa u sledećem formatu:

```

1  imeKladionicara svotaNovca
2  utakmica1
3  rezultat
4  ...
5  utakmica5
6  rezultat
7  ...

```

Ishod može biti 1 - prva ekipa je pobedila, x - nerešeno, 2 - druga ekipa je pobedila. Svaki ishod nosi određenu kvotu kojom se množi novac koji je kladioničar uplatio. U datoteci `utakmice.txt` se nalazi spisak utakmica sa njihovim kvotama u formatu:

```
1 imeUtakmice
2 kvota1 kvotaX kvota2
3 ...
```

Učitati podatke o utakmicama i kladioničarima. Svaki kladioničar čeka na ishod utakmica u posebnoj niti. Kladionica nakon što dobije ishod utakmica (implementirati nasumično biranje ishoda) obaveštava kladioničare da su utakmice završene. Kladioničari nakon toga računaju sopstvenu zaradu kao zbir kvota pogođenih utakmica pomnoženih sa odgovarajućim delom novca i ispisuju poruku o pogodjenom rezultatu. Na kraju ispisati ukupnu količinu novca koju kladionica treba da isplati kladioničarima.

[Rešenje 6.8]

6.5 Zadaci za vežbu sa rešenjima

Zadatak 6.9 Zbir vektora Napisati program koji konkurentno sabira dva vektora. Sa standardnog ulaza se učitava dimenzija vektora, elementi oba vektora i broj niti. Svakoj niti dodeliti indeks početka i kraja vektora nad kojim računa zbir a rezultat smeštati u prvi vektor. Indekse računati na osnovu dimenzije vektora i broja niti. Rezultujući vektor ispisati na standardni izlaz.

[Rešenje 6.9]

Zadatak 6.10 Broj petocifrenih brojeva Napisati program koji konkurentno računa broj pojavljivanja petocifrenih brojeva koji se nalaze u datotekama čija imena se učitavaju sa standardnog ulaza. Svakoj niti dodeliti po jednu datoteku nad kojom će računati. Rezultate brojanja smeštati u lokalne promenljive unutar niti i po završetku računanja za svaku datoteku ispisati broj pojavljivanja petocifrenih brojeva.

[Rešenje 6.10]

Zadatak 6.11 Berba Milovan ima voćnjak u kome gaji trešnje, kajsije, kruške i šljive. Došlo je vreme berbe i mora uposliti mlade studente da obru voćnjak. U datoteci `drvoredi.txt` se nalaze podaci o drvoredima voćnjaka u sledećem formatu:

```
1 vrstaVoća brojStabala
```

Sa standardnog ulaza učitati broj zaposlenih studenata a iz datoteke učitati podatke o drvoredima. Svaka nit predstavlja jednog studenta. Student odlazi do jednog drvoreda skidajući ga iz reda drvoreda koje je potrebno obrati i počinje branje. Ako pretpostavimo da jedno stablo voća može roditi od 30-50 kilograma voća, student za svako stablo iz drvoreda nasumično računa broj kilograma voća koji je obran i dodaje ih u skladište. Ukoliko su svi drvoredi obrani studenti prestaju sa radom i na standardni izlaz se ispisuje ukupna količina obranog voća svake vrste.

[Rešenje 6.11]

Zadatak 6.12 Turistička agencija Turistička agencija FlyProgrammer organizuje nagradnu igru i daje pet vaučera od 20% popusta na cenu kupljenje karte svojim klijentima. U datoteci `ucesnici.txt` se nalaze podaci o klijentima i cenama u sledećem formatu:

```
1 ime prezime
2 cena
```

Napisati program koji simulira nagradnu igru. Svaka nit čeka na rezultate nagradne igre za jednog klijenta. Turistička agencija izvlači dobitnike slučajnom selekcijom i nakon završenog izvlačenja obaveštava niti. Niti proveravaju da li je izvučen odgovarajući klijent i ispisuje poruku o ishodu.

[Rešenje 6.12]

6.5.1 Zadaci za vežbu

Zadatak 6.13 Množenje vektora skalarom Napisati program koji konkurentno množi vektor skalarom. Sa standardnog ulaza učitati dimenziju i elemente vektora, skalar i broj niti. Svakoj niti dodeliti deo vektora (indekse početnog i krajnjeg elementa). Nit računa proizvod vektora skalarom za elemente u zadatom opsegu i rezultat smešta u isti vektor. Na kraju ispisati rezultat na standardni izlaz.

Zadatak 6.14 Množenje matrice vektorom Napisati program koji konkurentno množi matricu vektorom. Sa standardnog ulaza učitati dimenziju i elemente vektora, dimenzije i elemente matrice i broj niti. Svakoj niti dodeliti deo matrice (indekse početne i krajnje vrste). Nit računa proizvod matrice vektorom za vrste u zadatom opsegu i rezultat smešta u nov vektor. Na kraju ispisati rezultat na standardni izlaz.

Zadatak 6.15 Transponovanje matrice Napisati program koji konkurentno transponuje matricu. U datoteci `matrica.txt` se nalaze dimenzije matrice nakon čega slede elementi matrice. Svaka nit transponuje po jednu vrstu matrice i smešta rezultat u rezultujuću matricu. Na kraju izračunavanja rezultujuću matricu upisati u datoteku `transponovana_matrica.txt`.

Zadatak 6.16 Matrice (dodatak) Implementirati zadatke 6.5 i 6.9 tako da se broj niti učitava sa standardnog ulaza, i svakoj niti dodeliti broj vrsta koje će obrađivati.

Zadatak 6.17 Funkcije nad vektorima Napisati program koji konkurentno računa proizvod, sumu, prosečnu vrednost, broj negativnih i broj pozitivnih elemenata vektora. Sa standardnog ulaza učitavati imena datoteka u kojima se nalaze vektori. Implementirati konkurentno računanje na dva načina, koristeći paralelizaciju zadataka i paralelizaciju podataka i uporediti vremena izvršavanja (obratiti pažnju na to da je korišćenjem paralelizacije zadataka najveće moguće ubrzanje jednako najsporijem zadatku dok kod paralelizacije podataka ubrzanje zavisi od broja procesora, broja niti,...). Na kraju ispisati rezultate svih izračunavanja na standardni izlaz.

Zadatak 6.18 Broj petocifrenih brojeva (dodatak) Implementirati zadatak 6.10 uz sledeće izmene. Brojeve učitati iz jedne datoteke `brojevi.txt` u niz a broj niti učitati sa standardnog ulaza. Svakoj niti dodeliti deo niza brojeva koji će obrađivati a rezultat čuvati u deljenoj promenljivoj tipa `AtomicLong`.

Zadatak 6.19 Broj karaktera DNK (dodatak) Implementirati zadatak 6.6 tako da se rezultati brojanja baza čuvaju u deljenim promenljivama tipa `AtomicLong` i na kraju ispisati procenat pojavljivanja svake baze u lancu.

Zadatak 6.20 Hotel Hotel Nightlife brine o svojim klijentima i trudi se da im smanji vreme čekanja dok se sređuje soba koju su rezervisali. U datoteci `sobe.txt` nalaze se brojevi soba koje je potrebno srediti. Sa standardnog ulaza učitati broj trenutno dostupnih čistačica. Svaka nit predstavlja jednu čistačicu. Čistačica skida sobe sa reda soba koje je potrebno srediti i odlazi do nje da je sredi (ispisati redni broj sobe u koju je ušla čistačica a nit uspavati na slučajan broj sekundi). Nakon što je sredila sobu, čistačica pronalazi ostavljen bakšiš (slučajan broj od 0-500 dinara) i po dogovoru bakšiš ubacuje u zajedničku kutiju. Kada završi sa jednom sobom, čistačica odlazi do sledeće sobe (skida je sa reda) i nastavlja sve dok red ne postane prazan. Nakon što završe sa čišćenjem, čistačice otvaraju kutiju sa bakšišem i dele novac na ravne časti tj. ispisuju na standardan izlaz dobijeni bakšiš.

Zadatak 6.21 Loto Napisati program koji simulira izvlačenje na Loto-u. Izvlače se tri broja iz opsega [1,37] a ukupna vrednost nagradnog fonda se unosi sa standardnog ulaza. Učesnici uplaćuju srećke i pokušavaju da pogode tri broja. U datoteci `ucesnici.txt` se nalaze informacije o uplaćenim srećkama u sledećem formatu:

```
1 ime
2 broj1 broj2 broj3
3 ...
```

Učitati podatke o učesnicima. Svaki učesnik čeka na kraj izvlačenja u posebnoj niti. Izvlačenje se odvija u glavnoj niti tako što se nasumično biraju tri broja iz opsega [1,37] i po završetku se

obaveštavaju niti učesnika. Niti učesnika nakon toga porede izvučene brojeve i ukoliko se sva tri poklapaju ispisuju poruku i u ukupnu sumu novca koji je potrebno isplatiti dodaju vrednost nagradnog fonda. Ukoliko se dva od tri broja poklapaju u ukupnu sumu se dodaje 40% nagradnog fonda. Ukoliko se samo jedan broj poklapa u ukupnu sumu se dodaje 10% nagradnog fonda. Na kraju ispisati količinu novca koju je potrebno isplatiti i količinu novca koju je lutrija zaradila (ukoliko zarada postoji).

6.6 Rešenja

Rešenje 6.4 Restoran

```

1 import java.util.concurrent._
2 import java.util.Scanner

4 // Pravljenje niti
5 //
6 // Da bismo napravili nit potrebno je da definisemo klasu koja nasledjuje klasu
   Thread
7 // i implementiramo metod run cije izvršavanje pocinje kada nad instancom nase klase
8 // pozovemo metod start.
9 //
10 // Drugi nacin je da nasa klasa implementira interfejs Runnable i implementira metod
   run.
11 // Medjutim, da bismo naglasili da zelimo da se metod run nase instance izvršava kao
   posebna nit
12 // potrebno je da napravimo instancu tipa Thread kojoj u konstruktoru treba da
   prosledimo
13 // instancu nase klase (koja implementira metod run.

14 class Konobar(ime : String, brStolova : Int) extends Thread {
15   override def run(){
16     for(i <- 0 until brStolova){
17       // Klasa ThreadLocalRandom predstavlja generator slucajnih brojeva
18       // jedinstven na nivou jedne niti.
19       // Metod current() vraca objekat ove klase za trenutnu nit.
20       Thread.sleep(ThreadLocalRandom.current().nextInt(1,10)*1000)
21       println("Konobar " + ime + " je usluzio " + i + ". sto.")
22     }
23     println("Konobar " + ime + " je završio sa posluživanjem.")
24   }
25 }

26

28 object Restoran {
29   def main(args : Array[String]) {
30     val sc : Scanner = new Scanner(System.in)

32     println("Unesite broj neusluzenih stolova u restoranu: ")
33     val brojStolova = sc.nextInt()

34     val korak = Math.ceil(brojStolova/5.0).toInt

36     // Pravimo instance niti
37     val stefan = new Konobar("Stefan", korak)
38     val nikola = new Konobar("Nikola", korak)
39     val filip = new Konobar("Filip", korak)
40     val nebojsa = new Konobar("Nebojsa", korak)
41     val djordje = new Konobar("Djordje", brojStolova - 4*korak)

42     // Ukoliko nasa klasa implementira interfejs Runnable
43     // val stefan = new Thread(new Konobar("Stefan", 10))

44     // Metod start zapocinje izvršavanje metoda run
45     stefan.start()
46     nikola.start()
47     filip.start()
48     nebojsa.start()
49     djordje.start()
50     // Nevalidno, tj ne radi kako ocekujemo: stefan.run()
51   }
52 }

```

}

Rešenje 6.5 Množenje matrica

```

1 import java.util.Scanner
2 import java.io.PrintWriter
3 import java.io.File
4 import scala.Array._
5
6 class Mnozilaac(vrstal : Array[Int],
7                 matrica2 : Array[Array[Int]],
8                 rezultat : Array[Int])
9   extends Thread {
10
11   var k : Int = matrica2.length*matrica2(1).length / vrstal.length
12   var m : Int = vrstal.length
13
14   override def run() {
15     for(i <- 0 until k)
16       rezultat(i) = skProizvod(i)
17   }
18
19   def skProizvod(j : Int) : Int = {
20     var res = 0
21     for(i <- 0 until m)
22       res += vrstal(i)*matrica2(i)(j)
23   }
24   res
25 }
26
27 object MnozenjeMatrica {
28   def main(args : Array[String]) {
29
30     val sc1 : Scanner = new Scanner(new File("matrica1.txt"))
31     val sc2 : Scanner = new Scanner(new File("matrica2.txt"))
32     val pw : PrintWriter = new PrintWriter(new File("matrica3.txt"))
33
34     // Ucitavamo dimenzije matrica
35     val n = sc1.nextInt()
36     val m1 = sc1.nextInt()
37     val m2 = sc2.nextInt()
38     val k = sc2.nextInt()
39
40     if(m1 != m2){
41       println("Greska! Dimenzije matrica se moraju poklapati!")
42       return
43     }
44
45     // Funkcija ofDim[Tip](n,m) pravi videdimenzioni niz dimenzija mxn
46     val matrica1 = ofDim[Int](n,m1)
47     val matrica2 = ofDim[Int](m2,k)
48     val rezultat = ofDim[Int](n,k)
49
50     // Ucitavamo elemente prve matrice
51     for(i <- 0 until n)
52       for(j <- 0 until m1)
53         matrica1(i)(j) = sc1.nextInt()
54
55     // Ucitavamo elemente druge matrice
56     for(i <- 0 until m2)
57       for(j <- 0 until k)
58         matrica2(i)(j) = sc2.nextInt()
59
60     val mnozioci = new Array[Mnozilaac](n)
61
62     // Pravimo niz niti koje ce da racunaju i-tu vrstu rezultata mnozenja matrica
63     for(i <- 0 until n)
64       mnozioci(i) = new Mnozilaac(matrica1(i), matrica2, rezultat(i))
65
66     // Zapocinjemo izvorsavanje niti
67     for(i <- 0 until n)
68       mnozioci(i).start()

```

```

69 // Cekamo da niti zavrse sa izracunavanjem
71 for(i <- 0 until n)
    mnozioci(i).join()
73
75 // Upisujemo rezultujucu matricu u datoteku
76 pw.append(s"$n $k \n")
77 for(i <- 0 until n){
78     for(j <- 0 until k)
79         pw.append(s"${rezultat(i)(j)} ")
80     pw.append("\n")
81 }
82 pw.close()
83 }
84 }

```

Rešenje 6.6 Broj karaktera DNK

```

import java.util.concurrent._
import java.util.Scanner
import java.io.File
import scala.collection.mutable.ArrayBuffer

class Brojac(poc : Int, kraj : Int,
             linije : ArrayBuffer[String],
             mapaKaraktera : ConcurrentHashMap[Char, Int])
    extends Thread {

    override def run() {
        for(i <- poc until kraj){
            // Racunamo broj svakog karaktera u liniji
            val a = linije(i).count(_=='a')
            val c = linije(i).count(_=='c')
            val g = linije(i).count(_=='g')
            val t = linije(i).count(_=='t')

            // Synchronized kljucna rec obelezava kriticku sekciju
            // i garantuje se da u svakom trenutku tacno jedna nit moze izvsavati naredbe
            // iz bloka.
            // Synchronized se moze koristiti na vise nacina:
            //
            // Metodi klase
            //
            // def f() = synchronized { teloFunkcije }
            //
            // Na ovaj nacin smo naglasili da je metod f jedne instance nase klase
            // kriticka sekcija
            // i u svakom trenutku tacno jedna nit moze izvsavati ovaj metod te instance.
            //
            // Blok instance
            //
            // instanca.synchronized { blok }
            //
            // Na ovaj nacin smo naglasili da za datu instancu u svakom trenutku
            // tacno jedna nit moze izvsavati naredbe bloka
            //
            // Ovakvo ponasanje mozemo posmatrati i iz drugacijeg ugla.
            //
            // Instanca predstavlja monitor objekat.
            //
            // U trenutku kada nit pozeli da izvsava blok kritickne sekcije,
            // ona zakljucava instancu, izvrsi kriticku sekciju i otkljucava instancu.
            //
            // Treba teziti ka tome da kriticka sekcija bude sto manja
            //
            // kako bismo sto vise iskoristili prednosti konkurentnog izvsavanja
            //
            // TODO: Zakomentarisati synchronized blok i videti sta se desava prilikom
            // pokretanja programa
            //
            // sa razlicitim brojem niti.
            //
            // Moguci scenario je da jedna nit procita vrednost iz mape, druga nit nakon
            // toga azurira mapu,
            //
            // a prva nit i dalje drzi vrednost koju je procitala pre azuriranja tako da
            // kada ona azurira mapu
            //
            // rezultat azuriranja nece biti ispravan.
        }
    }
}

```

```

50     //
51     mapaKaraktera.synchronized {
52         mapaKaraktera.replace('a', mapaKaraktera.get('a')+a)
53         mapaKaraktera.replace('c', mapaKaraktera.get('c')+c)
54         mapaKaraktera.replace('g', mapaKaraktera.get('g')+g)
55         mapaKaraktera.replace('t', mapaKaraktera.get('t')+t)
56     }
57 }
58 }
59
60 object BrojKarakteraDNK {
61     def main(args : Array[String]) {
62         val sc1 : Scanner = new Scanner(new File("bio_podaci.txt"))
63         val sc2 : Scanner = new Scanner(System.in)
64
65         println("Unesite broj niti: ")
66         println("Broj procesora na racunaru koji su na raspolaganju je : " + Runtime.
67             getRuntime.availableProcessors())
68
69         val brojNiti = sc2.nextInt()
70
71         val brojaci = new Array[Brojac](brojNiti)
72         // Klasa ArrayBuffer predstavlja niz promenljive duzine
73         // Neki od korisnih metoda su:
74         // - append(e) - dodaje element na kraj niza
75         // - isEmpty() - vraca true ukoliko je niz prazan, false inace
76         // - insert(i, e) - dodaje element na datu poziciju
77         // ...
78         val linije = new ArrayBuffer[String]()
79
80         while(sc1.hasNextLine)
81             linije.append(sc1.nextLine())
82
83         val brojLinija = linije.length
84         println(brojLinija)
85
86         // Klasa ConcurrentHashMap predstavlja implementaciju mape cije su operacije
87         // bezbedne u kontekstu konkurentnog izvršavanja.
88         // To znaci da u svakom trenutku tacno jedna nit moze izvršavati operacije nad
89         // mapom.
90         // Medjutim, operacije citanja (get) su neblokirajuće, tako da se mogu
91         // preklopiti sa drugim operacijama (npr. azuriranja)
92         // i u takvim slucajevima se ne garantuje azurnost rezultata.
93         //
94         // Konstruktor prima tri argumenta:
95         // - inicijalni kapacitet mape
96         // - faktor povecavanja mape
97         // - broj niti koji se pretpostavlja da ce konkurentno pristupati objektu mape
98         //
99         // Neki od korisnih metoda klase ConcurrentHashMap su:
100        // - get(kljuc) - vraca element sa zadatim kljucem, odnosno null ukoliko takav
101        // ne postoji
102        // - put(kljuc, vrednost) - dodaje element sa zadatim parametrima
103        // - remove(kljuc) - uklanja element sa zadatim kljucem
104        // - replace(kljuc, vrednost) - postavlja vrednost elementu sa zadatim kljucem
105        // - size() - vraca velicinu mape
106        // - isEmpty() - vraca true ukoliko je mapa prazna, false inace
107        // ...
108        //
109        val mapaKaraktera = new ConcurrentHashMap[Char, Int](4,4,brojNiti)
110        mapaKaraktera.put('a', 0)
111        mapaKaraktera.put('c', 0)
112        mapaKaraktera.put('g', 0)
113        mapaKaraktera.put('t', 0)
114
115        val korak = Math.ceil(brojLinija.toDouble/brojNiti.toDouble).toInt
116
117        for(i <- 0 until brojNiti)
118            brojaci(i) = new Brojac(i*korak, Math.min((i+1)*korak, brojLinija), linije,
119                mapaKaraktera)
120
121        for(b <- brojaci)

```



```

116         b.start()
118     for(b <- brojaci)
119         b.join()
120
121     println("Rezultati konkurentnog izvršavanja")
122     println("A: " + mapaKaraktera.get('a'))
123     println("C: " + mapaKaraktera.get('c'))
124     println("G: " + mapaKaraktera.get('g'))
125     println("T: " + mapaKaraktera.get('t'))
126
127     println("Pravi rezultati \nA: 1761 \nC: 1577 \nG: 1589 \nT: 1913")
128 }
}

```

Rešenje 6.7 Krediti

```

1 import java.util.concurrent.atomic._
2 import java.util.concurrent._
3 import java.util.Scanner
4 import java.io.File
5
6 class Sluzbenica(kamata : Int,
7                 kapital : AtomicLong,
8                 redKlijenata : ConcurrentLinkedQueue[Klijent],
9                 zaduzeniKlijenti : ConcurrentLinkedQueue[Klijent])
10
11 extends Thread {
12
13 override def run() {
14     while(true){
15         // Dohvatamo sledeceg klijenta iz reda
16         var k : Klijent = redKlijenata.poll()
17         // Ukoliko takav ne postoji završavamo
18         if(k == null)
19             return
20
21         println("Klijent " + k.getIme() + " razgovara sa sluzbenicom.")
22         Thread.sleep(ThreadLocalRandom.current().nextInt(1,10)*1000)
23         // Iako je kapital objekat AtomicLong i garantuje se atomicnost operacija
24         // azuriranja
25         // mogu nastati problemi prilikom konkurentnog pristupanja i azuriranja,
26         // i zbog toga je potrebno operacije sa ovim objektom obmotati synchronized
27         // blokom.
28         // Problem može nastati u delu koda (*****).
29         // Pretpostavimo da dve niti izvršavaju ovaj deo koda konkurentno.
30         // Prva nit pročita vrednost kapitala, nakon toga druga nit pročita vrednost
31         // kapitala
32         // pre nego što je prva nit izmenila vrednost, odmah posle prva nit promeni
33         // vrednost kapitala
34         // i postavi novu vrednost (staraVrednost - prvaPozajmica)/
35         // U ovom trenutku druga nit ima vrednost kapitala koja nije ispravna sa
36         // kojom dalje operise.
37         // Druga nit promeni vrednost kapitala i postavi novu vrednost (
38         // staraVrednost - drugaPozajmica)
39         // što nije realna vrednost (staraVrednost - prvaPozajmica - drugaPozajmica)
40         kapital.synchronized {
41             if(k.getPozajmica > kapital.get())
42                 println("Klijent " + k.getIme() + " ne može dobiti kredit.")
43             else{
44                 k.setDug(k.getPozajmica*((100+kamata.toFloat)/100))
45                 // *****
46                 val novKapital = kapital.get() - k.getPozajmica
47                 kapital.set(novKapital)
48                 // *****
49                 println("Klijent " + k.getIme() + " je dobio kredit u iznosu od "
50                     + k.getPozajmica + "e odnosno sa kamatom " + k.getDug + "e.")
51                 zaduzeniKlijenti.add(k)
52             }
53         }
54     }
55 }
56
57 }
58
59 }
60
61 }

```

```

51 class Kljent(ime : String, pozajmica : Int) {
    var dug : Float = 0
53
    def getIme : String = ime
55 def getPozajmica : Int = pozajmica
    def getDug : Float = dug
57
    def setDug(d : Float) : Unit = {
59     dug = d
    }
61 }

63 object Banka {
    def main(args : Array[String]) {
65         // Klasa AtomicLong predstavlja enkapsulaciju long integer vrednosti
        // nad kojom se operacije azuriranja izvrsavaju atomicno.
67         //
        // Neki od korisnih metoda ove klase su:
69         // - get() - vraca trenutnu vrednost
        // - set(v) - postavlja vrednost v
71         // - getAndAdd(v) - atomicki dodaje vrednost v i vraca prethodnu vrednost
        // - addAndGet(v) - atomicki dodaje vrednost v i vraca novu vrednost
73         // - getAndIncrement() - atomicki inkrementira vrednost i vraca prethodnu
        //   vrednost
        // - incrementAndGet() - atomicki inkrementira i vraca novu vrednost
75         // - getAndDecrement() - atomicki dekrementira vrednost i vraca prethodnu
        //   vrednost
        // - decrementAndGet() - atomicki dekrementira i vraca novu vrednost
77         // - compareAndSet(ocekivanaVrednost, novaVrednost) - postavlja novu vrednost
        //   ukoliko je stara jednaka ocekivanoj
79         //
        // U paketu java.util.concurrent.atomic postoje i druge korisne klase kao sto
        // su
81         // AtomicBoolean, AtomicIntegerArray, AtomicInteger itd.
        val sc1 : Scanner = new Scanner(System.in)
83
        println("Unesite pocetni kapital banke: ")
85         val kapital = new AtomicLong(sc1.nextLong())
        val sacuvanKapital : Float = kapital.get()
87
        println("Unesite kamatnu stopu: ")
89         val kamata = sc1.nextInt()

91         println("Unesite broj sluzbenica u ekspozituri: ")
        val sluzbenice = new Array[Sluzbenica](sc1.nextInt())
93
        val sc2 : Scanner = new Scanner(new File("red_klijenata.txt"))
95
        // Klasa ConcurrentLinkedQueue predstavlja implementaciju reda
97         // cije su operacije bezbedne u kontekstu konkurentnog izvršavanja.
        //
99         // Neki od korisnih metoda su:
        // - add(e) - dodaje element u red
101        // - poll() - skida element sa pocetka reda i vraca ga kao rezultat
        // - peek() - vraca element sa pocetka reda (ne skida ga)
103        // - remove(e) - uklanja element e iz reda
        // - isEmpty() - vraca true ukoliko je red prazan
105        // ...
        //
107        val redKlijenata = new ConcurrentLinkedQueue[Kljent]()
        val zaduzeniKlijenti = new ConcurrentLinkedQueue[Kljent]()
109
        while(sc2.hasNextLine)
111            redKlijenata.add(new Kljent(sc2.next(), sc2.nextInt()))

113        // .indices() vraca Range svih indeksa niza
        for(i <- sluzbenice.indices)
115            sluzbenice(i) = new Sluzbenica(kamata, kapital, redKlijenata, zaduzeniKlijenti)

117        for(s <- sluzbenice)
            s.start()
119

```

```

121     for(s <- sluzbenice)
        s.join()

123     // Iteriramo kroz red zaduženih klijenata i racunamo ukupno zaduženje
    var ukupnoZaduzenje : Float = 0
125     val iterator = zaduzeniKlijenti.iterator()
    while(iterator.hasNext)
127         ukupnoZaduzenje += iterator.next().getDug

129     println(s"Banka je zaradila ${ukupnoZaduzenje-sacuvanKapital}e.")
    }
131 }

```

Rešenje 6.8 Kladionica

```

1  import java.util.concurrent._
    import java.util.Scanner
3   import java.io.File
    import scala.collection.mutable.HashMap
5   import scala.collection.mutable.ArrayBuffer

7   class Kladionicar(ime : String,
                     novac : Int,
9                      tiket : HashMap[String, Char],
                     utakmice : HashMap[String, (Float, Float, Float, Char)])
11  extends Thread {

13      var zarada : Float = 0

15      override def run(){
        // Cekamo da se odigraju sve utakmice
17        // Funkcije wait(), notify() i notifyAll()
        // moraju biti zakljucane unutar bloka synchronized
19        utakmice.synchronized {
            utakmice.wait()
21        }

23        var pogodjeno = 0
        var ukupnaKvota : Float = 0
25        // Racunamo ukupnu zaradu
        for(t <- tiket)
27            if(t._2 == utakmice(t._1)._4){
                println(ime + " je pogodio utakmicu " + t._1 + " - " + utakmice(t._1)._4)
29                pogodjeno += 1
                if(utakmice(t._1)._4 == '1')
31                    ukupnaKvota += utakmice(t._1)._1
                else if(utakmice(t._1)._4 == 'x')
33                    ukupnaKvota += utakmice(t._1)._2
                else if(utakmice(t._1)._4 == '2')
35                    ukupnaKvota += utakmice(t._1)._3
            }
37        if(pogodjeno != 0)
            zarada = ukupnaKvota * novac/pogodjeno
39    }

41    def getIme : String = ime
    def getZarada : Float = zarada
43 }

45 object Kladionica {
    def main(args : Array[String]) {
47        val sc1 : Scanner = new Scanner(new File("utakmice.txt"))
        val sc2 : Scanner = new Scanner(new File("kladionicari.txt"))
49        // Klasa HashMap iz paketa scala.collection.mutable
        // predstavlja implementaciju mape koja se moze azurirati (eng. mutable)
51        //
        // Neke od korisnih funkcija su:
53        // -put(k,v) - dodaje vrednost u mapu sa zadatim kljucem
        // -size - vraca velicinu mape
55        // -contains(k) - vraca true ukoliko postoji element sa zadatim kljucem, false
        // inace
        // ...
    }
}

```

```

57 //
58 // Takodje mozemo iterirati kroz elemente mape for petljom
59 //
60 // Klasa Tuple4 je jedna u nizu klasa koje implementiraju torke (Tuple1, Tuple2,
61 // Tuple3,...)
62 // koje se mogu azurirati.
63 // Elementima torke pristupamo na sledeci nacin - torka._1, torka._2, torka._3,
64 // torka._4
65
66 val utakmice = new HashMap[String, (Float, Float, Float, Char)]()
67
68 // Rezultate utakmica postavljamo da budu karakter '-'
69 // kako bismo naglasili da se utakmice jos nisu odigrale
70 while(sc1.hasNextLine){
71     utakmice.put(sc1.nextLine(),(sc1.nextFloat(), sc1.nextFloat(), sc1.nextFloat(),
72     '-'))
73     sc1.nextLine()
74 }
75 val kladioniciari = new ArrayBuffer[Kladionicar]()
76
77 while(sc2.hasNextLine) {
78     val ime = sc2.next()
79     val novac = sc2.nextInt()
80     val tiket = new HashMap[String, Char]()
81     for(i <- 0 until 5){
82         sc2.nextLine()
83         tiket.put(sc2.nextLine(), sc2.next()(0))
84     }
85     kladioniciari.append(new Kladionicar(ime, novac, tiket, utakmice))
86 }
87
88 for(k <- kladioniciari)
89     k.start()
90
91 println("Cekamo da se utakmice odigraju.")
92 Thread.sleep(5000)
93
94 // Racunamo rezultate utakmica
95 val res = Array('1', 'x', '2')
96 for(u <- utakmice)
97     utakmice(u._1) = (u._2._1,
98     u._2._2,
99     u._2._3,
100     res(ThreadLocalRandom.current().nextInt(0, 3))
101 )
102
103 // Ulazimo u kriticku sekciju
104 // i obavestavamo niti koje cekaju
105 utakmice.synchronized {
106     utakmice.notifyAll()
107 }
108
109 for(k <- kladioniciari)
110     k.join()
111
112 var isplata : Float = 0
113 for(k <- kladioniciari){
114     isplata += k.getZarada
115     println(k.getIme + " ceka na isplatu " + k.getZarada + " dinara.")
116 }
117 println("Ukupno kladionica treba da isplati " + isplata + " dinara.")
118 }
119 }

```

Rešenje 6.9 Zbir vektora

```

1 import java.util.Scanner
2
3 class Sabirac(poc : Int,
4             kraj : Int ,
5             vektor1 : Array[Float],
6             vektor2 : Array[Float])

```

```

1  extends Thread {
2  // Svaka nit racuna zbir svog dela vektora [poc, kraj]
3  // i rezultat smesta u prvi vektor.
4  override def run() {
5      for(i <- poc until kraj)
6          vektor1(i) += vektor2(i)
7  }
8  }
9
10 object ZbirVektora {
11     def main(args : Array[String]){
12
13         val sc = new Scanner(System.in)
14
15         println("Unesite dimenziju vektora: ")
16         val n = sc.nextInt()
17
18         val vektor1 : Array[Float] = new Array(n)
19         val vektor2 : Array[Float] = new Array(n)
20
21         println("Unesite elemente prvog vektora: ")
22         for(i <- 0 until n)
23             vektor1(i) = sc.nextFloat()
24
25         println("Unesite elemente drugog vektora: ")
26         for(i <- 0 until n)
27             vektor2(i) = sc.nextFloat()
28
29         println("Unesite broj niti: ")
30         val brojNiti = sc.nextInt()
31
32         val niti = new Array[Sabirac](brojNiti)
33
34         val korak = Math.ceil(n/brojNiti.toDouble).toInt
35
36         // Pravimo niti i zadajemo im indekse - granice
37         for(i <- 0 until brojNiti)
38             niti(i) = new Sabirac(i*korak, Math.min((i+1)*korak,n),vektor1,vektor2)
39
40         // Pokrecemo racunanje
41         for(i <- 0 until brojNiti)
42             niti(i).start()
43
44         // Cekamo da niti zavrse sa racunanjem
45         for(i <- 0 until brojNiti)
46             niti(i).join()
47
48         // Kada sve niti zavrse sa racunanjem ispisujemo rezultat
49         print("Zbir vektora je: \n[")
50         for(i <- 0 until n-1)
51             print(vektor1(i) + ", ")
52         println(vektor1(n-1) + "]")
53     }
54 }

```

Rešenje 6.10 Broj petocifrenih brojeva

```

1  import java.io._
2  import java.util.Scanner
3  import scala.collection.mutable.ArrayBuffer
4
5  class BrojacPetocifrenih(dat : String) extends Thread {
6
7      var rezultat : Int = 0
8      // Citamo brojeve iz datoteke i povecavamo lokalni brojac
9      // ukoliko je broj petocifren
10     override def run() {
11         val sc : Scanner = new Scanner(new File(dat))
12
13         while(sc.hasNextInt()){
14             val broj = sc.nextInt()
15             if(broj >= 10000 && broj <= 99999)

```

```

16         this.rezultat+=1
17     }
18 }
19
20 def getRezultat: Int = rezultat
21 def getDatoteka: String = dat
22 }
23
24 object BrojPetocifrenih {
25     def main(args : Array[String]){
26
27         val sc = new Scanner(System.in)
28
29         val brojaci = ArrayBuffer[BrojacPetocifrenih]()
30         var kraj = false
31         var odg = ""
32         var dat = ""
33
34         while(!kraj) {
35             println("Da li zelite da zadate ime datoteke koja ce biti obradjena (y/n)?")
36             odg = sc.next()
37             if(odg.toLowerCase() == "n")
38                 kraj = true
39             else{
40                 println("Unesite ime datoteke: ")
41                 dat = sc.next()
42                 brojaci.append(new BrojacPetocifrenih(dat))
43             }
44         }
45
46         // Zapocinjemo izvršavanje
47         for(brojac <- brojaci)
48             brojac.start()
49
50         // Pozivom metoda join cekamo sve brojace da zavrse sa izracunavanjem
51         for(brojac <- brojaci)
52             brojac.join()
53
54         // Citamo rezultate brojanja svake niti
55         for(brojac <- brojaci)
56             println(s"Datoteka ${brojac.getDatoteka} sadrzi ${brojac.getRezultat}
57                 petocifrenih brojeva.")
58     }
59 }

```

Rešenje 6.11 Berba

```

1 import java.util.concurrent.atomic._
2 import java.util.concurrent._
3 import java.util.Scanner
4 import java.io.File
5
6 class Berac(drvoreci : ConcurrentLinkedQueue[(String, Int)],
7             skladiste : AtomicIntegerArray) extends Thread {
8
9     override def run() {
10
11         while(true) {
12             // Dohvatamo drvored za berbu iz reda
13             val drvored = drvoreci.poll()
14             // Ukoliko nema vise drvoreci za berbu završavamo
15             if(drvored == null)
16                 return
17             println("Berac bere drvo " + drvored._1 )
18             Thread.sleep(ThreadLocalRandom.current().nextInt(1,10)*1000)
19             // Dodajemo kilograme voca koje smo obrali u skladiste
20             for(_ <- 0 until drvored._2){
21                 val obrano = ThreadLocalRandom.current().nextInt(30, 50)
22                 if(drvored._1 == "tresnje")
23                     skladiste.getAndAdd(0, obrano)
24                 else if(drvored._1 == "kruske")
25                     skladiste.getAndAdd(1, obrano)
26             }
27         }
28     }
29 }

```

```

27         else if(drvoored._1 == "kajsije")
            skladiste.getAndAdd(2, obrano)
29         else if(drvoored._1 == "sljive")
            skladiste.getAndAdd(3, obrano)
31     }
32 }
33 }

35 object Berba {
    def main(args : Array[String]) {
37         val sc1 : Scanner = new Scanner(new File("drvoredi.txt"))
            val sc2 : Scanner = new Scanner(System.in)
39
41         // Pravimo skladiste voca koje imamo u vocnjaku
42         // i postavljamo inicijalne kolicine voca.
43         // Klasa AtomicIntegerArray sadzi niz integer vrednosti
44         // nad kojima se operacije izvrsavaju atomično.
45         // Slicno kao kod klasa AtomicInteger, AtomicLong i dr.
46         val skladiste = new AtomicIntegerArray(4)
47         skladiste.set(0,0)
48         skladiste.set(1,0)
49         skladiste.set(2,0)
50         skladiste.set(3,0)
51         // Pravimo red drvoreda za berbu
52         // Svaki drvored je jedan par (voce, brojStabala).
53         val drvoredi = new ConcurrentLinkedQueue[(String, Int)]()
54         while(sc1.hasNextLine)
55             drvoredi.add((sc1.next(), sc1.nextInt()))
56
57         println("Unesite broj beraca: ")
58         val brojBeraca = sc2.nextInt()
59
60         val beraci = new Array[Berac](brojBeraca)
61         for(i <- 0 until brojBeraca)
62             beraci(i) = new Berac(drvoredi, skladiste)
63
64         for(b <- beraci)
65             b.start()
66
67         for(b <- beraci)
68             b.join()
69
70         println("Tresanja je obrano: " + skladiste.get(0) + " kilograma.")
71         println("Krusaka je obrano: " + skladiste.get(1) + " kilograma.")
72         println("Kajsija je obrano: " + skladiste.get(2) + " kilograma.")
73         println("Sljiva je obrano: " + skladiste.get(3) + " kilograma.")
74     }
75 }

```

Rešenje 6.12 Turistička agencija

```

import java.util.concurrent._
2 import java.util.Scanner
import java.io.File
4
class Ucesnik(ime : String,
6         cena : Int,
        dobitnici : Array[String])
8 extends Thread {
10     override def run() {
11         // Cekamo dok se ne završi izvlačenje.
12         //
13         // Metod wait() suspenduje nit, oslobadja kritičnu sekciju obmotanu synchronized
14         // blokom
15         // i dozvoljava drugim nitima koje su zaključale isti objekat da udju u kritičnu
16         // sekciju
17         // sve dok neka druga nit ne pozove nad istim objektom metod notifyAll()
18         // cime se obavestavaju sve niti koje cekaju sa metodom wait()
19         // da mogu nastaviti sa radom
20         //

```

```

20     dobitnici.synchronized {
21         dobitnici.wait()
22     }
23     for(d <- dobitnici)
24         if(d == ime){
25             println("Cestitamo " + ime +
26                 "!!! Osvojili ste popust od 20% na cenu karte. Vasa karta sada kosta " +
27                 cena*0.8 + "e.")
28             return
29         }
30     println("Nazalost " + ime +
31         " niste osvojili popust, vise sreće drugi put. Vasa karta kosta " + cena + "e."
32     )
33 }
34
35 def getIme: String = ime
36
37 object TuristickaAgencija {
38     def main(args : Array[String]) {
39         val sc : Scanner = new Scanner(new File("ucesnici.txt"))
40
41         val dobitnici = new Array[String](5)
42         val n = sc.nextInt()
43         sc.nextLine()
44         val ucesnici = new Array[Ucesnik](n)
45         for(i <- 0 until n){
46             ucesnici(i) = new Ucesnik(sc.nextLine(), sc.nextInt(), dobitnici)
47             sc.nextLine()
48         }
49
50         for(u <- ucesnici)
51             u.start()
52
53         println("Izvlacenje je u toku.")
54         Thread.sleep(5000)
55         // Ulazimo u kriticku sekciju, i racunamo dobitnike nagradnih popusta
56         dobitnici.synchronized {
57             val izvuceniIndeksi = ThreadLocalRandom.current().ints(0, n).distinct().limit
58             (5).toArray
59             for(j <- 0 until izvuceniIndeksi.length)
60                 dobitnici(j) = ucesnici(izvuceniIndeksi(j)).getIme
61
62             // Kada su izracunati dobitnici, obavestavamo niti koje cekaju
63             // i izlazimo iz kriticke sekcije
64             dobitnici.notifyAll()
65         }
66     }
67 }

```


Distribuirano programiranje

7.1 O Apache Spark-u

Apache Spark je radni okvir (eng. framework) za distribuirano programiranje. Pruža interfejs za programiranje (eng. API) u jezicima Java, Scala, Python i R.

Svaka Spark aplikacija se sastoji od glavnog (eng. *driver*) programa koji pokreće funkciju `main` i izvršava paralelne operacije na povezanom klaster računar. Pristupanje klaster računar se vrši pomoću objekta kontekst tipa `SparkContext`. Prilikom konstrukcije kontekst objekta, potrebno je definisati koji klaster računar će se koristiti. Spark aplikacija može koristiti lokalni računar kao simulaciju klaster računara (svaka procesorska jedinica će simulirati jedan čvor klaster računara) ili neki udaljeni klaster računar.

Spark koristi posebne kolekcije podataka koje se mogu obrađivati paralelno (eng. *RDD - Resilient Distributed Datasets*). Paralelne kolekcije se mogu napraviti od već postojećih kolekcija u programu ili se mogu učitati iz spoljašnjeg sveta. Nad paralelnim kolekcijama možemo izvršavati dva tipa operacija *transformacije* i *akcije* (slika 7.1). Transformacije transformišu kolekciju na klaster računar i prave novu paralelnu kolekciju. Sve transformacije su lenje, što znači da rezultat izračunavaju u trenutku kada on postane potreban.

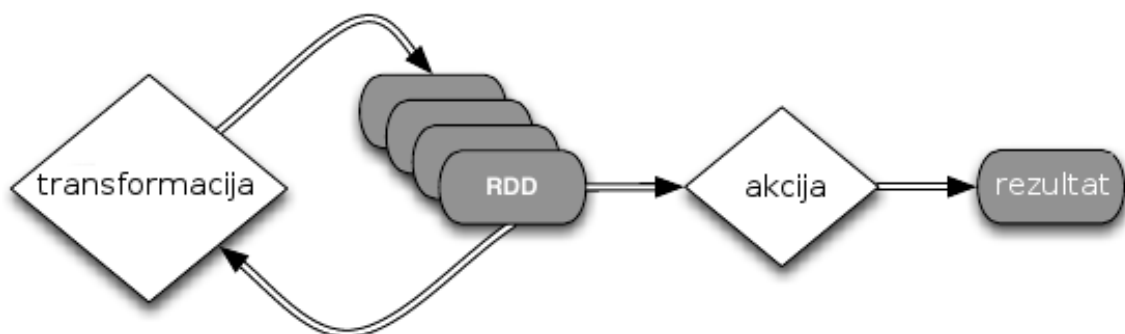
Akcije su operacije koje se izvršavaju na klaster računar nad paralelnim kolekcijama i njihov rezultat (izračunata vrednost) se vraća na lokalni računar.

Spark može da sačuva paralelne kolekcije u memoriji čvorova klaster računara i na taj način ubrzati izvršavanje narednih operacija.

Spark pruža mogućnost korišćenja deljenih podataka u vidu emitovanih promenljivih (eng. *broadcast variables*) i akumulatora (eng. *accumulators*).

Neke od funkcija transformacija su:

- `map(f)` - vraća novu kolekciju koja se dobija tako što se primeni funkcija `f` nad svakim elementom postojeće kolekcije
- `filter(f)` - primenjuje funkciju `f` nad svim elementima kolekcije i vraća novu kolekciju koja sadrži one elemente za koje je funkcija `f` vratila `true`



Slika 7.1: Operacije nad paralelnim podacima

- `flatMap(f)` - slična je funkciji `map`, razlika je to što primena funkcije `f` nad nekim elementom kolekcije može da vrati 0 ili više novih elemenata koji se smeštaju u rezultujuću kolekciju
- `groupByKey()` - poziva se nad kolekcijom parova (ključ, vrednost) i vraća kolekciju parova (ključ, `Iterable<vrednost>`) tako što grupiše sve vrednosti sa istim ključem i smešta ih u drugi element rezultujućeg para
- `reduceByKey(f)` - poziva se nad kolekcijom parova (ključ, vrednost) i vraća kolekciju parova (ključ, nova_vrednost), nova_vrednost se dobija agregiranjem svih vrednosti sa istim ključem koristeću zadatu funkciju agregacije `f`
- `aggregateByKey(pocetna_vrednost)(f1, f2)` - poziva se nad kolekcijom parova (ključ, vrednost) i vraća kolekciju parova (ključ, nova_vrednost), nova_vrednost se dobija agregiranjem pocetne vrednosti i svih vrednosti sa istim ključem koristeću zadatu funkciju agregacije `f1` u svakom čvoru klaster računara, a funkcija `f2` agregira vrednosti izračunate u čvorovima klaster računara u jednu vrednost - nova_vrednost
- `sortByKey()` - poziva se nad kolekcijom parova (ključ, vrednost) i vraća novu kolekciju sortiranu po ključu
- `cartesian(druga_kolekcija)` - spaja kolekciju sa drugom kolekcijom i vraća kolekciju svih parova (vrednost_iz_prve_kolekcije, vrednost_iz_druge_kolekcije)
- `zip(druga_kolekcija)` - spaja kolekciju sa drugom kolekcijom spajajući elemente na istim pozicijama i vraća kolekciju parova (vrednost_iz_prve_kolekcije, vrednost_iz_druge_kolekcije)

Neke od funkcija akcija su:

- `reduce(f)` - agregira elemente kolekcije koristeći funkciju `f` i vraća rezultat agregacije
- `collect()` - pretvara paralelnu kolekciju u niz (koji se nalazi na lokalnom računaru)
- `count()` - vraća broj elemenata kolekcije
- `countByKey()` - poziva se nad kolekcijom parova (ključ, vrednost), za svaki ključ broji koliko ima elemenata sa tim ključem i vraća neparalelnu kolekciju (ključ, broj_elementa)
- `first()` - vraća prvi element kolekcije
- `take(n)` - vraća prvih `n` elemenata kolekcije
- `takeSample(sa_vracanjem, n, seed)` - vraća prvih `n` nasumično izabranih elemenata kolekcije (sa ili bez vraćanja), `seed` predstavlja početnu vrednost generatora slučajnih brojeva
- `takeOrdered(n[, poredak])` - vraća prvih `n` elemenata sortirane kolekcije (koristeći prirodan poredak kolekcije ili zadati poredak)
- `saveAsTextFile(ime_direktorijuma)` - upisuje kolekciju u datoteke koje se nalaze u zadatom direktorijumu
- `foreach(f)` - poziva funkciju `f` nad svim elementima kolekcije (uglavnom se koristi kada funkcija `f` ima neke sporedne efekte kao što je upisivanja podataka u datoteku ili slično)

Konfiguraciju Spark aplikacije možemo podešavati dinamički prilikom pokretanja aplikacije na klaster računaru. Potrebno je upakovati aplikaciju zajedno sa svim njenim bibliotekama u .jar datoteku koristeći neki od alata (Maven ¹, SBT ² i sl.) i instalirati Spark upravljač na klaster računaru (Standalone ³, Mesos ⁴, Yarn ⁵ i sl.). Aplikaciju možemo pokrenuti pomoću `spark-submit` skripta koji se nalazi u `bin` direktorijumu instaliranog Spark alata i konfigurisati dinamički. Na primer:

¹<http://maven.apache.org/>

²<http://www.scala-sbt.org/>

³<http://spark.apache.org/docs/latest/spark-standalone.html>

⁴<http://spark.apache.org/docs/latest/running-on-mesos.html>

⁵<http://spark.apache.org/docs/latest/running-on-yarn.html>

```
1 ./bin/spark-submit --class Main --master local --num-executors 20
   Aplikacija.jar
```

Parametri koji se najčešće koriste prilikom konfiguracije su:

- `--master url` - URL klaster racunara
- `--class ime_klase` - glavna klasa naše aplikacije
- `--num-executors n` - broj čvorova koji izvršavaju našu aplikaciju (eng. *executors*)
- `--executor-cores n` - broj zadataka koje jedan čvor može izvršavati istovremeno
- `--executor-memory n` - veličina hip memorije svakog čvora

7.2 Uputstvo za Apache Spark

Uputstvo prikazuje kako konfigurisati projekat u okruženju IntelliJ Idea da koristi biblioteku Apache Spark.

Literatura:

- spark.apache.org/docs/0.9.1/scala-programming-guide.html

7.2.1 Potreban softver i alati

Potrebno instalirati:

- IntelliJ Idea
(jetbrains.com/idea/)
- Java JDK8
(oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html)
- Plugin Scala za IntelliJ Idea (instalacija iz okruženja)

7.2.2 Pravljenje projekta

Potrebno je napraviti `sbt` projekat na isti način kao i u prethodnim odeljcima (videti uputstvo za Konkurentno programiranje u delu 6.2.1).

7.2.3 Dodavanje biblioteke u `build.sbt`

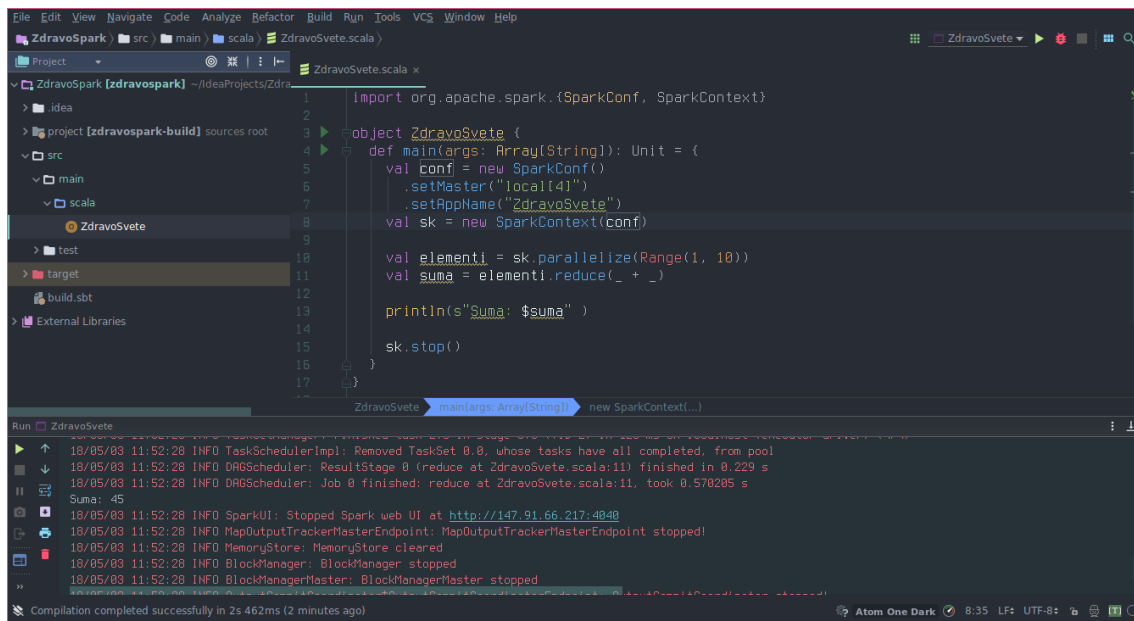
Potrebno je da sistemu `sbt` definišemo da naš projekat koristi spoljnu biblioteku. U datoteci `build.sbt` možemo definisati zavisnost (eng. *dependency*) od spoljne biblioteke.

Potrebno je dodati sledeći kod u datoteku `build.sbt`

```
libraryDependencies += {
  val sparkVer = "2.4.0"
  Seq(
    "org.apache.spark" %% "spark-core" % sparkVer
  )
}
```

Vaš `build.sbt` bi trebao imati sledeći oblik:

```
name := "ZdravoSpark"
version := "0.1"
scalaVersion := "2.12.8"
libraryDependencies += {
  val sparkVer = "2.4.0"
  Seq(
    "org.apache.spark" %% "spark-core" % sparkVer
  )
}
```



Slika 7.2: Pokretanje Spark programa

Pri čemu atribut `name` zavisi od imena projekta koje ste originalno odabrali.

Sačuvajte izmene. Kada Vas okruženje pita da ažurira projekat jer je izmenjena datoteka `build.sbt` prihvatite izmene. Alat `sbt` će u skladu sa Vašim izmenama preduzeti odgovarajuće akcije. U ovom slučaju to će biti preuzimanje biblioteke `spark-core` sa odgovarajućeg repozitorijuma i njeno uključivanje u Vaš projekat.

7.2.4 Pokretanje programa

Nakon što je `sbt` pripremio okruženje za rad, možemo da pristupimo Spark biblioteci. Na slici 7.2 je prikazano pokretanje Spark programa.

7.3 Zadaci sa rešenjima

Zadatak 7.1 Parni kvadrati Napisati program koji učitava ceo broj n veći od 2 i ispisuje sve kvadrate parnih brojeva počev od broja 2 do n .

[Rešenje 7.1]

Zadatak 7.2 Broj petocifrenih Napisati program koji ispisuje broj petocifrenih brojeva koji se nalaze u datoteci `brojevi.txt` (svaka linija sadrži jedan broj).

[Rešenje 7.2]

Zadatak 7.3 Skalarni proizvod Napisati program koji racuna skalarni proizvod dva vektora (pretpostavimo da su vektori uvek zadati ispravno tj. iste su dužine) i ispisuje ih na izlaz. Vektori se nalaze u datotekama `vektor1.txt` i `vektor2.txt` u formatu:

```
1 a1, a2, a3, a4, ... an
```

[Rešenje 7.3]

Zadatak 7.4 Broj pojavljivanja reči Napisati program koji za svaku reč iz knjige (datoteka `knjiga.txt`) broji koliko se puta ona pojavljuje i rezultat upisuje u datoteku.

[Rešenje 7.4]

Zadatak 7.5 Uređaji transakcije U datoteci *uredjaji.txt* se nalaze podaci o kupljenim uređajima u formatu:

```
1 marka_uredjaja ostali_podaci
```

Napisati program koji izdvaja podatke o svim transakcijama jedne marke i upisuje ih u posebnu datoteku sa nazivom *ime_marke.txt*.

[Rešenje 7.5]

Zadatak 7.6 Log poruke Datoteka *log.txt* sadrži podatke koji su generisani pokretanjem Java programa. Napisati program koji izdvaja poruke koje se odnose na pakete jezika Java grupisane po tipu poruke i ispisuje ih na izlaz.

Poruke mogu biti informacione, upozorenja ili greške.

Format poruka je:

```
1 tip ostatak_poruke
```

Tip može biti [warn], [info] ili [error].

[Rešenje 7.6]

Zadatak 7.7 Uspešna preuzimanja U datoteci *mavenLog.txt* se nalaze podaci o započetim/uspešnim preuzimanjima paketa prilikom pokretanja Maven alata (upravljač zavisnostima) u formatu:

```
1 Downloading: ostatak_poruke
```

ili

```
1 Downloaded: ostatak_poruke
```

Napisati program koji računa procenat uspešnih preuzimanja paketa u odnosu na započeta preuzimanja.

[Rešenje 7.7]

Zadatak 7.8 Pokloni Bliži se Božić i firma želi da pokloni svojim zaposlenim programerima tri paketića. Sin direktora firme je budući programer i želi da napravi program koji će simulirati izvlačenje troje dobitnika paketića. Kako još uvek nije dobro savladao programiranje, pomozimo mu tako što ćemo napisati program koji nasumično bira tri programera i ispisuje njihova imena, prezimena i email.

Podaci o zaposlenima se nalaze u datoteci *zaposleni.txt* u formatu:

```
1 ime prezime pol identifikator IP_adresa_racunara datum_zaposlenja
  sifra_pozicije plata
```

Šifra pozicije programera je IT_PROG.

[Rešenje 7.8]

Zadatak 7.9 Prosečna temperatura U datoteci *temperaturaBoston.txt* se nalaze podaci o prosečnim temperaturama u Bostonu od 1995 do 2016 godine u Farenhajtima. Napisati program koji ispisuje prosečne temperature u Bostonu za svaku godinu od 1995 do 2016 godine posebno u Celzijusima.

Format podataka je:

```
1 mesec dan godina temperatura
```

[Rešenje 7.9]

7.4 Zadaci za vežbu

Zadatak 7.10 Napisati program koji učitava ceo broj n i ispisuje faktorijele svih brojeva od 1 do n .

Zadatak 7.11 Napisati program koji učitava ceo broj n i ispisuje sumu prvih n elemenata harmonijskog reda (podsetimo se, harmonijski red je red oblika $1 + 1/2 + 1/3 + 1/4 \dots$).

Zadatak 7.12 Napisati program koji racuna zbir dva vektora (pretpostavimo da su vektori uvek zadati ispravno tj. iste su dužine) i ispisuje ih na izlaz.

Vektori se nalaze u datotekama *vektor1.txt* i *vektor2.txt* u formatu:

```
1 a1, a2, a3, a4, ... an
```

Zadatak 7.13 Napisati program koji racuna broj pojavljivanja svake cifre 0-9 u datoteci *knjiga.txt* i ispisuje rezultat sortiran po ciframa.

Zadatak 7.14 Napisati program koji prebrojava sve poruke o greškama koje se odnose na Spark alat i rezultat ispisuje na izlaz. Poruke o greškama se nalaze u datoteci *log.txt* u formatu:

```
1 tip ostatak_poruke
```

Tip poruke o grešci je [error].

Zadatak 7.15 Napisati program koji izdvaja dane sa najvišom temperaturom u Bostonu za svaku godinu posebno, počev od 1995 do 2016 i rezultat upisuje u direktorijum *MaxTemp*. Podaci o temperaturama se nalaze u datoteci *temperaturaBoston.txt* u formatu:

```
1 mesec dan godina temperatura
```

Zadatak 7.16 Napisati program koji izdvaja podatke o veličini preuzetih paketa koji se odnose na Apache server i rezultat upisuje u direktorijum *ApacheDownloaded*. Podaci o preuzimanjima se nalaze u datoteci *mavenLog.txt* u formatu:

```
1 Downloaded: putanja podaci_o_velicini
```

Zadatak 7.17 Ekonomski analitičari žele da analiziraju tržište tehničke robe sa nekim rasponom cena. Napisati program koji učitava ime marke i bira nasumično pet poruka o transakcijama koje se odnose na tu marku. Podaci o transakcijama se nalaze u datoteci *uredjaji.txt* u formatu:

```
1 marka_uredjaja ostali_podaci
```

Zadatak 7.18 Napisati program koji računa prosečnu platu programera u firmi. Podaci o zaposlenima se nalaze u datoteci *zaposleni.txt* u formatu:

```
1 ime prezime pol identifikator IP_adresa_racunara datum_zaposlenja
  sifra_pozicije plata
```

Šifra pozicije programera je IT_PROG.

7.5 Rešenja

Rešenje 7.1 Parni kvadrati

```
1 import org.apache.spark.SparkConf
2 import org.apache.spark.SparkContext
3 import org.apache.spark.rdd.RDD._
4
5 object ParniKvadrati {
```

```

7  def main(args: Array[String]) = {
9      println("Unesite broj n:")
      val n = Console.readInt()
11     /**
      * Podesavamo konfiguraciju Spark okruzenja
13     * tako sto dajemo ime aplikaciji
      * i dodeljujemo joj potencijalno 4 cvora
15     * (u našem slučaju procesorska jezgra)
      * */
17     val konf = new SparkConf()
      .setAppName("ParniKvadrati")
19     .setMaster("local[4]")

21     /**
      * Ako Spark izbacuje gresku da je heap size manji od potrebnog,
23     * otkomentarisati narednu liniju i proslediti odgovarajucu vrednost
      * */
25     // konf.set("spark.testing.memory", "2147480000")

27     /**
      * Pravimo objekat Spark konteksta
29     * koji pokrece i upravlja Spark okruzenjem
      * */
31     val sk = new SparkContext(konf)

33     /**
35     * Ukoliko zelimo da podesavamo parametre dinamicki
      * (broj cvorova koji izvorsavaju nasu aplikaciju,
37     * velicina hip memorije i sl.)
      * potrebno je da inicijalizujemo spark kontekst na sledeci nacin
39     *
      * val sk = new SparkContext(new SparkConf());
41     *
      * cime naznacavamo da ce se parametri konfiguracije
43     * podesiti dinamicki (koriscenjem spark-submit skripte).
      * */
45

47     val niz = (2 to n by 2).toArray

49     /**
      * Pravimo niz tipa RDD[Integer] od niza tipa Array[Integer]
      * */
51     val nizRDD = sk.parallelize(niz)

53     /**
      * Pravimo niz kvadrata parnih brojeva (uzimamo prvih 10)
55     * i rezultat pretvaramo u niz tipa Array[Integer]
      * */
57     val nizKvadrata = nizRDD.map(x => x*x)
      /* .take(10) */
59     .collect()

61     /**
      * Zaustavljamo Spark okruzenje
63     * */
      sk.stop()

65     /**
67     * Ispisujemo rezultujuci niz
      * */
69     println("Niz kvadrata parnih brojeva: ")
      println(nizKvadrata.mkString(", "))
71 }
}

```

Rešenje 7.2 Broj petocifrenih

```

1 import org.apache.spark.SparkConf
2 import org.apache.spark.SparkContext

```



```

import org.apache.spark.rdd.RDD._
4
6 object BrojPetocifrenih {
8   def main(args: Array[String]){
10    val konf = new SparkConf()
        .setAppName("BrojPetocifrenih")
12    .setMaster("local[4]")
14
        val sk = new SparkContext(konf)
16
        /**
18         * Otvaramo datoteku i njen sadrzaj cuvamo
        * u nizu tipa RDD[String].
        * Elementi niza su pojedinačne linije iz datoteke.
        */
20    val datRDD = sk.textFile("brojevi.txt")
22
        /**
24         * Filtriramo niz tako da nam ostanu samo petocifreni brojevi
        * i prebrojavamo ih.
        */
26    val brojPetocifrenihBrojeva = datRDD.filter(_.length() == 5)
        .count()
28
30    sk.stop()
32
        println("Petocifrenih brojeva ima: ")
        println(brojPetocifrenihBrojeva)
34    }
}

```

Rešenje 7.3 Skalarni proizvod

```

1 import org.apache.spark.SparkConf
import org.apache.spark.SparkContext
3 import org.apache.spark.rdd.RDD._
5
6 object SkalarniProizvod {
7   def main(args: Array[String]){
9    val konf = new SparkConf()
        .setAppName("SkalarniProizvod")
11    .setMaster("local[4]")
13
        val sk = new SparkContext(konf)
15
        /**
17         * Otvaramo datoteku i učitavamo vektor.
        */
        val vek1RDD = sk.textFile("vektor1.txt")
19
        /**
        * Razdvajamo elemente vektora iz niske koristeći separator ", "
        */
21    .flatMap(_.split(", "))
23
        /**
        * Kastujemo niske u tip Integer.
        */
25    .map(_.toInt)
27
        val vek2RDD = sk.textFile("vektor2.txt")
29
        .flatMap(_.split(", "))
        .map(_.toInt)
31
        /**
33         * Spajamo nizove vektora A i B funkcijom zip
        * i pravimo jedan niz parova (tipa Tuple)
        * tako da svaki par sadrži element vektora A i element vektora B
        * (a1,b1), (a2,b2), ... (an, bn).
35

```

```

37      * */
      val skProizvod = vek1RDD.zip(vek2RDD)
39          /**
40           * Mnozimo elemente para.
41           * */
42          .map(par => par._1 * par._2)
43          /**
44           * Pomnozene elemente parova sabiramo.
45           * */
46          .reduce((a, b) => a+b)
47
48      sk.stop()
49
50      println("Skalarni proizvod je: ")
51      println(skProizvod)
52  }
53 }

```

Rešenje 7.4 Broj pojavljivanja reči

```

1  import org.apache.spark.SparkConf
2  import org.apache.spark.SparkContext
3  import org.apache.spark.rdd.RDD._
4
5  object BrojPojavljivanjaReci {
6
7      def main(args: Array[String]){
8
9          val konf = new SparkConf()
10             .setAppName("BrojPojavljivanjaReci")
11             .setMaster("local[4]")
12
13          val sk = new SparkContext(konf)
14
15          val knjigaRDD = sk.textFile("knjiga.txt")
16
17          /**
18           * Ucitavamo linije i razlazemo ih separatorom " " tako da dobjemo niz reci
19           */
20          val reciBr = knjigaRDD.flatMap(_.split(" "))
21          /**
22           * Od svake reci pravimo par (rec, 1).
23           */
24          .map(rec => (rec, 1))
25          /**
26           * Sabiramo sve vrednosti drugog elementa para
27           * grupisane po prvom elementu para
28           * koji nam predstavlja kljuc.
29           */
30          .reduceByKey((_,_) )
31          /**
32           * Sortiramo reci leksikografski.
33           */
34          .sortByKey()
35          /**
36           * Cuvamo ih u datotekama koje se nalaze u direktorijumu
37           * BrojPojavljivanjaReci
38           */
39          .saveAsTextFile("BrojPojavljivanjaReci")
40
41          sk.stop()
42      }
43  }

```

Rešenje 7.5 Uredaji transakcije

```

1  import org.apache.spark.SparkConf
2  import org.apache.spark.SparkContext
3  import org.apache.spark.rdd.RDD._
4  import java.io._
5

```

```

object UredjajiTransakcije {
7
  def main(args: Array[String]){
9
    val konf = new SparkConf()
11    .setAppName("UredjajiTransakcije")
    .setMaster("local[4]")
13
    val sk = new SparkContext(konf)
15
    val transakcije = sk.textFile("uredjaji.txt")
17    /**
    * Razdvajamo podatke o uredjajima
    * i pravimo parove transakcija
    * (marka, ostali_podaci)
    */
21    .map(linija => {
23      val niz = linija.split(" ")
      (niz(0), niz.drop(1).mkString(" "))
25    })
    /**
27    * Grupisemo ih po marki tako da za svaku marku
    * cuvamo niz obavljenih transakcija
    * (tj. niz koji sadrzi ostale podatke za svaku transakciju)
    */
31    .groupByKey()
    /**
33    * Prolazimo kroz niz parova (marka, niz_transakcija)
    * i u datoteku ime_marke.txt
    * upisujemo podatke o transakcijama.
    *
37    * Parametar t u foreach konstrukciji predstavlja jedan par
    * (marka, niz_transakcija).
    */
39    .foreach(t => {
41      val dat = new PrintWriter(new File(t._1.toLowerCase() +
      ".txt" ))

43      dat.write("---" + t._1 + "---\n")

45      t._2.foreach(por => {
47        dat.append(por + "\n")
49      })
      dat.close()
51    })
53  }
}

```

Rešenje 7.6 Log poruke

```

1 import org.apache.spark.SparkConf
import org.apache.spark.SparkContext
3 import org.apache.spark.rdd.RDD._

5 object LogPoruke {

7   def main(args: Array[String]){

9     val konf = new SparkConf()
    .setAppName("LogPoruke")
11    .setMaster("local[4]")

13    val sk = new SparkContext(konf)

15    val poruke = sk.textFile("log.txt")
    /**
17    * Filtriramo podatke tako da nam ostanu
    * samo linije koje predstavljaju
    * upozorenja, informacije ili greske i odnose se na javu.
19

```

```

21         */
        .filter(linija =>
22             (linija.contains("[warn]")
23              || linija.contains("[info]")
24              || linija.contains("[error]"))
25             && (linija.contains("java")))
        /**
26         * Pravimo parove (tip_poruke, poruka).
27         */
28     .map(linija => {
29         val niz = linija.split(" ")
30         (niz(0), niz.drop(1).mkString(" "))
31     })
32     /**
33     * Grupisemo poruke po njihovom tipu (kljuc).
34     * tako da dobijemo niz parova (tip_poruke, niz_poruka).
35     */
36     .groupByKey()
37     /**
38     * Za svaki tip racunamo broj poruka tog tipa
39     * tako sto od parova (tip_poruke, niz_poruka)
40     * pravimo par (tip_poruke, broj_poruka)
41     */
42     .map( por => (por._1, por._2.size))
43     .collect()
44
45     println("Informacije o log porukama koje se odnose na Javu: ")
46
47     poruke.foreach( por => println(" " + por._1 + ": " + por._2 ))
48
49     sk.stop()
50 }
51 }

```

Rešenje 7.7 Uspešna preuzimanja

```

1 import org.apache.spark.SparkConf
2 import org.apache.spark.SparkContext
3 import org.apache.spark.rdd.RDD._
4
5 object UspesnaPreuzimanja {
6
7     def main(args: Array[String]){
8
9         val konf = new SparkConf()
10            .setAppName("UspesnaPreuzimanja")
11            .setMaster("local[4]")
12
13         val sk = new SparkContext(konf)
14
15         /**
16          * Ucitavamo podatke i smestamo ih u kes memoriju radi brzog pristupanja.
17          */
18         val preuzimanja = sk.textFile("mavenLog.txt")
19            .cache()
20
21         /**
22          * Racunamo broj zapocetih preuzimanja.
23          */
24         val zapoceta = preuzimanja.filter(_.contains("Downloading:"))
25            .count()
26
27         /**
28          * Racunamo broj završenih preuzimanja.
29          */
30         val završena = preuzimanja.filter(_.contains("Downloaded:"))
31            .count()
32
33         sk.stop()
34
35         println("%.2f".format(završena*100.0/zapoceta) + " procenata zapocetih
36         preuzimanja je završeno.")
37     }
38 }

```

Rešenje 7.8 Pokloni

```

1 import org.apache.spark.SparkConf
import org.apache.spark.SparkContext
3 import org.apache.spark.rdd.RDD._
import scala.compat._
5
6 object RandomZaposleni {
7
8   def main(args: Array[String]){
9
10     val konf = new SparkConf()
11       .setAppName("RandomZaposleni")
12       .setMaster("local[4]")
13
14     val sk = new SparkContext(konf)
15
16     /**
17      * Pronalazimo liniju koja sadrzi radnika programera,
18      * razdvajamo podatke o jednom radniku
19      * i cuvamo njegovo ime, prezime i email nalog.
20      * Nakon toga nasumicno biramo 3 programera.
21      *
22      * Funkcija takeSample kao argumente prihvata:
23      * - indikator da li zelimo izbora sa vracanjem
24      * - broj uzoraka
25      * - pocetnu vrednost (seed) za slucajni generator
26      */
27     val triProgramera = sk.textFile("zaposleni.txt")
28       /**
29        * Pronalazimo liniju koja sadrzi radnika programera,
30        */
31       .filter(_.contains("IT_PROG"))
32       /**
33        * Razdvajamo podatke o jednom radniku
34        * i cuvamo njegovo ime, prezime i email nalog.
35        */
36       .map(linija => {
37         val niz = linija.split(" ")
38         (niz(0), niz(1), niz(3))
39       })
40       /**
41        * Nasumicno biramo 3 programera.
42        * Prvi parametar (false) oznacava
43        * izbor bez vracanja a poslednji parametar
44        * predstavlja pocetnu vrednost (seed)
45        * generatora slucajnih brojeva.
46        */
47       .takeSample(false, 3, Platform.currentTimeMillis)
48
49     println("Tri zaposlena radnika u IT sektoru su: ")
50     triProgramera.foreach(prog => {
51       println("Ime i prezime: " + prog._1 + " " + prog._2 + "\n Email: " + prog._3
52         .toLowerCase()+"@firma.com")
53     })
54
55     sk.stop()
56   }
57 }

```

Rešenje 7.9 Prosečna temperatura

```

1 import org.apache.spark.SparkConf
import org.apache.spark.SparkContext
3 import org.apache.spark.rdd.RDD._
5
6 object ProsečnaTemperatura {
7
8   def main(args: Array[String]){
9
10     val konf = new SparkConf()
11       .setAppName("ProsečnaTemperatura")

```

```

11      .setMaster("local[4]")
13
14      val sk = new SparkContext(konf)
15
16      val tempRDD = sk.textFile("temperatureBoston.txt")
17      /**
18       * Pravimo torke (kljuc, vrednost) takve da je
19       * kljuc = godina
20       * vrednost = (mesec, dan, temperatura)
21       */
22      .map(linija => {
23          val niz = linija.split(" ")
24          (niz(3), (niz(1), niz(2), niz(4).toFloat))
25      })
26      /**
27       * Grupisemo torke po njihovom kljucu i za svaki kljuc racunamo
28       * sumu svih temperatura i broj temperatura koje smo sabrali.
29       *
30       * Funkcija aggregateByKey(pocetnaVrednostAkumulatora)(f1, f2)
31       * obradjuje niz parova (kljuc, vrednost)
32       * grupise ih po kljucu, akumulira vrednosti
33       * (inicijalna vrednost akumulatora
34       * se prosledjuje kao parametar)
35       * i kao rezultat vraca niz parova (k, akumuliranaVrednost).
36       *
37       * Funkcija f1(akumulator, vrednost) se primenjuje
38       * nad svim vrednostima koje se nalaze u jednom cvoru
39       * i rezultat se smesta u akumulator tog cvora
40       * Funkcija f2(akumulator1, akumulator2) se primenjuje
41       * nad svim izracunatim akumulatorima pojedinačnih cvorova
42       * i rezultat se smesta u globalni akumulator.
43       *
44       * U našem slučaju, akumulator će da sadrži dve vrednosti
45       * (sumaTemp, brojTemp) sa nulom kao početnom vrednošću.
46       * Funkcija f1(akumulator, vrednost)
47       * sabira vrednost (temperaturu)
48       * sa sumaTemp iz akumulatora, a brojTemp povećava za 1.
49       * Funkcija f2(akumulator1, akumulator2)
50       * sabira obe vrednosti ova dva akumulatora.
51       *
52       * Kao rezultat primene ove funkcije dobićemo niz parova
53       * (kljuc, (sumaTemp, brojTemp))
54       * gde nam je kljuc godina
55       * u kojoj želimo da izračunamo prosečnu temperaturu.
56       */
57      .aggregateByKey((0.0, 0))((ak, vr) => (ak._1 + vr._3, ak._2 +
58          1),
59          (a1, a2) => (a1._1 + a2._1, a1._2 + a2.
60              _2))
61      /**
62       * Pravimo niz parova (kljuc, prosečnaTemperatura)
63       */
64      .map(st => (st._1, st._2._1/st._2._2))
65      .sortByKey()
66      .collect()
67      .foreach(st => println("Godine "
68          + st._1
69          + " prosečna temperatura je iznosila "
70          + "%.2f".format(((st._2-32)/1.8))
71          + " celzijusa. "))
72
73      sk.stop()
74  }
75 }

```


8

Logičko programiranje

8.1 Jezik Prolog

Prolog (eng. *PROgramming in LOGic*) je deklarativan programski jezik namenjen rešavanju zadataka simboličke prirode. Prolog se temelji na teorijskom modelu logike prvog reda. Početkom 1970-ih godina Alain Kolmerauer (eng. *Alain Colmerauer*) i Filipe Rousel (eng. *Philippe Roussel*) na Univerzitetu u Marselju (eng. *University of Aix-Marseille*), zajedno sa Robertom Kovalskim (eng. *Robert Kowalski*) sa Odeljka Veštačke Inteligencije (eng. *Department of Artificial Intelligence*) na Univerzitetu u Edinburgu (eng. *University of Edinburgh*), razvili su osnovni dizajn jezika Prolog.

8.2 Instalacija BProlog-a

U okviru kursa će biti korišćena distribucija Prologa pod nazivom BProlog. BProlog se može preuzeti sa zvanične Veb strane <http://www.picat-lang.org/bprolog/>.

Potrebno je preuzeti adekvatnu verziju za Vaš sistem i otpakovati je. U dobijenom direktorijumu će postojati izvršiva datoteka `bp` kojom se može pokrenuti BProlog interpreter. Preporučeno je dodati `bp` u `PATH` kako bi BProlog bio dostupan iz komandne linije.

Na primer, pretpostavimo da imamo 64bitni Linux. Potrebno je preuzeti datoteku `bp81_linux64.tar.gz` i smestiti je u direktorijum po izboru, na primer `/home/korisnik/Downloads`. Potom treba izvršiti sledeće naredbe:

```
1 cd ~/Downloads
tar -xvf bp81_linux64.tar.gz
3 sudo mv BProlog /opt
sudo ln -s /opt/BProlog/bp /usr/bin/bprolog
```

Nakon toga, BProlog interpreter se iz konzole može pokrenuti komandom `bprolog`.

8.3 Uvod

8.3.1 Uvodni primeri

Zadatak 8.1 U bazu znanja uneti informacije o životinjama i njihovim odnosima po pitanju veličine. Napisati pravilo koje omogućava da se proverí koja je od dve životinje veća, kao i da generiše sve životinje za koje je neka životinja veća.

```
% ovo je jednolinijski komentar
2
/*
4 ovo je viselinijijski
komentar
6 */
8
/*
```



```

Programi se cuvaju sa ekstenzijom .pro ili .pl (na sistemima gde ekstenzije imaju
    globalno znacenje, kao sto je MS-Windows da ne bi bilo mesanja sa Perl programima
    treba uvek koristiti .pro).
10 Interpreter se pokrece komandom bp. Naredbe:
    help -- pomoc
12 compile('ime_programa') -- prevodi program i pravi izvrsni fajl ukoliko nema gresaka
    load('ime_izvrsnog_fajla') -- uvozi izvrsni fajl
14 cl('ime_programa') -- compile + load
    halt ili control+D -- za izlazak iz interpretera
16
Termovi: konstante, promenljive ili kompozitni termovi.
18 --- Konstante: atomi i brojevi.
    ----- Atomi: stringovi od najvise 1000 karaktera koji pocinju malim slovom ('abc', '
        a01', 'b_cd', 'l122k', ...).
20 ----- Brojevi: celi i realni.
    --- Promenljive: imena pocinju velikim slovom ili podvlakom (_). Specijalna, anonimna
        promenljiva: '_'.
22 --- Kompozitni (slozeni) termovi ili strukture: oblika f(t1, ..., tn) gde je f neka
        funkcija arnosti n ( 0<n<32768), a t1, ..., tn termovi.

24 Program: sekvenca Hornovih klauza. Postoje tri tipa Hornovih klauza: cinjenice,
    pravila i upiti.
    --- Cinjenice: atomicna formula oblika p(t1, ..., tn) gde je p predikat arnosti n, a
        t1, ..., tn termi. One opisuju svojstva i relacije izmedju objekata. Primer:
26     zivotinja(slon).
        veci(zebra,vuk).
28 --- Pravila: imaju sledecu formu
    H :- B1, ..., Bn. (n>0)
30 H, B1, ..., Bn su atomicne formule. H se zove GLAVA pravila, dok je sve sa desne
    strane :- TELO pravila. Citamo ih kao implikaciju sa desna na levo: vazi H, ako
    vaze B1, ..., Bn ("," u pravilu zamenjuje logicko "i").
    --- Cinjenice i pravila cine BAZU ZNANJA.
32 --- Upiti: konstrukcije kojima korisnik komunicira sa bazom znanja. Za ovo je
    neophodan interpetator kome se postavlja upit. Primer:
    ?- veci(slon, zec).
34     true.
    ?- zivotinja(veverica).
36     false.
    */
38
/* cinjenice, svojstva */
40 zivotinja(slon).
    zivotinja(vuk).
42 zivotinja(zec).
    zivotinja(zebra).
44
/* cinjenice, odnosi */
46 veci(slon,vuk).
    veci(vuk,zec).
48 veci(slon,zebra).
    veci(zebra,vuk).
50 veci(slon,zec).

52 /*
    upiti sa promenljivama:
54 -- daje jedno resenje, ako zelimo da prikaze jos resenja kucamo ; nakon prikazanog ?
        a za prekid control+C

56 ?- veci(slon, X).
    X = vuk ? ^C
58
    | ?- veci(slon, X).
60 X = vuk ?;
    X = zebra ?;
62 X = zec
    yes
64
    | ?- veci(X, Y).
66 X = slon
    Y = vuk ?;
68 X = vuk
    Y = zec ?;
70 X = slon

```

```

Y = zebra ?;
72 X = zebra
Y = vuk ?;
74 X = slon
Y = zec
76 yes
*/
78
/*
80 pravilo : za neko X i Y vazi je_veci(X,Y) ako postoji Z tako da vazi veci(X,Z) i veci
      (Z,Y)
*/
82 je_veci(X,Y):- veci(X,Z), veci(Z,Y).
/*
84 | ?- je_veci(X,Y)
X = slon
86 Y = zec ?;
X = slon
88 Y = vuk ?;
X = zebra
90 Y = zec ?;
no
92 */

```

Zadatak 8.2 Unifikacija. Jednakost.

```

/*
2
Provera tipa:
4
atom(X) - da li je term X atom
6 atomic(X) - da li je term X atom ili broj
number(X) - da li je term X broj
8 float(X) ili real(X) - da li je term X realan broj
integer(X) - da li je term X ceo broj
10 var(X) - da li je term X slobodna promenljiva
nonvar(X) - da li term X nije promenljiva
12
Primeri:
14 | ?- atom('abc')
yes
16 | ?- atomic(5)
yes
18
Unifikacija:
20
=      unifikabilni
22 \=    nisu unifikabilni
==     identicno jednaki termovi
24 \==   nisu identicno jednaki termovi
26 */
28 uni(X, Y):- X = Y.
30
/*
32 | ?- uni(4,5)
no
34 | ?- uni(4,X)
X = 4
yes
36 */
38 jed(X, Y):- X == Y.
40
/*
42 | ?- jed(4,X)
no
44 | ?- jed(4,5)
no
46 | ?- jed(4,4)
yes
*/

```

Zadatak 8.3 Aritmetički operatori. Operatori *is* i *cut*.

```

/*
2 is    aritmeticko izracunavanje
   :=   aritmeticki jednaki
4   \=   aritmeticki nisu jednaki
   <, <=, >, >=
6   +, -, *, /, // (celobrojno deljenje), div, mod, ** (stepenovanje)
   */

/*
10 Ako je X promenljiva, tada se njoj dodeljuje vrednost koju ima term Y (mora biti
    poznata vrednost), a ukoliko X nije promenljiva, X is Y se svodi na X := Y
   */
12 op1(X, Y):- X is Y.
/*
14 Termovima X i Y moraju biti poznate vrednosti, inace ce prijaviti gresku.
   */
16 op2(X, Y):- X := Y.

18 /*
   | ?- op1(3,4)
20 no
   | ?- op1(4,4)
22 yes
   | ?- op1(X,4)
24 X = 4
   yes
26 | ?- op1(4,X)
   *** error
28 | ?- op2(4,4)
   yes
30 | ?- op2(4,2)
   no
32 | ?- op2(4,X)
   *** error
34 | ?- op2(X,4)
   *** error
36 */

38 /*
   apsolutna vrednost, prvi argument je broj za koji trazimo apsolutnu vrednost, a drugi
   promenljiva gde se smesta rezultat
40
   losa implementacija, za pozitivne brojeve oba pravila prolaze
42
   | ?- abs1(1,X)
44 X = 1 ?;
   X = -1
46 yes
   | ?- abs1(-1,X)
48 X = 1
   yes
50
   */
52 abs1(X, X):- X >= 0.
   abs1(X, Y):- Y is -X.
54

/*
56 dobre implementacije abs2 i abs3

58 | ?- abs2(1,X)
   X = 1 ?
60 yes
   | ?- abs2(-1,X)
62 X = 1
   yes
64 */
66 abs2(X, X):- X >= 0.
   abs2(X, Y):- X < 0, Y is -X.

```

```

68 /*
koriscenje operatora "cut" koji se oznacava sa "!" uklanjamo alternativne klauze,
tako da se u slucaju pozitivnih nece primeniti drugi predikat cim uspe prvi
70
| ?- abs3(1,X)
72 X = 1
yes
74 | ?- abs3(-1,X)
X = 1
76 yes
78 */
abs3(X, X):- X >= 0, !.
80 abs3(X, Y):- Y is -X.

```

Zadatak 8.4 Rekurzivni predikat, primer porodičnog stabla.

```

% porodicno stablo
2
% svojstva
4 musko(mihajlo).
musko(stevan).
6 musko(petar).
musko(mladen).
8 musko(rajko).
zensko(milena).
10 zensko(milica).
zensko(jelena).
12 zensko(senka).
zensko(mina).
14 zensko(maja).

16 % odnosi
roditelj(mihajlo,milica).
18 roditelj(mihajlo,senka).
roditelj(milena,rajko).
20 roditelj(maja,petar).
roditelj(maja,mina).
22 roditelj(stevan,mladen).
roditelj(stevan,jelena).
24 roditelj(milica,mladen).
roditelj(milica,jelena).

26 % pravila
28 majka(X,Y):- roditelj(X,Y), zensko(X).
otac(X,Y):- roditelj(X,Y), musko(X).
30 brat(X,Y):- musko(X), majka(Z,X), majka(Z,Y), X\==Y.
sestra(X,Y):- zensko(X), majka(Z,X), majka(Z,Y), X\==Y.
32 ujak(X,Y):- brat(X,Z), majka(Z,Y).
tetka(X,Y):- sestra(X,Z), majka(Z,Y).
34

% rekurzivno pravilo
36 % roditelj je predak
predak(X,Y):- roditelj(X,Y).
38 % roditelj pretka je takodje predak
predak(X,Y):- roditelj(X,Z), predak(Z,Y).

```

Zadatak 8.5 Napisati Prolog predikate:

- prestupna koji određuje da li je godina prestupna
- brdana koji određuje koliko dana ima prosleđeni mesec

```

1 % godina je prestupna ako je deljiva sa 4 i nije deljiva sa 100 ili je deljiva sa 400
prestupna(X):- X mod 4 == 0, X mod 100 /= 0.
3 prestupna(X):- X mod 400 == 0.

5 /*
anonimna promenljiva _ se koristi da oznaci da nam vrednost koja se prosledi za
godinu nije bitna, moze biti bilo sta, ali tu vrednost ne koristimo
7

```

```

| ?- brdana(januar, _, X)
9 X = 31
yes
11 | ?- brdana(januar, 2017, X)
X = 31
13 yes
*/
15 brdana(januar, _, 31).
brdana(februar, X, 28):- not(prestupna(X)).
17 brdana(februar, X, 29):- prestupna(X).
brdana(mart,_,31).
19 brdana(april,_,30).
brdana(maj,_,31).
21 brdana(jun,_,30).
brdana(jul,_,31).
23 brdana(avgust,_,31).
brdana(septembar,_,30).
25 brdana(oktobar,_,31).
brdana(novembar,_,30).
27 brdana(decembar,_,31).

```

8.3.2 Zadaci za samostalni rad sa rešenjima

Zadatak 8.6 Napisati sledeće predikate:

- maksimum(A, B, M) - određuje maksimum za dva broja A i B
- suma(N, S) - za dati prirodan broj N računa sumu prvih N brojeva
- sumaParnih(N, S) - za dati paran prirodan broj N računa sumu parnih brojeva od 2 do N
- proizvod(N, P) - za dati prirodan broj N računa proizvod prvih N prirodnih brojeva
- proizvodNeparnih(N, P) - za dati neparan prirodan broj N računa proizvod neparnih brojeva od 1 do N
- cifre(N) - ispisuje cifre prirodnog broja N rečima

[Rešenje 8.6]

8.3.3 Zadaci za vežbu

Zadatak 8.7 Napisati sledeće predikate:

- sumaCifara(N, SC) - određuje sumu cifara prirodnog broja N
- brojCifara(N, BC) - određuje broj cifara prirodnog broja N
- maxCifra(N, MC) - određuje maksimalnu cifru prirodnog broja N
- sumaKvadrata(N, SK) - računa sumu kvadrata prvih N prirodnih brojeva
- fakt(N, F) - računa faktorijel prirodnog broja N
- sumaDel(X, D) - računa sumu pravih delilaca broja X

Zadatak 8.8 Ako su date činjenice oblika:

- ucenik(SifraUcenika, ImeUcenika, Odeljenje)
- ocene(SifraUcenika, SifraPredmeta, Ocena)
- predmet(SifraPredmeta, NazivPredmeta, BrojCasova)

Napisati sledeće predikate:

- a) `bar2PeticeSifra(S)` - određuje šifru `S` učenika koji ima bar dve petice iz različitih predmeta
- b) `bar2PeticeIme(X)` - određuje ime `X` učenika koji ima bar dve petice iz različitih predmeta
- c) `odeljenjePetice(X,Y)` - određuje odeljenje `X` u kome postoje bar dve petice iz predmeta sa šifrom `Y`

Zadatak 8.9 Ako su date činjenice oblika:

- `film(NazivFilma, ZanrFilma, ImeReditelja, SifraGlumca)`
- `glumac(SifraGlumca, ImeGlumca, GodRodj, MestoRodj)`

Napisati sledeće predikate:

- a) `filmskiUmetnik(X)` - `X` je filmski umetnik ako je reditelj nekog filma i igra u nekom filmu
- b) `glumacBarDva(X)` - određuje ime glumca `X` koji igra u bar dva različita filma
- c) `opstiGlumac(X)` - određuje ime glumca `X` koji igra u bar dva filma različitog žanra
- d) `zanrovskiGlumac(X,Y)` - određuje ime glumca `X` koji igra u filmu žanra `Y`

8.4 Liste

8.4.1 Uvodni primeri

Zadatak 8.10 Osnovni pojmovi i predikati za rad sa listama.

```

1  /*
   Lista - niz uredjenih elemenata, tj. termova.
3  Lista moze biti:
   [] - prazna
5  .(G,R) - struktura, gde je G ma koji term i naziva se glava liste, a R lista i
   naziva se rep liste

7  Primeri:
   [] - prazna
9  .(a, []) - jednoclana lista, gde je a bilo koji term
   .(a, .(b, [])) - dvoclana lista, gde su a i b termi ...

11 Zapis pomocu zagrada (prvi element predstavlja glavu, a ostali cine listu koja je rep
   ):
13 [a,b,c] <=> .(a, .(b, [c]))

15 Zapis liste u kom su jasno razdvojeni glava i rep (pogodan za unifikaciju): [G|R].

17 Primeri unifikacije listi:
[X, Y, Z] [jabuka, kruska, banana] -----> X = jabuka, Y = kruska, Z = banana
[racunar] [X|Y] -----> X = racunar, Y = []
[maja, ana, jovana] [X, Y|Z] -----> X = maja, Y = ana, Z = [jovana]

21 */

23 % predikat proverava da li element pripada listi (ako joj pripada jednak je glavi ili
   nekom elementu iz repa liste)
25 sadrzi(X, [X|_]) :- !.
   sadrzi(X, [G|R]) :- G == X, sadrzi(X, R).

27 % drugi nacin, predikat kao disjunkcija:
29 % sadrzi(X, [G|R]) :- G == X; sadrzi(X, R).

31 % predikat koji racuna duzinu liste (prazna je duzine nula, nepraznu dekomponujemo na
   glavu i rep, pa je duzina liste = 1 + duzina repa)
   duzina([], 0).
33 duzina([G|R], L) :- duzina(R, L1), L is L1+1.

35 % predikat racuna sumu elemenata liste brojeva

```

```

suma([], 0).
37 suma([G|R], S):- number(G), suma(R, S1), S is S1+G.

39 % predikat racuna aritmeticku sredinu elemenata liste brojeva
arsr([],0).
41 % ako ne koristimo sablon za nepraznu listu [G|R], moramo proveriti da li je K nula
    jer se sada L moze unifikovati sa []
arsr(L, A):- duzina(L, K), K \= 0, suma(L, S), A is S/K.

43
45 % predikat ucitava listu duzine N ciji elementi mogu biti proizvoljni termovi
% za negativno N ne ucitava listu
ucitaj(N,_) :- N < 0, !.
47 % za nulu vraća praznu listu
ucitaj(0, []).
49 % read(X) ucitanu vrednost sa ulaza dodeljuje promenljivoj X
% ako je N > 1, lista ima glavu i rep, ucitamo glavu, pa pozovemo predikat za
    ucitavanje repa
51 ucitaj(N, [G|R]):- N >= 1, write('unesi element '), read(G), nl, M is N-1, ucitaj(M,R
    ).

53 /*
    prilikom unosa vrednosti obavezna je tacka kao oznaka kraja ulaza za read
55 | ?- ucitaj(3,L)
    unesi element | 5.

57
    unesi element | 4.

59
    unesi element | 3.

61
    L = [5,4,3]
63 yes

65 | ?- ucitaj(3,L)
    unesi element | [1,2,3].

67
    unesi element | 4.

69
    unesi element | [].

71
    L = [[1,2,3],4,[]]
73 yes
75 */

```

8.4.2 Zadaci za samostalni rad sa rešenjima

Zadatak 8.11 Napisati sledeće predikate:

- dodajPocetak(X, L, NL) - dodaje X na početak liste L
- dodajKraj(X, L, NL) - dodaje X na kraj liste L
- obrisiPrvi(L, NL) - briše prvi element, tj. glavu liste
- obrisiPoslednji(L, NL) - briše poslednji element liste
- obrisi(X, L, NL) - briše sva pojavljivanja elementa X u listi L
- obrisiPrvo(X, L, NL) - briše samo prvo pojavljivanje elementa X u listi L
- obrisiK(L, K, NL) - briše K-ti element liste L

[Rešenje 8.11]

Zadatak 8.12 Napisati predikat `podeli(L, L1, L2)` koji deli listu L na dve liste, listu pozitivnih elemenata L1 i listu negativnih elemenata L2.

[Rešenje 8.12]

Zadatak 8.13 Napisati predikat `dupliraj(L, NL)` koji od date liste `L` formira novu listu `NL` tako što svaki negativan element duplira, tj. dva puta upisuje u novu listu.

[Rešenje 8.13]

Zadatak 8.14 Napisati predikat `zameni(X, Y, L, NL)` koji od date liste `L` formira novu listu `NL` zamenom elemenata `X` i `Y`.

[Rešenje 8.14]

Zadatak 8.15 Napisati predikat `pretvori(L, X)` koji za datu listu cifara `L` formira broj određen tim ciframa.

[Rešenje 8.15]

Zadatak 8.16 Napisati predikat `maxEl(L, X)` koji određuje maksimalni element liste `L`.

[Rešenje 8.16]

Zadatak 8.17 Napisati predikate za sortiranje liste rastuće:

- a) `insertionSort(L, SL)` - insertion sort algoritam se zasniva na ubacivanju redom svakog elementa liste na svoje pravo mesto (mesto u sortiranoj listi)
- b) `mergeSort(L, SL)` - merge sort algoritam se zasniva na dekompoziciji liste, tj. listu delimo na dva jednaka dela, te delove sortiramo i posle toga ih objedinjujemo

[Rešenje 8.17]

8.4.3 Zadaci za vežbu

Zadatak 8.18 Napisati predikat `parNepar(L, L1, L2)` koji deli listu `L` na dve liste, listu parnih elemenata `L1` i listu neparnih elemenata `L2`.

Zadatak 8.19 Napisati predikat `podeli(L, N, L1, L2)` koji deli listu `L` na dve liste `L1` i `L2`, pri čemu je zadata dužina prve liste `L1`.

Zadatak 8.20 Napisati predikat `ogledalo(L1, L2)` koji proverava da li je lista `L1` jednaka obrnutoj listi liste `L2`.

Zadatak 8.21 Napisati predikat `interval(X, Y, L)` koji kreira listu `L` koja sadži sve cele brojeve iz intervala zadanog sa prva dva argumenta.

Zadatak 8.22 Napisati predikat `skalar(L1, L2, S)` koji određuje skalarni proizvod dva vektora, tj. listi brojeva `L1` i `L2`.

Zadatak 8.23 Napisati predikat `sortirana(L)` koji proverava da li je lista `L` sortirana, bilo opadajuće ili rastuće.

Zadatak 8.24 Napisati predikat `spoji(L1, L2, L)` koji spaja dve rastuće sortirane liste `L1` i `L2` u treću tako da i ona bude sortirana rastuće.

8.5 Razni zadaci

8.5.1 Zadaci sa rešenjima

Zadatak 8.25 Napisati predikate `nzd(N, M, NZD)` i `nzs(N, M, NZS)` koji određuju najveći zajednički delilac i najmanji zajednički sadržalac prirodnih brojeva `N` i `M` redom.

[Rešenje 8.25]

Zadatak 8.26 Ako su date činjenice oblika:

- `stan(Porodica, KvadraturaStana)`
- `clan(Porodica, BrojClanova)`

Napisati predikat `poClanu(Porodica, Prosek)` koji određuje prosečan broj kvadrata stana po članu porodice koja živi u njemu.

[Rešenje 8.26]

Zadatak 8.27 Ako je data baza znanja:

- `automobil(SifraAutomobila, NazivAutomobila)`
- `vlasnik(ImeVlasnika, SifraAutomobila)`
- `brziSifra(SX, SY)` - automobil šifre `SX` je brži od automobila šifre `SY`

Napisati predikate:

- a) `brziNaziv(X, Y)` - automobil naziva `X` je brži od automobila naziva `Y`
- b) `imaAutomobil(X)` - `X` je vlasnik nekog automobila
- c) `imaBrzi(X, Y)` - `X` je vlasnik bržeg automobila od onog čiji je vlasnik `Y`.

[Rešenje 8.27]

Zadatak 8.28 Napisati predikat `savrsen(N)` koji proverava da li je prirodan broj `N` savršen, tj. da li je jednak sumi svojih pravih delilaca. U slučaju da se prosledi neispravan argument, predikat treba da ispiše poruku o grešci i prekine program.

[Rešenje 8.28]

Zadatak 8.29 Napisati predikat `izbaci3(N, X)` koji iz prirodnog broja `N` izbacuje sve cifre manje 3. U slučaju da se prosledi neispravan argument, predikat treba da prekine program.

[Rešenje 8.29]

Zadatak 8.30 Napisati predikate za liste brojeva:

- a) `duplikati(L, L1)` - izbacuje duplikate iz liste `L`
- b) `unija(L1, L2, L)` - određuje uniju listi `L1` i `L2`
- c) `presek(L1, L2, L)` - određuje presek listi `L1` i `L2`
- d) `razlika(L1, L2, L)` - određuje razliku listi `L1` i `L2`

[Rešenje 8.30]

Zadatak 8.31 Napisati program koji rešava sledeću zagonetku. Postoji pet kuća, svaka različite boje u kojoj žive ljudi različitih nacionalnosti koji piju različita pića, jedu različita jela i imaju različite kućne ljubimce. Važi sledeće:

- Englez živi u crvenoj kući
- Španac ima psa
- kafa se pije u zelenoj kući
- Ukrajinac pije čaj
- zelena kuća je odmah desno uz belu
- onaj koji jede špagete ima puža
- pica se jede u žutoj kući
- mleko se pije u srednjoj kući

- Norvežanin živi u prvoj kuci s leva
- onaj koji jede piletinu živi pored onoga koji ima lisicu
- pica se jede u kući koja je pored kuće u kojoj je konj
- onaj koji jede brokoli pije sok od narandže
- Japanac jede suši
- Norvežanin živi pored plave kuće

Čija je zebra, a ko pije vodu?

[Rešenje 8.31]

Zadatak 8.32 Napisati program koji rešava sledeću zagonetku. Svakog vikenda, Milan čuva petoro komšijske dece. Deca se zovu Kata, Lazar, Marko, Nevenka i Ognjen, a prezivaju Filipović, Grbović, Hadžić, Ivanović i Janković. Svi imaju različit broj godina od dve do šest. Važi sledeće:

- jedno dete se zove Lazar Janković
- Kata je godinu dana starija od deteta koje se preziva Ivanović koje je godinu dana starije od Nevenke
- dete koje se preziva Filipović je tri godine starije od Marka
- Ognjen je duplo stariji od deteta koje se preziva Hadžić

Kako se ko zove i koliko ima godina?

[Rešenje 8.32]

8.5.2 Zadaci za vežbu

Zadatak 8.33 Napisati predikat `uzastopni(X, Y, Z, L)` koji proverava da li su prva tri argumenta uzastopni elementi u listi `L`.

Zadatak 8.34 Napisati predikat `kompresuj(L, KL)` koji u datoj listi `L` eliminiše uzastopne duplikate.

Zadatak 8.35 Napisati predikat `prefiksi(L, P)` koji određuje sve liste koje su prefiksi date liste `L`.

Zadatak 8.36 Napisati predikat `sufiksi(L, S)` koji određuje sve liste koje su sufiksi date liste `L`.

Zadatak 8.37 Napisati predikat `opadajuće(N, L)` koji za dat prirodan broj `N` formira listu brojeva od `N` do 1.

Zadatak 8.38 Napisati predikat `form(N, L)` kojim se formira lista od prirodnih brojeva deljivih sa 5 i manjih od datog prirodnog broja `N`.

Zadatak 8.39 Napisati program koji rešava sledeću zagonetku. Četiri žene se zovu Petra, Milica, Lenka i Jovana, a prezivaju Perić, Mikić, Lazić i Jović. One imaju četiri kćerke koje se takodje zovu Petra, Milica, Lenka i Jovana. Važi sledeće:

- nijedna majka nema prezime koje počinje istim slovom kao ime
- nijedna kćerka nema prezime koje počinje istim slovom kao ime
- nijedna kćerka se ne zove kao majka
- majka koja se preziva Perić se zove isto kao Milićina kćerka
- Lenkina kćerka se zove Petra

Odrediti imena majki i kćerki.

Zadatak 8.40 Napisati program koji rešava sledeću zagonetku. Četiri para je došlo na maskenbal:

- Markova žena se maskirala kao macka
- dva para su stigla pre Marka i njegove žene, a jedan muskarac je bio maskiran u medveda
- prvi koji je stigao nije bio Vasa, ali je stigao pre onoga koji je bio maskiran u princa
- žena maskirana u vešticu (nije Bojana) je udata za Peru, koji se maskirao kao Paja patak
- Marija je došla posle Laze, a oboje su stigli pre Bojane
- žena maskirana u Ciganku je stigla pre Ane, pri čemu nijedna od njih nije udata za muškarca maskiranog u Betmena
- žena maskirana u Snežanu je stigla posle Ivane

Odrediti kako je bio obučen koji par.

Zadatak 8.41 Izračunavanje vrednosti aritmetičkog izraza korišćenjem listi možete pogledati ovde:

https://rosettacode.org/wiki/Arithmetic_evaluation#Prolog

Zadatak 8.42 Implementacije raznih problema u Prologu možete pogledati ovde:

<https://rosettacode.org/wiki/Category:Prolog>

9

Programiranje ograničenja - Prolog

Potrebno je imati instaliran B-Prolog na računaru.

Literatura:

- (a) <http://www.picat-lang.org/bprolog/>
- (b) <http://www.picat-lang.org/bprolog/download/manual.pdf>

9.1 Programiranje ograničenja

9.1.1 Uvodni primeri

Zadatak 9.1 Osnovni pojmovi i postavka problema.

```
2  /*
3
4  Programiranje ogranicenja nad konacnim domenom:
5  1) generisanje promenljivih i njihovih domena
6  2) generisanje ogranicenja nad promenljivima
7  3) instanciranje promenljivih ili obelezavanje
8
9  Definisanje domena (D) promenljivih:
10 -- X in D ili X :: D -> promenljiva X uzima samo vrednosti iz konacnog domena D
11 -- Vars in D ili Vars :: D -> sve promenjive iz liste Vars uzimaju samo vrednosti iz
12    konacnog domena D
13 Domen se definise kao interval u obliku Pocetak..Korak..Kraj (Korak je opcion i
14    ukoliko se ne navede, podrazumeva se da je Korak = 1)
15 Primeri za domen:
16 1..10 -> 1,2,3,4,5,6,7,8,9,10
17 1..2..10 -> 1,3,5,7,9
18
19 Osnovni predikati za ogranicenja
20
21 1) opsta:
22 -- alldifferent(Vars) ili alldistinct(Vars) - sve vrednosti razlicite u listi termova
23    Vars
24 -- atmost(N,L,V) - najvise N elemenata iz skupa L jednako sa V (N mora biti Integer,
25    V term, a L lista termova)
26 -- atleast(N,L,V) - najmanje N elemenata iz skupa L jednako sa V (N mora biti Integer
27    , V term, a L lista termova)
28 -- excatly(N,L,V) - tacno N elemenata iz skupa L jednako sa V (N mora biti Integer, V
29    term, a L lista termova)
30
31 2) aritmeticka:
32 -- E1 R E2 gde su E1 i E2 aritmeticki izrazi, a R iz skupa {#=, #\=, #>=, #>, #<=,
33    #<}
34 -- min(L) - minimalni element iz liste termova L
35 -- max(L) - maksimalni element iz liste termova L
```

```

28 -- max(E1, E2)/min(E1, E2) -- manji/veci od izraza E1 i E2
-- sum(L) - suma elemenata liste termova L

30
Instanciranje i prikazivanje promenljivih: labeling(Vars).
32 Funkcija labeling se moze pozvati i sa razlicitim opcijama u obliku labeling(Options,
    Vars)
gde je Options lista opcija. Ukoliko se pozove sa labeling(Vars), podrazumevano je
Options = []. Neke od opcija:
34 -- minimize(E) - trazi instance za Vars pri kojima je vrednost celobrojnog izraza E
    minimalna
-- maximize(E) - trazi instance za Vars pri kojima je vrednost celobrojnog izraza E
    maksimalna

36
*/

38
/*
40 Primer: X pripada skupu {1,2,3}, Y skupu {2,4,6,8,10}, Z skupu {5,6,7,8} i vazi Z>=Y
*/

42
primer(Vars) :- Vars = [X, Y, Z], % generisanje promenljivih
44     X :: 1..3, % definisanje domena
     Y :: 2..2..10,
46     Z :: 5..8,
     Z #>= Y, % ogranicenje
48     labeling(Vars). % instanciranje

50
/*
| ?- primer(Vars).
52 Vars = [1,2,5] ?;
Vars = [1,2,6] ?;
54 Vars = [1,2,7] ?;
Vars = [1,2,8] ?;
56 Vars = [1,4,5] ?;
Vars = [1,4,6] ?;
58 Vars = [1,4,7] ?;
Vars = [1,4,8] ?;
60 Vars = [1,6,6] ?;
Vars = [1,6,7] ?;
62 Vars = [1,6,8] ?;
Vars = [1,8,8] ?;
64 ...
*/

66
/*
68 Primer: ispisati sve brojeve od 1..100 koji su puni kvadrati
*/

70
puni(Vars) :- Vars = [X],
72     Vars :: 1..100,
     Y*Y #= X ,
74     labeling(Vars).

76
/*
| ?- puni(Vars)
78 Vars = [1] ?;
Vars = [4] ?;
80 Vars = [9] ?;
Vars = [16] ?;
82 Vars = [25] ?;
Vars = [36] ?;
84 Vars = [49] ?;
Vars = [64] ?;
86 Vars = [81] ?;
Vars = [100] ?;
88 no
90
*/

```

9.1.2 Zadaci za samostalni rad sa rešenjima

Zadatak 9.2 Napisati program koji pronalazi petocifren broj ABCDE za koji je izraz $A+2*B-3*C+4*D-5*E$

minimalan i A, B, C, D i E su različite cifre.

[Rešenje 9.2]

Zadatak 9.3 Dati su novčići od 1, 2, 5, 10, 20 dinara. Napisati program koji pronalazi sve moguće kombinacije tako da zbir svih novčića bude 50 i da se svaki novčić pojavljuje bar jednom u kombinaciji.

[Rešenje 9.3]

Zadatak 9.4 Napisati program koji reda brojeve u magičan kvadrat. Magičan kvadrat je kvadrat dimenzija 3x3 takav da je suma svih brojeva u svakom redu, svakoj koloni i svakoj dijagonali jednak 15 i svi brojevi različiti. Na primer:

```
4 9 2
3 5 7
8 1 6
```

[Rešenje 9.4]

Zadatak 9.5 Napisati program koji pronalazi sve vrednosti promenljivih X, Y, Z za koje važi da je $X \geq Z$ i $X * 2 + Y * X + Z \leq 34$ pri čemu promenljive pripadaju narednim domenima $X \in \{1, 2, \dots, 90\}$, $Y \in \{2, 4, 6, \dots, 60\}$ i $Z \in \{1, 10, 20, \dots, 100\}$

[Rešenje 9.5]

Zadatak 9.6 Napisati program koji dodeljuje različite vrednosti različitim karakterima tako da suma bude zadovoljena:

```
TWO
+TWO
-----
FOUR
```

[Rešenje 9.6]

Zadatak 9.7 Napisati program koji pronalazi sve vrednosti promenljivih X, Y, Z i W za koje važi da je $X \geq 2 * W$, $3 + Y \leq Z$ i $X - 11 * W + Y + 11 * Z \leq 100$ pri čemu promenljive pripadaju narednim domenima $X \in \{1, 2, \dots, 10\}$, $Y \in \{1, 3, 5, \dots, 51\}$, $Z \in \{10, 20, 30, \dots, 100\}$ i $W \in \{1, 8, 15, 22, \dots, 1000\}$.

[Rešenje 9.7]

Zadatak 9.8 Napisati program koji raspoređuje brojeve 1-9 u dve linije koje se seku u jednom broju. Svaka linija sadrži 5 brojeva takvih da je njihova suma u obe linije 25 i brojevi su u rastućem redosledu.

```
1 3
2 4
5
6 8
7 9
```

[Rešenje 9.8]

Zadatak 9.9 Pekara *Kiflica* proizvodi hleb i kifle. Za mešenje hleba potrebno je 10 minuta, dok je za kiflu potrebno 12 minuta. Vreme potrebno za pečenje ćemo zanemariti. Testo za hleb sadrži 300g brašna, a testo za kiflu sadrži 120g brašna. Zarada koja se ostvari prilikom prodaje jednog hleba je 7 dinara, a prilikom prodaje jedne kifle je 9 dinara. Ukoliko pekara ima 20 radnih sati za mešenje peciva i 20kg brašna, koliko komada hleba i kifli treba da se umesi kako bi se ostvarila maksimalna zarada (pod pretpostavkom da će pekara sve prodati)?

[Rešenje 9.9]

Zadatak 9.10 Napisati program pronalazi vrednosti A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S (svako slovo predstavlja različit broj) koje su poredane u heksagon na sledeći način:

A,B,C
D,E,F,G
H,I,J,K,L
M,N,O,P
Q,R,S

tako da zbir vrednosti duž svake horizontalne i dijagonalne linije bude 38 ($A+B+C = D+E+F+G = \dots = Q+R+S = 38$, $A+D+H = B+E+I+M = \dots = L+P+S = 38$, $C+G+L = B+F+K+P = \dots = H+M+Q = 38$).

[Rešenje 9.10]

Zadatak 9.11 Kompanija Start ima 250 zaposlenih radnika. Rukovodstvo kompanije je odlučilo da svojim radnicima obezbedi dodatnu edukaciju. Da bi se radnik obučio programskom jeziku Elixir potrebno je platiti 100 evra po osobi za kurs, ali bi njegovo produktivno znanje ovog programskog jezika donelo 150 projekat/sati mesečno, što bi za kompaniju značilo dobit od 5 evra po projekat/satu. Da bi se radnik obučio programskom jeziku Dart potrebno je platiti 105 evra po osobi za kurs, ali bi njegovo produktivno znanje ovog programskog jezika donelo 170 projekat/sati mesečno, koji bi za kompaniju značili dobit od 6 evra po satu. Ukoliko Start ima na raspolaganju 26000 evra za obuku i maksimalan broj 51200 mogućih projekat/sati mesečno, odrediti na koji način kompanija treba da obuči svoje zaposlene kako bi ostvarila maksimalnu dobit.

[Rešenje 9.11]

Zadatak 9.12 Napisati program koji raspoređuje n dama na šahovsku tablu dimenzije $n \times n$ tako da se nikoje dve dame ne napadaju.

[Rešenje 9.12]

Zadatak 9.13 Napisati program koji za ceo broj n ispisuje magičnu sekvencu S brojeva od 0 do $n-1$. $S = (x_0, x_1, \dots, x_{n-1})$ je magična sekvenca ukoliko postoji x_i pojavljivanja broja i za $i = 0, 1, \dots, n-1$.

[Rešenje 9.13]

9.1.3 Zadaci za vežbu

Zadatak 9.14 Za svaku narednu zagonetku, napisati program koji dodeljuje različite vrednosti različitim karakterima tako da suma bude zadovoljena:

GREEN + ORANGE = COLORS
MANET + MATISSE + MIRO + MONET + RENOIR = ARTISTS
COMPLEX + LAPLACE = CALCULUS
THIS + IS + VERY = EASY
CROSS + ROADS = DANGER
FATHER + MOTHER = PARENT
WE + WANT + NO + NEW + ATOMIC = WEAPON
EARTH + AIR + FIRE + WATER = NATURE
SATURN + URANUS + NEPTUNE + PLUTO = PLANETS
SEE + YOU = SOON
NO + GUN + NO = HUNT
WHEN + IN + ROME + BE + A = ROMAN
DONT + STOP + THE = DANCE
HERE + THEY + GO = AGAIN
OSAKA + HAIKU + SUSHI = JAPAN
MACHU + PICCHU = INDIAN
SHE + KNOWS + HOW + IT = WORKS
COPY + PASTE + SAVE = TOOLS

Zadatak 9.15 Za svaku narednu zagonetku, napisati program koji dodeljuje različite vrednosti različitim karakterima tako da suma bude zadovoljena:

```
THREE + THREE + ONE = SEVEN
NINE + LESS + TWO = SEVEN
ONE + THREE + FOUR = EIGHT
THREE + THREE + TWO + TWO + ONE = ELEVEN
SIX + SIX + SIX = NINE + NINE
SEVEN + SEVEN + SIX = TWENTY
ONE + ONE + ONE + THREE + THREE + ELEVEN = TWENTY
EIGHT + EIGHT + TWO + ONE + ONE = TWENTY
ELEVEN + NINE + FIVE + FIVE = THIRTY
NINE + SEVEN + SEVEN + SEVEN = THIRTY
TEN + SEVEN + SEVEN + SEVEN + FOUR + FOUR + ONE = FORTY
TEN + TEN + NINE + EIGHT + THREE = FORTY
FOURTEEN + TEN + TEN + SEVEN = FORTYONE
NINETEEN + THIRTEEN + THREE + TWO + TWO + ONE + ONE + ONE = FORTYTWO
FORTY + TEN + TEN = SIXTY
SIXTEEN + TWENTY + TWENTY + TEN + TWO + TWO = SEVENTY
SIXTEEN + TWELVE + TWELVE + TWELVE + NINE + NINE = SEVENTY
TWENTY + TWENTY + THIRTY = SEVENTY
FIFTY + EIGHT + EIGHT + TEN + TWO + TWO = EIGHTY
FIVE + FIVE + TEN + TEN + TEN + TEN + THIRTY = EIGHTY
SIXTY + EIGHT + THREE + NINE + TEN = NINETY
ONE + NINE + TWENTY + THIRTY + THIRTY = NINETY
```

Zadatak 9.16 Za svaku narednu zagonetku, napisati program koji dodeljuje različite vrednosti različitim karakterima tako da jednakost bude zadovoljena:

```
MEN * AND = WOMEN
COGITO = ERGO * SUM
((JE + PENSE) - DONC) + JE = SUIS
FERMAT * S = LAST + THEOREM.
WINNIE / THE = POOH
TWO * TWO + EIGHT = TWELVE
```

Zadatak 9.17 Uraditi sve zadatke koji su pobrojani ovde:
<http://www.primepuzzle.com/leeslatest/alphameticpuzzles.html>

Zadatak 9.18 Čistačica Mica sređuje i čisti kuće i stanove. Da bi sredila i počistila jedan stan potrebno joj je 1 sat, dok joj je za kuću potrebno 1.5 sati. Prilikom čišćenja, Mica potroši neku količinu deterdženta, 120ml po stanu, odnosno 100ml po kući. Mica zaradi 1000 dinara po svakom stanu, odnosno 1500 dinara po kući. Ukoliko Mica radi 40 sati nedeljno i ima 5l deterdženta na raspolaganju, koliko stanova i kuća je potrebno da očisti kako bi imala najveću zaradu?

Zadatak 9.19 Marija se bavi grnčarstvom i pravi šolje i tanjire. Da bi se napravila šolja, potrebno je 6 minuta, dok je za tanjir potrebno 3 minuta. Pri pravljenju šolje potroši se 75 gr, dok se za tanjir potroši 100 gr gline. Ukoliko ima 20 sati na raspolaganju za izradu svih proizvoda i 250 kg gline, a zarada koju ostvari iznosi 2 evra po svakoj šolji i 1.5 evra po tanjiru, koliko šolja i tanjira treba da napravi kako bi ostvarila maksimalnu zaradu?

Zadatak 9.20 Jovanin komšija preprodaje računare i računarsku opremu. Očekuje isporuku računara i štampača. Pri tom, računari su spakovani tako da njihova kutija zauzima 360 kubnih decimetara prostora, dok se štampači pakuju u kutijama koje zauzimaju 240 kubnih decimetara prostora. Komšija se trudi da mesečno proda najmanje 30 računara i da taj broj bude bar za 50% veći od broja prodatih štampača. Računari koštaju 200 evra po nabavnoj ceni, a prodaju se po ceni od 400 evra, dok štampači koštaju u nabavci 60 evra i prodaju se za 140 evra. Magacin kojim komšija raspolaže ima svega 30000 kubnih decimetara prostora i mesečno može da nabavi robu u iznosu od najviše 14000 evra. Koliko računara, a koliko štampača komšija treba da proda kako bi se maksimalno obogatio?

9.2 Rešenja

Rešenje 8.6

```

1  % maksimum dva broja
   % I nacin:
3  maksimum(A,B,M):- A>=B, M is A.
   maksimum(A,B,M):- A<B, M is B.
5  % II nacin bez trece promenjive:
   % maksimum(A,B,A):- A>=B.
7  % maksimum(A,B,B):- A<B.

9  % suma prvih N prirodnih brojeva
   suma(1,1).
11 suma(N,S):- N>1, N1 is N-1, suma(N1,S1), S is S1+N.

13 % suma parnih prirodnih brojeva od 2 do N
   % moze se dodati provera N mod 2 == 0 u pravilu, ali i bez toga sam prepoznaje za
   % neparne da je netacan upit jer rekurzijom dodje do sumaParnih(1,S) sto je netacna
   % cinjenica u bazi
15 sumaParnih(2,2).
   sumaParnih(N,S):- N>2, N1 is N-2, sumaParnih(N1,S1), S is S1+N.

17 % proizvod prvih N prirodnih brojeva
19 proizvod(1,1).
   proizvod(N,P):- N>1, N1 is N-1, proizvod(N1,P1), P is P1*N.

21 % proizvod neparnih prirodnih brojeva od 1 do N
23 proizvodNeparnih(1,1).
   proizvodNeparnih(N,P):- N>1, N1 is N-2, proizvodNeparnih(N1,P1), P is P1*N.

25 % ispis cifara unetog prirodnog broja N
27 cifra(0, nula).
   cifra(1, jedan).
29 cifra(2, dva).
   cifra(3, tri).
31 cifra(4, cetiri).
   cifra(5, pet).
33 cifra(6, sest).
   cifra(7, sedam).
35 cifra(8, osam).
   cifra(9, devet).

37 % ukoliko nije prirodan broj, cut operatorom sprecavamo poziv poslednjeg predikata
39 cifre(N):- N < 1, !.

41 % ukoliko je jednocifren svodi se na poziv predikata cifra
   % write(t) gde je t neki term, ispisuje term
43 % nl (newline) - ispisuje se novi red
   % obratiti paznju na upotrebu cut operatora ! - sprecavamo poziv poslednjeg predikata
   % za jednocifrene
45 cifre(N):- N > 1, N < 10, cifra(N, C), write(C), nl, !.

47 % ukoliko nije jednocifren, racunamo tekucu cifru koju ispisujemo i ostatak broja za
   % koji se ponovo poziva predikat
   cifre(N):- N1 is (N // 10), cifre(N1), N2 is (N mod 10), cifra(N2, C), write(C), nl.

```

Rešenje 8.11

```

1  % dodaje element na pocetak liste
   dodajPocetak(X, L, [X|L]).

3  % dodaje element na kraj liste
5  dodajKraj(X, [], [X]).
   dodajKraj(X, [G|R], [G|LR]):- dodajKraj(X, R, LR).

7  % brise prvi element liste
9  % za praznu listu ce uvek vratiti no jer ne moze unifikovati sa sablonom [G|R], a
   % mozemo napraviti i sami za taj slucaj da je predikat netacan
   % fail je uvek netacan pa ce nam ovako definisan predikat za slucaj prazne liste
   % vratiti no

```

```

11 obrisiPrvi([], _):- fail.
   obrisiPrvi([_|R], R).
13
14 % brise poslednji element liste
15 obrisiPoslednji([], _):- fail.
   % bitan cut operator jer se jednoclana moze upariti sa sablonom [G|R]
17 obrisiPoslednji([], []):- !.
   obrisiPoslednji([G|R], [G|R1]):- obrisiPoslednji(R, R1).
19
20 % brise element X iz liste ako postoji (svako pojavljivanje elementa X)
21 obrisi(_, [], []).
   obrisi(X, [X|R], R1):- obrisi(X, R, R1), !.
23 obrisi(X, [G|R], [G|R1]):- G \== X, obrisi(X, R, R1).

24 % brise element X iz liste ako postoji (samo prvo pojavljivanje elementa X)
   obrisiPrvo(X, [], []).
27 obrisiPrvo(X, [X|R], R):- !.
   obrisiPrvo(X, [G|R], [G|R1]):- G \== X, obrisiPrvo(X, R, R1).
29
30 % brise K-ti element liste, brojimo od 1, ako je K vece od duzine liste, treci
   argument je jednak prvom
31 obrisiK([], K, []):- K > 0.
   obrisiK([G|R], 1, R):- !.
33 obrisiK([G|R], K, [G|R1]):- K>1, K1 is K-1, obrisiK(R, K1, R1).

34 % druga varijanta predikata: brise K-ti element liste, broji od 1, ali ukoliko zadata
   lista nema K-ti element, predikat vraca no kao odgovor i nema unifikacije za
   treci argument
   % obrisiK([], K, []):- fail.
37 % obrisiK([G|R], 1, R):- !.
   % obrisiK([G|R], K, [G|R1]):- K>1, K1 is K-1, obrisiK(R, K1, R1).

```

Rešenje 8.12

```

1 % podeli1 - deli listu na dve liste - listu pozitivnih i listu negativnih elemenata
   % L1 - lista pozitivnih, L2 - lista negativnih
3 podeli([], [], []).
   podeli([G|R], [G|R1], L2):- G >= 0, podeli(R, R1, L2), !.
5 podeli([G|R], L1, [G|R2]):- G < 0, podeli(R, L1, R2).

```

Rešenje 8.13

```

1 % svaki negativan element dodajemo dva puta u novu listu, a pozitivne samo jednom
2 dupliraj([], []).
   dupliraj([G|R], [G,G|R1]):- G<0, dupliraj(R, R1), !.
4 dupliraj([G|R], [G|R1]):- G>=0, dupliraj(R, R1).

```

Rešenje 8.14

```

1 % menja elemente X i Y u listi
2 zameni(X, Y, [], []).
   zameni(X, Y, [X|R], [Y|R1]):- zameni(X, Y, R, R1), !.
4 zameni(X, Y, [Y|R], [X|R1]):- zameni(X, Y, R, R1), !.
   zameni(X, Y, [G|R], [G|R1]):- G \== X, G \== Y, zameni(X, Y, R, R1).

```

Rešenje 8.15

```

1 % potreban nam je dodatni predikat za izdvajanje poslednjeg elementa liste
2 izdvojPoslednji([G], G, []):- !.
   izdvojPoslednji([G|R], X, [G|R1]):- izdvojPoslednji(R, X, R1).
4
5 % formira broj od date liste cifara
6 pretvori([], 0):- !.
   pretvori(L, X):- izdvojPoslednji(L, Poslednji, Ostatak),
8       pretvori(Ostatak, Y),
       X is Poslednji + 10 * Y.
10

```

```
/*
12 | ?- pretvori([7,0,7], X)
   X = 707
14 yes
   | ?- pretvori([0,2,3], X)
16 X = 23
   yes
18 | ?- pretvori([], X)
   X = 0
20 yes
22 */
```

Rešenje 8.16

```
% maksimalni element liste
2 maxEl([X], M):- M is X, !.
% pozivamo za rep (idemo u dubinu), pa poredimo maksimalni element repa i glavu liste
4 maxEl([G|R], X):- maxEl(R, Y), G < Y, X is Y, !.
   maxEl([G|R], X):- maxEl(R, Y), G >= Y, X is G.
```

Rešenje 8.17

```
/*
2 Insertion sort algoritam se zasniva na ubacivanju redom svakog elementa liste na
   svoje pravo
4 mesto.

6 IH (Induktivna hipoteza) - umemo da sortiramo listu od n-1 elementa.
   IK (Induktivni korak) - n-ti element ubacujemo na odgovarajucu lokaciju.
8
*/
10 insertionSort([], []).
   insertionSort([G|R], SL):- insertionSort(R, S1), ubaciS(G, S1, SL).
12
% ubacuje sortirano element X u listu
14 ubaciS(X, [], [X]).

16 % ubaciS(X, [G|R], [X, G|R]):- X=<G.
% ubaciS(X, [G|R], [G|SL]):- X>G, ubaciS(X, R, SL).
18
/*
   alternativno sa cut operatorom ne moramo
20 da pisemo X>G u drugom predikatu, jer
   kada stavimo cut operator kod prvog, cim
22 on uspe drugi predikat se neće ni pokušavati
*/
24 ubaciS(X, [G|R], [X, G|R]):- X=<G, !.
   ubaciS(X, [G|R], [G|SL]):- ubaciS(X, R, SL).
26
/*
28 Merge sort algoritam se zasniva na dekompoziciji. Naime, ulaznu listu delimo na dva
   jednaka
30 dela, te delove sortiramo i posle toga ih objedinjujemo.

32 IH - umemo da sortiramo liste velicine n/2.
   IK - u linearnom vremenu objedinjujemo dve sortirane liste od po n/2 elemenata.
34
*/
36 mergeSort([], []).
   mergeSort([X], [X]).
38 mergeSort(N, SL):- podeli(N, L, R),
   mergeSort(L, L1),
40 mergeSort(R, R1),
   objedini(L1, R1, SL).
42
% delimo tako sto prvi element stavljamo u prvu particiju
44 % drugi u drugu pri cemu su particije dobijene
   % vec rekursivno podelom repa
```

```

46 podeli([], [], []).
47 podeli([X], [X], []) :- !.
48 podeli([G1, G2|R1], [G1|R1], [G2|R2]) :- podeli(R1, R1, R2).

50 /*
   Primer: ako objedinjujemo [1,3,5,7] i [5,6,7,8,9] znamo da 1 prethodi svima pa mozemo
         rekurzivno da objedinimo [3,5,7] sa [5,6,7,8,9] i dodamo 1 na pocetak tako
         objedinjene liste
52 */
53 objedini(L, [], L).
54 objedini([], L, L).
55 objedini([G1|R1], [G2|R2], [G1|R]) :- G1 < G2, objedini(R1, [G2|R2], R), !.
56 objedini([G1|R1], [G2|R2], [G2|R]) :- G1 >= G2, objedini([G1|R1], R2, R).

```

Rešenje 8.25

```

1 % poredjenjem glava listi zakljucujemo u kom redosledu dodajemo elemente u spoenu
   listu
2 spoji([], L, L) :- !.
3 spoji(L, [], L) :- !.
4 spoji([G1|R1], [G2|R2], [G1|R]) :- G1 < G2, spoji(R1, [G2|R2], R), !.
5 spoji([G1|R1], [G2|R2], [G2|R]) :- G1 >= G2, spoji([G1|R1], R2, R).

```

Rešenje 8.26

```

1 /*
   za odredjivanje nzd i nzs dva prirodna broja koristicemo Euklidov algoritam
3
   bitno je obezbediti da se pogram ispravno ponasa - kada nadje jedno resenje treba
   obezbediti prekid koriscenjem cut operatora u 1. klauzi nzdBom, jer bi u
   suprotnom program zasao u skup negativnih brojeva (pa bi nastao beskonacan ciklus
   ) ili pokusaj deljenja sa nulom (sto bi prijavilo gresku)
5 */
6 nzdB(A,B,NZD) :- A >= B, B > 0, nzdBom(A,B,NZD), !.
7 nzdB(A,B,NZD) :- A < B, A > 0, nzdBom(B,A,NZD).

9 nzdBom(N, 0, N) :- !.
10 nzdBom(N, M, NZD) :- P is N mod M, nzdBom(M, P, NZD).

11
12 % nzs dobijamo mnozenjem oba prirodna broja i deljenjem sa nzdB
13 nzs(N, M, NZS) :- nzdB(N, M, P), NZS is N*M//P.

15 /*
   | ?- nzdB(15,9,NZD)
17 NZD = 3
   yes
19 | ?- nzdB(12,8,NZD)
   NZD = 4
   yes
21 | ?- nzs(13,17,NZS)
23 NZS = 221
   yes
25 */

```

Rešenje 8.27

```

1 % cinjenice
2 stan(petrovic, 76).
3 stan(ciric, 93).
4 stan(aleksic, 55).
5 stan(lisic, 123).
6 stan(peric, 67).

7
8 clan(ciric, 3).
9 clan(peric, 5).
10 clan(aleksic, 2).
11 clan(lisic, 3).
12 clan(petrovic, 4).
13

```

```

% koristimo operator za realno deljenje / (// je za celobrojno, ne mesati ova dva
  operatora) da bi se dobio tacan rezultat
15 poClanu(Porodica, Prosek):- stan(Porodica, X), clan(Porodica, Y), Prosek is X/Y.

```

Rešenje 8.28

```

% baza znanja
2 automobil(a1, audi).
  automobil(h1, honda).
4 automobil(m1, mercedes).
  automobil(m2, mercedes).
6 automobil(c1, citroen).
  automobil(c2, citroen).
8
10 vlasnik(milan, h1).
  vlasnik(maja, m1).
  vlasnik(nemanja, m2).
12 vlasnik(aleksandar, a1).
  vlasnik(andjela, c1).
14 vlasnik(petar, c2).

16 brziSifra(a1, c1).
  brziSifra(m1, c1).
18 brziSifra(m2, h1).
  brziSifra(a1, c2).
20
22 brziNaziv(X, Y):- automobil(SX, X), automobil(SY, Y), brziSifra(SX, SY).
  imaAutomobil(X):- vlasnik(X, _).
24
  imaBrzi(X, Y):- vlasnik(X, S1), vlasnik(Y, S2), brziSifra(S1, S2).

```

Rešenje 8.29

```

% fail ce prouzrokovati da 2. klauza uvek bude netacna jer zelimo da se poziv
  predikata za neispravan argument vidi kao netacna cinjenica u bazi
2 proveraN(N):- N > 0.
  proveraN(N):- N <= 0, write('Broj nije prirodan'), nl, fail.
4
% predikat sumaPom za treci argument ima kandidata delioca broja N koji postepeno
  uvecavamo (pocinjemo od jedinice koja deli svaki broj)
6 sumaDelilaca(N, S):- sumaPom(N, S, 1).
% dovoljno je ici do N/2, suma je inicijalno 0, tako da na primer za N=1 vraca 0
8 sumaPom(N, 0, I):- I>N//2.
% ako I deli, dodajemo ga na tekucu sumu i pozivamo za sledeceg kandidata delioca
10 sumaPom(N, S, I):- I<=N//2, N mod I:=0, I1 is I+1, sumaPom(N, S1, I1), S is S1+I.
% ako I ne deli, ne dodajemo ga na tekucu sumu vec samo pozivamo za sledeceg
  kandidata delioca
12 sumaPom(N, S, I):- I<=N//2, N mod I:=0, I1 is I+1, sumaPom(N, S, I1).

14 % ako proveraN prodje, tada se izvršava predikat sumaDelilaca i proverava da li je
  dobijena suma jednaka broju N, ako proveraN ne prodje, ispisace se poruka i
  prekinuti program
savršen(N):- proveraN(N), sumaDelilaca(N, S), N:=S.
16
/*
18 | ?- savrsen(6)
yes
20 | ?- savrsen(-6)
Broj nije prirodan
22 no
*/

```

Rešenje 8.30

```

% maksimum dva broja
2 % I nacin:
maksimum(A,B,M):- A>=B, M is A.
4 maksimum(A,B,M):- A<B, M is B.

```

```

% II nacin bez trece promenjive:
6 % maksimum(A,B,A):- A>=B.
% maksimum(A,B,B):- A<B.
8
% suma prvih N prirodnih brojeva
10 suma(1,1).
suma(N,S):- N>1, N1 is N-1, suma(N1,S1), S is S1+N.
12
% suma parnih prirodnih brojeva od 2 do N
14 % moze se dodati provera N mod 2 =:= 0 u pravilu, ali i bez toga sam prepoznaje za
neparne da je netacan upit jer rekurzijom dodje do sumaParnih(1,S) sto je netacna
cinjenica u bazi
sumaParnih(2,2).
16 sumaParnih(N,S):- N>2, N1 is N-2, sumaParnih(N1,S1), S is S1+N.

% proizvod prvih N prirodnih brojeva
18 proizvod(1,1).
20 proizvod(N,P):- N>1, N1 is N-1, proizvod(N1,P1), P is P1*N.

% proizvod neparnih prirodnih brojeva od 1 do N
22 proizvodNeparnih(1,1).
24 proizvodNeparnih(N,P):- N>1, N1 is N-2, proizvodNeparnih(N1,P1), P is P1*N.

% ispis cifara unetog prirodnog broja N
26 cifra(0, nula).
28 cifra(1, jedan).
cifra(2, dva).
30 cifra(3, tri).
cifra(4, cetiri).
32 cifra(5, pet).
cifra(6, sest).
34 cifra(7, sedam).
cifra(8, osam).
36 cifra(9, devet).

% ukoliko nije prirodan broj, cut operatorom sprecavamo poziv poslednjeg predikata
38 cifre(N):- N < 1, !.
40
% ukoliko je jednocifren svodi se na poziv predikata cifra
42 % write(t) gde je t neki term, ispisuje term
% nl (newline) - ispisuje se novi red
44 % obratiti paznju na upotrebu cut operatora ! - sprecavamo poziv poslednjeg predikata
za jednocifrene
cifre(N):- N > 1, N < 10, cifra(N, C), write(C), nl, !.
46
% ukoliko nije jednocifren, racunamo tekucu cifru koju ispisujemo i ostatak broja za
koji se ponovo poziva predikat
48 cifre(N):- N1 is (N // 10), cifre(N1), N2 is (N mod 10), cifra(N2, C), write(C), nl.

```

Rešenje 8.31

```

1 % pomocni predikat za proveru pripadnosti elementa listi
sadrzi(X, [X|_]):- !.
3 sadrzi(X, [_|R]):- G \== X, sadrzi(X, R).

% izbacivanje duplikata iz liste
5 duplikati([], []).
7 duplikati([G|R], [G|R1]):- not(sadrzi(G, R)), duplikati(R,R1), !.
duplikati([_|R], R1):- duplikati(R,R1).
9
% pomocni predikat za spajanje dve liste
11 spoji([], L, L).
spoji([G|R1], L2, [G|R3]):- spoji(R1, L2, R3).
13
% unija listi - jedna ideja: spajamo liste pa uklanjamo duplikate
15 % za vezbu implementirati ovaj predikat tako da se duplikati uklanjaju pri samom
spajanju listi
unija(L1, L2, L):- spoji(L1, L2, L3), duplikati(L3, L).
17
% presek listi
19 presek([], _, []).
presek([G|R1], L2, [G|R3]):- sadrzi(G, L2), presek(R1, L2, R3), !.

```

```

21 presek([_|R1], L2, L):- presek(R1, L2, L).
23 % razlika listi L1 i L2
razlika([], _, []).
25 razlika([G1|R1], L2, R3):- sadrzi(G1, L2), razlika(R1, L2, R3), !.
razlika([G|R1], L2, [G|R3]):- razlika(R1, L2, R3).

```

Rešenje 8.32

```

/*
2 strukturama oblika k(boja, nacionalnost, jelo, pice, kucniLjubimac) opisujemo date
  cinjenice, a u listi L su kuće poredjane jedna pored druge, tako da po redosledu
  u listi imamo informaciju da li je kuća desno od neke druge kuće i da li su kuće
  jedna pored druge
*/
4
/*
6 pomocni predikat koji proverava da li je X clan liste, njemu prosledjujemo strukturu
  k sa poznatim vrednostima iz teksta i opisujemo kakva kuća treba da bude u listi
*/
8 clan(X, [X|_]).
clan(X, [_|R]):- clan(X,R).
10
% predikat smesta u listu L kuće koje zadovoljavaju uslove iz teksta, tj. predikat L
  unifikuje sa resenjem zagonetke
12 % u listu ubacujemo cinjenice koje su vezane za raspored kuća i to samo one koje
  jednoznacno odredjuju poziciju kuće u listi
kuce(L):- L = [ k(_,norvezanin,_,_,_),
14   k(plava,_,_,_,_),
   k(.,.,.,mleko,.),
16   k(.,.,.,_,_),
   k(.,.,.,_,_) ],
18   % dodajemo kakve sve kuće treba da budu u listi
   clan(k(crvena, englez,.,_,_),L),
   clan(k(., spanac,.,.,pas),L),
20   clan(k(zelena,.,.,kafa,.),L),
   clan(k(., ukrajinac,.,caj,.),L),
22   % kada informacija daje relaciju za neke dve kuće iz liste koristimo predikate
   desno i pored
24   desno(k(zelena,.,.,kafa,.),k(bela,.,.,_,_),L),
   clan(k(.,.,spagete,.,puz),L),
26   clan(k(zuta,.,pica,.,_),L),
   pored(k(.,.,piletina,.,_),k(.,.,.,lisica),L),
28   pored(k(.,.,pica,.,_),k(.,.,.,konj),L),
   clan(k(.,.,brokoli,narandza,.),L),
30   clan(k(.,japanac,susi,.,_),L),
   % medju datim informacijama se ne pominje zebra niti voda, ali posto je krajnje
   pitanje vezano za ove pojmove, moramo dodati da takvi clanovi treba da postoje u
   resenju zagonetke
32   clan(k(.,.,.,zebra),L),
   clan(k(.,.,.,voda,.),L).
34
% proverava da li su kuće X i Y jedna pored druge u listi L
36 pored(X,Y,[X,Y|_]).
pored(X,Y,[Y,X|_]).
38 pored(X,Y,[_|R]):- pored(X,Y,R).
40
% proverava da li je kuća X desno od kuće Y u listi L
desno(X,Y,[Y,X|_]).
42 desno(X,Y,[_|R]):- desno(X,Y,R).
44
% predikat zagonetka daje odgovor na pitanje cija je zebra, a ko pije vodu, tako sto
  prvo trazi resenje zagonetke pa iz njega izdvaja samo potrebne clanove
zagonetka(X,Y):- kuce(L), clan(k(.,X,.,.,zebra),L), clan(k(.,Y,.,voda,.),L) .
46
/*
48 resenje:
50 | ?- kuce(L)
L = [k(zuta,norvezanin,pica,voda,lisica),k(plava,ukrajinac,piletina,caj,konj),k(
  crvena,englez,spagete,mleko,puz),k(bela,spanac,brokoli,narandza,pas),k(zelena,
  japanac,susi,kafa,zebra)]

```

```

52 odgovor na pitanje:
54 | ?- zagonetka(X,Y)
56 X = japanac
   Y = norvezanin
58
*/

```

Rešenje 9.2

```

% funkciji labeling prosledjujemo zahtev za minimizaciju trazenog izraza u listi
2 petocifren :- Vars = [A,B,C,D,E],
   % definisemo domen
4   A :: 1..9,
   B :: 0..9,
6   C :: 0..9,
   D :: 0..9,
8   E :: 0..9,
   % dodajemo uslov
10  alldifferent(Vars),
   % prilikom obelezavanja prosledjujemo i uslov minimizacije
12  labeling([minimize(A+2*B-3*C+4*D-5*E)],Vars),
   % prevodimo dobijene vrednosti u broj
14  Broj is 10000*A+1000*B+100*C+10*D+E,
   % ispisujemo resenje
16  write(Broj), nl.

```

Rešenje 9.3

```

% promenljive oznacavaju broj novcica u kombinaciji redom za A - 1 din, B - 2 din, C
- 5 din, D - 10 din, E - 20 din.
2 kombinacije(Vars) :- Vars = [A,B,C,D,E],
   % definisemo domen
4   A :: 1..50,
   B :: 1..25,
6   C :: 1..10,
   D :: 1..5,
8   E :: 1..2,
   % dodajemo uslov
10  A+2*B+5*C+10*D+20*E #= 50,
   labeling(Vars),
12  % ispisujemo resenje
   write(A+2*B+5*C+10*D+20*E = 50), nl.

```

Rešenje 9.4

```

magicni(Vars):- Vars=[X1,X2,X3,X4,X5,X6,X7,X8,X9],
2   % domen za sve je isti
   Vars :: 1..9,
4   % uslov razlicitosti
   alldifferent(Vars),
6   % uslovi za zbirove
   X1+X2+X3#=15,
8   X4+X5+X6#=15,
   X7+X8+X9#=15,
10  X1+X4+X7#=15,
   X2+X5+X8#=15,
12  X3+X6+X9#=15,
   X1+X5+X9#=15,
14  X3+X5+X7#=15,
   labeling(Vars),
16  write(X1), write(' '), write(X2), write(' '), write(X3), nl,
   write(X4), write(' '), write(X5), write(' '), write(X6), nl,
18  write(X7), write(' '), write(X8), write(' '), write(X9), nl.
/*
20 | ?- magicni(Vars).
   2 7 6
22  9 5 1

```



```

4 3 8
24 Vars = [2,7,6,9,5,1,4,3,8] ?;
2 9 4
26 7 5 3
6 1 8
28 Vars = [2,9,4,7,5,3,6,1,8] ?;
4 3 8
30 9 5 1
2 7 6
32 ...
*/

```

Rešenje 9.5

```

1 pronadji(Vars):- Vars=[X,Y,Z],
    % definisemo domen
3   X :: 1..90,
    Y :: 2..2..60,
5   Z :: 1..10..100,
    % definisemo ogranicenja
7   Z #=< X,
    2*X+Y*X+Z #=< 34,
9   % instanciramo promenljive
    labeling(Vars).

11
/*
13 | ?- pronadji(Vars)
Vars = [1,2,1] ?;
15 Vars = [1,4,1] ?;
Vars = [1,6,1] ?;
17 Vars = [1,8,1] ?;
Vars = [1,10,1] ?;
19 ...
21 */

```

Rešenje 9.6

```

1 jednakost(Vars):- Vars=[T,W,O,F,U,R],
2   Vars :: 0..9,
    % isključujemo za T i F nulu iz domena
4   T#\=0,
    F#\=0,
6   alldifferent(Vars),
    2*(T*100+W*10+O) #= F*1000+O*100+U*10+R,
8   labeling(Vars),
    write(' '), write(T), write(W), write(O), nl,
10  write('+'), write(T), write(W), write(O), nl,
    write('-----'), nl,
12  write(F), write(O), write(U), write(R), nl.

/*
14 | ?- jednakost(Vars)
734
16 +734
-----
18 1468
Vars = [7,3,4,1,6,8] ?;
20 765
+765
-----
22 1530
24 ...
26 */

/*
28 BITNO: Ukoliko se na kraju predikata doda fail, ispisace se sva resenja pri pozivu,
    inace staje cim pronadje jedno resenje, pa sledece dobijamo kucanjem ; za
    nastavljjanje pretrage
30 kao u prethodnom primeru

```

```

32 */
jednakostSvi(Vars):- Vars=[T,W,O,F,U,R],
34   Vars :: 0..9,
   % isključujemo za T i F nulu iz domena
36   T#\=0,
   F#\=0,
38   alldifferent(Vars),
   2*(T*100+W*10+O) #= F*1000+O*100+U*10+R,
40   labeling(Vars),
   write(' '), write(T), write(W), write(O), nl,
42   write('+'), write(T), write(W), write(O), nl,
   write('-----'), nl,
44   write(F), write(O), write(U), write(R), nl,
   % odvajamo resenja jer zbog fail ne staje kod prvog
46   % resenja vec ce nastaviti pretragu
   write('-----'), nl, fail.

```

Rešenje 9.7

```

pronadji(Vars):- Vars=[X,Y,Z,W],
2   % definisemo domen
   X :: 1..10,
4   Y :: 1..2..51,
   Z :: 10..10..100,
6   W :: 1..7..1002,
   % definisemo ogranicenja
8   2*W #=< X,
   3+Y #=< Z,
10  X-11*W+Y+11*Z #=< 100,
   % instanciramo promenljive
12  labeling(Vars).
/*
14 Ovo je primer sistema nejednacina bez resenja:
16 | ?- pronadji(Vars)
no
18 */

```

Rešenje 9.8

```

% A1, B1, C1, D1, E1 predstavljaju brojeve na glavnoj dijagonali
% A2, B2, C1, D2, E2 predstavljaju brojeve na glavnoj dijagonali
2 dijagonale(Vars):- Vars=[A1,B1,C1,D1,E1,A2,B2,D2,E2],
4   % domen za sve je isti
   Vars :: 1..9,
6   % uslov razlicitosti
   alldifferent(Vars),
8   % uslovi za zbirove
   A1+B1+C1+D1+E1#=25,
10  A2+B2+C1+D2+E2#=25,
   % uslovi za poredak
12  A1#<B1, B1#<C1, C1#<D1, D1#<E1,
   A2#<B2, B2#<C1, C1#<D2, D2#<E2,
14  labeling(Vars),
   write(A1), write(' '), write(A2), nl,
16  write(' '), write(B1), write(' '), write(B2), nl,
   write(' '), write(C1), nl,
18  write(' '), write(D2), write(' '), write(D1), nl,
   write(E2), write(' '), write(E1), nl.

```

Rešenje 9.9

```

1 /*
   H - broj komada hleba, K - broj komada kifli
3
   H>=0
5   K>=0

```

```
7  S obzirom na to da imamo 20kg brasna na raspolaganju, mozemo napraviti:
9  - najvise 20000/120 kifli
   - najvise 20000/300 hleba

11 K <= 20000/120 ~ 166
   H <= 20000/300 ~ 66

13
15 S obzirom na to da imamo 20h na raspolaganju, mozemo napraviti:
   - najvise 1200/12 kifli
   - najvise 1200/10 hleba

17
19 H <= 1200/10 = 120
   K <= 1200/12 = 100

21 najoptimalnije je za gornju granicu domena postaviti minimum od dobijenih vrednosti,
   tj. sve ukupno H <= 66, K <= 100

23 */

25 pekara(Vars) :- Vars = [H, K],
   H :: 0..66,
27   K :: 0..100,

29 /*
   Ogranicenja vremena:
31 - vreme potrebno za mesenje jednog hleba je 10 min,
   tj. za mesenje H komada hleba potrebno je 10*H minuta
33 - vreme potrebno za mesenje jedne kifle je 12 min,
   tj. za mesenje K komada kifli potrebno je 12*K minuta

35
37 Ukupno vreme koje je na raspolaganju iznosi 20h, tako da je:
   10*H + 12*K <= 1200
39 */

41
43 10*H + 12*K #=< 1200,

45
47 /*
   Ogranicenje materijala:
   - za jedan hleb potrebno je 300g brasna, a za H komada hleba potrebno je H*300 grama
49 - za jednu kifli potrebno je 120g brasna, a za K komada kifli potrebno je K*120
   grama

51
53 Ukupno, na raspolaganju je 20kg brasna, tako da je:
   300*H + 120*K <= 20000
55 */

57 300*H + 120*K #=< 20000,

59
61 /*
   Zarada iznosi:
55 - 7din/hleb, tj. zarada za H komada hleba bice 7*H
   - 9din/kifla tj. zarada za K komada kifli bice 9*K

57
59 Ukupna zarada iznosi:
   7*H + 9*K - funkcija koju treba maksimizovati - ovo dodajemo prilikom obelezavanja
61 */

63 labeling([maximize(7*H+9*K)], Vars),
   Zarada is 7*H+9*K,
   write('Maksimalna zarada od '), write(Zarada), write(' dinara se ostvaruje za '),
   write(H), write(' komada hleba i '), write(K), write(' komada kifli. '), nl.
```

Rešenje 9.10

```
heksagon(Vars):- Vars=[A,B,C,D,E,F,G,H,I,J,K,L,M,N,O,P,Q,R,S],
2   % domen za sve je isti
   Vars :: 1..38,
4   % uslov razlicitosti
   alldifferent(Vars),
6   /*
   Dodajemo ogranicenja za svaku horizontalnu liniju
8   A,B,C
```

```

10 D,E,F,G
11 H,I,J,K,L
12 M,N,O,P
13 Q,R,S
14 */
15 A+B+C#=38,
16 D+E+F+G#=38,
17 H+I+J+K+L#=38,
18 M+N+O+P#=38,
19 Q+R+S#=38,
20 % Dodajemo ogranicenja za svaku od glavnih dijagonala
21 H+M+Q#=38,
22 D+I+N+R#=38,
23 A+E+J+O+S#=38,
24 B+F+K+P#=38,
25 C+G+L#=38,
26 % Dodajemo ogranicenja za svaku od sporednih dijagonala
27 A+D+H#=38,
28 B+E+I+M#=38,
29 C+F+J+N+Q#=38,
30 G+K+O+R#=38,
31 L+P+S#=38,
32
33 labeling(Vars),
34 write(' '), write(A), write(' '), write(B), write(' '), write(C), nl,
35 write(' '), write(D), write(' '), write(E), write(' '), write(F), write(' '), write(G), nl,
36 write(H), write(' '), write(I), write(' '), write(J), write(' '), write(K), write(' '),
37 write(L), nl,
38 write(' '), write(M), write(' '), write(N), write(' '), write(O), write(' '), write(P), nl,
39 write(' '), write(Q), write(' '), write(R), write(' '), write(S), nl.

```

Rešenje 9.11

```

1 /*
2 kompanija ima 250 zaposlenih radnika
3 za sve njih organizuje dodatnu obuku
4 ako je E promenjiva za Elixir, a D za Dart
5 mora da vazi (gruba procena)  $E \leq 250$ ,  $D \leq 250$  i  $E + D = 250$ 
6 */
7
8 kompanija(Vars) :- Vars = [E, D],
9     Vars :: 0..250,
10
11     % dodajemo uslov za ukupan broj radnika
12     E+D#=250,
13
14     % dodajemo ogranicenje za broj projekat sati
15     150*E+170*D#=<51200,
16
17     % dodajemo ogranicenje za raspoloziva sredstva
18     100*E+105*D#=<26000,
19
20 /*
21 ukupna zarada se dobija kada od ostvarene dobiti preko broja projekat/sati oduzmemo
22 gubitak za placanje kurseva radnicima, tj. funkcija koju treba maksimizovati je:
23
24 150*5*E + 170*6*D - (100*E + 105*D) --> ovo dodajemo kod obelezavanja
25 */
26 labeling([maximize(150*5*E + 170*6*D - (100*E + 105*D)),Vars),
27 Zarada is (150*5*E + 170*6*D - (100*E + 105*D)),
28 write('Maksimalna zarada je '), write(Zarada),
29 write(', broj radnika koje treba poslati na kurs Elixir je '), write(E),
30 write(', a broj radnika koje treba poslati na kurs Dart je '), write(D),nl.

```

Rešenje 9.12

```

1 /*

```

```

3 Kraljica se može pomerati u pravoj liniji vertikalno, horizontalno ili dijagonalno,
  za bilo koliko slobodnih polja. Rešenje zahteva da nikoje dve dame ne dele istu
  vrstu, kolonu ni dijagonalu. Problem je opstiji od poznatog problema osam
  sahovskih dama jer u ovom problemu treba N dama postaviti na sahovsku tablu
  dimenzije NxN koje se zadaje kao ulazni podatak.

5 *** U rešenju se koristi petlja FOREACH. Opsti oblik:

7   foreach(E1 in D1, . . ., En in Dn, LocalVars, Goal)

9 - E1 - može biti bilo koji term, najcesce promenljiva
  - D1 - lista ili opseg celih brojeva oblika L..R (L i R su ukljuceni)
11 - LocalVars (opciono) - lista lokalnih promenljivih
  - Goal - predikat koji se poziva

13 Primeri:

15 | ?- foreach(I in [1,2,3], write(I))
17 123
18 yes

19 Domen za X i Y u ovom slucaju mora biti iste kardinalnosti
21 | ?- foreach((X,Y) in ([a,b], 1..2), writeln(X==Y))
22 a==1
23 b==2
24 yes

25 | ?- foreach(X in [a,b], Y in [1,2], writeln(X==Y))
26 a==1
27 a==2
28 b==1
29 b==2
30 yes
31 */

33 % Indeksi niza Qs I i J predstavljaju kolone u kojima su kraljice, a elementi niza Qs
  [I] i Qs[J] predstavljaju vrste u kojima se nalaze kraljice
35 kraljice(N):- length(Qs,N), % Qs je niz, tj. lista od n promenljivih
  Qs :: 1..N,

37 % I je implicitno razlicito od J -> razlicite kolone
  % Qs[I] treba da bude razlicito od Qs[J] -> razlicite vrste
39 % apsolutna vrednost razlike vrsta treba da bude razlicita od apsolutne vrednosti
  razlike kolona -> razlicite dijagonale
  foreach(I in 1..N-1, J in I+1..N,
41 (Qs[I] #\= Qs[J], abs(Qs[I]-Qs[J]) #\= J-I)),
  labeling(Qs),
43 writeln(Qs), fail.

45 % stavljanjem predikata fail na kraj, dobicemo sve moguće kombinacije za kraljice
  % bez fail, program staje posle prvog pronadjenog rezultata

```

Rešenje 9.13

```

1 /*
  Primer magicne sekvence za N=5:
3 [2,1,2,0,0]

5 Izvršavanje programa za gornji primer (1 sa desne strane kad je uslov S[J]#=I
  ispunjen, 0 kad nije):
  sum([(S[J]#=I) : J in 1..N])#=S[I+1])
7 I=0
  S[1]==0 0
9 S[2]==0 0
  S[3]==0 0
11 S[4]==0 1
  S[5]==0 1
13 -----
  ukupno 2
15 S[1]==2 tacno! (imamo dve nule)
I=1
17 S[1]==1 0
  S[2]==1 1
19 S[3]==1 0

```

```

21     S[4]==1 0
    S[5]==1 0
    -----
23     ukupno 1
    S[2]==1 tacno! (imamo jednu jedinicu)
25 I=2
    S[1]==2 1
27     S[2]==2 0
    S[3]==2 1
29     S[4]==2 0
    S[5]==2 0
31     -----
    ukupno 1
33     S[3]==2 tacno! (imamo dve dvojke)
    I=3
35     S[1]==3 0
    S[2]==3 0
37     S[3]==3 0
    S[4]==3 0
39     S[5]==3 0
    -----
41     ukupno 0
    S[4]==0 tacno! (imamo 0 trojki)
43
    I=4
45     S[1]==4 0
    S[2]==4 0
47     S[3]==4 0
    S[4]==4 0
49     S[5]==4 0
    -----
51     ukupno 0
    S[5]==0 tacno! (imamo 0 cetvorki)
53
    */
55
    /*
57     Zadavanje listi u obliku: [T : E1 in D1, . . . , En in Dn, LocalVars, Goal]
59     - za svaku kombinacija vrednosti E1,...,En, ako predikat Goal uspe, T se
    dodaje u listu
61     - LocalVars i Goal su opcioni argumenti
63
    */
    magicna(N):- length(S,N),
65         S :: 0..N-1,
    /* dodajemo ogranicenja:
67     element na poziciji S[I+1] treba da bude jednak broju elemenata liste koji su jednaki
        sa I, taj broj dobijamo sumiranjem liste ciji su elementi poredjenja na
        jednakost elementa liste sa trazanim brojem I
    */
69     foreach(I in 0..N-1,
        sum([(S[J] #= I) : J in 1..N]) #= S[I+1]),
71     labeling(S),
    writeln(S), fail. % da bismo dobili sve, a ne samo jednu magicnu sekvencu
    dodajemo fail predikat na kraj

```